



ALMANAC

Engineering Handbook

Shiv Vyas · Revision E

Almanac is a personal iOS app that gathers every signal from the devices you already wear and carry (Apple Watch, iPhone, Whoop, Fitbit, a Renpho scale, your bank, your calendar) into one local store, scores your life across nine sectors, and puts a coach called LIFO on top of all of it. This handbook is the single reference for how it is built, why it is built that way, what keeps it fast and cheap, and what it does when things go wrong. It is written to be studied: every chapter ends with a reading order through the code and questions to check yourself against.

REVISION E 15 SEPTEMBER 2026 IOS 26 · WATCHOS 26 · SWIFT 6 ARCHITECTURE: MVVM
STATUS: TESTFLIGHT BUILD 17 AUDIENCE: THE AUTHOR

Table of contents

Every line is a link as well as a page number. Part two is the features, one chapter per screen, each in the same shape.

PART ONE: FOUNDATIONS

1	Architecture	7
1.1	System context	7
1.2	Module map	8
1.3	The DailyMetrics spine	10
1.4	The arbiter: merge by rank	12
1.5	Local first topology, and what syncs now	13
1.6	Conflict resolution	14
1.7	One store per account	14
1.8	The privacy boundary	15
1.9	Study notes	16
2	MVVM and project structure	17
2.1	Why MVVM here	17
2.2	What each layer owns	18
2.3	The screen first directory layout	19
2.4	Anatomy of a feature folder	20
2.5	The composition root	22
2.6	Shared components	23
2.7	The preview and checks harness	23
2.8	Rules and review checklist	25
2.9	Study notes	25
3	Design system	27
3.1	Principle	27
3.2	Colour tokens	28
3.3	One type scale	29
3.4	Space, radius and the glass panel	31
3.5	Layout metrics: phone and iPad	31
3.6	Component inventory	32
3.7	Motion pieces	35
3.8	Dark mode	35
3.9	The typography check	36
3.10	Study notes	36
4	The app shell, accounts and sign-in	38
4.1	Intent	38
4.2	Files and folders	39
4.3	Launch: from @main to the first tab	40
4.4	Signing in: a code, then a name	42
4.5	The returning-user door	44
4.6	Keeping a session alive	44
4.7	Several accounts on one device	46
4.8	The app delegate and push registration	47
4.9	Two shells, one set of screens	48
4.10	Edge cases and tests	49
4.11	Study notes	50

PART TWO: THE SCREENS

5	Onboarding	53
5.1	Intent	53
5.2	Files and folders	54
5.3	Model	55
5.4	ViewModel	57
5.5	View	59
5.6	How it works	62
5.7	Performance and cost	63
5.8	Edge cases and tests	64
5.9	Study notes	65
6	Today and the calendar	67
6.1	Intent	68
6.2	Files and folders	69
6.3	Model	70
6.4	ViewModel	73
6.5	View	76
6.6	How it works	78
6.7	Performance and cost	79
6.8	Edge cases and tests	80
6.9	Study notes	81
7	Health	83
7.1	Intent	83
7.2	Files and folders	85
7.3	Model	86
7.4	ViewModel	89
7.5	View	91
7.6	How it works	94
7.7	Performance and cost	95
7.8	Edge cases and tests	97
7.9	Study notes	98
8	Live sessions	100
8.1	Intent	100
8.2	Files and folders	101
8.3	Model	102
8.4	ViewModel	105
8.5	View	110
8.6	How it works	112
8.7	Performance and cost	113
8.8	Edge cases and tests	114
8.9	Study notes	115

9	The Watch companion	117	14	LIFO, the coach	197
9.1	Intent	117	14.1	Intent	198
9.2	Files and folders	118	14.2	Files and folders	199
9.3	Model	119	14.3	Model	200
9.4	ViewModel	121	14.4	ViewModel	202
9.5	View	126	14.5	View	203
9.6	How it works	128	14.6	How a turn is answered	204
9.7	Performance and cost	129	14.7	Speech in, speech out	207
9.8	Edge cases and tests	130	14.8	LIFO speaks first	209
9.9	Study notes	131	14.9	The privacy boundary	211
10	The workout library	133	14.10	Performance and cost	212
10.1	Intent	133	14.11	Edge cases and tests	214
10.2	Files and folders	134	14.12	Study notes	215
10.3	Model	136	15	The calendar assistant	217
10.4	ViewModel	138	15.1	Intent	217
10.5	View	141	15.2	Files and folders	217
10.6	How it works	144	15.3	Model	218
10.7	Performance and cost	145	15.4	ViewModel	220
10.8	Edge cases and tests	146	15.5	View	221
10.9	Study notes	148	15.6	How it works	222
11	Money	150	15.7	Performance and cost	223
11.1	Intent	150	15.8	Edge cases and tests	224
11.2	Files and folders	151	15.9	Study notes	225
11.3	Model	152	16	Social	226
11.4	ViewModel	154	16.1	Intent	227
11.5	View	156	16.2	Files and folders	228
11.6	How it works	158	16.3	Model	228
11.7	Performance and cost	159	16.4	ViewModel	231
11.8	Edge cases and tests	161	16.5	View	234
11.9	Study notes	162	16.6	How it works	235
12	Notes and the plan	164	16.7	Performance and cost	237
12.1	Intent	165	16.8	Edge cases and tests	238
12.2	Files and folders	166	16.9	Study notes	239
12.3	Model	167	17	Profile and settings	241
12.4	ViewModel	170	17.1	Intent	241
12.5	View	171	17.2	Files and folders	242
12.6	How it works	174	17.3	Model	243
12.7	Performance and cost	176	17.4	ViewModel	244
12.8	Edge cases and tests	178	17.5	View	246
12.9	The plan that remains	179	17.6	How it works	248
12.10	Study notes	180	17.7	Performance and cost	249
13	The Life board and the nine sectors	182	17.8	Edge cases and tests	251
13.1	Intent	182	17.9	Study notes	252
13.2	Files and folders	182			
13.3	Model	184			
13.4	ViewModel	187			
13.5	View	189			
13.6	How it works	190			
13.7	Performance and cost	192			
13.8	Edge cases and tests	194			
13.9	Study notes	195			

18	Widgets, Live Activities and notifications	254	22	Production edge cases	317
18.1	Intent	255	22.1	How to read this register	317
18.2	Files and folders	255	22.2	Time and calendars	318
18.3	Model	257	22.3	Apple Health	319
18.4	ViewModel	260	22.4	Whoop	321
18.5	View	262	22.5	Fitbit	322
18.6	How it works	263	22.6	Banks and Plaid	323
18.7	Performance and cost	264	22.7	Calendar and EventKit	324
18.8	Edge cases and tests	265	22.8	Sessions and accounts	325
18.9	Study notes	266	22.9	Notes sync	326
PART THREE: UNDER THE HOOD					
19	Integrations	269	22.10	Live sessions and sensors	326
19.1	Apple HealthKit	269	22.11	The watch hand-off	328
19.2	Whoop	273	22.12	Video and the catalog	329
19.3	Fitbit	276	22.13	LIFO and the cloud tier	330
19.4	The arbiter: who wins a column	279	22.14	Nudges and push	332
19.5	Plaid	280	22.15	Social	333
19.6	EventKit and the calendar	281	22.16	UI and data display	334
19.7	Supabase auth and REST	282	22.17	Study notes	334
19.8	Apple Foundation Models	284	23	Operations	336
19.9	ElevenLabs: hearing and speaking	285	23.1	Setup runbook	336
19.10	Apple Speech and the microphone	285	23.2	Secrets	339
19.11	YouTube embeds	286	23.3	Shipping to TestFlight	340
19.12	APNs and the morning nudge	286	23.4	Testing strategy	340
19.13	Screen Time: still cut	287	23.5	The design preview and native checks harness	343
19.14	What the integrations share	287	23.6	Driving the simulator from the terminal	344
19.15	Study notes	288	23.7	Working in parallel	344
20	The backend	289	23.8	Guards that run without a test	345
20.1	Configuration	289	23.9	Study notes	345
20.2	The migrations, in order	291	24	Cost and the allowance	346
20.3	The shared door	295	24.1	The ceiling	346
20.4	The Edge Functions	296	24.2	Where the money goes	346
20.5	Tests	301	24.3	The token ledger	347
20.6	Email templates	302	24.4	The lines that are not metered	350
20.7	The static site	302	24.5	Why the ceiling holds	351
20.8	Study notes	303	24.6	What it costs today	351
21	Data reference	304	24.7	Study notes	353
21.1	Four rules every model obeys	304			
21.2	DailyMetrics	304	APPENDICES: REFERENCE		
21.3	UserGoals, goal evaluation and anomalies	306	A	Product thesis and roadmap	355
21.4	Source records and the raw archive	307	A.1	The thesis	355
21.5	Money	308	A.2	Product PR FAQ	356
21.6	Plan entries and habit ticks	309	A.3	What shipped since Revision D	357
21.7	Calendar events and chat messages	310	A.4	What is still open	359
21.8	Notes	310	A.5	Study notes	362
21.9	Sectors	311			
21.10	The workout catalog and bookmarks	312			
21.11	Accounts, scopes and the container	313			
21.12	The stores	313			
21.13	Adding a field without breaking installed stores	315			
21.14	Study notes	316			

B	Decision log	363
B.1	Product and scope	363
B.2	Architecture and data	364
B.3	Accounts and sign-in	366
B.4	Design system and layout	367
B.5	Health and wearables	368
B.6	Live sessions and the watch	370
B.7	Workout library	371
B.8	Money	372
B.9	Notes and the plan	373
B.10	The Life sectors	374
B.11	LIFO and the assistant	375
B.12	Social	377
B.13	Backend and operations	378
B.14	Study notes	379

C	Glossary and reading order	380
C.1	Glossary	380
C.2	Reading order	385

PART ONE

Foundations

The shape of the system before any screen: how the modules depend on each other, where state lives, how a screen is built, what every screen is built from, and how a person gets through the door.

- 1. Architecture
- 2. MVVM and project structure
- 3. Design system
- 4. The app shell, accounts and sign-in

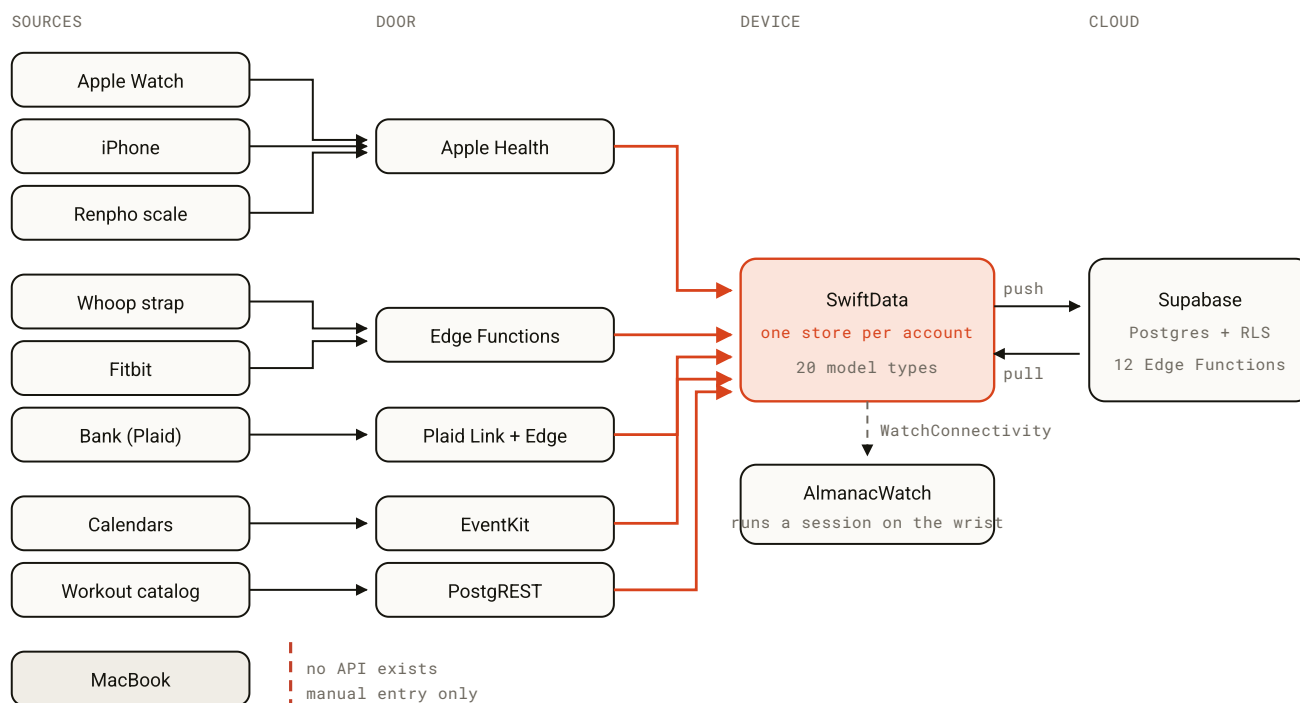
1 Architecture

Almanac is a local-first iOS app built on one bet: **the value is in the join, not the collection.** Seven sources of data now feed it, one table keyed by calendar day still receives all of the health signal, and every screen reads from the device before anything touches the network.

This chapter describes the system as it is on 15 September 2026, not the one the first edition planned. Since Revision D the package has grown from three targets to eight, the app has gained a watch, a bank, a second wearable, a calendar, a notes library, a nine-sector life board and a cloud tier for the coach. The architecture survived all of that because three early decisions held: one row per day, a compiler-enforced module graph, and a device that never waits for a server. Where the code and this handbook disagree, the code is right and this handbook is a bug.

1.1 System context

Every source reaches the app through one of four doors: Apple Health, a vendor REST API behind a Supabase Edge Function, a system framework on the device itself, or a table in Supabase that the app reads directly over PostgREST. The Apple Watch is now both a source, through Health, and a second screen, through WatchConnectivity. Everything still converges on the local SwiftData store before a single pixel is drawn.



Eight sources, four doors, one store. The MacBook still has no programmatic path at any privilege level; that is a platform fact, not a missing feature. The watch is the only box that both feeds the store (through Health) and reads from it (a live session mirrored over WatchConnectivity).

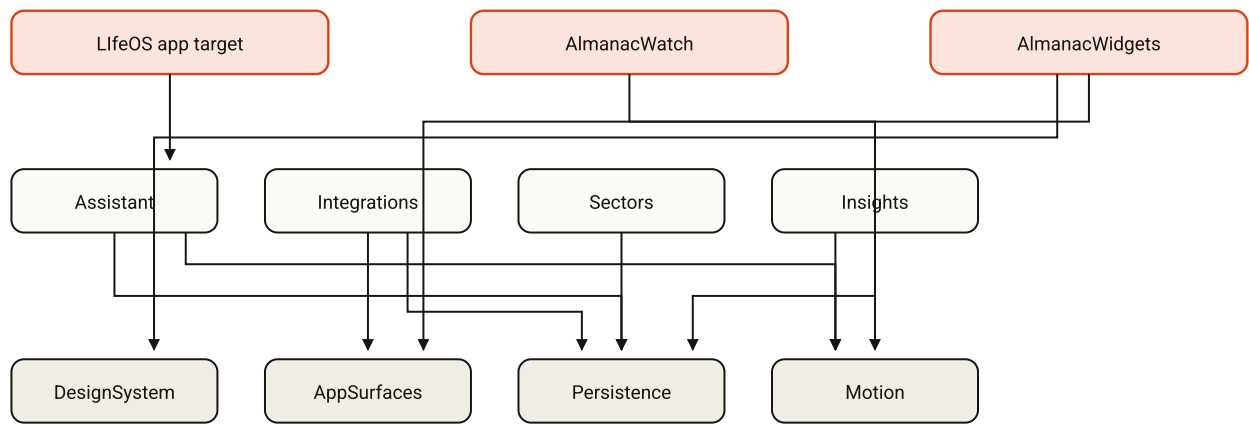
Each source, its door, and where its data lands:

SOURCE	DOOR	LANDS IN	CHAPTER
Apple Watch, iPhone, Renpho scale	<code>HealthKitReader</code> reads every <code>HealthMetric</code> Apple Health holds	<code>DailyMetrics</code> named columns and the <code>extras</code> bag	7, 19
Whoop strap	OAuth via the <code>whoop-callback</code> bridge, direct REST from the device, tokens in the keychain	<code>WhoopRawRecord</code> first, then derived into <code>DailyMetrics</code> , <code>SleepRecord</code> , <code>WorkoutRecord</code>	19
Fitbit	OAuth exchanged server side; <code>fitbit-sync</code> pulls under a refresh lease; the device never holds a Fitbit token	<code>DailyMetrics</code> through the same arbiter	19
Bank accounts	Plaid Link in the app, four Edge Functions hold the credentials	<code>MoneyEntry</code> , <code>MoneyAccount</code> , cursor per item on the device	11
Calendars	<code>EventKitSource</code> behind a <code>CalendarSource</code> protocol	<code>CalendarEvent</code> cache with windowed queries	6
Workout catalog	Read-only PostgREST client, cached locally	<code>CatalogVideo</code>	10
Live sensors	CoreBluetooth heart rate, CoreMotion on the wrist	<code>WorkoutRecord</code> on save; the readout is never persisted	8, 9
You	Quick log, journal, notes, check-ins	Whichever model owns it, with <code>MetricSource.manual</code> where it competes with a sensor	everywhere

1.2 Module map

`LifeOSKit` is a local Swift package providing compile-time boundaries. The app target contains only features and wiring, laid out per screen as described in Chapter 2. The dependency rules are enforced by the compiler, not by convention: a target that does not declare a dependency cannot import it, and the package manifest is the whole rule book.

```
// LifeOSKit/Package.swift
platforms: [.iOS("26.0"), .macOS("26.0"), .watchOS("26.0")],
targets: [
    .target(name: "AppSurfaces"),
    .target(name: "DesignSystem"),
    .target(name: "Persistence"),
    .target(name: "Integrations", dependencies: ["Persistence", "AppSurfaces"]),
    .target(name: "Insights", dependencies: ["Persistence"]),
    .target(name: "Sectors", dependencies: ["Persistence"]),
    .target(name: "Assistant", dependencies: ["Insights", "Persistence"]),
    .target(name: "Motion"),
    // plus one test target per library
]
```



Arrows read "depends on". The bottom row depends on nothing, which is what keeps it previewable and testable from a terminal.

Eight targets in two tiers. The four at the bottom have no dependencies at all; the four above them may reach down but never sideways. The app target imports all of them; the watch target imports `AppSurfaces` and `Motion`; the widget target imports `AppSurfaces` and `DesignSystem`.

TARGET	OWNS	DEPENDS ON	WHY IT IS ITS OWN TARGET
Persistence	Every <code>@Model</code> class, the stores, pure logic over rows (streaks, goal evaluation, anomaly evaluation), <code>UserScope</code>	nothing	The schema must be importable by anything without dragging a network client in with it
DesignSystem	Tokens, the type scale, spacing, layout metrics and every stateless primitive	nothing	Components stay previewable and reusable; it also builds for macOS so tests run without a simulator
AppSurfaces	The wire types shared with the watch and the widgets: <code>SurfaceSnapshot</code> , <code>LiveSessionReadout</code> , <code>WatchWire</code> , <code>WorkoutActivityAttributes</code> , <code>HUDPlacement</code> , <code>SessionRingMath</code>	nothing	Three processes decode these bytes; a shared target is the only way they cannot drift
Motion	<code>RepCounter</code> , a pure rep counter over wrist acceleration	nothing	Runs on the watch; must not pull <code>SwiftData</code> or networking onto the wrist
Integrations	Whoop, Fitbit, Apple Health, Plaid, EventKit, Supabase auth and REST, social, profiles, push tokens, notes sync, the live effort maths, the arbiter	Persistence, AppSurfaces	Every vendor is contained here; nothing above it knows a vendor exists
Insights	The coach: tasks, engines, router, context bundle, digest, chat wire, speech endpointing, spend series	Persistence	It may read rows and nothing else, which is the privacy boundary as a dependency edge
Sectors	The nine sectors, their scorers, the monthly close, projections, levers, the workout planner and library filter	Persistence	Pure scoring over stored rows; testable with fixture months
Assistant	The calendar tools the on-device model calls, and the context they run in	Insights, Persistence	Tools are domain logic with a strict read-only rule; they must not reach a network client mid-conversation

THE RULE THAT MAKES THE GRAPH WORTH HAVING

`Persistence` sits below `Integrations` and must not import it. That is why `DailyMetrics.sourcesData` stores a raw `String` rather than a `MetricSource`: the vocabulary of "who wrote this" lives one tier up, and the row only carries its name. The dependency arrow decided the type.

1.3 The DailyMetrics spine

This is still the single most consequential decision in the system. **One row per calendar day.** Every health source writes into it; every health reader reads from it. The row has grown two bags since the first edition, and both exist to keep that sentence true as the number of sources went from two to four.

```
// LifeOSKit/Sources/Persistence/DailyMetrics.swift
@Model
public final class DailyMetrics {
    #Unique<DailyMetrics>([\`date\`])

    /// Always `Calendar.startOfDay`. Never a timestamp.
    public var date: Date

    public var weightKg: Double?
    public var steps: Int?
    public var sleepMinutes: Int?
    // ... eleven named columns, all optional ...

    /// Everything Apple Health can supply that does not have a column of
    /// its own, as a JSON dictionary keyed by `HealthMetric.rawValue`.
    public var extrasData: Data?

    /// Who wrote each value, as a JSON dictionary keyed by `HealthMetric.rawValue`.
    public var sourcesData: Data?

    public var updatedAt: Date
    public var syncedAt: Date?
}
```

The named columns came first and are load-bearing: the sector scorers, the Whoop merge and the charts all read them by name. The `extras` bag is the long tail of Apple Health, more than thirty metrics most people have no data for, stored as `[String: Double]` so that adding a metric is one enum case and one mapping rather than a schema migration. The `sources` bag runs parallel to it and records provenance per metric per day.

Both bags decode to an empty dictionary rather than throwing when the bytes are unreadable. A day whose extras cannot be parsed still has its named columns, and losing the screen over the tail would be the worse failure. Both also treat writing `nil` as removing the key, never as storing a zero, for the reason the whole model is built on: **absent and zero are different things, and a false zero is worse than a blank in a health app.**

WHAT THE SINGLE-TABLE CHOICE STILL BUYS

- Trends become a sort, not a multi-source asynchronous join. `TodayViewModel.load` fetches one bounded window of rows and derives the grid, the streak and today's figures in one pass.
- The coach queries one table instead of orchestrating four network APIs mid-conversation, which is what makes an on-device model viable and what makes the cloud tier's context bundle small.
- Offline works by default, because nothing renders from a live network call.
- Adding a source means writing into the same row through the same arbiter. Fitbit shipped as a second wearable without a new pipeline; it is a loop around one function.
- The widgets and the watch decode a `SurfaceSnapshot` that is a projection of one row, so the same steps figure appears on the lock screen, the wrist and the hub without three queries.

`MetricsStore` is the only way anything writes to the row. Its `upsert` fetches or creates the day and applies a closure, and it is a merge: fields the closure does not touch are preserved. `upsertBatch` exists because a

save per row turned sixty days into sixty disk transactions on the main actor; a sync loop now makes one.

1.4 The arbiter: merge by rank

With one wearable the merge rule was a boolean called `fillGapsOnly`: Health fills what Whoop left blank. With two straps that rule cannot tell a Whoop value from a typed one, which is exactly the distinction a second wearable forces. The replacement is a rank, and it is a pure function so every branch can be tested without a device, a permission prompt or a store.

```
// LifeOSKit/Sources/Integrations/MetricArbiter.swift
public static func resolve(
    metric: HealthMetric,
    existing: Double?, existingSource: MetricSource?,
    incoming: Double?, incomingSource: MetricSource,
    ranking: SourceRanking
) -> (value: Double?, source: MetricSource?) {
    // A source having no reading is not evidence of zero.
    guard let incoming else { return (existing, existingSource) }
    guard existing != nil else { return (incoming, incomingSource) }

    let held = existingSource ?? MetricSource.legacy(for: metric)

    // Greater than or equal, not greater than. Equal rank means the same
    // source syncing again, which must be allowed to correct itself.
    guard ranking.rank(incomingSource, for: metric) >= ranking.rank(held, for: metric) else {
        return (existing, held)
    }
    return (incoming, incomingSource)
}
```

The ranks are decided per metric, because a source that is the authority on one number is a fallback on another:

RANK	WHO	WHEN
4	manual	Always. A figure a person typed is never overwritten by a background sync.
3	appleHealth	For metrics where <code>healthIsAuthoritative</code> : not claimed by a wearable and not typed, such as steps, active energy, weight from the scale. The latest reading is simply the truth, which is what lets an afternoon sync correct the morning's step count.
3	the primary wearable	For metrics <code>claimedByWearable</code> : resting HR, HRV, SpO2, respiratory rate, every sleep stage. Whoop by default; Fitbit if the person named it primary.
2	the other wearable	Fills a gap the primary left, never overwrites it.
1	appleHealth	For wearable-claimed metrics: the fallback when no strap has spoken.
0	unattributed	A value written before provenance was recorded. The first sync after upgrade re-attributes it.

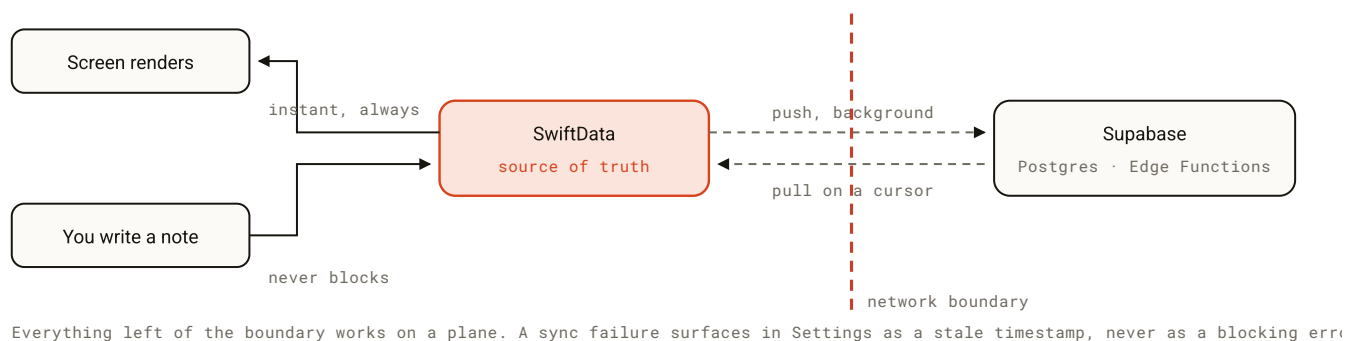
Rank 0 has one exception, and the exception matters. Weight and water are the two metrics a person can type. A figure typed before provenance existed is unattributed too, and reading it as unknown would let the next Health sync silently replace it. That is the one path that destroys data a person entered by hand, so `MetricSource.legacy(for:)` reads an unattributed value for a typed metric as typed. The test is named for the bug it prevents: `anUnattributedWeightIsNotReplacedByASync`.

`HealthApply.write` is the one place the `HealthMetric` vocabulary meets `DailyMetrics` columns. It loops the arbiter over a day's readings inside a single `upsert`, records the winning source in the bag as a side effect, and routes named metrics to their column and everything else to `extras`. The arbiter can be perfectly correct and still be wired into the wrong column; `HealthApplyTests` run against a real in-memory store precisely to catch that.

1.5 Local first topology, and what syncs now

SwiftData on the device is the source of truth for the UI. Supabase is identity, backup for some models, the only place a secret may live, and the place to run jobs a sleeping phone cannot. The first edition had one thing syncing (nothing, in practice). This edition has a precise list, and the split is worth knowing because it decides what a second device sees.

DATA	WHERE IT LIVES	HOW IT MOVES
Notes: folders, documents, blocks	Device and <code>note_folders</code> , <code>note_documents</code>	<code>NoteSync</code> pushes and pulls under a per-account cursor on every pass; the token is read per pass, never captured
Profile and avatar	Device cache and <code>profiles</code> , the <code>avatars</code> bucket	<code>ProfileClient</code> upserts the whole row; the avatar downloads only when its stamp changed
Friends, messages, groups, shared stats	Server only, read live	<code>SocialAPI</code> , <code>GroupAPI</code> , <code>SharedStatsClient</code> over PostgREST; RLS does the guarding
Plaid items	Credentials server side in <code>plaid_items</code> ; cursor and item ids on the device	Four Edge Functions; the device never sees an access token
Fitbit connection	Server side in <code>fitbit_connections</code>	<code>fitbit-sync</code> pulls under a lease; results land in <code>DailyMetrics</code>
Whoop tokens	Keychain, per account	The device calls Whoop directly; only the code exchange touches the server
Coach usage	<code>lifo_usage</code>	Atomic debit per cloud turn; the ceiling in Chapter 24
Push registrations and nudges	<code>device_tokens</code> , <code>nudge_log</code>	Token and timezone re-pushed on every foreground
Health rows, sleep, workouts, money entries, sector scores, calendar cache, chat	Device only	The <code>daily_metrics</code> tables exist in the schema from the first migration, but no client writes them today. A second device rebuilds them from the sources.



Reads and writes both terminate locally. The dashed paths are the only network-dependent edges in the whole app, and none of them sits on a user-facing critical path.

Two mechanisms make this cheap. `RootView` reloads every snapshot on `ModelContext.didSave`, so a sync that lands repaints the screen without anyone polling. And `SyncStalenessPolicy` decides whether an automatic pull is due at all: data older than an hour, or from before today's midnight, is worth refreshing; anything fresher is not. The day boundary is the clock that matters most, because the first open of the morning must pull the night's sleep, and a sync from 23:50 is only minutes old while knowing nothing about the night.

1.6 Conflict resolution

Within the health row, the arbiter above is the conflict rule: field-level, by rank, with a same-rank writer allowed to correct itself. It replaces the field-level last-write-wins the first edition described, and it is better for the same reason: a row-level overwrite would let the 04:00 Whoop pass discard the water you logged at 23:00, and a timestamp cannot tell a strap from a person.

Across devices, for the models that sync, the rule is simpler and per model. Notes carry an origin id and an `updated_at`; the pull applies rows newer than the cursor and the push sends rows changed since the last push, which is last-write-wins by row on a single-writer assumption that holds because one person edits one note at a time. The profile is an upsert of the whole row with explicit nulls, because an omitted key would leave the old value standing and make deleting a birthday impossible. Social rows never conflict because they are never edited locally; only status is updatable on a friendship, and the database says so.

KNOWN LIMITATION

The notes cursor depends on server clocks, not device clocks, so the skew problem the first edition warned about is gone for notes. It remains for anything that compares `updatedAt` on the device with a server timestamp, and today nothing does. If a model starts syncing health rows, the arbiter's rank must travel with the row or the merge will regress to a timestamp race.

1.7 One store per account

The first edition assumed a single user and said so. That assumption is gone: several people can sign in on one device, and the way that works is the most important structural change since Revision D. **Isolation is by store file, not by an owner column.**

```
// LifeOSKit/Sources/Persistence/UserScope.swift
public struct UserScope: Hashable, Sendable, Codable {
    /// The signed-in Supabase user id.
    public let id: String

    public func storeURL(base: URL) -> URL {
        directory(base: base).appendingPathComponent("store.sqlite")
    }

    public func directory(base: URL) -> URL {
        base.appendingPathComponent("Accounts", isDirectory: true)
            .appendingPathComponent(safeDirectoryName, isDirectory: true)
    }

    /// A defaults suite of this account's own.
    public var defaultsSuiteName: String { "account.\(safeDirectoryName)" }
}
```

The reasoning in the file's own comment is the whole argument: a column means a predicate on every read across twenty models, and a predicate forgotten once is a leak that looks like working software. A separate file cannot leak, because the rows are not there to be read. `LifeOSContainer.make(for:)` opens `Application Support/Accounts/<uid>/store.sqlite`, and every account also gets its own `UserDefaults` suite for the things that turned out to be per account rather than per device: sync cursors, connection tokens, whether the tour has been seen. Appearance, the assistant's voice and which engine answers stay in `standard` on purpose, because they belong to the phone.

Three pieces cooperate. `AccountStore` keeps the roster of accounts in defaults (ids and labels are not secrets) and one keychain item per account for the session (a refresh token is a credential). `AccountSession` owns which scope is open and the `ModelContainer` for it, and tears down every cross-account surface, the widget snapshot, the Live Activity, the watch bridge, before adopting the next one. And `LIfeOSApp` re-roots the whole view tree with `.id(scope.id)` so no screen carries the previous account's view models or scroll position into the next; a stale snapshot on screen after a switch is the same class of leak as a stale row. Chapter 4 walks through the sequence.

The upgrade path is a file move. Every install before accounts kept one unscoped `default.store`. `adoptLegacyStore` moves that file, and its `-wal` and `-shm` companions, into the first account that signs in, once, and refuses when the destination already has a store. Moving the write-ahead log matters: the database alone would leave the most recent writes behind in a log the new file cannot find, and `adoptionCarriesTheWriteAheadLog` pins it. Signing out leaves the store on disk; removing an account from the device is a different, louder action that does not exist yet.

1.8 The privacy boundary

Three things leave the device, and each crosses a boundary that is a type or a dependency edge rather than a code review promise.

- **Derived numbers to the coach's cloud tier.** `RemoteChatEngine` takes a `ContextBundle`, which is assembled on the device from digests: aggregates, deltas, sector scores, the last thirty transactions as

merchant, category and amount. `DailyMetrics` is a `SwiftData` class that is not `Sendable`, so the compiler prevents a raw row from crossing. Chapter 14 has the shape of the bundle.

- **Content, not numbers, to the calendar assistant.** Event titles cannot be aggregated without destroying their use, so this is the one deliberate exception, and it is gated behind a confirmation broker before any write. Chapter 15.
- **Three figures to friends.** Streak, days tracked and workouts, plus a wellness score, and only when the person has turned sharing on in a group. `publishWellnessIfShared` in `RootView` checks the group's flag before every publish. Chapter 16.

Everything with a vendor secret runs in an Edge Function: the Whoop client secret, the Fitbit client secret, the Plaid client id and secret, the Twilio credentials, the OpenAI key, the APNs key. The app bundle holds only what is public by OAuth design, and `AppConfig` says so in its comments. Chapter 23 has the three-tier table.

1.9 Study notes

READ IN THIS ORDER

1. `LifeOSKit/Package.swift`, for the graph. Twenty lines, and every rule in this chapter is one of its edges.
2. `LifeOSKit/Sources/Persistence/DailyMetrics.swift`, for the row and the two bags.
3. `LifeOSKit/Sources/Persistence/MetricsStore.swift`, for the only write path.
4. `LifeOSKit/Sources/Integrations/MetricSource.swift` then `MetricArbiter.swift`, for the rank and the merge.
5. `LifeOSKit/Sources/Integrations/HealthApply.swift`, for where the vocabulary meets the columns.
6. `LifeOSKit/Sources/Persistence/UserScope.swift` and `LifeOSContainer.swift`, for one store per account.
7. `LifeOSKit/Tests/IntegrationsTests/MetricArbiterTests.swift`, which reads as a list of the rules in plain English.

CHECK YOURSELF

- **Why does the arbiter use greater-than-or-equal rather than greater-than?** So that the same source syncing again can correct its own earlier value; an afternoon Health sync must be able to raise the morning's step count.
- **Why is `sourcesData` a string bag rather than a typed `MetricSource`?** Because `Persistence` sits below `Integrations` in the package graph and cannot import the enum.
- **What does an unattributed weight become on the first sync after upgrade?** It is read as typed, rank 4, so Health cannot overwrite it; every other unattributed metric is rank 0 and is re-attributed.
- **Why a store file per account rather than a `userID` column?** A column needs a predicate on every read of every model and one omission leaks; a file cannot leak rows it does not contain.
- **Which health data reaches Supabase today?** None of the rows. Notes, profile, social, connection credentials and coach usage sync; `DailyMetrics`, sleep, workouts, money and sectors stay on the device.
- **What must move with a store when it is adopted, and why?** The `-wal` and `-shm` files, or the most recent writes are stranded in a log the moved database cannot find.

2 MVVM and project structure

Every screen in Almanac is built the same way: a Model of value types, a ViewModel that owns all state and every decision, and a View that renders and forwards intent. The folder layout mirrors that exactly, one folder per screen, so a file's path tells you its layer before you open it.

The first edition described nine view models and four tabs. There are now thirty-three observable classes, most of them view models, across sixteen feature folders and five tabs, plus a composition root that has grown to 844 lines because it is the one place every feature is wired to every other. The shape did not change. This chapter explains why it held, and adds the two things that were not in the first edition: the composition root itself, and the preview and checks harness that lets a screen be captured and verified without an account.

2.1 Why MVVM here

SwiftUI will happily let a view query the database. `@Query` in a `View` is one line and works immediately, and it is the reason this app does not do it. Three specific things break:

1. **Nothing is testable without a simulator.** Goal evaluation, streak arithmetic, sector scoring and the arbiter are the parts most likely to be wrong, and they would live inside a view body that only a running device can exercise.
2. **Derivation gets re-run per render.** Status-per-day, the month grid, the streak, the money rollup: computed in `body`, they recompute on every state change, including ones that have nothing to do with them.
3. **Persistence types leak into layout code.** Once a view knows what a `@Model` class is, changing the schema means editing views. The schema has changed many times since Revision D.

MVVM fixes all three for the price of one extra file per screen. The ViewModel does the fetching and all derivation once, hands the View a plain value type, and can be exercised in a unit test with an in-memory container in milliseconds.

THE RULE THAT MAKES IT WORK

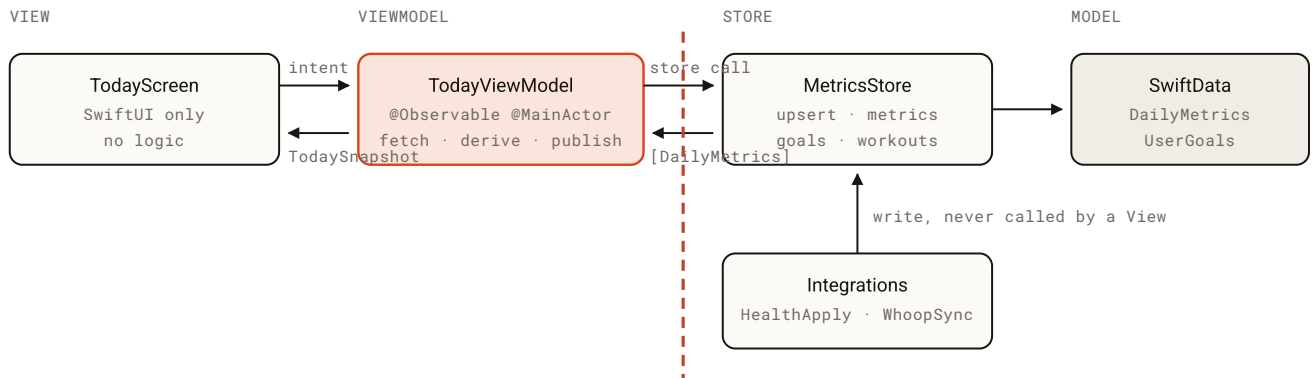
A ViewModel never imports SwiftUI, and a View never imports SwiftData. Both are checkable by reading the first ten lines of a file, and between them they force everything else into place. `grep -rn "import SwiftData" LIfeOS/Features/*/View` returns nothing today, and it should stay that way.

WHERE ARE THE VIEW CONTROLLERS?

There are none, and that is not an omission. `UIViewController` is UIKit's unit of screen ownership: it manages a view hierarchy's lifecycle by hand. SwiftUI replaces that whole job with declarative views that re-render from state, so the controller's two remaining responsibilities split cleanly: owning state and making decisions went to the ViewModel, and describing what is on screen went to the View struct. The one place

UIKit still appears is `PushDelegate`, because APNs registration and the orientation query have no SwiftUI entry point, and the note editor's `UITextView`, because block editing needs a text view SwiftUI does not offer.

2.2 What each layer owns



nothing left of this line imports SwiftData; the store hands back value types, so the two boxes on the left stay testable and previous

The View sends intent and renders a snapshot. The ViewModel is the only place that decides anything. The store owns the invariant (one row per day) so callers cannot get it wrong.

LAYER	LIVES IN	SHAPE	MAY IMPORT
Domain model	LifeOSKit/Sources/Persistence	@Model classes, stores, pure logic	Foundation, SwiftData
Feature model	Features/<Screen>/Model	struct, Sendable, Equatable : the screen's rendered state	Foundation, Persistence value types, DesignSystem value types
ViewModel	Features/<Screen>/ViewModel	@Observable @MainActor final class	Foundation, Observation, Persistence, Integrations, Sectors, Insights; never SwiftUI
View	Features/<Screen>/View	struct: View, plus its private subviews	SwiftUI, DesignSystem, its own ViewModel; never SwiftData
Shared component	Components/{Model, ViewModel, View}	Same three shapes, reused by two or more screens	As above
Design primitive	LifeOSKit/Sources/DesignSystem	Stateless View structs, tokens, pure layout maths	SwiftUI only; knows nothing about the app
Wiring	App/ and App/Surfaces/	The scene, the shell, the root, the account session, push, the watch bridge, the harness	Everything; this is the only place that may

The distinction between the shared component and the design primitive is the one people get wrong. `DesignSystem` holds things that would make sense in any app: `HeroNumeral`, `DotGrid`, `PillNavBar`.

Components holds things that are reusable but specifically about this app's data: the quick-log sheet, the Whoop connect modal, the avatar button that knows where the photo is stored.

2.3 The screen first directory layout

Folders are named after screens, not after layers. **Features/Money/** holds everything the Money tab is, and inside it the three MVVM folders classify each file. Finding code starts with "which screen," which is how anyone actually thinks about a bug report.

```
LifeOS/
├── App/
│   ├── LifeOSApp.swift          app target, file-system synchronised
│   ├── AppShell.swift          @main; owns AccountSession; re-roots per account
│   ├── RootView.swift          onboarding or app; URL callbacks; session keep-alive
│   ├── AccountSession.swift    five tabs, every view model, every sheet
│   ├── ChatTier.swift          which store is open; sign in, switch, sign out
│   ├── OrientationLock.swift   the one place the cloud engine is built
│   ├── PushService.swift       portrait on phone, except the video player
│   └── Surfaces/
│       ├── SurfaceCoordinator.swift publishes SurfaceSnapshot to widgets and watch
│       ├── WatchSessionBridge.swift the mirrored workout session
│       ├── WorkoutLiveActivityController.swift
│       ├── SurfaceDesignPreview.swift widget gallery harness (flag-gated)
│       ├── SurfaceNativeChecks.swift inbox and snapshot checks (DEBUG)
│       ├── SurfaceSettingsScreen.swift "Widgets & Watch" settings page
│       └── LaunchFilmPreview.swift synthetic captures for the launch film (flag-gated)
├── Features/                   one folder per screen
│   ├── Onboarding/            intro, signup, profile step, tour, connections
│   ├── Today/                 hub, agenda, month, day sheet, event sheet, metric detail
│   ├── Health/                the hub with Health and Fitness halves
│   ├── Body/                  weight and wellness sections
│   ├── Activity/              the live session recorder and Begin Activity
│   ├── Recovery/              Whoop detail and trend chart
│   ├── Workouts/              the library, planner sheet, video player
│   ├── Money/                 Plaid, budgets, charts, drill-down
│   ├── Notes/                 the PARA library, editor, inbox, drawer, Pencil
│   ├── Plan/                  habits, still hosted inside Notes
│   ├── Life/                  the nine-sector board, close, in-flight, detail
│   ├── Coach/                 LIFO: screen, orb, speech, voices
│   ├── Assistant/             the calendar assistant sheet
│   ├── Social/                friends, chat, groups, leaderboards
│   ├── Settings/              profile screen, edit sheet, connections, Plaid Link
│   └── Notifications/         the nudge inbox screen
├── Components/                shared across two or more screens
│   ├── Model/ProfilePhotoDownsizing.swift
│   ├── ViewModel/QuickLogViewModel.swift
│   └── View/ QuickLogSheet, AddEntrySheet, JournalEntrySheet,
│       CapsuleButton, ProfileAvatar, WhoopConnectModal
└── Resources/ Assets.xcassets, PrivacyInfo.xcprivacy

LifeOSKit/Sources/            eight targets; see 1.2
AlmanacWatch/                 the watch app: controller, screen, delegate
```

AlmanacWidgets/	HealthWidget and WorkoutLiveActivity
AlmanacWatchWidgets/	the watch complication
supabase/	migrations, functions, templates, tests
scripts/catalog/	the catalog verifier
web/	plaid-oauth.html and the app-site association

Every feature folder has a `View/` ; most have all three. `Health/` and `Life/` have no `Model/` because their snapshots are assembled from other features' snapshots or from package types. `Notifications/` is a single file because the inbox's state lives in `PushService` . A folder with fewer than three subfolders is not a violation; it is a screen that did not need the layer.

WHY THIS WORKS WITHOUT TOUCHING THE PROJECT FILE

The app target uses a `PBXFileSystemSynchronizedRootGroup` . Any folder created under `LIfeOS/` is picked up by Xcode automatically, because the directory structure is the project structure. Moving a screen between folders is a `git mv` . The corollary still holds: do not put documentation in these folders, because every file is a build input and a `README.md` is copied into the bundle as a resource.

2.4 Anatomy of a feature folder

Every feature folder answers the same questions in the same order. Chapters 5 to 18 are each one instance of this template.

FOLDER	CONTAINS	TEST
<code>Model/</code>	One snapshot struct per screen, plus any small enums it needs. Value types only. No methods that fetch, no dates computed from <code>.now</code> .	Compiles without SwiftUI or SwiftData
<code>ViewModel/</code>	One class per screen. Holds <code>private(set) var snapshot</code> , the context, and the funcs the view calls.	Instantiable in a test with an in-memory container
<code>View/</code>	The screen struct and its private subviews. Reads only from its view model; owns <code>@State</code> for presentation concerns like sheet visibility and focus.	A <code>#Preview</code> per subview; a design-preview page for the screen

The ViewModel shape is identical everywhere, which is the point:


```
// LifeOS/Features/Today/ViewModel/TodayViewModel.swift
@MainActor @Observable
final class TodayViewModel {
    private(set) var snapshot = TodaySnapshot()
    private(set) var detail: DayDetailSnapshot?

    private var context: ModelContext?
    private let calendar: Calendar

    init(calendar: Calendar = .current) { self.calendar = calendar }

    func attach(_ context: ModelContext) { self.context = context }

    /// One pass over the day rows: the day-to-status map is built once and used
    /// for both the grid and the streak.
    func load() {
        guard let context else { return }
        let store = MetricsStore(context: context, calendar: calendar)
        // Bounded: the grid needs this month and the streak needs recent
        // history, never the whole table.
        let start = calendar.date(byAdding: .day, value: -400, to: .now) ?? .distantPast
        ...
    }
}
```

The context arrives through `attach` rather than the initialiser because `RootView` constructs every view model as `@State` before the environment is available, and a snapshot with its empty default is a valid thing to render in the meantime. Two view models break the pattern for a reason each states in a comment: `AssistantViewModel` takes its context in `init` because it builds a `ChatStore` immediately, so `RootView` holds it as an optional and creates it in `attachAll()`; and `ActivityRecorder` takes a `UserDefaults` suite so the design preview and the checks harness can run it against disposable defaults.

A calendar is injected everywhere a date is normalised. It defaults to `.current` and a test passes a fixed UTC calendar, which is what makes "which day does 23:30 belong to" a deterministic question in `PersistenceTests`.

INJECTION IS BY CONTEXT, NOT BY PROTOCOL

Every view model builds a concrete store from a `ModelContext`, so testing one means standing up an in-memory `ModelContainer` rather than passing a stub. The consequence the first edition named is still true: the package has 1,298 tests across 130 suites and the app target has none. What changed is the mitigation. Everything that can be decided by arithmetic now lives in the package where a test can reach it (`SessionKeepAlive`, `LiveEffort`, `HUDPlacement`, `CodeDigits`), and the view models that cannot be unit tested are driven by the checks harness in 2.7 instead.

2.5 The composition root

`RootView` is where every feature is wired to every other, and it is the one file in the app that may know about all of them. Five tabs, stated rather than inferred from ordering:

```
// LIfеOS/App/RootView.swift
private enum AppTab: Hashable { case today, health, money, notes, life }

@State private var tab: AppTab = .today // deliberately not persisted: a cold launch lands
on Today

private var navItems: [PillNavItem<AppTab>] {
    [
        PillNavItem(value: .today, systemImage: "circle.grid.3x3.fill", label: "Today"),
        PillNavItem(value: .health, systemImage: "heart.fill", label: "Health"),
        PillNavItem(value: .money, systemImage: "dollarsign", label: "Money"),
        PillNavItem(value: .notes, systemImage: "text.book.closed.fill", label: "Notes"),
        PillNavItem(value: .life, systemImage: "square.grid.3x3.fill", label: "Life"),
    ]
}
```

Health absorbed activity, weight, recovery and wellness so the bar stays free for the other life domains. Plan lost its tab to Notes, where habits now live as a section. Settings sits behind the profile avatar on Today rather than spending a slot. The Life board maps three of its nine sectors back onto tabs, and `LifeSector.ownsTab` in the `Sectors` package is the one decision about which three; `RootView` reads it and asserts if the switch ever drifts.

The root does five jobs, and each is one mechanism:

1. **Owns every view model as `@State`**, nineteen of its own plus the four connection view models the shell hands it, and attaches the context to all of them in `attachAll()` from a single `.task`. Cross-feature wiring happens here and only here: the recorder's `onSaved` triggers a Health sync, the coach's `bundleExtras` closure reads the money and life snapshots, notes get their sync client.
2. **Reloads on save, never on a timer.** One `.task` listens for `ModelContext.didSave` and calls `reloadAll()`. A save is the only thing that can change what a screen shows while the app is running, so nothing polls. Detail pages reload only while open, because off screen they are six months of rows nobody is looking at.
3. **Draws two shells over one set of screens.** A compact width gets the pill bar floating over the content; a regular width gets the same bar stood on end as a rail. The screens are identical in both; 4.9 has the rail's behaviour.
4. **Presents every sheet and cover** from one place, keyed on optionals rather than parallel booleans where the sheet has content: `today.detail` is the only source of truth for the day sheet, `openMetric` is a value rather than four flags, `eventSheet` is an enum of create or edit.
5. **Routes what arrives from outside:** a widget deep link through `SurfaceCoordinator.pendingRoute`, a tapped nudge through `PushService.pending`, a scene activation that syncs the calendar and restarts the Health watcher.

The three quick actions (calendar assistant, LIFO, begin activity) are defined once in `RootView` and injected through `EnvironmentValues.quickActions`. Notes and Life own their own `NavigationStack`s several levels down, and a toolbar has to be attached inside the stack it belongs to; threading an array through both would put the same three buttons in four call sites, which is how a set drifts apart.

2.6 Shared components

`Components/` is split into the same three folders, for the same reason: a shared piece of UI that owns state has a view model, and it belongs somewhere obvious.

COMPONENT	STATE	USED BY
<code>QuickLogSheet</code>	<code>QuickLogViewModel</code> : writes water and weight through <code>MetricsStore</code> , source <code>manual</code>	Begin Activity, the root
<code>AddPlanEntrySheet</code> , <code>AddMoneySheet</code>	Stateless; a closure with the typed fields	Notes (habits), Money
<code>JournalEntrySheet</code>	Stateless; text only	Health, wellness section
<code>WhoopConnectModal</code>	Reads <code>WhoopConnectionViewModel</code>	Health hub, Settings
<code>CapsuleButton</code>	Stateless; quiet or prominent	Every card with a CTA
<code>ProfileAvatar</code>	Takes photo bytes; falls back to a glyph, never initials	Today's toolbar, the profile
<code>ProfilePhotoDownsizing</code>	Pure: resizes to the bucket's cap	Onboarding, the edit sheet

A component earns its place in `Components/` the second time it is needed, not the first. `SessionHUD` and `ActivityControls` are the recent example: they were private to Begin Activity until the video player needed them, and they were moved rather than copied.

WHERE THE DASH RULE LIVES

"A missing metric renders as a dash, never as 0" is the single most repeated rule in this app. It is enforced by type: `MetricTile`, `IconBubbleTile`, `PastelFillCard`, `TrendStatTile` and `HeroNumeral` all take `String?` and render the dash themselves. A screen cannot pass a zero it does not have, because it has nothing to pass.

2.7 The preview and checks harness

Most screens need an account, a store with weeks of rows, a connected strap and a signed-in session before they show anything worth looking at. That made design review and QA slow and made the state on screen unrepeatable. The harness fixes both. `LIfe0SApp` checks one launch argument in `DEBUG`:

```
// LIfeOS/App/LIfeOSApp.swift
#if DEBUG
if ProcessInfo.processInfo.arguments.contains("--design-preview") {
    HealthActivityDesignPreview()
} else {
    shell
}
#endif
```

`HealthActivityDesignPreview` builds an in-memory container, a disposable `UserDefaults` suite, and an `ActivityRecorder` that never saves to Health, then switches on `--page=`. Every page is a real screen fed a fixture; nothing is a mock-up. The fixture uses six real rows lifted from the catalog seed so thumbnails resolve, a WHOOP recovery of 82 and a live reading of 152 bpm, and it labels itself as sample data.

ARGUMENT	RENDERS	MODIFIERS
<code>--page=health</code> (default)	The Health hub	<code>--fitness</code> opens the Fitness half; <code>--dark</code> ; <code>--large</code> for accessibility type
<code>--page=activity</code>	Begin Activity	<code>--live</code> starts a backdated run and feeds twelve minutes of readings; <code>--strength</code> ; <code>--hud-collapsed</code> ; <code>--live-activity</code> requests a real Live Activity
<code>--page=library</code> , <code>library-empty</code>	The workout library	<code>--schedule-sheet</code>
<code>--page=player</code>	The video player	<code>--video=<id></code> picks a seed row; <code>--live</code>
<code>--page=profile</code> , <code>settings</code> , <code>connections</code> , <code>sharing</code> , <code>categories</code>	The named screen	
<code>--page=checks</code>	Runs <code>ActivityRecorderChecks</code> and writes <code>recorder-checks.txt</code> to Documents	

Two more harnesses sit behind build flags rather than arguments, so they cannot ship even in a debug build by accident: `SurfaceDesignPreview` (flag `SURFACE_DESIGN_PREVIEW`) renders the widget gallery, the inbox and the "Widgets & Watch" page and runs `SurfaceNativeChecks`; `LaunchFilmPreview` (flag `LAUNCH_FILM_PREVIEW`) renders the synthetic frames the launch film was cut from.

NATIVE CHECKS

The checks are the answer to "the feature layer has no tests." They are not unit tests: they run inside the app process, against real `UserDefaults` suites and a real in-memory container, exercising the view models the way the app does. `ActivityRecorderChecks` starts a session, feeds readings, counts reps, hands off to a watch that never answers, and asserts on each step; `SurfaceNativeChecks` proves that a notification for one account is rejected once another is signed in and that sign-out empties the widget snapshot. Each prints

PASS or FAIL and a title, the QA reports paste the output verbatim, and the rule for a failing check is the same as for a failing test: fix the cause, never the check.

2.8 Rules and review checklist

RULE	ENFORCED HOW	WHY
A View never imports SwiftData	Review: visible in the imports	All reads go through a ViewModel; the schema can change without touching layout
A ViewModel never imports SwiftUI	Review: visible in the imports	Keeps it testable and prevents View-shaped thinking leaking in
A View never holds a @Model instance	Review: screens take snapshots	Rendering cannot fault an object or trigger a fetch mid-layout
A ViewModel publishes value types only	Review: the snapshot struct is the surface	Nothing above the view model can mutate the database by accident
No feature folder imports another feature folder	Review	Sharing goes through Components, which forces the shared thing to be named
Cross-feature wiring lives in rootView	Review	One file knows the whole graph; a feature never reaches sideways
Derivation happens in the ViewModel, never in body	Review	body runs far more often than data changes
Snapshot types are Equatable	Compiler	Cheap re-render skipping, and ViewModel tests become one-liners
Every font comes from LifeOSType	scripts/check-typography.sh	The app once had 376 hand-written font calls across 28 sizes
A new stored property on an existing model is optional or defaulted	Review; a hang on the spinner if missed	Installed stores refuse to open otherwise. The equipment column shipped without a default and had to be fixed the same day
Anything decidable by arithmetic lives in the package	Review	A test can reach it; a view cannot be asserted on
A screen that needs an account has a design-preview page	Review; the QA report lists the captures	Repeatable state, and a capture without loading anyone's data

2.9 Study notes

READ IN THIS ORDER

- LifeOS/App/LifeOSApp.swift : forty lines that decide whether a store is open and re-root the tree.
- LifeOS/App/AppShell.swift : onboarding or app, and every URL that arrives.
- LifeOS/App/RootView.swift : read attachAll() and reloadAll() first, then the content switch, then the two shells.
- LifeOS/Features/Today/ViewModel/TodayViewModel.swift : the canonical view model shape.

5. `LIfeOS/Features/Today/Model/TodaySnapshot.swift` : what a snapshot is and is not.
6. `LIfeOS/Features/Health/View/HealthActivityDesignPreview.swift` : how a screen is rendered without an account.
7. `LIfeOS/Features/Activity/ViewModel/ActivityRecorderChecks.swift` : what a native check looks like.

CHECK YOURSELF

- **Why does a view model take its context through `attach` rather than `init` ?** Because `RootView` constructs them as `@State` before the environment context exists, and an empty snapshot is a valid first render.
- **What triggers `reloadAll()` ?** `ModelContext.didSave` , scene activation, and a finished recorder save. Nothing polls.
- **Where is the decision about which sectors map to a tab?** `LifeSector.ownsTab` in the Sectors package; `RootView` only maps the answer onto its private enum.
- **Why are the quick actions an environment value?** Notes and Life own their own navigation stacks, and a toolbar must be attached inside the stack it belongs to.
- **What is the difference between a native check and a unit test?** A check runs inside the app process against real defaults and view models; a test runs in the package with no simulator. Both print a verdict; only the check can reach a view model.
- **Which two harnesses are behind build flags rather than launch arguments, and why?** The surface gallery and the launch film; a flag cannot be tripped by a stray argument in a debug build.

3 Design system

Sixteen feature folders have to read as one app. That happens because they share a small set of primitives and differ by exactly one variable, the module hue, rather than because each screen was designed carefully on its own. This chapter is the inventory of those primitives and the three scales they are built on: colour, type and space.

Everything here lives in `Life0SKit/Sources/DesignSystem`, a target with no dependencies that also builds for macOS. That second fact shapes more of the code than it looks: it is why colours are explicit light and dark pairs rather than UIKit dynamic colours, why the width class is the package's own enum rather than `UserInterfaceSizeClass`, and why the six-box code field keeps its keyboard configuration behind an `#if os(iOS)`. The price is a few lines per component; the return is `swift test` running from a terminal in seconds with no simulator.

3.1 Principle

The hub is neutral; the domains are tinted. Today is a warm off-white canvas. Every domain screen is the same canvas with a whisper of its hue washed in across the top 260 points. This is a change from the first edition, where domain screens were saturated-at-top gradients: the restyle kept the hue as a tint rather than a paint job, so cards became solid white with one soft shadow instead of frosted glass fighting a gradient. The two visual languages resolved into one hierarchy, and adding a module is still a token, not a design exercise.

Three rules hold the whole thing together. The accent never varies by module and has exactly two meanings: today, and act. Numerals are the rounded face and words never are. And a missing value is a dash, drawn by the component, never by the screen.

3.2 Colour tokens

```
// LifeOSKit/Sources/DesignSystem/Tokens.swift
public enum ModuleHue: String, CaseIterable, Sendable {
    case body, activity, recovery, nutrition, money, habits

    public var top: Color { ... } // saturated; survives only as chart and accent ink
    public var pastel: Color { ... } // carries the module identity on the light canvas
    public var pastelDark: Color { ... } // the muted counterpart for dark mode
}

public enum LifeOSTokens {
    /// The single accent. Means "today" and "act". Identical in both schemes.
    public static let accent = Color(red: 0.94, green: 0.34, blue: 0.18)

    /// Live session push states, on the live gradient, the same in both schemes.
    public static let pushEasy = Color(red: 0.20, green: 0.72, blue: 0.45)
    public static let pushNear = Color(red: 0.96, green: 0.65, blue: 0.14)
    public static let pushOver = Color(red: 0.90, green: 0.27, blue: 0.23)

    public static let liveGradientTop = ModuleHue.recovery.top
    public static let liveGradientBottom = Color(white: 0.97)

    public static let canvas = AdaptiveColor(light: Color(red: 0.949, green: 0.945, blue:
0.933),
                                                dark: Color(white: 0.07))
    public static let cardSurface = AdaptiveColor(light: .white, dark: Color(white: 0.13))
    ...
}
```


TOKEN	VALUE	MEANING
<code>ModuleHue.body</code>	Teal	Weight, composition, the Health hub
<code>ModuleHue.activity</code>	Amber	Movement, the workout library
<code>ModuleHue.recovery</code>	Blue	Whoop, HRV, sleep; also the live session gradient
<code>ModuleHue.nutrition</code>	Purple	Declared; no screen owns it yet
<code>ModuleHue.money</code>	Green	The Money tab and its charts
<code>ModuleHue.habits</code>	Orange	Habit strips, the heart-rate stat group
<code>LifeOSTokens.accent</code>	Orange, fixed	Today, the selected tab, the prominent action, the code field's focus ring
<code>pushEasy</code> / <code>pushNear</code> / <code>pushOver</code>	Green, amber, red	How hard the live session is pushing against the day's capacity
<code>canvas</code> / <code>cardSurface</code> / <code>tileSurface</code>	Off-white and white; near-black and dark grey	The three surfaces. Tiles are solid because translucent white over a near-white canvas is invisible
<code>dotMissed</code> / <code>dotFuture</code> / <code>dotPadding</code> / <code>dotOutline</code>	Greys	The month grid's four non-day states; padding is faint rather than invisible so the month reads as one block
<code>alertBackground</code> / <code>alertText</code>	Soft red, deep red	The anomaly banner; red enough to interrupt, soft enough to live on cream
<code>fabFill</code> / <code>fabGlyph</code>	Ink on white, white on ink	Never the same luminance; a white plus on the ink token vanishes the moment the scheme flips
<code>cardShadow</code>	6% black, light mode only	Dark mode separates surfaces by tone; a black shadow on a black canvas is invisible cost

Colours are `AdaptiveColor` pairs, resolved explicitly by the consuming view with `token.resolve(scheme)`. It is one more line per use than a dynamic colour, and it is the line that makes `TokensTests` possible: `alertColorsDifferByScheme` and `fabFillAndGlyphStayOppositeInBothSchemes` read both halves of a pair and compare them, which a UIKit dynamic colour would not let a terminal test do.

3.3 One type scale

Before `Typography.swift` existed there were 376 hand-written `.system(size:weight:)` calls across 28 distinct sizes between 9 and 104, and three of those sizes accounted for 178 of them. Nobody chose that; it is what happens when every screen picks its own number, and the drift is invisible until two screens sit side by side. The scale has nine text steps and three numeral steps, fewer than the app had, on purpose: a scale only does its job if the steps are far enough apart that picking the wrong one is obvious.

```
// LifeOSKit/Sources/DesignSystem/Typography.swift
public enum LifeOSType {
    public static let eyebrow      = Font.system(size: 11, weight: .semibold) // tracked
    uppercase markers
    public static let caption       = Font.system(size: 12, weight: .regular)
    public static let label        = Font.system(size: 13, weight: .medium) // dense
    rows, chips
    public static let secondary     = Font.system(size: 15, weight: .regular)
    public static let rowTitle      = Font.system(size: 15, weight: .semibold)
    public static let body         = Font.system(size: 17, weight: .regular) // reading
    text
    public static let sectionTitle = Font.system(size: 20, weight: .semibold)
    public static let screenTitle  = Font.system(size: 24, weight: .bold)
    public static let display       = Font.system(size: 34, weight: .bold) // one per
    screen at most

    /// The date on the hub, and nothing else. Its tracking travels with it.
    public static let masthead = Font.system(size: 104, weight: .bold)
    public static let mastheadTracking: CGFloat = -5

    public static let numeral      = Font.system(size: 28, weight: .bold, design: .rounded)
    public static let numeralLarge = Font.system(size: 44, weight: .bold, design: .rounded)
    /// The only place a size may be passed in. Hero numerals size to their space.
    public static func numeral(_ size: CGFloat, weight: Font.Weight = .bold) -> Font
}
```

Two families, each with exactly one job. **Text is the system sans, always. Numerals are rounded, and only numerals.** Rounded digits are easier to read at a glance, which is why the money screen reached for the face in the first place, and a rounded label next to a sans label is the drift the scale exists to stop. A serif was briefly the notes tab's title face and is deliberately gone: it looked right on that one screen and wrong beside every other.

The masthead is the one step outside the pattern and the reason it exists is instructive. The hub's date was flattened into `display` to fit the scale, which took it from 104 to 34 points and turned the masthead the whole screen is built around into an ordinary heading. A scale that cannot express the biggest thing in the app is missing a step rather than being violated by one, so the step was added, with its negative tracking attached, because a caller who took the font without the tracking would get a subtly wrong masthead.

Two companions keep the scale from being bypassed. `LifeOSType.UIKitScale` restates the same sizes as `UIFont` for the note editor's text view, so changing a step changes both. `LifeOSTextRole` and the `.lifeOSText(_:)` modifier give the same steps Dynamic Type behaviour through `@ScaledMetric`, so screens can adopt accessibility sizes one at a time without touching the masthead or the numerals.

3.4 Space, radius and the glass panel

```
// LifeOSKit/Sources/DesignSystem/Space.swift
public enum Space {
    public static let half: CGFloat = 4    // the only sub-grid value, for optical nudges
    inside a control
    public static let x1: CGFloat = 8
    public static let x2: CGFloat = 16
    public static let x3: CGFloat = 24
    public static let x4: CGFloat = 32
    public static let x5: CGFloat = 40
    public static let x6: CGFloat = 48
    public static let x8: CGFloat = 64
    public static let x10: CGFloat = 80
}

public enum Radius {
    public static let small: CGFloat = 12
    public static let medium: CGFloat = 20
    public static let large: CGFloat = 28
    public static let pill: CGFloat = 999
}
```

Named by multiples rather than by role, because roles drift (someone always needs a "medium-large") while multiples cannot. Radii sit on the same grid so cards nest without optical drift. The same file holds `GlassPanel`, `PrimaryButton` and `SecondaryButton`. `GlassPanel` is where the app takes Liquid Glass from the system rather than imitating it with a material: one `.glassEffect(.regular, in:)`, so every frosted panel in the app gets real refraction and highlights from one place, and the notes drawer, the profile's frosted panel over the photo and the coach's canvas all read as the same surface.

3.5 Layout metrics: phone and iPad

Every number that varies with width lives in `LayoutMetrics`, for the same reason `Space` exists. The app distinguishes exactly two widths and maps the platform's size class onto them once, in `RootView`, so the package never imports UIKit's enum.

```
// LifeOSKit/Sources/DesignSystem/LayoutMetrics.swift
public static func metrics(for width: LayoutWidth) -> LayoutMetrics {
    switch width {
    case .compact:
        LayoutMetrics(gutter: 20, sectionSpacing: 22,
            maxContentWidth: .infinity, heroScale: 1.0,
            fabBottomInset: 72, contentBottomInset: 130,
            railInset: 0,
            statColumns: 2, tileColumns: 3, isRegular: false)
    case .regular:
        LayoutMetrics(gutter: 32, sectionSpacing: 30,
            maxContentWidth: 1060, heroScale: 1.3,
            fabBottomInset: 28, contentBottomInset: 32,
            railInset: 112,
            statColumns: 4, tileColumns: 6, isRegular: true)
    }
}
```

METRIC	COMPACT	REGULAR	WHY THE REGULAR VALUE
<code>maxContentWidth</code>	unbounded	1060	Above a portrait iPad's 1024, on purpose: portrait is already one comfortable column and capping it bought only dead margin. The cap is for landscape and wide Stage Manager panes.
<code>contentBottomInset</code>	130	32	Compact clears the floating pill bar, which overlaps the scroll view. Regular has a side rail instead, and the old inset left a band of dead space.
<code>railInset</code>	0	112	The rail's real footprint is its 32 gutter plus its 64 width. At 80 every regular screen had its leading edge tucked under the rail; the Notes sidebar showed it plainly. The extra beyond 96 is the rail's shadow.
<code>statColumns</code> / <code>tileColumns</code>	2 / 3	4 / 6	Derived rather than measured: <code>GridItem(.adaptive)</code> cannot cap the count, and a 1366-point pane would take eight tiles across.

`railInset` is stated as a number the screens apply rather than as a safe area inset, because a safe area does not survive the trip through a `NavigationStack` into a `ScrollView` whose background ignores it; the rail ended up drawn over the first column of the month. `LayoutMetricsTests` pin every relationship in the table: `onlyRegularWidthPaysForTheRailOnTheLeadingEdge`, `theContentCapOnlyBitesWiderThanAPortraitIPad`, and a guard that every metric is positive.

3.6 Component inventory

Thirty-four files. Each is stateless apart from animation phase, takes value types, and knows nothing about the app's models. The inventory is grouped by what the component is for.

CANVAS AND CARDS

PRIMITIVE	WHAT IT IS	NOTE
<code>GradientCanvas(hue:)</code>	The one canvas: neutral ground with the hue's pastel washed in over the top 260 points	Adding a module is a hue token
<code>SoftCard</code>	The one card: white, 24-point continuous corners, one soft shadow in light mode, a tonal step in dark	The shadow is an offscreen pass; never the cell of a long repeated list
<code>PastelFillCard</code>	Pastel fill, white icon bubble, quiet label, huge dark number; takes a <code>SplitNumeral</code> too	Nil renders a dash, never a zero
<code>IconBubbleTile</code>	White card, pastel icon chip, big number	The Fitness look; distinct from the pastel-fill card
<code>MetricTile</code>	Small solid tile for a repeated row: label above value	Deliberately not a <code>SoftCard</code> , for the shadow reason
<code>TrendStatTile</code>	An <code>IconBubbleTile</code> with the week behind it as a <code>RoundedBarChart</code>	A step count alone says nothing about whether it was a good day
<code>GlassPanel</code>	Content on Liquid Glass with a hairline edge	Without the edge the panel dissolves into a light gradient
<code>InsightCallout</code>	What LIFO said, drawn as a dashed card rather than a chat bubble	Dashed reads as generated, not stored; shared with the nudge
<code>AlertBanner</code>	The red interruption: facts only, never advice	No anomalies is silence, not an empty red frame

NUMBERS AND CHARTS

PRIMITIVE	WHAT IT IS	NOTE
<code>HeroNumeral</code> / <code>HeroEmptyState</code>	An 84-point rounded numeral with its unit and label; the empty state shown in its place	A numeral is never shown without its unit
<code>SplitNumeral</code>	A hero figure and a quieter remainder, the way "120 / 80" is set; <code>.sleep(minutes:)</code> makes hours the hero	Pure struct, tested
<code>TrendSeries</code> / <code>TrendPoint</code>	A metric over a window of days with the baseline a single reading needs; <code>value</code> is optional so a gap stays a gap to the chart	Every derived figure ignores gaps rather than counting them
<code>RoundedBarChart</code>	Full-height soft track per day with a solid rounded bar; baseline is zero or the window's minimum	Weight against zero makes seven indistinguishable bars
<code>SleepComposition</code>	One night in stages, deepest at the bottom, fixed order	A five-hour night and a fragmented seven look identical on a duration chart
<code>RecoveryBand</code>	Whoop's own thresholds: below 34, 34 to 66, 67 and above	The colour is the point; 31% and 81% read alike as text

CALENDAR VOCABULARY

PRIMITIVE	WHAT IT IS	NOTE
<code>DotGrid</code> / <code>DotCell</code> / <code>DotState</code>	The shared vocabulary of the whole app: filled is target met, hollow is missed, orange is today, grey is future, faint outline is no data, and <code>blank</code> is padding	Columns built once in <code>init</code> ; padding cells hidden from VoiceOver
<code>MonthGridLayout</code>	Pure calendar maths producing the cells, and <code>stepping(from:by:day:)</code> which keeps the requested day so walking a year forward from the 31st does not arrive at the 28th	Fully tested against a fixed UTC calendar
<code>MonthCalendarView</code> / <code>WeekdayHeader</code>	The assembled month with narrow weekday letters rotated to the calendar's first weekday	Takes cells and a date, no data access
<code>WeekStrip</code>	Seven days across the top of a domain screen with an optional progress ring per day	Selection is a binding; the view model decides what it means

NAVIGATION AND CONTROLS

PRIMITIVE	WHAT IT IS	NOTE
<code>PillNavBar</code> / <code>PillNavItem</code>	A floating capsule of 52-point circular buttons; horizontal along a phone's bottom, vertical as an iPad rail	One component in two orientations; splitting them is how they drift
<code>ActionFan</code> / <code>QuickAction</code>	The three actions as a flat row in a navigation bar, or a fan that arcs out of a corner	Fan radius 106 is set by geometry: under 76 two circles pass through each other
<code>QuickActionsToolbar</code>	The environment value and modifier that puts the injected actions in a screen's bar	Placement is <code>.primaryAction</code> because macOS has no top bar to name
<code>SegmentedPills</code>	A two-or-more way switch with a sliding capsule	Replaced the last piece of stock iOS chrome on the sign-in screen
<code>UnderlinePicker</code>	Text tabs with an underline; collapses to a menu at accessibility sizes	The Health hub's Health and Fitness switch
<code>CodeField</code> / <code>CodeDigits</code>	Six boxes over one real hidden text field, so autofill and paste keep working	The digit logic is a free function so a test never touches a view
<code>InlineStatus</code> / <code>InlineStatusView</code>	The one thing a screen is complaining about, resolved by precedence	Three stacked coloured lines read as a broken app
<code>FocusableField</code> , <code>.shake(_:)</code>	The text-field look with a focus ring; a decaying horizontal shake for a wrong code	
<code>PrimaryButton</code> / <code>SecondaryButton</code>	The one solid full-width action and its quiet companion	52 points tall
<code>ColorSchemePreference</code>	System, light or dark; <code>system</code> is nil so SwiftUI keeps following the device	Stored per device, not per account

3.7 Motion pieces

Four components animate, and each earns it by being made of the app's own vocabulary rather than decoration dropped on top.

- `DotBloom` is the onboarding animation: a month of dots filling in one by one with a spring per column. The first thing a new person sees is the grid they will read every morning, so the animation teaches the interface instead of entertaining. It respects reduce motion.
- `ScanlineCat` is the intro's greeting, a cat drawn as horizontal scanlines or as dots on the grid's own pitch. All geometry is in `ScanlineCatSilhouette` (three primitives: ellipse, rect, triangle, in a unit box) and `ScanlineSlicer` (which cuts shapes into merged x-spans at a given y, in closed form, so slicing costs no rasterisation). Both are pure and tested; `ScanlineSlicerTests` is the largest suite in `DesignSystemTests` at 443 lines. The view only decides colour, timing and how the cat assembles on appear: rows drift in from the side they are nearest, the face goes on last and solid because a striped eye cannot read as an eye.
- `GlassGlobe` is the coach's orb: a Siri-like glass slime, clear and silver, morphing on a `TimelineView` at 30 frames per second. Audio `intensity` makes it breathe. Hue is kept out of it; the colour is the light on glass.
- `SlateAurora` is the coach's backdrop, a three-by-three `MeshGradient` from onyx through blue slate to an alabaster bloom. Fixed rather than scheme-resolved, because the coach is an immersive dark surface in both appearances.

THE LIVE GRADIENT AND PUSH-STATE TINTS

A running session paints the recovery blue from the top to near white at the bottom, on the in-session hero, the lock screen Live Activity and the Dynamic Island alike, so the numbers read as the same data wherever they appear. On that gradient the three push colours say how hard the session is pushing against the day's capacity: green is easy going, amber is near the ceiling, red is over it. They are the same in both schemes because the gradient is. Chapter 8 has the maths that picks one.

3.8 Dark mode

Every hue has a dark variant, deep top and near-black bottom, and every surface token has both halves. These were defined before any screen existed, because retrofitting a gradient system to dark mode later is significantly harder than doing it once at the start, and the restyle to a light canvas proved the point: the dark tokens changed values but no screen had to learn that dark mode exists.

NO COLOUR IS DEFINED ONLY INSIDE A SCHEME BRANCH

Every token has a light value and a dark value, always both. A colour that exists only in the dark branch renders as whatever the parent inherited in light mode, usually black on black, and usually discovered by a user rather than a preview. The two exceptions are deliberate and named:

`SlateAurora` and the live gradient are the same in both schemes because the surfaces they paint are immersive.

3.9 The typography check

The scale is only the scale if nothing else declares a font. `scripts/check-typography.sh` is what keeps it that way: it greps every Swift file under `LifeOS` and `LifeOSKit/Sources`, excluding `Typography.swift` itself, and fails on four patterns.

```
# scripts/check-typography.sh
check "Literal font sizes outside the scale (use a LifeOSType step):" \
  '\.system(size: [0-9]'

check "UIKit fonts outside the scale (use LifeOSType.UikitScale):" \
  '\(systemFont\|monospacedSystemFont\)(ofSize:'

check "Rounded or serif faces declared outside the scale:" \
  'design: \. \(rounded\|serif\)'

check "Apple semantic text styles (use a LifeOSType step):" \
  '\.font(\. \(largeTitle\|title[0-9]\* \
  |headline\|subheadline\|body\|callout\|footnote\|caption[0-9]*\))'
```

The first pattern matches a literal point size and not a computed one, because an icon drawn at a fraction of its container is not type and has no step to pick. The fourth is the subtle one: Apple's semantic styles are a second scale nobody chose, and they do not line up with ours. The script exits non-zero on any hit, so it belongs in the same pre-push habit as `swift test`. It is also honest about the present: the intro and signup screens still carry three `.subheadline` and `.body` calls that predate the scale, and running the script today lists exactly those three. That list is the work remaining, not a reason to loosen the check.

3.10 Study notes

READ IN THIS ORDER

1. `Tokens.swift`: the hues, the accent, the surfaces, and the comment on every one.
2. `Typography.swift`: the scale and the story of the 376 calls it replaced.
3. `Space.swift`: the grid, the radii, and `GlassPanel`.
4. `LayoutMetrics.swift`: read the doc comment on `railInset`; it is a complete bug post-mortem.
5. `PillNavBar.swift` and `ActionFan.swift`: one component, two orientations, and why.
6. `SignInChrome.swift`: `CodeField` over a hidden text field, and `InlineStatus.resolve`.
7. `ScanlineSlicer.swift` then `ScanlineCatSilhouette.swift`: geometry a test can assert on.
8. `scripts/check-typography.sh`: run it, and read what it finds.

CHECK YOURSELF

- **Why are colours** `AdaptiveColor` **pairs rather than UIKit dynamic colours?** The package builds for macOS so tests run from a terminal, and a test can read both halves of a pair.
- **Which face may a word never be set in?** Rounded. It is for numerals only, and a rounded label beside a sans label is the drift the scale exists to stop.

- **Why does `masthead` exist as its own step?** Flattening the hub's date into `display` took it from 104 to 34 points; a scale that cannot express the biggest thing in the app is missing a step.
- **Why is `railInset` 112 and not 96?** The rail's footprint is 32 gutter plus 64 width; the rest is breathing room for its shadow. At 80 every screen's leading edge was under the rail.
- **Why is the code field one hidden text field rather than six?** So the system keyboard, `.oneTimeCode` autofill and paste keep working; the boxes are a rendering of that field and hold no state.
- **Which two surfaces are the same colour in both schemes, and why is that allowed?** The coach's aurora and the live session gradient; both are immersive surfaces whose tokens resolve as dark by design.

4 The app shell, accounts and sign-in

Before any tab exists, three objects decide whose data is open, whether a session still signs them in, and which shell to draw around the screens. This chapter follows a launch from `@main` to the first rendered tab, then the sign-in that gets a new person there, then what keeps them signed in and how several people share one phone.

4.1 Intent

Signing in is required and it is the only entry point. That was already true in the first edition and the reason has only strengthened: a store belongs to an account, notes sync to one, and every connection is held per account. There is nowhere for a signed-out person's data to live that would not become somebody else's the moment they signed in. What is new is that the shell must handle more than one account on a device, must keep a session alive for someone who never closes the app, and must survive an upgrade from the single-store world without looking like it deleted a year of notes.

WORKING BACKWARDS

Almanac opens on the intro the first time and on Today every time after. Signing in takes an email or a phone number and a six-digit code, then a name. A returning person types the code and lands straight in the app. Two people can sign in on one phone and switch between them without either seeing the other's notes, journal or health.

Why no password?

A password is a credential to lose, store and reset. A one-time code delivered to a channel the person already controls proves the same thing with no secret at rest and no reset flow to build.

Email or phone?

Email is the default. It is free through Resend to 3,000 a month, needs no A2P registration, and carries none of the SMS failure modes. Phone goes through Twilio Verify behind an Edge Function, and when a text cannot be delivered the app offers email rather than retrying.

How does the app know I am returning?

The account tells it, not the door you came through. After the code is accepted,

`AuthSession.hasProfile` says whether a name was ever saved; with OTP there is no other difference between signing up and signing in.

What happens if I stay in the app for two hours?

An access token lives an hour. A loop keyed on the sign-in state sleeps until 45 seconds before expiry and renews. Before that loop existed, every authenticated call after the first hour failed with a JWT expired error and nothing was watching the clock.

4.2 Files and folders

LifeOS/App/	
├─ LifeOSApp.swift	@main; owns AccountSession; re-roots per account
├─ AccountSession.swift	which scope and container are open; sign in, switch, sign
├─ AppShell.swift	onboarding or RootView; URL callbacks; session keep-alive
├─ RootView.swift	the two shells over five tabs (Chapter 2)
├─ OrientationLock.swift	portrait on phone; the video player asks for more
├─ PushService.swift	APNs registration and the nudge inbox; PushDelegate
└─ ChatTier.swift	the one place the cloud engine is built
LifeOS/Features/Onboarding/	
├─ Model/OnboardingModels.swift	OnboardingStep, AuthMode, IntroPage, SignupDraft, DialCou
├─ ViewModel/OnboardingViewModel.swift	send, verify, profile, restore, sign out
└─ View/SignupScreens.swift	SignupScaffold, IdentityScreen, CodeScreen, ProfileStepSc
LifeOSKit/Sources/Integrations/	
├─ SupabaseAuth.swift	the three auth calls over REST; AuthSession; AuthError
└─ AuthSessionStore.swift	AuthSessionStoring; KeychainAuthSessionStore; InMemoryAut

- └─ AccountStore.swift
- └─ SessionRefresh.swift
- └─ SessionKeepAlive.swift
- └─ PhoneOTP.swift

the roster of accounts and one keychain item per account
 SessionRefresher: restore on launch and foreground
 when to renew a session that has not lapsed yet
 E.164 checks before a code is requested

LifeOSKit/Sources/Persistence/

- └─ UserScope.swift
- └─ LifeOSContainer.swift
- └─ AccountDeviceAccess.swift

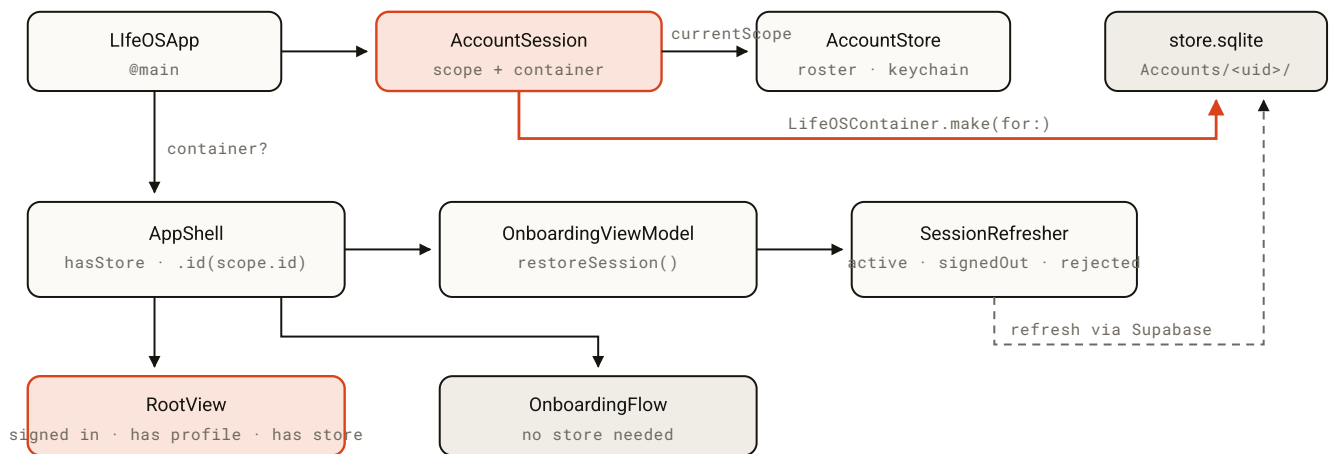
where an account's store and defaults live
 the schema; make(for:); adoptLegacyStore
 a device permission must also be granted by the account

LifeOSKit/Sources/DesignSystem/

- └─ SignInChrome.swift

InlineStatus, SegmentedPills, CodeField, FocusableField,

4.3 Launch: from @main to the first tab



Two locks guard the tab hierarchy: a session, and a store to read through. Either alone produces a wrong screen; both together produce the app.

`LifeOSApp` holds one piece of state, an `AccountSession`, and switches on whether it has a container:

```
// LIfiOS/App/LIfiOSApp.swift
@ViewBuilder private var shell: some View {
    if let container = session.container {
        AppShell(onSignedIn: { session.signIn($0, session: $1) },
            onSignedOut: { session.signOut() },
            hasStore: true, accountID: session.scope?.id)
            .modelContainer(container)
            // Re-rooted per account, so no screen carries the previous
            // account's view models or scroll position into the next one.
            .id(session.scope?.id ?? "none")
            .environment(\.accountSession, session)
            .task(id: session.scope?.id) { purgeSeededHistoryOnce() }
    } else {
        // Nobody signed in. The onboarding flow needs no store.
        AppShell(onSignedIn: ..., onSignedOut: ..., hasStore: false, accountID: nil)
    }
}
```

`AccountSession.init` does three things before the first frame. It adopts a pre-accounts session if the roster is empty and the legacy keychain slot holds one, moving the old `default.store` into the new account's directory in the same breath. It removes a current account whose keychain session does not match its id, so a stale pointer can never open somebody else's store. Then it adopts the current scope, which opens the container and hands the owner id to `PushService` and `SurfaceCoordinator`. A container that cannot open is not something the person can fix, and carrying on with the previous account's container would show them somebody else's data, so the safe failure is nothing open.

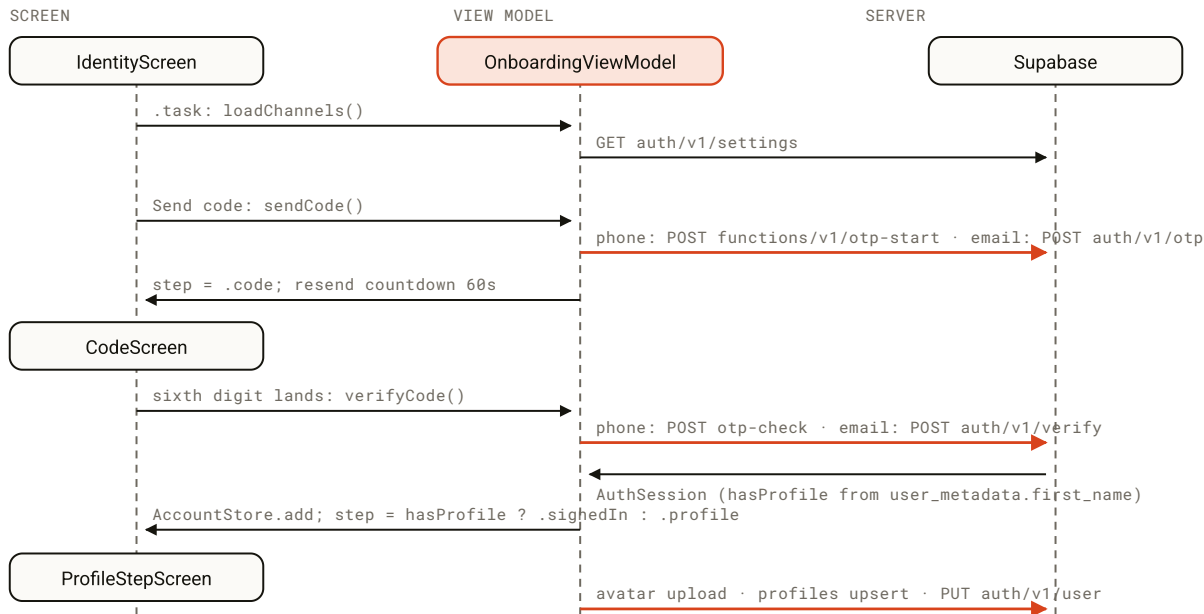
`AppShell` is the second lock. A session alone is not enough to build the tab hierarchy: every screen below `RootView` reads through a `ModelContext`, and mounting it in a scene with no container gets as far as the first fetch before CoreData throws an `NSException` for an entity it cannot find, which is not catchable from Swift and takes the whole process. So the gate is three-way:

```
// LIfiOS/App/AppShell.swift
if onboarding.isSignedIn && hasFinishedOnboarding && hasStore {
    RootView(whoop: whoop, fitbit: fitbit, plaid: plaid, health: health, onSignOut: { ... })
} else {
    OnboardingFlow(model: onboarding, ...)
}
```

`hasFinishedOnboarding` is an `@AppStorage` in the account's own defaults suite. Its job is narrower than its name: `isSignedIn` resolves synchronously from a keychain read, so on a relaunch that restores a session the flag is already true on the first render, instead of flashing the intro carousel for a frame before the launch task settles. The first-run tour is keyed on its own flag so it fires exactly once per account however the person arrived, and it is an overlay rather than a `fullScreenCover` because iOS can restore a previously presented cover at launch and two covers contending for one window silently drops one.

4.4 Signing in: a code, then a name

`OnboardingStep` is a single enum rather than a pile of booleans, so an impossible combination cannot be represented: `intro`, `identity`, `code`, `profile`, `connections`, `signedIn`. `AuthMode` is which door the person came through, and it changes copy and nothing else.



Three round trips for a new person, two for a returning one. The server never sees a password and the app never stores one.

`SupabaseAuth` is the whole auth client: no SDK, because the three calls needed here are small and a dependency would pull in far more surface than that. The anon key is sent as the API key on every call; it is public by design and Row Level Security is what protects rows. Phone is always offered because SMS goes through the `otp-start` Edge Function and Twilio Verify, not GoTrue's own provider; email availability still comes from `auth/v1/settings`, and the identity screen waits for that answer rather than offering a channel that cannot deliver.

```
// LifeOSKit/Sources/Integrations/SupabaseAuth.swift
public func sendCode(to destination: String, channel: Channel) async throws {
    if channel == .phone {
        _ = try await postFunction("otp-start", body: ["phone": destination])
        return
    }
    _ = try await post("otp", body: ["email": destination, "create_user": true])
}

public func verify(code: String, destination: String, channel: Channel) async throws ->
AuthSession {
    if channel == .phone {
        let data = try await postFunction("otp-check", body: ["phone": destination, "code":
code])
        return try decode(data)
    }
    let data = try await post("verify", body: ["token": code, "type": "email", "email":
destination])
    return try decode(data)
}
```

Before a phone number reaches the server, `PhoneOTP.isAcceptable` refuses what will never be sent to: only Indian mobiles (`+91` then ten digits starting 6 to 9) and NANP numbers (`+1` then ten digits) are accepted, and `pumpingPrefix` produces the country-plus-early-digits key the server uses to catch SMS pumping against a neighbouring range. The Edge Function enforces the same rules; this copy exists so the form can refuse without a round trip.

`AuthError.fromHTTP` maps every failure the classifier knows about onto a sentence a person can act on, and the Twilio cases are the interesting ones. A trial-account failure (21608) blames the account, not the number, because the number is not the problem. An opted-out number (21610) is given the remedy: text START. An unregistered campaign (30034) and a carrier block (60410) are named rather than lumped into a generic failure. And an empty-bodied 502 with no message stays channel-neutral, because email is now the default and the copy must not assume SMS. `AuthErrorMappingTests` has one test per case, each named for the sentence it guards.

THE CODE SCREEN

This is the screen people fail on, so it does the work instead of asking for it. Six boxes rather than one tracked field. Verification fires the moment the sixth digit lands. A wrong code shakes and buzzes through `.sensoryFeedback(.error, trigger:)` rather than quietly turning a line red, and the trigger is a counter rather than a Bool because two bad codes in a row must play twice. The Verify button stays even though the sixth digit auto-verifies, because autofill can land a full code without a keystroke and a screen with no button leaves that person nothing to press.

The resend countdown is 60 seconds, not 45: the phone Edge Function's cooldown in `otp_guard.ts` is 45 seconds, but email posts straight to GoTrue, governed by `auth.email.max_frequency`, which is 60. The countdown has to clear the stricter of the two or "Resend code" invites a 429.

4.5 The returning-user door

With OTP there is no password, so signing in and signing up are the same three steps. The intro offers two doors, "Let's get started" on the last page and "Already have an account? Sign in" beneath it, and picking the wrong one is harmless by design: the branch after verify is what routes.

```
// LIfiOS/Features/Onboarding/ViewModel/OnboardingViewModel.swift
let session = try await auth.verify(code: draft.code.filter(\.isNumber),
                                   destination: draft.destination, channel: draft.channel)
try AccountStore().add(Account(userID: session.userID,
                               label: session.email ?? session.phone ?? "Account"), session:
session)
AccountStore().clearLegacySession()
store = KeychainAuthSessionStore(account: session.userID)
self.session = session
isSignedIn = true
// The only thing that separates a returning user from a new one, and it is
// known only now. The door they came through does not decide this; the account does.
step = session.hasProfile ? .signedIn : .profile
```

`hasProfile` is derived from `user_metadata.first_name` in the verify response. That is why `saveProfile` still writes the name to user metadata even though the profile row now lives in the `profiles` table: the flag that decides whether a returning person is sent through signup again is read from the session, not from a table the session has not loaded yet. `AuthSession` decodes the flag by hand with a default of false, because sessions already in the keychain have no such key and a synthesised decoder would throw on them and sign every existing person out on upgrade.

The profile step saves in a deliberate order: the picture goes to storage before the row, so the row is never saved pointing at an object that does not exist; a failed upload costs the picture and not the profile. A failed profile write does not strand the person at the last step either: the account exists either way, so the error is shown and the step advances to connections. `onboarding.step` is written to the account's defaults at that point so a relaunch mid-signup resumes at connections rather than at the profile.

4.6 Keeping a session alive

Two pure pieces decide, and one loop acts. `SessionRefresher.restore()` is asked on launch and on every return to the foreground. Its governing rule is that signing someone out is destructive and cannot be undone without a fresh code, so it happens only when the server has definitively refused the credential:


```
// LifeOSKit/Sources/Integrations/SessionRefresh.swift
public func restore() async -> Outcome {
    guard let stored = store.load() else { return .signedOut }
    guard stored.isExpired() else { return .active(stored) }
    guard let refreshToken = stored.refreshToken, !refreshToken.isEmpty else {
        store.clear(); return .rejected
    }
    do {
        let refreshed = try await auth.refresh(refreshToken: refreshToken)
        guard !Task.isCancelled,
              store.load()?.refreshToken == stored.refreshToken,
              store.load()?.userID == stored.userID else { return .signedOut }
        guard refreshed.userID.isEmpty || refreshed.userID == stored.userID else {
            store.clear(); return .rejected
        }
        let session = refreshed.carryingForward(stored)
        try? store.save(session)
        return .active(session)
    } catch let error as AuthError where Self.isRefusal(error) { // 400, 401, 403, 422
        guard store.load()?.accessToken == stored.accessToken else { return .signedOut }
        store.clear(); return .rejected
    } catch {
        return .active(stored) // offline, outage, rate limit: keep it and try later
    }
}
```

Every guard in that function is a bug that happened. Re-reading the store after the network call and comparing the refresh token is what stops a logout during a refresh from resurrecting the session. Comparing the user id is what stops a refresh from replacing the account's identity. `carryingForward` exists because a refresh response is allowed to be sparse and must not erase what is already known. And the `authURL` helper exists because `appendingPathComponent` percent-encoded the `?` in `token?grant_type=refresh_token`, GoTrue answered 404, and for a while no stored session could ever be renewed: an hour after signing in every authenticated call began failing, and signing in always looked fine. `AuthRefreshURLTests` pins the query staying a query.

`OnboardingViewModel.restoreSession` coalesces concurrent callers onto one task, because Supabase rotates the refresh token on use and two concurrent refreshes would have the second present a token the first just retired, signing the person out for doing nothing but foregrounding the app during launch.

`SessionKeepAlive` covers the case neither event reaches: someone who keeps the app open past the hour. It is two numbers and a function, pure so the schedule can be tested without waiting an hour for it:

```
// LifeOSKit/Sources/Integrations/SessionKeepAlive.swift
public static let margin: TimeInterval = 45 // inside isExpired's own minute of slack
public static let floor: TimeInterval = 5 // never a spin against the auth endpoint

public static func delay(untilExpiry expiresAt: Date, now: Date = .now) -> TimeInterval {
    max(expiresAt.timeIntervalSince(now) - margin, floor)
}
```

`AppShell` runs the loop in a `.task(id: onboarding.isSignedIn)`, so it starts the moment somebody signs in and stops the moment they sign out rather than waking on a timer behind the signup screen. The margin is deliberately inside `isExpired`'s sixty seconds of slack: waking earlier finds a session the refresher considers healthy, does nothing, and spends the wake for nothing. The floor is what stops a session that arrives already expired, or a clock that jumps backwards, from turning the loop into a spin.

4.7 Several accounts on one device

Section 1.7 explained why isolation is a file per account. This section is the bookkeeping that makes switching safe. `AccountStore` keeps two things: a roster in defaults, because a list of who has used this phone is not a secret and enumerating accounts must not need a keychain read each, and a keychain item per user id, because a refresh token is a credential in its own right.

```
// LifeOSKit/Sources/Integrations/AccountStore.swift
public var currentScope: UserScope? {
    // Nil rather than "the first account we can find": a current id naming an
    // account that has been signed out must never silently open somebody else's store.
    guard let id = defaults.string(forKey: Self.currentKey),
        accounts.contains(where: { $0.userID == id }) else { return nil }
    return UserScope(id: id)
}

public func add(_ account: Account, session: AuthSession) throws {
    guard !account.userID.isEmpty, account.userID == session.userID else {
        throw AccountStoreError.identityMismatch
    }
    try keychain(for: account.userID).save(session)
    ...
}

@discardableResult
public func setCurrent(_ userID: String) -> Bool {
    // Refuses an account with no stored session rather than switching to one that cannot
    sync.
    guard accounts.contains(where: { $0.userID == userID }),
        keychain(for: userID).load()?.userID == userID else { return false }
    defaults.set(userID, forKey: Self.currentKey)
    return true
}
```

`KeychainAuthSessionStore()` with no arguments defaults to whichever account is current, read straight from defaults rather than through `AccountStore` because the store is main-actor bound and this type is used from wherever a network call happens to be made. This matters more than it looks: every network client in the app builds one of these to get a token, and before accounts there was one slot, so after a switch they would all have carried on presenting the previous account's token and reading the previous account's rows.

Switching and signing out are the two places every cross-account surface is torn down, and the list is the map of what is per account:

SURFACE	ON SWITCH	ON SIGN OUT
The store	Container replaced; the view tree re-rooted by <code>.id</code>	Container closed; file left on disk
Widget snapshot	<code>SurfaceCoordinator.clear()</code> then adopt the next owner	Cleared; an empty snapshot is written
Live Activity and watch bridge	<code>endAll()</code> ; bridge ended with discard	Same
The recorder	<code>beforeAccountChange</code> deactivates it	Same
Push	Owner id re-attached; the inbox reloads from the new suite	Row deleted with the old token before the keychain is cleared; delivered notifications removed; APNs unregistered
Pending OAuth (Whoop, Fitbit)	Left alone; keyed per account	In-flight browser credentials cleared; saved provider links stay with their owner
Calendar and Health opt-ins	Read from the new suite	Removed from the suite; the device permission stays
Plaid Link	<code>resetForAccountChange()</code>	Same
The legacy keychain slot	Cleared	Cleared, belt and braces

The legacy slot deserves its own sentence because it is what made logging out impossible for a build. It has no user id, so it is not "one account's" session and the per-account clear never reached it; it survived every sign out and the next launch adopted it and signed the person back in, into a scene with no store open, and the first note fetch threw. A credential that can restore an account is that account's session whatever slot it sits in.

`AccountDeviceAccess` is the last piece: device permissions are app-wide in iOS, so Calendar access granted while one account was open must also be granted by the next one. `allowsCalendar` requires both the system authorisation and the account's own flag.

4.8 The app delegate and push registration

`PushDelegate` is the whole reason the app has a delegate: the device token arrives through `didRegisterForRemoteNotificationsWithDeviceToken` and nowhere else, the orientation query arrives through `supportedInterfaceOrientationsFor`, and the watch mirroring handler must be installed at launch. `PushService` is one object rather than two because registration and the inbox share the state that matters: the token, which arrives long after the sign-in that asked for it and is needed again at sign out to delete the row.

- **Authorisation is never requested at launch.** A permission prompt on first run, before the app has shown anyone anything worth being interrupted about, is the fastest way to a permanent no. It is asked from the notification inbox, the first time that screen is opened.
- **The registration is keyed to the active account** and re-pushed on every foreground, because the row carries the timezone the send hour is read in; somebody who has flown somewhere would otherwise keep

being nudged at eight in the morning where they used to live.

- **Deregistration happens before the session goes**, with the token captured by value, because deleting the row needs the access token of the account whose row it is. Left behind, that row would push one person's sleep onto a phone now showing somebody else's name.
- **A nudge that arrives while the app is open is still shown**. The point of the channel is that LIFO speaks first; suppressing it because someone happens to be looking at the app would drop the message entirely. A tap puts the nudge in the inbox and `RootView` opens the coach on it, seeding the sentence but not the numbers, which are worked out now against the store as it stands.

4.9 Two shells, one set of screens

`RootView` reads the platform's size class once and draws one of two shells. A compact width gets the pill bar floating over the content along the bottom, and it stays put when the keyboard opens, letting the keyboard cover it: without `.ignoresSafeArea(.keyboard)` the whole stack is lifted and the bar rides up to sit on top of the note being written, in the strip someone is actually looking at.

A regular width gets the same bar stood on end as a left rail, as an overlay so a screen's canvas runs the full width underneath it. Overlays claim no layout room, which is why `railInset` exists for the screens to apply themselves. The rail withdraws while the pane is being worked with and comes back when that stops:

```
// LIfiOS/App/RootView.swift
content
    // Scrolling or dragging anywhere in the pane counts as working with the
    // content. Simultaneous, not exclusive: it observes the touch rather than
    // claiming it, so scroll views, text selection and Pencil strokes all still see it.
    .simultaneousGesture(
        DragGesture(minimumDistance: 12)
        .onChange { _ in withdrawRail() }
        .onEnded { _ in scheduleRailReturn() } // 1400 ms later
    )
    .overlay(alignment: .leading) {
        PillNavBar(selection: $tab, items: navItems, axis: .vertical)
        .opacity(isRailVisible ? 1 : 0)
        .offset(x: isRailVisible ? 0 : -(metrics.gutter + 76)) // parked off the edge
        .allowsHitTesting(isRailVisible)
    }
    .overlay(alignment: .leading) {
        if !isRailVisible {
            Color.clear.frame(width: 28).contentShape(.rect)
            .onTapGesture { showRail() } // a way back without scrolling something
first
        }
    }
}
```

The keyboard is the other way of working with the pane, and the one where the rail is most in the way: on a landscape iPad the keyboard takes half the height and the note being written is what is left. So `keyboardWillShow` withdraws the rail and `keyboardWillHide` brings it back, and a tab change brings it back too, because a tab change is navigation, not content work. The 1400 millisecond return delay is what

stops it flickering between the flicks of a fast scroll. Compact width never hides the bar: along the bottom it overlaps nothing that is being read.

ORIENTATION

The iPhone target allows landscape in its build settings so that a workout video can be watched sideways, but almost nothing else in the app is laid out for it. `OrientationLock` therefore defaults to portrait on a phone and every orientation on an iPad, and the one screen that wants more asks for it while it is on screen and hands it back when it leaves. `release()` also requests a geometry update back to portrait, because a phone still held sideways would otherwise stay in a landscape the next screen cannot draw.

4.10 Edge cases and tests

CONDITION	BEHAVIOUR
Session found, no store open	<code>AppShell</code> keeps the person in onboarding; <code>restoreSession</code> is not allowed to promote past the gate without <code>hasStore</code>
Offline at launch with an expired token	<code>.active(stored)</code> : the session is kept and the next foreground tries again
Server refuses the refresh token (400, 401, 403, 422)	Store cleared, <code>.rejected</code> , the person returns to signup with "Your session expired. Sign in again"
Rate limited or 5xx on refresh	Kept; a refusal must be definitive
Sign out during an in-flight refresh	The refresher re-reads the store after the network call and finds the token gone; returns <code>.signedOut</code> without saving
Refresh response names a different user id	Cleared and rejected; a refresh cannot replace the account's identity
Two restores at once (launch and foreground on a cold start)	Coalesced onto one task, because the refresh token rotates on use
App kept open past the hour	The keep-alive loop wakes 45 seconds before expiry and renews
Twilio cannot deliver	The specific cause is named and "Use email instead" is offered; the flag survives the trip back to the identity screen
Wrong code, twice	Shakes and buzzes both times; the counter, not a Bool, drives it
Autofill lands a code with no keystroke	Non-digits trimmed in the field, the Verify button remains
Upgrade from the single-store build	The legacy session is adopted into the roster and <code>default.store</code> moved into the account's directory, once
Current account id with no matching keychain session	Removed in <code>AccountSession.init</code> ; nothing opens
Store cannot be opened	Container nil, surfaces cleared, onboarding shown; never the previous account's data
Second account signs in on the same device	Own store file, own defaults suite, own keychain item; the roster keeps the first device
Cover already presented at launch	The tour is an overlay, so nothing contends for the window

TESTS

- `SessionRefreshTests` : twelve cases, one per row of the refresh policy, including `logoutDuringRefreshCannotResurrectTheSession` and `aRefreshCannotReplaceTheAccountIdentity` .
- `SessionKeepAliveTests` : the wake is late enough for the refresher to act, an already expired session still waits the floor, a clock jump does not spin.
- `AccountStoreTests` : nobody signed in means no scope, a mismatched identity cannot open another account's store, a restored roster uses its own stored session, and the bug the type exists to fix: a second sign-in no longer overwrites the first.
- `UserScopeTests` : two accounts get different files, an id cannot climb out of the accounts directory, the legacy store is adopted once and carries its write-ahead log, rows do not cross between accounts.
- `AuthErrorMappingTests` , `AuthRefreshURLTests` , `PhoneOTPTests` , `SignInChromeTests` (status precedence and `CodeDigits`).
- Native: `SurfaceNativeChecks` proves a delayed notification from a previous account is rejected and the next account starts with missing readings, never previous values.

4.11 Study notes

READ IN THIS ORDER

1. `LIfеOS/App/LIfеOSApp.swift` : the two-way switch on a container.
2. `LIfеOS/App/AccountSession.swift` : `init` , then `adopt(_:)` , then `signOut()` .
3. `LifeOSKit/Sources/Integrations/AccountStore.swift` : the roster and the keychain per account.
4. `LIfеOS/App/AppShell.swift` : the three-way gate, the keep-alive loop, and `handle(_ url:)` .
5. `LifeOSKit/Sources/Integrations/SessionRefresh.swift` : read every guard as a bug report.
6. `LifeOSKit/Sources/Integrations/SupabaseAuth.swift` : the three calls and `AuthError.fromHTTP` .
7. `LIfеOS/Features/Onboarding/ViewModel/OnboardingViewModel.swift` : `verifyCode` and `saveProfile` .
8. `LIfеOS/App/RootView.swift` , the `wideShell` and the rail functions.

CHECK YOURSELF

- **Why are there two locks before `RootView` mounts?** A session without a store reaches the first fetch and CoreData throws an exception Swift cannot catch; a store without a session is somebody's data with nobody signed in.
- **Which HTTP statuses sign a person out, and why only those?** 400, 401, 403 and 422: the server refused the credential itself. 429 and 5xx are failures to answer, and a transport error is not an answer at all.
- **Why is the keep-alive margin 45 seconds rather than 90?** `isExpired` has 60 seconds of slack; waking before that finds a healthy session and does nothing.
- **Why does `KeychainAuthSessionStore()` read the current account id from defaults directly?** It is built from wherever a network call is made; going through the main-actor `AccountStore` would not be possible there, and defaulting to the current account is what stops every client presenting the old token after a switch.

- **What decides whether a person sees the profile step?** `AuthSession.hasProfile` , derived from user metadata in the verify response. The door they tapped decides only copy.
- **Why is the resend countdown 60 seconds when the phone cooldown is 45?** Email is governed by GoTrue's 60-second frequency; the countdown must clear the stricter of the two.

PART TWO

The screens

One chapter per feature. Each follows the same shape: intent, files, model, view model, view, how it works end to end, what keeps it fast and cheap, what breaks and how it is handled, and how to study it.

- 5. Onboarding
- 6. Today and the calendar
- 7. Health
- 8. Live sessions
- 9. The Watch companion
- 10. The workout library
- 11. Money
- 12. Notes and the plan
- 13. The Life board and the nine sectors
- 14. LIFO, the coach
- 15. The calendar assistant
- 16. Social
- 17. Profile and settings
- 18. Widgets, Live Activities and notifications

5 Onboarding

Five pages of pitch with a cat that waves, then an identity, a six-digit code, a name, and the connections. Signup is mandatory and it is the only way in: a store belongs to an account, so there is nowhere for a signed-out person's data to live.

5.1 Intent

WORKING BACKWARDS

Almanac opens on a lit 3D cat and five pages that say what the app is, one domain per page in the order the tabs appear. Signing in takes an email address or a phone number and a code, a first and last name, and then offers WHOOP, Fitbit, a bank through Plaid and Apple Health. Every connection is skippable. Nothing on the way in asks for a password. A returning user is sent straight into the app once the code is accepted; a new one gets one welcome screen and then Today.

Why is there an account at all?

Because the data outlives the phone, and because the app now supports several people on one device. The account is what lets Supabase hold notes, a profile and provider links, what Row Level Security keys every row to, and what names the SwiftData file this person's data lives in (Chapter 4). Without it a lost device is a lost year, and a second person signing in would inherit the first person's everything.

Why no password?

A password is a credential to lose, store and reset. A one-time code delivered to a channel the user already controls proves the same thing with no secret at rest and no reset flow to build.

Email or phone?

Email first. It is free through Resend to 3,000 a month, needs no A2P registration, and carries none of the SMS failure modes. An SMS costs about five cents and can fail four different ways. Phone is offered as the alternative, to the US and India only, and the app asks Supabase which channels are actually enabled before it renders the choice.

Why can every connection be skipped?

A screen that blocks on a WHOOP login is a screen people abandon. Everything Almanac shows degrades to a blank rather than a wrong number, so an unconnected app is honest, not broken. Every connection is offered again in Settings.

FLOW ANATOMY

- **IntroScreen** : five pages over one persistent SceneKit scene, with a "Skip" and a sign-in door for a returning user.
- **IdentityScreen** : email and phone tabs, a dial-code picker, one field, one complaint at a time.
- **CodeScreen** : six boxes, verification fired on the sixth digit, a shake on a wrong code.
- **ProfileStepScreen** : photo, first and last name, birthday, height, gender, country. Only the name is required.
- **ConnectionsScreen** : WHOOP, Fitbit, bank accounts, Apple Health, each with its live state.
- **FirstRunTour** : one welcome screen after the account exists, shown once per account, however the person arrived.

5.2 Files and folders

```
Features/Onboarding/
├─ Model/
│   ├── OnboardingModels.swift      step enum, auth mode, intro pages, draft, dial codes, Gend
│   └─ ProfilePhotoStore.swift      per-account avatar cache, LocalProfile, ProfileStore, Prof
├─ ViewModel/
│   └─ OnboardingViewModel.swift    channels, send, verify, profile, restore, sign out
└─ View/
    ├── IntroScreen.swift           the pager and the hero
    ├── OnboardingCat.swift          the SceneKit character
    ├── SignupScreens.swift          SignupScaffold, IdentityScreen, CodeScreen, ProfileStepScr
    ├── ConnectionsScreen.swift      the four connection rows, plus OnboardingFlow
    └─ FirstRunTour.swift            the one-time welcome

App/AppShell.swift                  chooses onboarding or RootView; owns the tour flag
Components/Model/ProfilePhotoDownsizing.swift  512pt JPEG under the bucket limit

LifeOSKit/Sources/DesignSystem/SignInChrome.swift  InlineStatus, CodeField, shake, focusableFie
LifeOSKit/Sources/Integrations/SupabaseAuth.swift  /auth/v1/settings, otp, verify, user
LifeOSKit/Sources/Integrations/PhoneOTP.swift      E.164 rules, shared with the Edge Function
LifeOSKit/Sources/Integrations/AuthSessionStore.swift  the keychain slot, per account
LifeOSKit/Sources/Integrations/ProfileClient.swift  the profiles row and the avatars bucket
```

OnboardingFlow lives beside **ConnectionsScreen** rather than in its own file because it is a twenty-line switch over **OnboardingStep** and nothing else. **AppShell** chooses between it and **RootView**, so the tab hierarchy is never constructed for a signed-out user. The session and account machinery it leans on is described in Chapter 4; this chapter is about the screens and the rules they enforce.

5.3 Model

```
// LIfiOS/Features/Onboarding/Model/OnboardingModels.swift
enum OnboardingStep: Equatable {
    case intro
    case identity           // phone or email
    case code               // OTP (phone or email)
    case profile            // name, country
    case connections        // Whoop and Health, all optional
    case signedIn           // returning user; nothing left to ask
}

enum AuthMode: Equatable {
    case signUp
    case signIn
}
```

`OnboardingStep` is a single value because the alternative, a pile of booleans, can represent states that do not exist. A user cannot be on the code screen and the profile screen at once, and the type should say so.

`AuthMode` is the door the user came through, and it changes copy and nothing else. With OTP there is no password, so sign-in and signup are the same three steps; picking the wrong door is harmless by design, because the branch after verify is what actually routes.

```
// LIfeOS/Features/Onboarding/Model/OnboardingModels.swift
struct SignupDraft: Equatable {
    var channel: SupabaseAuthChannel = .email
    var phone = ""
    var dialCode = "+1"
    var email = ""
    var code = ""
    var firstName = ""
    var lastName = ""
    var country = Locale.current.region?.identifier ?? "US"

    var photo: Data? // already downsized
    var birthDate: Date?
    var heightCM: Double? // always centimetres, whatever the field shows
    var gender: Gender = .unspecified

    var e164: String { dialCode + phone.filter(\.isNumber) }

    var canSendCode: Bool {
        channel == .phone
        ? PhoneOTP.isAcceptable(e164)
        : email.contains("@") && email.contains(".")
    }
    var canVerify: Bool { code.filter(\.isNumber).count >= 6 }
    var canFinishProfile: Bool {
        !firstName.trimmingCharacters(in: .whitespaces).isEmpty
        && !lastName.trimmingCharacters(in: .whitespaces).isEmpty
    }
}
```

The draft holds what the user typed, before any of it is trusted. The three `can...` properties are the only validation in the feature, and they live on the model rather than in a view so a disabled button and a rejected submit can never disagree. Three details are worth noticing:

- **The photo is `Data`, not `UIImage`.** That keeps the draft a value type that can be compared for equality, and it means the bytes are already the downsized JPEG that will be uploaded, so no view holds a multi-megabyte camera-roll original.
- **Height is always centimetres internally.** The imperial fields on the profile step are bindings that translate for display. Storing the number in whichever unit the user happened to be typing is how a height ends up silently wrong after a locale change.
- **Only the name is required.** Everything else on the profile step sharpens what the app can say, and none of it is worth blocking someone at the door for.

`Gender` is deliberately not a biological-sex field wearing a friendlier label. It is how someone wants to be addressed, "Prefer not to say" is the default and a real answer, and `stored` returns nil for it so nothing is written to the server. If health baselines ever need sex, that is a different question with different answers, and `HealthConnectionViewModel` already reads it from Health's own characteristic rather than from here (17.4).

`DialCountries.all` is built from `Locale.Region.isoRegions` crossed with a hand-typed table of known dial codes, so the list is complete and localised while a region with no known code is dropped rather than

guessed at. Offering the wrong prefix sends an SMS into the void, and the user has no way to tell that is what happened. `otpRegions` is `["US", "IN"]`: the country picker on the identity screen is filtered to those two, and the view model resets a draft whose locale is elsewhere to the US.

5.4 ViewModel

```
// LIfiOS/Features/Onboarding/ViewModel/OnboardingViewModel.swift
@MainActor @Observable
final class OnboardingViewModel {
    private(set) var step: OnboardingStep = .intro
    private(set) var mode: AuthMode = .signUp
    var draft = SignupDraft()

    private(set) var isBusy = false
    private(set) var errorMessage: String?
    private(set) var phoneSendFailed = false
    private(set) var resendIn = 0
    private(set) var availableChannels: Set<SupabaseAuthChannel> = []
    private(set) var isSignedIn: Bool
    private(set) var session: AuthSession?

    private var store: any AuthSessionStoring
    private var auth: SupabaseAuth?
    private var restoreTask: Task<Bool, Never>?

    func loadChannels() async
    func restoreSession() async -> Bool
    func sendCode() async
    func verifyCode() async
    func saveProfile() async
    func signOut()
}
```

The rules that live here, and nowhere else:

- **Channel availability is asked, not assumed.** `loadChannels()` reads `auth/v1/settings`. Email stays the default when it is configured and the draft quietly falls back to phone when it is not; `phoneUnavailableNote` explains a missing SMS provider rather than leaving a dead tab on screen.
- **Email sends a six-digit code, not a magic link.** Including `{{ .Token }}` in the project's email template is what makes GoTrue send a code, and `verifyCode()` posts it to `auth/v1/verify` exactly as phone does, with no per-channel branch.
- **The account, not the door, decides what comes next.** After `verify` the step is `session.hasProfile ? .signedIn : .profile`. A "Sign in" tap with no account still creates one; a "Get started" tap on an existing account skips the profile step it already has answers for.
- **Verify also records the account.** The same call adds the session to `AccountStore`, clears the pre-accounts keychain slot, and re-points the view model's own store at the per-account slot. From here on every network client in the app reads this account's token (Chapter 4).

- **A failed profile write does not strand the user.** The account already exists by then, so `saveProfile()` records the error and advances to `.connections` anyway. Blocking the last step of signup on a metadata write would trade a cosmetic failure for an unusable app.
- **The photo goes to storage before the row.** `saveProfile()` uploads the avatar first and only then saves the `profiles` row, so the row never points at an object that does not exist. A failed upload costs the picture and not the profile; it is a `try?`.
- **Names still go to `user_metadata` as well.** The profile row is the source of truth for display, but `hasProfile` is derived from `user_metadata.first_name` when a session is refreshed, and that is what decides whether a returning user is sent through signup again. Both writes are needed.
- **Resend is rate-limited in the UI.** A sixty-second countdown starts on every send. Sixty, not forty-five: phone's Edge Function cooldown is 45 seconds, but email posts straight to GoTrue, whose `auth.email.max_frequency` is 60. The countdown has to clear the stricter of the two or "Resend code" invites a 429.
- **`isSignedIn` is stored, not computed.** A computed `store.load() != nil` is invisible to `@Observable`, which once left the shell showing the app after a sign-out until something else happened to redraw it.
- **Restoring is one call, and it is shared.** `restoreSession()` holds the in-flight `Task` so a launch, a foreground and the keep-alive loop arriving together await one refresh. Supabase rotates the refresh token on use, so the second of a concurrent pair would present a token the first had just retired and be refused.

```
// LifeOS/Features/Onboarding/ViewModel/OnboardingViewModel.swift
func verifyCode() async {
    guard let auth, draft.canVerify else { return }
    isBusy = true
    errorMessage = nil
    defer { isBusy = false }

    do {
        let session = try await auth.verify(
            code: draft.code.filter(\.isNumber),
            destination: draft.destination,
            channel: draft.channel
        )
        try AccountStore().add(Account(userID: session.userID,
            label: session.email ?? session.phone ?? "Account"), session: session)
        AccountStore().clearLegacySession()
        store = KeychainAuthSessionStore(account: session.userID)
        self.session = session
        isSignedIn = true
        step = session.hasProfile ? .signedIn : .profile
    } catch let error as AuthError {
        errorMessage = error.readable
    } catch {
        errorMessage = "That code didn't work"
    }
}
```

The initialiser has one more rule worth knowing. It reads the keychain synchronously and, if a session is there, decides the opening step at once: `.connections` if the per-account defaults say the person got that far and

quit, otherwise `.signedIn` or `.profile` by `hasProfile`. That is what lets a relaunch land on the app without flashing the intro for a frame.

HASPROFILE DEFAULTS TO FALSE ON DECODE, BY HAND

A session already in the keychain from before this field existed has no `hasProfile` key. A synthesised `Decodable` would throw on it and sign every existing user out on the upgrade, so `AuthSession` writes its own initialiser and defaults the missing key to false. That default is safe only because the flag is read once, immediately after `verify()`; a restored session never consults it again.

5.5 View

THE INTRO AND THE CAT

`IntroScreen` renders `IntroPage.all` in a `TabView` pager under a hero that stays put while the pages slide. The pages are data rather than five hand-written views for the same reason as before: a loop cannot let page three fall out of step with page one. At accessibility text sizes the pager is replaced with a single continuous reading surface, so a nested horizontal pager does not fight vertical accessibility scrolling. The copy height is measured with `onGeometryChange` and the largest page wins, so the hero does not jump between pages of different length.

The hero is `OnboardingCat`, a `UIViewRepresentable` over an `SCNView`. It is a real lit character rather than an image sequence: an orthographic camera, an ambient light and one omni key light, a physically based coat material, and a body built from about thirty scaled spheres plus two extruded ear shapes and two extruded arcs for the smile. The horizontal ink lines that make it read as the app's dotted illustration style come from a surface shader modifier:

```
// LIfiOS/Features/Onboarding/View/OnboardingCat.swift
coat.shaderModifiers = [surface: """
    #pragma body
    float row = _surface.position.y * 16.0;
    float edge = abs(fract(row) - 0.5);
    float aa = max(fwidth(row), 0.025);
    float ink = 1.0 - smoothstep(0.14 - aa, 0.14 + aa, edge);
    _surface.diffuse.rgb *= mix(1.0, 0.035, ink);
    """]
```

Rows are computed from view-space `y`, so they stay horizontal as the cat turns, and `fwidth` softens each edge at the rendered resolution so the lines do not shimmer. Motion is a handful of `SCNAction`s: a breathing rise, a head tilt, a tail curl, an eye blink on a 4.6 second keyframe loop, and a wave that plays on page change and on tap. All of it is cheap by construction: the view is capped at 30 frames per second, the scene is paused whenever the screen is not active or Reduce Motion is on, and `dismantleUIView` drops the scene entirely. With Reduce Motion the cat still turns to face each page, it just snaps instead of animating.

THE SIGNUP SCAFFOLD

```
// LIfiOS/Features/Onboarding/View/SignupScreens.swift
struct SignupScaffold<Content: View, Action: View>: View {
    let title: String
    let subtitle: String
    var step = 1
    var totalSteps = 3
    var showsProgress = true
    var onBack: (() -> Void)?
    @ViewBuilder let content: Content
    @ViewBuilder let action: Action
}
```

Every signup step is the same shape: a back control, the wordmark, a progress rule, a title, a subtitle, one field group, and one primary action pinned to the bottom in a `safeAreaInset`. `SignupScaffold` owns that shape so the three screens differ only in their content, which is what keeps them from drifting apart visually as they are edited. Sign-in mode hides the progress rule and counts two steps rather than three; a returning user has no profile step to count.

THE CODE SCREEN DOES THE WORK INSTEAD OF ASKING FOR IT

This is the screen people fail on, so it is built to remove the ways of failing. The single tracked text field is gone; `CodeField` from `DesignSystem` draws six boxes and calls back the moment the sixth digit lands, so verification fires without a tap. Autofill can land a whole code with no keystroke, so a Verify button is kept for that user. A wrong code bumps a counter, and the counter drives both a spring shake and an error haptic:

```
// LIfiOS/Features/Onboarding/View/SignupScreens.swift
@State private var wrongAttempts = 0
...
CodeField(text: $model.draft.code, count: 6) { verify() }
    .shake(wrongAttempts)
    .animation(.spring(response: 0.28, dampingFraction: 0.4), value: wrongAttempts)
...
.onChange(of: model.errorMessage) { _, new in
    guard new?.isEmpty == false else { return }
    wrongAttempts += 1
}
.sensoryFeedback(.error, trigger: wrongAttempts)
```

A counter rather than a Bool, because two bad codes in a row must shake twice, and a Bool that is already true animates nothing.

ONE COMPLAINT AT A TIME

The identity screen can have three things to say at once: a validation error, a configuration problem, and a note that SMS is off. Stacked, they read as a broken app. `InlineStatus.resolve` picks the single most actionable one, with precedence error, then configuration, then warning, and treats whitespace as absence so

an empty string never paints a red gap. The "Use email instead" escape is kept below the status rather than folded into it: it is an action the user can take, not another thing going wrong.

THE PROFILE STEP

A `Picker` circle at the top, two name fields, then four labelled rows that read as a set: birthday, height, gender, country. The birthday picker opens on a plausible adult date thirty years back rather than today, so the wheel is not three decades of scrolling. Height shows feet and inches where the locale is imperial and centimetres elsewhere, through bindings that only translate; the stored truth is centimetres. The picked photo is downsized before it is kept, because a camera-roll original is several megabytes and this is displayed at 96 points (17.6 for the maths).

CONNECTIONS, AND THE FLOW

`ConnectionsScreen` is the only onboarding screen that reads live state. It takes the four shared connection view models that `AppShell` owns, so a connection made here is already made when Settings opens later. Each row shows the provider's own `statusDetail`, a spinner while it connects or syncs, a "Connected" label once it has, and a disabled button with "Available when this service is enabled" when the build lacks the provider's configuration. The Apple Health row calls `health.connect()`, which is the permission prompt and the first sync in one; the same view model serves Today's empty-state button and the Connections page in Settings.

```
// LifeOS/Features/Onboarding/View/ConnectionsScreen.swift
struct OnboardingFlow: View {
    @Bindable var model: OnboardingViewModel
    ...
    var body: some View {
        Group {
            switch model.step {
            case .intro:      IntroScreen(onStart: { model.beginSignup() },
                                         onSignIn: { model.beginSignIn() })

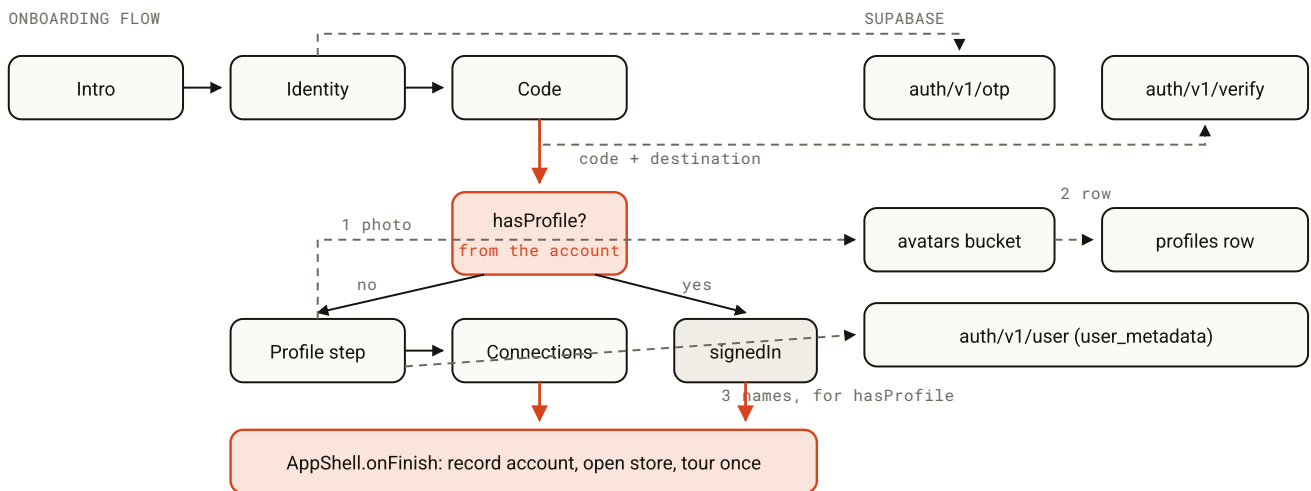
            case .identity:   IdentityScreen(model: model)
            case .code:      CodeScreen(model: model)
            case .profile:   ProfileStepScreen(model: model)
            case .connections: ConnectionsScreen(model: model, whoop: whoop, fitbit: fitbit,
                                                health: health, plaid: plaid, onFinish:
onFinish)
            case .signedIn:   ProgressView().controlSize(.large)
            }
        }
        .animation(.easeInOut(duration: 0.28), value: model.step)
        .onChange(of: model.step) { _, step in
            if step == .signedIn { onFinish() }
        }
    }
}
```

`onFinish` is called from `onChange` rather than from the `.signedIn` view body, because a body can run more than once for a single state and `onFinish` flips persisted app state.

THE WELCOME

`FirstRunTour` replaced a six-page second carousel with one scrollable screen: the cat, "Welcome home.", three rows pointing at Today, connections and the coach, and an "Open my day" button. It is presented by `AppShell` as an overlay rather than a `fullScreenCover`, because iOS can restore a previously presented cover at launch and two covers contending for one window silently drops one. Its flag, `hasSeenFirstRunTour`, is stored in the per-account defaults and is separate from `hasFinishedOnboarding`, so it fires exactly once per account however the person arrived: signup, sign-in, or skip. A 400 millisecond delay and a cancellation check keep a stale tour from presenting over a shell that is already going away.

5.6 How it works



The door the user came through never routes. The account does, once the code is verified. The profile step writes three things in a fixed order so a row can never point at a missing picture, and so a refreshed session can still tell a returning user from a new one.

Step by step, for a new user on a fresh install:

1. `LifeOSApp` finds no account, so `AccountSession.container` is nil and `AppShell` is mounted with `hasStore: false`. It renders `OnboardingFlow`.
2. The identity screen's `.task` asks `auth/v1/settings` which channels exist and logs, rather than shows, a missing anon key.
3. "Send code" posts to `auth/v1/otp` for email or to the `otp-start` Edge Function for phone (Chapter 20). The step becomes `.code` and the sixty-second countdown starts.
4. The sixth digit fires `verifyCode()`. On success the session goes into the per-account keychain slot, the account into the roster, and the step branches on `hasProfile`.
5. The profile step uploads the avatar, saves the `profiles` row, mirrors it into the per-account local caches, writes the names into `user_metadata`, and records `"onboarding.step" = "connections"` so a quit here resumes here.
6. Connections are offered. "Continue to LifeOS" calls `onFinish`.
7. `AppShell.onFinish` hands the account and session to `AccountSession.signIn`, which opens this account's store, clears the resume marker, and animates `hasFinishedOnboarding` to true. The whole

scene re-roots on the new account id and `RootView` mounts for the first time.

8. Four hundred milliseconds later the tour fades in, once.

For a returning user the shell's launch task calls `restoreSession()`. If the refresh succeeds, a store is open and the step is `.signedIn`, it sets `hasFinishedOnboarding` and pulls the profile from the server with `ProfileSync.pull()`, because the profile belongs to the account and not to the phone that typed it. A refused refresh is the only thing that sends someone back to signup.

5.7 Performance and cost

- **No store, no tab hierarchy.** Onboarding never touches `SwiftData`. The tabs, their view models and their first fetches are constructed only after an account has a store, which is also what prevents the uncatchable Core Data exception a fetch against an empty schema would raise (Chapter 4).
- **The keychain is read synchronously at init.** `isSignedIn` and the opening step are decided before the first render, so a relaunch that restores a session never paints the intro carousel for a frame.
- **The cat costs what a 30 fps SceneKit view costs, and only while it is on screen.** It pauses when the scene phase is not active, when Reduce Motion is on, and when the view disappears. Geometry is thirty low-segment spheres; the ink lines are a few shader instructions rather than a texture.
- **One refresh, however many callers.** The shared `restoreTask` is the difference between one token rotation and a refused second one.
- **Email is the free channel.** The default channel costs nothing per send; phone is rate-limited on both sides and restricted to two regions so SMS pumping has nowhere to aim (`PhoneOTP.pumpingPrefix` exists for the server's benefit).
- **Avatars are 512 points, scale 1, JPEG 0.8.** A profile photo is a few tens of kilobytes by the time it is held in the draft, uploaded, and cached.
- **The profile pull is two small requests.** One row, and a bucket listing scoped to the account's own folder; the picture itself is only downloaded when the listing's timestamp differs from the recorded one (17.6).
- **Nothing about the intro leaves the device.** The first network request is the settings probe, made when the identity screen appears, not at launch.

5.8 Edge cases and tests

CONDITION	BEHAVIOUR
No <code>SUPABASE_ANON_KEY</code> in the build	<code>isConfigured</code> is false; the identity screen shows "Sign-in is unavailable right now" and the key name goes to the log, not the screen
SMS not enabled on the project	The phone tab stays but a warning explains it; sending is still possible by email
Email not enabled, phone is	<code>loadChannels()</code> flips the draft to phone silently
Locale outside the US and India	The draft resets to <code>US</code> , <code>+1</code> ; the country picker offers only the two OTP regions
Wrong or expired code	Inline error, shake, haptic; the step stays <code>.code</code> so the code can be retried without resending
SMS send fails	Error shown and <code>phoneSendFailed</code> set; "Use email instead" appears and survives backing out to the identity screen, but is cleared on the way to the intro
Resend tapped early	Disabled until the sixty-second countdown clears; no 429 is ever provoked from the client
Profile write fails	Error recorded, flow advances to connections; the account is already valid
Avatar upload fails	Row saved without <code>avatar_path</code> ; the local copy still shows the picture
Quit during the profile or connections step	Relaunch resumes at connections; a quit before the profile step resumes at the profile step, by <code>hasProfile</code>
Returning user, session in keychain	Renewed at launch, then straight into the app; profile pulled from the server
Returning user, session refused	Signed out with "Your session expired. Sign in again"
Returning user, offline	Stays signed in. The credential is still good; only the network is missing
Foregrounded mid-signup after an hour	Session renewed, but <code>hasFinishedOnboarding</code> is never promoted here, so nobody is yanked out of the profile step for switching apps
Second person signs in on the same device	Gets their own store, keychain slot, defaults suite, avatar file and profile cache; the first person's remain, unopened
Reduce Motion on	The cat still faces each page, without animating; the pager selects without easing
Accessibility text sizes	The pager becomes one continuous column; the header caps its own type at <code>xxxLarge</code> so the wordmark and Skip stay on one line

TESTS

- `SessionRefreshTests` (twelve tests) drives every branch of the refresher against a stub `URLProtocol`: an expired session is refreshed and saved, a rejected refresh token signs out, being offline, an outage and a 429 all keep the stored session, and a logout during a refresh cannot resurrect it.
- `AuthSessionProfileTests` pins the `hasProfile` rules: a verified user with a first name has one, a whitespace-only name does not, a stored session without the key still decodes, and a refresh does not forget it.

- `AuthErrorMappingTests` maps every Edge Function failure kind, plus the generic 502 fallback on both channels, to a specific readable sentence.
- `PhoneOTPTests` : Indian mobiles accepted, Indian landline lengths rejected, US and the local test number accepted, other countries and short values rejected.
- `SignInChromeTests` : `InlineStatus` precedence and blank handling; the code field fills from the left, ignores non-digits, and drops overflow.
- `AccountStoreTests` (seventeen tests) covers the roster: a second account does not evict the first, signing out clears the legacy slot too, removing an account leaves its store on disk, a mismatched identity cannot open another account's store.
- `SessionKeepAliveTests` : the wake is late enough to act on, an already expired session still waits the floor, a clock jump cannot produce a spin.

The signup screens themselves have no automated test; they are exercised by hand against the project's test-number map, where `+1 123-456-7890` with code `123123` returns a real session and never reaches Twilio. Email has no equivalent bypass and depends on the Resend SMTP block being pushed to the hosted project.

THE GUEST BYPASS IS GONE

Earlier editions described a "Skip for now" guest mode. It no longer exists. A store belongs to an account, notes sync to one, and connections are held per account, so there is nowhere for a signed-out person's data to live that would not become somebody else's the moment they signed in. The intro's "Skip" only skips the pitch; it lands on the identity screen.

5.9 Study notes

READ IN THIS ORDER

1. `Features/Onboarding/Model/OnboardingModels.swift` : the step enum and the draft are the whole contract.
2. `Features/Onboarding/ViewModel/OnboardingViewModel.swift` : read `init` , `verifyCode` , `saveProfile` , `restoreSession` in that order.
3. `App/AppShell.swift` : the gate on line 57 and the three `.task` blocks.
4. `LifeOSKit/Sources/Integrations/SupabaseAuth.swift` : `AuthSession` and its hand-written decoder.
5. `Features/Onboarding/View/SignupScreens.swift` : `CodeScreen` first, then the profile step's height bindings.
6. `LifeOSKit/Sources/DesignSystem/SignInChrome.swift` : `InlineStatus.resolve` and `CodeField` .
7. `Features/Onboarding/Model/ProfilePhotoStore.swift` : `ProfileSync.pull` and the stamp.
8. `Features/Onboarding/View/OnboardingCat.swift` : the shader and the idle actions.

CHECK YOURSELF

- **Why does tapping "Sign in" with no account still create one?** Because OTP has no password; the only thing that distinguishes a new user is `hasProfile` , read after verify, and the mode only changes copy.

- **Why is the resend countdown 60 seconds and not 45?** Phone's Edge Function cooldown is 45s, but email goes straight to GoTrue whose `max_frequency` is 60s; the client has to clear the stricter one.
- **What would break if `AuthSession` used a synthesised decoder?** Every session stored before `hasProfile` existed would fail to decode and sign its user out on upgrade.
- **Why is the tour an overlay and not a `fullScreenCover`?** iOS can restore a cover at launch, and two covers contending for one window silently drops one.
- **Why does the profile step upload the photo before saving the row?** So the row can never point at an object that does not exist; a failed upload loses the picture, not the profile.
- **Why does the identity screen probe `auth/v1/settings` rather than assuming both channels?** A phone tab whose SMS provider is off is a button that silently fails; asking first turns that into a warning with a working alternative.

6 Today and the calendar

The hub. A month of dots, a streak, today's schedule, and four tiles that each carry their own week. Every number on it comes from one local table and one local cache, and every tap opens something rather than explaining something.

6.1 Intent

WORKING BACKWARDS

Today opens on a grid of dots, one per day this month. Filled dots are days you hit your targets, hollow dots are misses, faint dots are days with no data, and today is orange. Under it sit your schedule for the day and the week ahead, drawn from the phone's calendar, then a streak, then four tiles: steps, sleep, weight, recovery, each with the last seven days as bars. Tap a dot and the day opens in full; tap a tile and the metric opens on a page of its own with a week, a month or a year behind it; tap the calendar icon and a month you can walk through opens.

What makes a dot fill?

Meeting 3 of 4 daily goals: steps 8,000, sleep 7h, exercise 30 min, water 2,500 ml. All five values are editable in Settings (Chapter 17). The rule is deliberately explicit; a vague definition makes the entire grid meaningless.

What happens on a day with no data at all?

It renders as no-data, not as a miss, and it is excluded from streak arithmetic entirely. Leaving your watch on the charger is not a moral failure, and a system that punishes it teaches you to distrust the streak. A day with some data is judged on what it has: if steps are logged at 500 and nothing else was recorded, that is a miss.

If I change a goal, does history change?

Yes. Status is computed on read, never stored, so past dots re-evaluate against the new target immediately. The grid answers "how am I doing against my current standard".

Where does the schedule come from?

From EventKit, through a local SwiftData cache. The permission prompt fires from the agenda card's own connect button, never at launch, and the phone's calendar app remains the source of truth: Almanac reads a window of 30 days back and 90 forward, writes go straight back to EventKit, and the cache is re-synced afterwards so it shows what the provider actually stored.

Why is there a separate month screen?

Today's month grid shows how the streak has been going and cannot leave the current month. The month screen answers the other question, what is on, across any month, with day numbers and up to three event dots per day. Two questions, two grids.

6.2 Files and folders

```
Features/Today/
├─ Model/
│   ├── TodaySnapshot.swift      everything the hub renders, plus CalendarAccessState and Up
│   ├── TodayMetric.swift        steps · sleep · weight · recovery: icon, hue, unit, format,
│   ├── MetricDetailSnapshot.swift MetricRange (week/month/year), MetricBar, the page's values
│   └─ DayDetailSnapshot.swift   one day: metrics, habits, events
├─ ViewModel/
│   ├── TodayViewModel.swift     the 400-day pass: grid, streak, week series, agenda, day sh
│   ├── MetricDetailViewModel.swift one fetch, bucketed by day or month
│   └─ MonthViewModel.swift      one fetch per month, grouped by day, reversible stepping
└─ View/
    ├── TodayScreen.swift        layout only; one column or two
    ├── AgendaCard.swift         today's schedule and Upcoming, journey rows, DayPart
    ├── CalendarEventRow.swift   the month screen's row (multi-day spans)
    ├── DayDetailSheet.swift     the day in full; only today gets controls
    ├── EventSheet.swift         create or edit one event; recurring is read-only
    ├── MetricDetailScreen.swift chart, summary, all readings, tracking help
    └─ MonthScreen.swift         the month grid, swipe to step, date jump

Components/View/QuickLogSheet.swift      water and weight, the two metrics with no automatic
Components/ViewModel/QuickLogViewModel.swift
Components/View/JournalEntrySheet.swift   free text, no title

LifeOSKit/Sources/Persistence/
├─ CalendarEvent.swift           @Model, CalendarEventSnapshot (Sendable), CalendarEventDraft
├─ CalendarStore.swift          windowed reads, apply() reconcile, freeSlots()
├─ CalendarDayIndex.swift        overlapping events grouped per visible day
├─ GoalEvaluation.swift          DayStatus, evaluate(_:against:)
└─ Streak.swift                 currentStreak(statuses:)

LifeOSKit/Sources/Integrations/
├─ CalendarSource.swift          the provider protocol
├─ EventKitSource.swift          EventKit behind it, an actor
├─ CalendarSync.swift            pull every authorised source, merge, reconcile; route writes
├─ CalendarMerge.swift           title + start dedup, EventKit wins
└─ HealthSyncWindow.swift        which days a Health sync re-reads

LifeOSKit/Sources/DesignSystem/
├─ MonthGridLayout.swift         DotState, DotCell, the padding maths, stepping()
├─ MonthCalendarView.swift       DateHeadline, WeekdayHeader, DotGrid composition
├─ TrendSeries.swift             TrendPoint, the seven-slot window
├─ TrendStatTile.swift           the tile with the week
└─ RoundedBarChart.swift         tracks, bars, gaps, two baselines
```

6.3 Model

THE HUB'S SNAPSHOT

```
// LIfеOS/Features/Today/Model/TodaySnapshot.swift
struct TodaySnapshot: Equatable {
    var date: Date = .now
    var cells: [DotCell] = []
    var streak: Int = 0

    var steps: Int?
    var stepsProgress: Double?
    var sleepMinutes: Int?
    var sleepProgress: Double?
    var weightKg: Double?
    var recoveryPct: Double?

    var stepsWeek: TrendSeries = TrendSeries(points: [])
    var sleepWeek: TrendSeries = TrendSeries(points: [])
    var weightWeek: TrendSeries = TrendSeries(points: [])
    var recoveryWeek: TrendSeries = TrendSeries(points: [])
    var stepsTarget: Double?
    var sleepTargetMinutes: Double?

    var calendarAccess: CalendarAccessState = .notDetermined
    var agenda: [CalendarEventSnapshot] = []
    var upcoming: [UpcomingEvent] = []
    var scheduledWorkoutTitle: String?

    var hasNoHealthData: Bool {
        steps == nil && sleepMinutes == nil && weightKg == nil && recoveryPct == nil
        && stepsWeek.points.isEmpty && sleepWeek.points.isEmpty
    }
}
```

Every metric is an optional, and each one that has a goal carries its progress alongside it, already divided. The view receives numbers it can render and nothing it has to interpret: it never sees a `DailyMetrics`, so rendering cannot fault a `SwiftData` object or trigger a fetch mid-layout. Two things are new since the previous edition. Each figure carries its week as a `TrendSeries` with a slot for every day, so a gap stays a gap in the chart rather than closing up. And `hasNoHealthData` exists because the app used to seed sixty days of invented history at first launch, so an empty hub was unreachable; now that the numbers are real, an unconnected day is the normal first thing a person sees and the screen has to offer a connection rather than a grid of dashes.

ONE METRIC, DESCRIBED ONCE

```
// LIfiOS/Features/Today/Model/TodayMetric.swift
enum TodayMetric: String, Identifiable, Hashable, CaseIterable {
    case steps, sleep, weight, recovery

    var icon: String { ... }           // figure.walk, moon.fill, scalemass.fill,
    bolt.heart.fill
    var hue: ModuleHue { ... }         // activity, nutrition, body, recovery
    var unit: String? { ... }          // nil, nil, "kg", "%"
    var baseline: RoundedBarChart.Baseline { self == .weight ? .windowMinimum : .zero }
    var higherIsBetter: Bool { self != .weight }

    func value(in row: DailyMetrics) -> Double? { ... }
    func goal(in targets: GoalTargets) -> Double? { ... }
    func format(_ value: Double) -> String { ... }
}
```

The tile and the detail page used to be able to disagree: the grid decided steps were `figure.walk` in the activity hue, and a second screen would have decided again. Icon, hue, unit, formatting, the reading a row yields and the goal a day is judged against all live here once, and both surfaces read them. It also gives the navigation destination something to be: a value the stack can carry, rather than four booleans. Weight is the one exception that runs through everything. It is read against its own window rather than against nought, because it moves about a percent over a week and bars measured from zero would be indistinguishable, and it has no goal, because it is not a target to hit.

THE CALENDAR EVENT, IN THREE SHAPES

```
// LifeOSKit/Sources/Persistence/CalendarEvent.swift
public struct CalendarEventSnapshot: Equatable, Identifiable, Sendable {
    public let id: UUID           // local; provisional on a provider fetch
    public let source: CalendarEventSource
    public let sourceID: String    // EKEvent.eventIdentifier, qualified for recurring rows
    public let calendarTitle: String
    public let title: String
    public let startDate: Date
    public let endDate: Date
    public let isAllDay: Bool
    public let isRecurring: Bool
    public let location: String?
    public let notes: String?
}

public struct CalendarEventDraft: Equatable, Sendable {
    public var title: String
    public var startDate: Date
    public var endDate: Date
    public var isAllDay: Bool
    public var location: String?
    public var notes: String?
}

@Model public final class CalendarEvent { ... var sourceRaw: String ... var lastSyncedAt:
Date }
```

The `@Model` is the cache row. The snapshot is what views and assistant tools receive, and `Sendable` is the point: a tool result crosses into an inference engine (Chapter 15), and a `CalendarEvent` cannot make that trip. The draft is what a caller supplies to create or edit, with no id and no source, because routing decides those. The natural key of a row is `(source, sourceID)`; the local `id` never leaves the device. For a recurring occurrence the source id is qualified with the occurrence's start, because EventKit reuses one identifier across a whole series. A qualified id never resolves through `event(withIdentifier:)`, so a write against an occurrence fails with `unknownEvent` instead of silently editing the wrong one, and the sheet refuses such writes before that can happen.

THE DAY SHEET AND THE METRIC PAGE

`DayDetailSnapshot` is `Identifiable` on its date because `.sheet(item:)` presents it. It lists every habit that currently exists, done or not, deliberately not filtered by creation date, which is why the sheet's header is a count and never a grade: an old day can list a habit that did not exist then. `MetricDetailSnapshot` carries a `MetricRange` of week, month or year and a `MetricBar` per bucket. The grain is not a separate choice: a week is seven days, a month thirty days, a year twelve months, because a year drawn as 365 bars is a smear three points wide and a week drawn as one bar is not a chart.

6.4 ViewModel

TODAYVIEWMODEL.LOAD(): ONE PASS OVER THE ROWS

```
// LIfiOS/Features/Today/ViewModel/TodayViewModel.swift
func load() {
    guard let context else { return }
    let store = MetricsStore(context: context, calendar: calendar)
    do {
        let targets = try store.goals().targets
        let start = calendar.date(byAdding: .day, value: -400, to: .now) ?? .distantPast
        let rows = try store.metrics(from: start, to: .now) // most recent first

        var statusByDay: [Date: DayStatus] = [:]
        var ordered: [DayStatus] = []
        for row in rows {
            let status = evaluate(row.reading, against: targets)
            statusByDay[calendar.startOfDay(for: row.date)] = status
            ordered.append(status)
        }

        let cells = MonthGridLayout.cells(monthContaining: .now, calendar: calendar, today:
.now,
            status: { date in
                switch statusByDay[calendar.startOfDay(for: date)] {
                case .onTarget: .onTarget
                case .missed: .missed
                case .noData, nil: .noData
                }
            })

        var rowByDay: [Date: DailyMetrics] = [:]
        for row in rows { rowByDay[calendar.startOfDay(for: row.date)] = row }
        let week = TrendSeries.days(endingOn: .now, count: 7, calendar: calendar)
        func series(_ value: (DailyMetrics) -> Double?) -> TrendSeries {
            TrendSeries(points: week.map { day in
                TrendPoint(date: day, value: rowByDay[day].flatMap(value))
            })
        }
        ...
        snapshot = TodaySnapshot(... streak: currentStreak(statuses: ordered), ...)
        if let detail { select(detail.date) }
    } catch {
        assertionFailure("Today load failed: \(error)")
    }
}
```

Inside `load()`, in order:

1. Read `GoalTargets` from the store, which materialises defaults on first access.
2. Fetch four hundred days ending now, most-recent-first. Bounded on purpose: the grid needs this month and the streak needs recent history, never the whole table.

3. `evaluate` each row once into a date-keyed dictionary. Rebuilding that map inside the per-day closure would re-evaluate every row about thirty times per load.
4. Ask `MonthGridLayout` for the cells, mapping `DayStatus` to `DotState` here because Persistence must not depend on DesignSystem.
5. Compute `currentStreak` from the same ordered statuses.
6. Build the four week series from the rows already in hand: a second dictionary and seven lookups per metric rather than a second query. The rows stay inside this method; only plain numbers reach the snapshot.
7. Read today's agenda and the next seven days from `CalendarStore`, two windowed fetches.
8. Read the title of any workout scheduled for today from the bookmark store and the catalog cache (Chapter 10). Two reads rather than a relationship: the catalog is a cache the next refresh can drop a row from, so a schedule pointing at a vanished video simply shows nothing.
9. Publish one `TodaySnapshot`. A single assignment means a single render.
10. If the day sheet is open, re-derive it, so an external write, a WHOOP sync or a habit tick, updates the sheet as well as the grid behind it.

A read failure trips an `assertionFailure` and leaves the previous snapshot in place rather than blanking the screen. Nothing about a failed local fetch is recoverable by the user, so the last good state is the most useful thing to keep showing.

`select(date)` builds the day sheet from three stores: the metrics row for that day, the habit ticks on it, and the events overlapping it. A day with no metrics row is not an early return; its habits are still worth showing and the metric rows render as blanks. `toggleHabit` writes through `PlanStore` and then does nothing else: the save fires `ModelContext.didSave`, `RootView` reloads every view model, and `load()` re-derives the open sheet. That is the one refresh path; calling `select()` again here would repeat the 400-day pass a second time per tap.

METRICDETAILVIEWMODEL: ITS OWN OBJECT, ON PURPOSE

A view model of its own rather than more fields on `TodayViewModel`, because that one runs a 400-day pass on every save in the app, and hanging a year of bucketing off it would repeat that work for a screen nobody is looking at. `load(metric)` makes one fetch bounded by the range, then buckets in memory:

```
// LIfiOS/Features/Today/ViewModel/MetricDetailViewModel.swift
let buckets = Self.buckets(range: range, endingOn: now, calendar: calendar)
guard let start = buckets.first else { return }
let rows = try store.metrics(from: start, to: now)    // one fetch, not one per bucket

var readingsByBucket: [Date: [Double]] = [:]
for row in rows {
    guard let value = metric.value(in: row),
          let bucket = Self.bucket(for: row.date, range: range, calendar: calendar)
    else { continue }
    readingsByBucket[bucket, default: []].append(value)
}
let bars = buckets.map { bucket in
    MetricBar(date: bucket,
              label: Self.label(for: bucket, range: range, calendar: calendar),
              value: readingsByBucket[bucket].map { $0.reduce(0, +) / Double($0.count) })
}
```

Every bucket is present whether or not it has a reading, for the same reason `TrendSeries.days` keeps gaps: compacting the buckets that do have readings draws a chart whose neighbouring bars are not a bucket apart, which is a quieter lie than a false zero but a lie. A day has at most one row, so the mean is that row's own value in the two short windows; it only averages anything in the year view, where a bucket is a month. The month view labels one bar in five, because thirty labelled bars on a phone is a grey smudge. The headline figure is the most recent actual reading in the window, which is usually but not always today: a night's sleep or a recovery score can be a day behind, and the year view's monthly aggregate must never replace it.

MONTHVIEWMODEL: ONE FETCH PER MONTH

The month screen fetches the visible month padded by a week either side, then groups with `CalendarDayIndex.group` so a cell asks a dictionary rather than the store. A month is up to 42 cells and every one wants a count; one fetch, not one per cell. The padding is what makes the leading and trailing cells answerable, since the first row of a month usually starts in the previous one. Stepping remembers the day number the person last picked separately from the selection, so walking from 31 March to February and back returns to the 31st rather than the 28th; `MonthGridLayout.stepping` clamps only for display.

6.5 View

```
// LIfiOS/Features/Today/View/TodayScreen.swift
struct TodayScreen: View {
    let snapshot: TodaySnapshot
    let onSelectDay: (Date) -> Void
    let onConnectCalendar: () -> Void
    let onAddEvent: () -> Void
    let onTapEvent: (CalendarEventSnapshot) -> Void
    let onOpenToday: () -> Void
    var onConnectHealth: () -> Void = {}
    var isHealthConnected = false
    var onSelectMetric: (TodayMetric) -> Void = { _ in }
    ...
}
```

The screen takes a value and a set of intents and owns nothing: no view model, no store, no context. Every tap names what was tapped and stops there. What a day opens, what a metric opens, whether the permission sheet fires, are all the shell's decisions, which is what keeps this a pure function of its inputs and a preview a single literal.

ONE COLUMN OR TWO

On a phone the order is month, scheduled workout, agenda, streak, health prompt, stat grid, with `layout.statColumns` deciding whether tiles are one or two across. On a wide pane the month, workout, agenda and streak stand in a left column capped at 520 points, and the stats stand beside them two across. The cap is the point, not merely a column count: a dot grid divides whatever width it is given by seven, so a full-width month on a 1300 point pane draws 170 point dots and shoves every stat below the fold, the opposite of what more room should buy. There is no `maxContentWidth` on this screen, because both arrangements are grids and the cap exists for prose.

THE PIECES

- `MonthCalendarView` composes `DateHeadline` (the day numeral at masthead size), `WeekdayHeader` (narrow symbols rotated to the calendar's first weekday, today's column accented) and `DotGrid`. Cells arrive already computed; deriving them in a body would re-run the calendar maths on every render.
- `AgendaCard` always renders, because it carries the three permission states. Not determined shows "Connect calendar", denied shows "Open Settings", authorised shows up to four rows grouped by `DayPart` (all day, morning, afternoon, evening, each with its own icon and pastel) and a "+N more" that opens the day sheet. Upcoming is a second card, shown only with access and something to show, five rows grouped by day label. Rows are capped before grouping so the card's height budget stays the same whatever shape the day has.
- `TrendStatTile` is the tile with the week: label, pastel icon bubble, the numeral, and a `RoundedBarChart` of seven bars at 74 points on a phone and 88 on a wide pane. A week with no readings draws the plain figure and still holds the chart's height, so a dataless tile stands as tall as its siblings. Every tile is a plain button into `MetricDetailScreen`, built from `TodayMetric.allCases` rather than written out four times.

- **The health prompt** appears only when there is nothing to show and nothing attached. A connected strap that has not synced yet is a different situation, and telling that person to connect something would be wrong.
- **The scheduled workout card** is informational: the library is where a workout is played, and a card that opened a video would put the player two taps from a calendar grid.

THE SHEETS

`DayDetailSheet` renders past days as history with static marks and gives only today controls, so the grid can never be rewritten after the fact. A not-done habit on a past day is a dash rather than a cross, because the list may include habits that did not exist that day and a cross would read as a verdict the sheet has no standing to make. A missing metric renders as a dash rather than hiding the row, so the sheet's height does not jump between days.

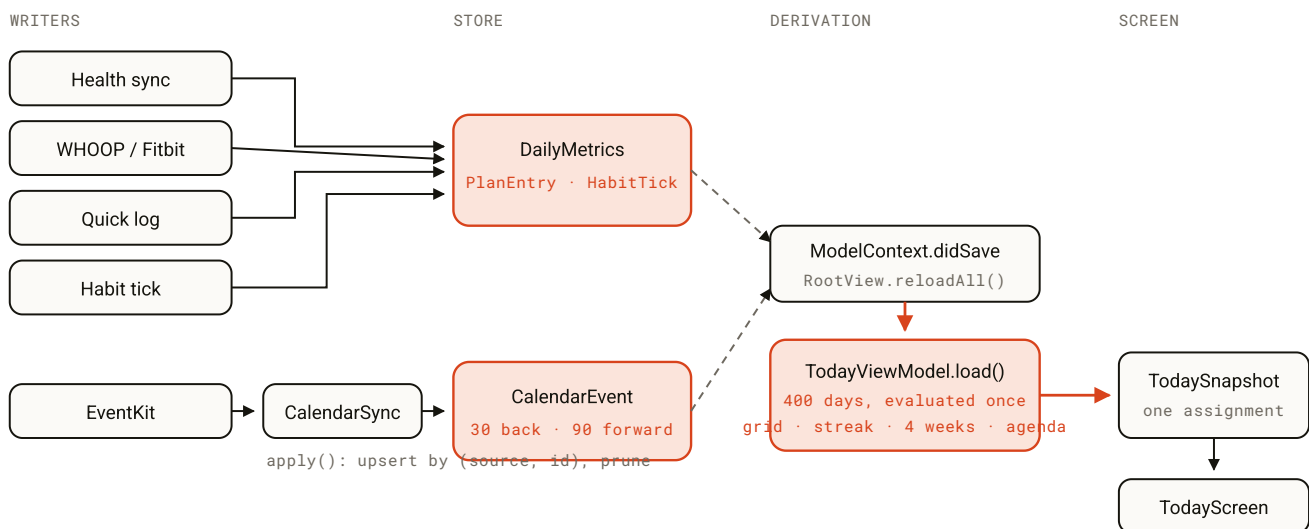
`EventSheet` is a pure function of its `EventSheetPresentation`, create or edit. Every write goes out through `onSave` and `onDelete`, which is what keeps it previewable without `EventKit`. A recurring occurrence renders read-only, fields disabled, Save and Delete hidden, with "This repeats. Edit the series in your calendar app." Editing one occurrence needs series semantics the app does not implement, and pointing at the calendar app is honest where silently mutating the occurrence would not be. The end date is clamped on save rather than blocked: a timed event floors at fifteen minutes after its start, an all-day event only at its start, since end equals start is the normal single-day shape. A new event defaults to the next full hour today, or 9am on any other day, because "next Tuesday at 16:41" is nobody's intent.

`MonthScreen` carries a swipe gesture on the grid to step months, chevrons for anyone who would rather aim than swipe, a date jump sheet, and a "Today" button disabled while today is selected. On a wide enough pane `ViewThatFits` puts the 400 point calendar panel and the agenda side by side. It never writes: create and edit leave through the caller, which owns `CalendarSync`, and it re-reads on `ModelContext.didSave` so a write from the sheet it opened lands in its dots.

THE QUICK LOG AND THE JOURNAL

`QuickLogSheet` logs water and weight, the two metrics with no automatic source, and is now reached from inside Begin Activity rather than from a bar button. Water accumulates: "add 250 ml" is the intent, not "today's total is 250 ml". `JournalEntrySheet` is a bare text editor with no title and no fields, because asking for either is friction on the one thing that has to stay effortless.

6.6 How it works



Nothing polls. Every writer saves through the store, the save notification reloads every view model, and the hub's view model publishes one value. The calendar cache is filled by an ambient sync on scene activation and after every write, and pruned only for sources that actually answered.

THE CALENDAR PIPELINE

1. `RootView.syncCalendar()` runs on the launch task, on every scene activation, and after any calendar write. It does nothing unless `CalendarAccessState.current` is `.authorized`, which requires both the system permission and the per-account opt-in flag: a device permission granted for one Almanac account does not authorise another.
2. `CalendarSync.sync()` asks each source whether it is authorised, fetches the window from those that are, and skips any that throw. Partial data beats an empty screen, and errors are not surfaced because sync is ambient, never a user-visible request.
3. `CalendarMerge.merge` dedups across sources on lowercased title plus start date, EventKit winning because that copy is writable offline and is what the OS shows elsewhere. Google is the second source the protocol is shaped for; today only EventKit is wired.
4. `CalendarStore.apply` reconciles the fetch into the cache by natural key: an existing row is updated in place and keeps its local id, a new one adopts the snapshot's, and in-window rows absent from the fetch are deleted only when their source was authoritative this pass. Rows outside the window are never touched.
5. The save fires `didSave` and the hub's agenda repaints.

Writes go the other way. `EventSheet` hands a draft to `RootView`, which calls `calendarSync.create`, `update` or `delete`. Sync routes the write to the source that owns the event, and then runs a full `sync()` so the cache reflects what the provider actually stored. Providers normalise all-day handling and time zones, so trusting the local draft would drift.

KEEPING TODAY'S NUMBERS MOVING

A step count climbs until midnight, and a sync at breakfast is stale by lunch. Two mechanisms keep the tiles honest while the app is open. `HealthSyncWindow.days(since:)` decides which days a sync re-reads: always today, plus the day of the last sync and every day since, because a night's sleep is recorded against the morning it ends and the day already covered can still gain a value. A first sync backfills thirty days, the month the dot grid shows, and no further, because Health keeps the data regardless and a longer backfill can be added the day something needs it. A last-sync stamp from the future is treated as a clock change and syncs today alone.

Then `HealthConnectionViewModel.startWatching()` runs an `HKObserverQuery` per metric for as long as the app is in front. Each change re-runs the sync, then sleeps fifteen seconds; the stream buffers one event, so a day's steps arriving in dozens of batches becomes one re-read per quarter minute rather than one per batch. The observers are torn down when the app leaves the foreground, so a phone in a pocket is not running ten queries a quarter minute. The sync writes through `MetricsStore`, the save reloads the hub, and the numeral moves. Nothing in the watcher knows about a screen.

6.7 Performance and cost

- **Bounded fetches.** The hub reads 400 days; the metric page reads only its range; the month screen reads one month plus a week either side; the agenda reads one day and the next seven. Nothing reads the whole table.
- **Evaluate once, look up many times.** Status and rows are keyed by day in two dictionaries with reserved capacity; the grid, the streak and the four week series are all lookups against them.
- **Bucket in memory, not in the store.** The year view is one fetch and a dictionary, not twelve round trips.
- **Reload only what is on screen.** `reloadAll()` reloads the metric page only while it is pushed, and the money detail only while it is open.
- **Event-driven, never polled.** The single `didSave` subscription in `RootView` is the whole refresh mechanism; the Health observer stream is throttled to one re-read per fifteen seconds and stopped in the background.
- **One assignment, one render.** The snapshot is built whole and assigned once; no intermediate state is published mid-load.
- **The chart maths is pure and static.** `RoundedBarChart.fraction` and `MonthGridLayout.cells` take values and return values, so they are tested from the terminal and never recomputed inside a body.
- **The cache means the calendar works on a plane.** Reads never touch EventKit directly; a sync that fails leaves the previous rows in place.
- **Nothing leaves the device.** Calendar contents and the day sheet stay local. What an escalated assistant turn sends is described in Chapter 15 and Chapter 24.

6.8 Edge cases and tests

CONDITION	BEHAVIOUR
Month boundaries	Leading blanks from <code>firstWeekday</code> , padded to whole rows; only padding cells lack a date
Future days in the current month	Rendered <code>.future</code> by position, never evaluated against goals
Today with partial data	Rendered <code>.today</code> by position, so a half-logged day never shows as a miss
No data for a day	Faint dot; skipped by the streak; leading no-data days do not break it
Goal changed in Settings	Next <code>load()</code> re-evaluates all history; dots change
No health source connected and no data	The tiles show dashes and a "Connect Apple Health" card appears; a connected strap that has not synced suppresses the card
Metric with no reading in the week	Tile shows the figure or a dash and holds the chart's height blank
Weight week that is flat	Half-height bars: every reading is both the high and the low, and half reads as steady
Latest sleep or recovery is yesterday's	The detail headline shows the latest actual reading and its date, never today's blank
Calendar never asked	Agenda offers "Connect calendar"; the prompt fires on that tap only
Calendar denied	Agenda offers "Open Settings"
Calendar granted on the device, but for another account	Not authorised for this account until it opts in; the prompt is not shown again, the flag is
Event deleted in the Calendar app	Gone from the cache on the next sync, because EventKit was authoritative for its window
A source throws during sync	Its previously synced rows are kept; it is not treated as having reported zero
Same event in EventKit and Google	One row, the EventKit copy
Recurring occurrence tapped	Read-only sheet pointing at the calendar app
End dragged before start	Clamped on save: 15 minutes for a timed event, start itself for all-day
Multi-day or overnight event	Appears on every overlapping day in the month grid, excluding a midnight end, across daylight-saving changes
Habit tick on a past day	Impossible: past days render without controls, and <code>toggleHabit</code> guards on <code>isToday</code> as the second lock
Last Health sync stamp in the future	Read as a clock change; today alone is re-read
Read failure in any view model	Previous snapshot kept; assertion in debug

TESTS

- `GoalEvaluationTests` and `StreakTests`: three of four is on target, two is a miss, exactly at goal counts, partial data is judged not excused, no-data days are skipped without breaking the streak.

- `MonthGridLayoutTests` (eleven tests): August 2026 starts with five blanks, a Sunday-first month takes six, today overrides its status, days after today are future, stepping keeps the day number, clamps to a short month, reaches a leap 29th, and crosses a year boundary backwards.
- `TrendSeriesTests` and `RoundedBarChartTests` : the average ignores gaps, the latest reading is not the last slot, a lone reading has no delta, points keep their gaps, the window minimum baseline spreads a metric that never nears zero, a flat week draws half bars.
- `CalendarStoreTests` : applying the same fetch twice does not duplicate, an existing row keeps its local id, in-window rows absent from a fetch are deleted, out-of-window rows survive, pruning only touches authoritative sources.
- `CalendarSyncTests` : a throwing source does not block the others and keeps its previously synced events, an unauthorised source is skipped, create dispatches to the first authorised source and resyncs, update routes to the owner.
- `CalendarMergeTests` , `CalendarFreeSlotTests` , `CalendarDayIndexTests` : the dedup rule, gap subtraction with all-day events not blocking time, and overnight and daylight-saving spans.
- `HealthSyncWindowTests` (eight tests): a first sync backfills a month, today is always included, a recent sync only re-reads today, a gap is backfilled including the day before it, a future stamp still syncs today.

THE ONE TEST THAT MATTERS MOST

For every screen, an empty store must produce a snapshot whose readings are all nil. The hub, the metric page and the day sheet all obey it by construction: each optional is filled from a row that may not exist, and there is no default of zero anywhere in the path.

6.9 Study notes

READ IN THIS ORDER

1. `Features/Today/ViewModel/TodayViewModel.swift` : `load()` top to bottom.
2. `LifeOSKit/Sources/DesignSystem/MonthGridLayout.swift` : the padding and the today/future rule.
3. `Features/Today/Model/TodayMetric.swift` : why one enum describes both the tile and the page.
4. `Features/Today/ViewModel/MetricDetailViewModel.swift` : the bucketing.
5. `LifeOSKit/Sources/Persistence/CalendarStore.swift` : `apply()` and `freeSlots()` .
6. `LifeOSKit/Sources/Integrations/CalendarSync.swift` and `EventKitSource.swift` : the recurring-id qualification.
7. `LifeOSKit/Sources/Integrations/HealthSyncWindow.swift` , then `HealthConnectionViewModel.startWatching()` .
8. `Features/Today/View/AgendaCard.swift` : `DayPart` and the row caps.

CHECK YOURSELF

- **Why is the status map built before the grid closure and not inside it?** The closure runs once per cell; building the map inside it would evaluate every row about thirty times per load.

- **Why do the week series reuse the 400-day rows?** The rows are already in hand for the streak, so the week costs a dictionary and seven lookups rather than a second query.
- **What happens to a cached event whose source threw during sync?** Nothing. Pruning only applies to sources that answered this pass, so the row survives.
- **Why is a recurring event's source id qualified with its start?** EventKit shares one identifier across a series; qualifying it makes each occurrence a distinct cache row and makes accidental writes fail loudly.
- **Why does the metric page have its own view model?** Because the hub reloads on every save in the app, and a year of bucketing must not ride along for a screen nobody is looking at.
- **Why does a Health sync re-read the day of the last sync as well as today?** Sleep is attributed to the morning it ends, so a day already read can gain a value afterwards.

7 Health

One tab for everything the body reports: how it is doing, and what it did. Two halves behind an underline picker, one week strip that moves both to the same day, and a rule that runs through every card on the screen: a missing reading is a dash, never a zero.

7.1 Intent

Revision D called this tab Body and split it four ways: Activity, Weight, Recovery, Wellness. The restructure of 25 August folded those four into two halves that answer two different questions. **Health** is how the body is doing: recovery, sleep, vitals against their own baselines, weight, and the journal. **Fitness** is what it did: calories burnt, steps, strain, the sessions themselves, and the training week. The tab bar has five slots and this one takes one of them, because Almanac is still not a fitness app; it is the place a life's signals meet, and the body is one of nine sectors (Chapter 13).

Open Health and the day is already chosen for you: today, with a week strip to step back. Four tracking tiles across the top show steps, sleep, weight and recovery for that day, each a door to its own history. Below them, Health or Fitness. Health opens with anything that is off, then recovery and calories, a sleep card with the night's stages, HRV and resting heart rate, blood oxygen and skin temperature read against your own fortnight, and every other reading Apple Health had for the day, grouped and listed only where a value exists. Fitness opens with the week's burn as bars, then today's movement, strain and heart rates, the day's workouts, and a card that says how the training week is adding up with a row into the workout library.

Why does the whole tab move to one selected day?

Because both halves are readings of the same day. If Health kept Tuesday and Fitness kept Thursday, a reader could compare Tuesday's recovery with Thursday's session and draw a conclusion from a comparison that was never real. `RootView` owns one date and fans it out to all four view models at once.

Both my iPhone and Watch count steps. Does that double?

No. Steps arrive through `HKStatisticsQuery` with a cumulative-sum option, which de-duplicates overlapping samples across sources. Summing raw samples client-side would roughly double every step count on a day you carried both devices.

What does "everything Apple Health holds" mean in practice?

Forty-seven metrics in a vocabulary enum, read every day the sync touches. Ten of them have named columns on the day row because scorers and charts read them by name; the other thirty-seven live in a JSON bag on the same row. The screen lists whichever groups actually have a value, so a person with a blood pressure cuff sees blood pressure and a person without never learns the row existed.

Where does weight come from?

The Renpho scale, via Apple Health. Renpho already syncs there, so Almanac never talks to Renpho directly: one less integration, one less credential to store.

WHAT THE TAB DELIBERATELY DOES NOT DO

- It never diagnoses. An anomaly banner says "Resting HR is well above your 2-week baseline" and stops. "See a doctor" is a claim this data cannot support.
- It never invents a verdict from missing data. No sleep all week means no sleep verdict, not "Low".
- It never colours strain with the recovery bands. High recovery is good and green; high strain is hard work, and green would read as praise. Strain gets one plain accent.

7.2 Files and folders

Features/Health/	
└─ Model/	
└─ HealthSection.swift	health · fitness, each with its hue
└─ View/	
└─ HealthHubScreen.swift	the hub: title, date, week strip, tiles, picker, switch
└─ HealthSegmentView.swift	the Health half
└─ FitnessSegmentView.swift	the Fitness half, plus WorkoutRow
└─ SleepDashboardCard.swift	night duration, stages, quality rows
└─ HealthReadingCard.swift	one reading on a pastel card
└─ StatGroup.swift	a featured reading and a grid of supporting values
└─ HealthActivityDesignPreview.swift	fixture pages: health, activity, library, player, ch
└─ CalendarHealthDesignPreview.swift	fixture for the calendar and health captures
Features/Body/	the Weight card and the wellness rollup
└─ Model/BodySnapshot.swift	weight, weekly delta, 14 points
└─ Model/WellnessSnapshot.swift	week rollup, verdicts, journal, Health groups
└─ View/BodyScreen.swift	WeightSection and its bar chart
└─ ViewModel/BodyViewModel.swift	sixty days to find one delta
└─ ViewModel/WellnessViewModel.swift	week averages, journal writes, the Health groups
Features/Activity/	the day's movement and the week of calories
└─ Model/ActivitySnapshot.swift	steps, minutes, energy, workouts, weekCalories
└─ ViewModel/ActivityViewModel.swift	seven rows, one day picked out
Features/Recovery/	recovery, sleep, vitals, the fortnight
└─ Model/RecoverySnapshot.swift	the day plus eight TrendSeries and the nights
└─ ViewModel/RecoveryViewModel.swift	one fetch, trends, anomalies
└─ View/WhoopDetailScreen.swift	the 14-day screen
└─ View/TrendChart.swift	one metric as a line, gaps kept
Features/Settings/ViewModel/	
└─ HealthConnectionViewModel.swift	the permission prompt, the sync, the foreground watch
LifeOSKit/Sources/Integrations/	
└─ HealthKitReader.swift	the actor that talks to HKHealthStore
└─ HealthMetric.swift	the 47-metric vocabulary and the merge rules
└─ HealthDaySummary.swift	a day's readings, grouped for display
└─ HealthApply.swift	HealthMetric meets DailyMetrics columns
└─ HealthSyncWindow.swift	which days a sync re-reads
└─ MetricArbiter.swift	the merge rule, pure
└─ MetricSource.swift	who wrote a value, and the ranking
LifeOSKit/Sources/Persistence/	
└─ DailyMetrics.swift	the day row, with the extras and sources bags
└─ AnomalyEvaluation.swift	baseline and threshold maths
└─ MetricsStore.swift	metrics(from:to:), sleepRecords, workouts
LifeOSKit/Sources/DesignSystem/	
└─ TrendSeries.swift	points with gaps, average, latest, delta
└─ SleepComposition.swift	a night's stages and their fixed order
└─ WeekStrip.swift	the seven-day selector
└─ RecoveryBand.swift	green, yellow, red from a percentage

FOUR VIEW MODELS, ONE SCREEN

Activity, Body, Recovery and Wellness keep their own folders even though none of them owns a tab any more. A section is a view concern; what each one derives is not. Merging them into one view model would produce a class that loads a day, a week, a fortnight and sixty days for four unrelated readings, and would reload all of it whenever any part changed. Four small loaders means a date change costs exactly four fetches and nothing else.

7.3 Model

Every snapshot is a value type with optional fields. That is the rule the whole app is built on, enforced by the types rather than by discipline at each call site. A day with the watch on the charger produces nils, and every nil renders as a dash.

```
// LifeOS/Features/Recovery/Model/RecoverySnapshot.swift
struct RecoverySnapshot: Equatable {
    // The day
    var recoveryPct: Double?
    var hrvMs: Double?
    var restingHR: Double?
    var dayStrain: Double?

    // Sleep quality
    var sleepMinutes: Int?
    var sleepPerformancePct: Double?
    var sleepEfficiencyPct: Double?
    var sleepConsistencyPct: Double?
    var sleepDebtMinutes: Int?

    // Vitals, each read against its own baseline
    var spo2Percentage: Double?
    var skinTempCelsius: Double?
    var respiratoryRate: Double?

    // The fortnight
    var recoveryTrend = TrendSeries(points: [])
    var hrvTrend = TrendSeries(points: [])
    var spo2Trend = TrendSeries(points: [])
    // ... five more series, the nights, naps, anomalies, weekRecovery

    var hasAnyReading: Bool {
        recoveryPct != nil || sleepMinutes != nil || dayStrain != nil || hrvMs != nil
    }
}
```

The fortnight rides inside the day's snapshot because half of these readings mean nothing alone. An SpO2 of 97.2 tells a reader nothing; a tenth above their own baseline does. So every vital carries a `TrendSeries`, and the card renders the delta from that series' average rather than the bare number.

`hasAnyReading` is how the screens tell "not connected" apart from "connected and quiet". Without it the Connect Whoop card would show to someone whose strap simply had a slow night.

TRENDSERIES KEEPS ITS GAPS

```
// LifeOSKit/Sources/DesignSystem/TrendSeries.swift
public struct TrendPoint: Sendable, Equatable, Identifiable {
    public let date: Date
    public let value: Double? // nil is a day with no reading
}

public struct TrendSeries: Sendable, Equatable {
    public let points: [TrendPoint]
    public var hasAnyReading: Bool { points.contains { $0.value != nil } }
    public var average: Double? { // mean of the readings that exist
        guard !readings.isEmpty else { return nil }
        return readings.reduce(0, +) / Double(readings.count)
    }
    public var latest: Double? { // most recent reading, not the last slot
        points.reversed().first { $0.value != nil }?.value ?? nil
    }
    public var deltaFromAverage: Double? {
        guard readings.count > 1, let latest, let average else { return nil }
        return latest - average
    }
}
```

Three decisions live here. The average ignores gaps rather than counting them as zero. The latest reading is the last non-nil point, because the window usually ends on a day Whoop has not scored yet. And a series with one reading has no delta: a lone value is its own average, and "0 from baseline" would imply a history that has not been collected.

THE DAY ROW AND ITS TWO BAGS

DailyMetrics keeps its ten named health columns from Revision D (steps, exercise minutes, sleep minutes, active energy, weight, water, resting HR, HRV, SpO2, respiratory rate) and the Whoop columns. What is new is a long tail of Apple Health metrics that have no column of their own, and a record of who wrote each value.

```
// LifeOSKit/Sources/Persistence/DailyMetrics.swift
/// Everything Apple Health can supply that does not have a column of its
/// own, as a JSON dictionary keyed by `HealthMetric.rawValue`.
public var extrasData: Data?

/// Who wrote each value, as a JSON dictionary keyed by `HealthMetric.rawValue`.
public var sourcesData: Data?

public func extra(_ key: String) -> Double? { extras[key] }
public func setExtra(_ key: String, _ value: Double?) {
    var bag = extras
    if let value { bag[key] = value } else { bag.removeValue(forKey: key) }
    extras = bag
}
```

Why a bag and not thirty-seven more columns: most people have no data for most of the tail, and a column each would mean a schema change per metric for fields nothing queries. The bag keeps adding a metric to one enum case and one mapping. The one thing a bag must not lose is the distinction this model is built on: a missing key means no reading, and writing `nil` removes the key rather than storing a zero. Both bags are optional so a store written before they existed opens without a migration, which matters after the lesson in 10.8.

`sourcesData` stores raw strings rather than a typed source because `Persistence` sits below `Integrations` and must not import it. The vocabulary lives up there.

THE VOCABULARY

`HealthMetric` is its own enum, deliberately not a set of HealthKit types, so the file builds and tests without the framework. It carries nine groups (movement, heart and fitness, body, sleep, vitals, intake, mind, walking, cycle), a title, a unit label, a decimal count, and three merge properties the arbiter reads:

```
// LifeOSKit/Sources/Integrations/HealthMetric.swift
/// Measured by a strap worn on the body, so a wearable outranks the phone.
public var claimedByWearable: Bool {
    switch self {
    case .restingHR, .hrvMs, .spo2Percentage, .respiratoryRate,
        .sleepMinutes, .deepSleepMinutes, .remSleepMinutes,
        .coreSleepMinutes, .awakeMinutes, .timeInBedMinutes,
        .sleepEfficiencyPercentage, .readinessScore:
        true
    default:
        false
    }
}

/// A person can type this into the app, so a background sync must not
/// replace what they entered on purpose.
public var acceptsManualEntry: Bool {
    switch self {
    case .weightKg, .waterML: true
    default: false
    }
}

/// Health is the top authority among syncing sources: nothing else measures
/// this and nobody types it, so the latest reading is simply the truth.
public var healthIsAuthoritative: Bool {
    !claimedByWearable && !acceptsManualEntry
}
```

Cycle metrics (basal temperature, menstrual flow) are their own group and are gated by `needsCycleConsent`. The first authorisation request never asks for them. Not because they are more private than a heart rate, but because a permission sheet that asks every person for menstrual data is telling most of them the app has misread who they are.

7.4 ViewModel

Four view models, each with the same shape: `attach(context)`, `select(date)`, `load()`. Each reads a different window and derives a different thing.

VIEW MODEL	WINDOW	DERIVES
<code>ActivityViewModel</code>	7 days	Today's steps, minutes, energy, resting HR; the day's workouts; seven <code>DayValue</code> s of calories, Whoop's figure first, Health's as fallback
<code>BodyViewModel</code>	60 days	The latest weigh-in, the weekly delta, 14 points for the chart
<code>RecoveryViewModel</code>	14 days	The day's readings, eight trend series, the nights, naps, five anomaly checks, the week strip's recovery rings
<code>WellnessViewModel</code>	7 days, plus the selected day	Average sleep and exercise, workout days, two verdicts, the journal, the Apple Health groups

RECOVERY: ONE FETCH, MANY READINGS

```
// LifeOS/Features/Recovery/ViewModel/RecoveryViewModel.swift
let end = selectedDate
let start = calendar.date(byAdding: .day, value: -(Self.windowDays - 1), to: end) ?? end

// One fetch for the whole window. The selected day is picked out of
// it rather than fetched again.
let rows = try store.metrics(from: start, to: end)
let day = rows.first { calendar.isDate($0.date, inSameDayAs: end) }

let restingTrend = trend(rows, from: start, to: end) { $0.restingHR }

// History excludes the selected day: a spike must not drag the
// baseline toward itself on the very day it is being judged.
func history(_ series: TrendSeries) -> [Double?] {
    series.points.dropLast().map(\.value)
}

let anomalies = [
    AnomalyEvaluation.evaluate(metric: "Resting HR", today: day?.restingHR,
        history: history(restingTrend), threshold: .relativeAbove(0.10)),
    AnomalyEvaluation.evaluate(metric: "Skin temp", today: day?.skinTempCelsius,
        history: history(skinTrend), threshold: .absoluteAbove(1.0)),
    AnomalyEvaluation.evaluate(metric: "Blood oxygen", today: day?.spo2Percentage,
        history: history(spo2TrendSeries), threshold: .absoluteBelow(3.0)),
    AnomalyEvaluation.evaluate(metric: "HRV", today: day?.hrvMs,
        history: history(hrvTrendSeries), threshold: .relativeBelow(0.30)),
    // ... respiratory rate at .relativeAbove(0.08)
].compactMap { $0 }
```

The window is fourteen days because the Whoop sync window is fourteen days, so a chart never has a gap the sync could have filled. The `trend` helper builds a point for every calendar day in the window, not only the days with rows, which is what keeps a day Whoop never reported as a visible gap. Thresholds are relative

where the metric scales with the person (heart rate, HRV) and absolute where it has a physiological unit (a degree of skin temperature means the same at any baseline). `AnomalyEvaluation` refuses to judge with fewer than seven readings in the history, because a noisy baseline raises false alarms.

BODY: SIXTY DAYS TO FIND ONE NUMBER

```
// LifeOS/Features/Body/ViewModel/BodyViewModel.swift
let start = calendar.date(byAdding: .day, value: -60, to: selectedDate) ?? .distantPast
let rows = try store.metrics(from: start, to: selectedDate) // most-recent-first

let weighed = rows.compactMap { row in row.weightKg.map { (date: row.date, kg: $0) } }
var delta: Double?
if let newest = weighed.first,
    let weekAgo = weighed.first(where: { $0.date <= newest.date.addingTimeInterval(-6 *
86_400) }) {
    delta = newest.kg - weekAgo.kg
}
```

The hero is the most recent row that actually has a weight, which is not necessarily the most recent row. The delta is that weight minus the most recent weigh-in at least six days older; if none exists the delta is suppressed rather than computed over a shorter window, because a "weekly change" measured over two days is a wrong number wearing a right label. Sixty days rather than fourteen because people skip weeks, and a fortnight would suppress the delta for anyone who steps on the scale fortnightly.

WELLNESS: THE JOURNAL IS A NOTE NOW

Revision D stored a journal entry as a `PlanEntry` of kind `.journal`. It is now a note page, one per day, in the notes store (Chapter 12). Writing twice in one evening appends to the day's page rather than filing a second entry, because two rows dated the same day would break the "did I write today" question the streak is built on.

```
// LifeOS/Features/Body/ViewModel/WellnessViewModel.swift
func addJournal(_ text: String) {
    let store = NotesStore(context: context, calendar: calendar)
    let entry = try store.journalEntry()
    var blocks = entry.blocks.filter { !($0.kind == .paragraph && $0.isEmpty) }
    blocks.append(contentsOf: NoteBlockParser.blocks(fromMarkdown: trimmed))
    try store.update(entry, blocks: blocks)
    load()
}
```

The same view model builds the Apple Health groups for the selected day. The week of rows above cannot answer "what did Health have for this day", so it fetches the one row separately and hands it to `HealthDaySummary.groups(for:includingCycle:)`. The cycle switch it passes is the same `UserDefaults` key the sync reads, so what is on screen and what is being read can never disagree.

THE CONNECTION VIEW MODEL

`HealthConnectionViewModel` is deliberately thinner than its Whoop counterpart: Health has no OAuth, no tokens and no server. What it does have is a permission model that refuses to tell the app anything, which shapes everything in it.

```
// LIfiOS/Features/Settings/ViewModel/HealthConnectionViewModel.swift
enum State: Equatable {
    case unavailable           // iPad without Health, or a device that has none
    case notAsked              // never asked; the row offers to connect
    case syncing
    case synced(days: Int)     // days touched, not days with data
    case noData                // asked, and nothing came back
    case failed(String)
}
```

`.noData` deliberately does not say "denied". HealthKit never reports whether read access was granted, because admitting a refusal would leak that the user has no data of that type. Claiming a denial the app cannot see would be a guess presented as a fact, so the copy says "No data yet. Check Almanac in Health > Sharing" and points at the one place that knows.

7.5 View

`HealthHubScreen` takes four value snapshots, two bindings, and a handful of closures. It owns no view model, which is what lets its preview be a single literal and every section be looked at in any state without a store.

```
// LIfiOS/Features/Health/View/HealthHubScreen.swift
struct HealthHubScreen: View {
    let activity: ActivitySnapshot
    let weight: BodySnapshot
    let recovery: RecoverySnapshot
    let wellness: WellnessSnapshot
    var onAddJournal: () -> Void = {}
    var onConnectWhoop: () -> Void = {}
    var isWhoopConnected = false
    var onSelectMetric: (TodayMetric) -> Void = { _ in }
    var library: WorkoutLibraryViewModel? // nil in previews
    var recorder: ActivityRecorder?       // travels with the library
    @Binding var section: HealthSection
    @Binding var selectedDate: Date
    @Environment(\.layout) private var layout
```

The layout, top to bottom: "Your health" as a large title with a date picker capped at today; `WeekStrip`; a `LazyVGrid` of four tracking tiles built from `TodayMetric.allCases`, each a button into `MetricDetailScreen` (Chapter 6); an `UnderlinePicker` over the two sections; and the section view. The content column reads `layout.maxContentWidth`, `layout.gutter` and `layout.railInset` from the environment, so on an iPad the same screen sits inside a bounded column beside the side rail without a second layout.

THE HEALTH HALF

Order matters here, and each card is a decision:

1. `AlertBanner` with the anomalies. First, because "is anything off" is the question a person opens the tab to answer.
2. A Connect Whoop card, shown only when there is no reading and Whoop is not connected.
3. Recovery (with its `RecoveryBand` label in caps) and calories, two `HealthReadingCard`s in an adaptive grid.
4. `SleepDashboardCard`: the night's duration at large-title size, a `SleepStageBar` when stages exist, then quality rows (performance, efficiency, consistency, debt, naps), each row present only when its value is.
5. HRV and resting HR; then blood oxygen and skin temperature, each captioned with its signed delta from the fortnight average.
6. A Vitals `StatGroup` for respiratory rate.
7. `appleHealthGroups`: one `StatGroup` per `HealthDaySummary.Group` that has a reading, coloured by category.
8. `WeightSection`, the journal card, a "14-day trends" link, and a "Synced 3h ago" line.

```
// LifeOS/Features/Health/View/HealthSegmentView.swift
/// Listed rather than designed: a panel appears for each group that has a
/// reading and the rest are simply absent, so a person with a blood
/// pressure cuff sees blood pressure and a person without never learns the
/// row existed.
@ViewBuilder
private var appleHealthGroups: some View {
    ForEach(wellness.healthGroups) { group in
        StatGroup(
            title: group.title,
            rows: group.items.map { StatGroup.Row(label: $0.title, value: $0.formatted) },
            hue: categoryStyle(group.title).0, icon: categoryStyle(group.title).1
        )
    }
}
```

`StatGroup` renders nothing at all when no row has a value; otherwise the first row with a value becomes the featured reading and the rest form an adaptive grid whose minimum cell width grows to 240 points at accessibility text sizes.

THE FITNESS HALF

Calories first, as seven `RoundedBarChart` bars for the week ending on the selected day with the total in the header. Then steps and active time; strain, average HR and max HR; the day's workouts as `WorkoutRow`s (only when there are any, so most days render no empty frame); the training card; and the trends link. The training card carries the way into the workout library with today's plan already on it (Chapter 10), but only when both the library view model and the recorder were handed down, because the player the library pushes records the session.

`WorkoutRow` shows a "partial" label when Whoop captured less than 95 percent of a session. A workout recorded at 40 percent is not an easy workout, and letting its low strain speak for it would say exactly that. It also shows the split chip when the session followed a catalog video, so a pull day in the library reads as a pull day in the list.

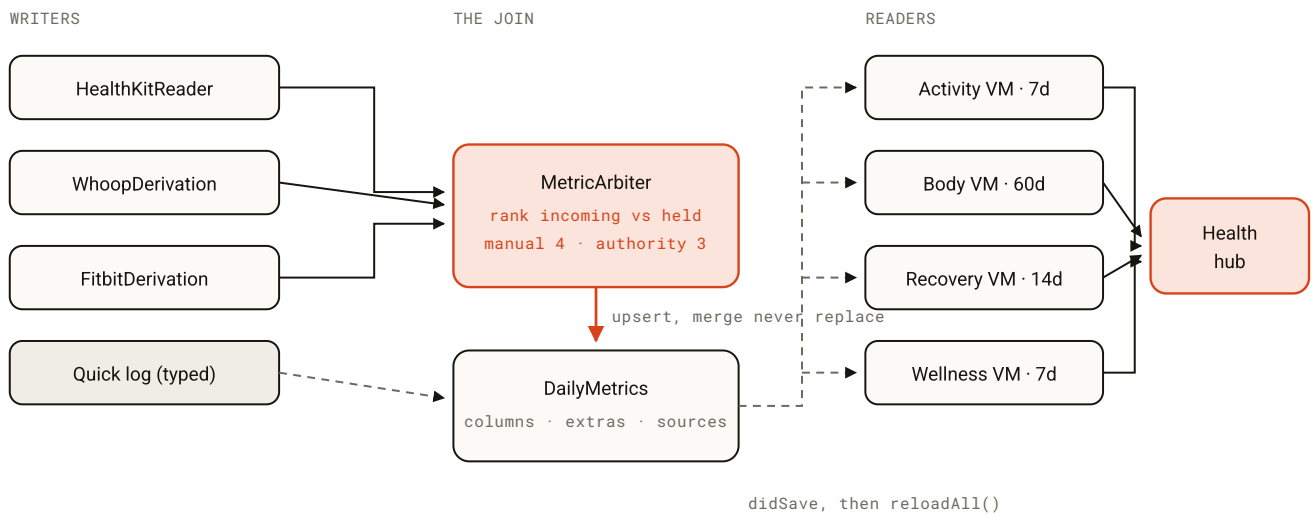
THE FORTNIGHT SCREEN

`WhoopDetailScreen` opens with `SleepCompositionChart`, one column per night, because it is the only chart showing structure rather than quantity: a short night and a fragmented night look identical on the duration chart and obviously different here, which is the reason the stages are ingested at all. Below it, five `TrendChart`s.

```
// LifeOS/Features/Recovery/View/TrendChart.swift
/// A run of consecutive days that all have readings.
///
/// Swift Charts joins a line straight across a day it was given no mark
/// for, which would draw a reading that never happened. Splitting the
/// window into runs and giving each its own series keeps the gap a gap.
private var segments: [Segment] {
    var runs: [Segment] = []
    var current: [TrendPoint] = []
    for point in series.points {
        if point.value != nil {
            current.append(point)
        } else if !current.isEmpty {
            runs.append(Segment(id: runs.count, points: current))
            current = []
        }
    }
    if !current.isEmpty { runs.append(Segment(id: runs.count, points: current)) }
    return runs
}
```

The chart is a line, not bars, because the shape of a fortnight is a direction and a line draws a direction. The chrome is almost absent: no gridlines, no y axis, two date labels in the footer. The y domain is the data's own range with a quarter of headroom, never anchored at zero, because a resting heart rate plotted from zero is a flat line with no shape in it. The current figure lives in the header in text ink, because the warmer module hues fall under 3:1 contrast on this surface and a visible label is the relief.

7.6 How it works



Three syncing sources and a typed value all pass through one pure arbiter before they touch the day row. Readers never see a source; they see a row and a snapshot.

STEP BY STEP: A FOREGROUND SYNC

1. `RootView` comes to the foreground and calls `health.startWatching()`. If permission has ever been asked, the view model opens `reader.changes()`, an `AsyncStream` fed by one `HKObserverQuery` per universal metric type.
2. Health records a batch of steps. The observer yields once; the stream buffers at most one event, so thirty batches landing together are one reason to re-read, not thirty.
3. The view model runs `sync()`. If a sync is already in flight it awaits that one rather than starting a second: a launch task and a foreground transition arriving together do one pass.
4. `HealthSyncWindow.days(since:)` decides which days to re-read: always today; the day of the last sync as well (a night's sleep is recorded against the morning it ends, so a day already covered can gain a value); thirty days on a first sync or after a gap that long; only today when the stamp is in the future, because that is a clock change, not a missing sync.
5. For each day, `reader.day(_ :includingCycle:)` runs one query per metric. Counts use `.cumulativeSum`; measurements use `.discreteAverage`, because summing a heart rate would produce a number in the thousands. Sleep is a category, not a quantity, so one `HKSampleQuery` answers all six sleep metrics by hand. A metric that errors is dropped rather than failing the day: one unreadable type must not cost the other forty-six.
6. A day Health knows nothing about is skipped rather than written. An upsert would create an empty row and put a blank day on the calendar that nothing ever fills.
7. `HealthApply.write` upserts the row. For each incoming value it reads the held value and the held source, asks `MetricArbiter.resolve`, writes the winner into the named column or the bag, and records the winning source.
8. `MetricsStore` saves. `ModelContext.didSave` fires. `RootView.reloadAll()` reloads every snapshot. The tiles repaint through the path they already used; nothing here knows about a screen.

9. The watch sleeps fifteen seconds before listening again.

THE ARBITER

```
// LifeOSKit/Sources/Integrations/MetricArbiter.swift
public static func resolve(
    metric: HealthMetric,
    existing: Double?, existingSource: MetricSource?,
    incoming: Double?, incomingSource: MetricSource,
    ranking: SourceRanking
) -> (value: Double?, source: MetricSource?) {
    // A source having no reading is not evidence of zero.
    guard let incoming else { return (existing, existingSource) }
    guard existing != nil else { return (incoming, incomingSource) }

    let held = existingSource ?? MetricSource.legacy(for: metric)

    // Greater than or equal, not greater than. Equal rank means the same
    // source syncing again, which must be allowed to correct itself.
    guard ranking.rank(incomingSource, for: metric) >= ranking.rank(held, for: metric) else {
        return (existing, held)
    }
    return (incoming, incomingSource)
}
```

Revision D merged with a boolean: fill gaps only. That could not tell a Whoop value from a typed one, which is exactly the distinction a second wearable forces. `SourceRanking` turns a source into a rank per metric: 4 for a typed value, 3 for the authority (Apple Health for passively counted metrics, the primary strap for wearable-claimed ones), 2 for a secondary reading, 1 for a fallback, 0 for unknown. The primary wearable defaults to Whoop, so an unset preference reproduces the old behaviour exactly.

One legacy rule matters: a value written before provenance existed reads as rank 0 for almost everything, so the next sync re-attributes it. Weight and water are the exception. A figure the user typed before this shipped is unattributed too, and treating it as unknown would let the next Health sync silently replace it. That is the one path that destroys data a person entered by hand, so an unattributed value for a typed metric is read as typed.

7.7 Performance and cost

- **One fetch per view model per date change.** Recovery fetches fourteen rows once and picks the selected day out of them. Activity fetches seven. Nothing fetches the same window twice.
- **Observer, not timer.** A step count changes dozens of times an hour and not at all overnight. A polling interval short enough to look live wastes queries; one cheap enough to run is not live. `HKObserverQuery` fires when Health already knows something arrived.
- **Buffer of one, sleep of fifteen.** The stream uses `.bufferingNewest(1)` and the consumer sleeps fifteen seconds after each pass. A day's steps arrive in dozens of batches, and re-reading forty-seven metrics for each one would spend the afternoon in HealthKit to move a numeral.

- **Foreground only.** `enableBackgroundDelivery` needs an entitlement and wakes the app to write rows nobody is looking at. The launch and foreground syncs already cover the time the app was away. `stopWatching()` runs on every scene deactivation so a phone in a pocket is not running ten queries a quarter minute.
- **Bounded backfill.** Thirty days is the month the dot grid shows. Going further would spend forty-seven queries a day on history no screen displays, and Health keeps the data regardless.
- **Every observer completes.** The handler calls `completion()` on the error path too. HealthKit retries a query whose handler never completed, and a silent leak of those is how an app ends up with a hundred live observers.
- **The actor.** `HealthKitReader` is an actor because `HKHealthStore` is a shared resource and the sync can be entered from two places at once. The observer array is actor state rather than a local because `HKObserverQuery` is not `Sendable`.
- **Nothing leaves the device.** Apple Health readings are written to the local store and never synced to Supabase. The only health figures that ever leave are the three a person chooses to share with friends (Chapter 16).
- **No chart draws from nothing.** `TrendChart` returns no view at all when a series has no reading, so a fresh install renders no empty axes.

A UNIT BUG WORTH REMEMBERING

`HKUnit.percent()` yields 0.98 for a 98 percent blood oxygen reading. The app stores percentages as percentages everywhere else, Whoop included, so a Health-sourced SpO2 was landing two orders of magnitude below a Whoop-sourced one and rendering as "1.0". It went unnoticed because Whoop owned the field and Health only filled its gaps. `HealthKitReader.scale(for:)` now multiplies every percent-unit metric by 100 on the way in.

7.8 Edge cases and tests

CONDITION	BEHAVIOUR
Watch on the charger all day	Every Activity reading nil; dashes, never zeros
Never weighed	Hero reads "No weigh-in", delta suppressed, chart bars all short and grey
Fewer than 7 days between weigh-ins	Delta suppressed entirely rather than shortened
Gap in the middle of the fortnight	Point kept as nil; the line breaks into two runs; the axis position is preserved
Whoop not connected and nothing read	Connect Whoop card shows; recovery, HRV and strain nil; sleep may still arrive from Health
Whoop connected but a quiet night	<code>hasAnyReading</code> is true, so no Connect card; the readings show dashes
Health permission refused	Indistinguishable from no data by Apple's design; state is <code>.noData</code> and the row points at Health > Sharing
Health sync while a sync is running	The second caller awaits the first; one pass
Last-sync stamp in the future	Read as a clock change; only today is re-read
A day Health has nothing for	Skipped, not upserted; no blank row on the calendar
Health and Whoop both report resting HR	Whoop wins (claimed by wearable, rank 3 vs 1); Health fills the gap on days Whoop missed
Health and the user both report weight	The typed value wins (rank 4); Health may still correct a weight it wrote itself
A pre-provenance typed weight	Read as typed via <code>MetricSource.legacy</code> ; never replaced by a sync
Cycle tracking turned off after data was read	<code>HealthDaySummary</code> hides the cycle group anyway; showing it would betray the switch
Fewer than 7 readings behind a vital	No baseline, no anomaly, no banner
Today's reading is the spike	History drops the selected day so a spike cannot drag its own baseline
Whoop captured 40 percent of a session	The row is labelled "partial" so the low strain is not read as an easy day
Extras bag bytes are unreadable	Decoded as empty; the named columns still render rather than losing the screen over the tail

TESTS

- `MetricArbiterTests` (10 tests): a missing reading never clears a value; Health does not overwrite Whoop; Health fills a Whoop gap; Health overwrites its own counts; a hand-entered value survives; the chosen strap overwrites the other; the secondary strap still fills a gap; an unattributed typed value is treated as typed.
- `HealthApplyTests` (9 tests) against a real in-memory store: every column fills; Whoop values survive a Health sync; a later sync corrects a step count; counts are stored as integers; a write records its

provenance; the long tail is attributed too. These exist because the arbiter can be perfectly correct and still be wired into the wrong column.

- `WhoopArbitrationTests` (5): Whoop owns what it measures, does not overwrite the phone's exercise minutes or a typed weight, yields to the chosen strap.
- `HealthSyncWindowTests` (8): a first sync backfills a month; today is always included; a recent sync re-reads only today; a gap is backfilled including the day before it; a very old sync is capped; the window never reaches the future.
- `HealthMetricVocabularyTests` and `CycleConsentTests` : every metric has a title and group; raw values are unique; the first request never asks for cycle data; universal and cycle partition every metric.
- `AnomalyEvaluationTests` (10): the baseline needs seven readings; relative and absolute thresholds flag at the boundary and stay quiet below it.
- `TrendSeriesTests` (13) and `SleepCompositionTests` (6): averages ignore gaps; a single reading has no delta; points keep their gaps; a night with no stages is not renderable; awake counts toward the bar but not toward sleep.

STILL NOT COVERED

The four feature view models have no tests, for the reason recorded in the decision log: they take a context through `attach` rather than a protocol through `init` . The delta suppression in `BodyViewModel` and the verdict thresholds in `WellnessViewModel` (420 sleep minutes reads "Optimal", 20 exercise minutes counts as a workout day, 4 workout days reads "Consistent") are the ones most worth writing first.

7.9 Study notes

READ IN THIS ORDER

1. `LifeOSKit/Sources/Integrations/HealthMetric.swift` , the vocabulary and the three merge properties.
2. `LifeOSKit/Sources/Integrations/MetricSource.swift` and `MetricArbiter.swift` , the ranking and the rule.
3. `LifeOSKit/Sources/Integrations/HealthApply.swift` , where the rule meets the row.
4. `LifeOSKit/Sources/Integrations/HealthKitReader.swift` , particularly `changes()` and `read(_ :on:)` .
5. `LifeOS/Features/Settings/ViewModel/HealthConnectionViewModel.swift` , the sync loop and the watch.
6. `LifeOS/Features/Recovery/ViewModel/RecoveryViewModel.swift` , the one-fetch pattern and the anomaly checks.
7. `LifeOS/Features/Health/View/HealthSegmentView.swift` , then `TrendChart.swift` .

CHECK YOURSELF

- **Why does the arbiter use greater-than-or-equal rather than greater-than?** Equal rank means the same source syncing again, and a source must be allowed to correct itself, which is how an afternoon sync fixes

the morning's step count.

- **Why is a Health-sourced SpO2 multiplied by 100?** `HKUnit.percent()` returns a fraction; the app stores percentages as percentages, and the mismatch rendered "1.0" for a 98 percent reading.
- **What does the observer stream's buffer size of one buy?** A batch of samples is one reason to re-read the day, not thirty, and a throttled consumer never returns to a queue of stale events.
- **Why does the recovery baseline exclude the selected day?** A spike on the day being judged would drag its own baseline toward itself and hide itself.
- **Why is a legacy unattributed weight treated as typed?** It is the one path where a sync could destroy a value a person entered by hand.
- **Why does the app never say Health access was denied?** HealthKit deliberately hides the answer, so any claim of denial would be a guess presented as fact.

8 Live sessions

Press Begin and a workout is recorded on the phone: a wall-clock timer that survives relaunch, a heart-rate strap over Bluetooth, an effort estimate against today's battery, and one readout struct that every surface renders, from the glass rings on the screen to the lock screen and the Dynamic Island.

8.1 Intent

Until 14 September a person starting a workout saw a timer, and if they scanned and tapped for a sensor, a heart rate. They could not see how hard they were working, how much they had left, or whether they should push. Everything needed to answer those questions already reached the app: Whoop recovery and sleep performance sit in `DailyMetrics`, the profile carries a birth date, the workout builder collects energy, and a strap delivers beats every second. Nothing joined them. This slice is the join.

WORKING BACKWARDS

Tap the plus, choose Run, tap Begin. The screen turns blue. A big rounded clock counts up, and beneath it three glass rings: your heart rate against today's ceiling zone, your estimated effort against today's target band, and your battery, what you have left to spend. The last sensor you used reconnects on its own. Lock the phone and the same numbers sit on the lock screen over the same blue gradient, and in the Dynamic Island as a heart, a number, and the clock. Pause and everything freezes. Finish and the session is saved to Almanac, and to Apple Health if you asked.

Is the effort number Whoop's strain?

No, and the screen always says "estimated" next to it. It is a model on the same 0 to 21 scale, calibrated so a steady hour in zone 3 reads about 12 and a hard ninety minutes about 18. Whoop's real strain for the workout arrives later on its own record through the existing sync, and nothing here overwrites it.

What if I have not entered a birth date?

Then there are no zones, no effort and no battery, and the screen says so with one line: "Add your birth date in Profile for zones, effort and battery." A zone built on an assumed age would be a reading the person never gave, and this app refuses those. Heart rate and calories still show.

Does the timer keep going if the app is killed?

Yes. The timer is wall-clock based and written to a per-account draft at most every ten seconds, so a relaunch restores it exactly. If Health was recording, the recorder asks HealthKit to recover the active session; if it cannot, the timer is kept and the person is told the Health session is gone.

OUT OF SCOPE HERE

Apple Watch heart rate and rep counting are Chapter 9. Following a video with the rings over it is Chapter 10. There are no server changes; this chapter is entirely on the device.

8.2 Files and folders

Features/Activity/	
└─ Model/	
└─ ActivitySnapshot.swift	the Fitness half's day (Chapter 7); not the live ses
└─ View/	
└─ BeginActivityScreen.swift	the sheet: picker, timer card, live hero, tiles, sen
└─ ActivityControls.swift	begin · pause · finish · discard · done
└─ SessionRings.swift	the glass HUD: rings, bar, rep controls
└─ ViewModel/	
└─ ActivityRecorder.swift	the session: HealthKit, draft, readout, save
└─ ActivityRecorder+Watch.swift	the watch half (Chapter 9)
└─ ActivityRecorderChecks.swift	34 in-simulator checks, DEBUG only
└─ LiveHeartRateSensor.swift	CoreBluetooth: scan, pair, remember, reconnect
App/Surfaces/	
└─ WorkoutLiveActivityController.swift	requests, throttles and ends the Live Activity
AlmanacWidgets/	
└─ WorkoutLiveActivity.swift	lock screen and Dynamic Island views
LifeOSKit/Sources/Integrations/	
└─ LiveEffort.swift	zones, effort, capacity, ceiling, battery, push, bui
└─ CapacityInputs.swift	nine days of rows into CapacityMath
└─ ActivitySessionState.swift	the wall-clock timer, SessionSource, the BLE frame d
└─ WhoopAutoPair.swift	"exactly one WHOOP nearby"
LifeOSKit/Sources/AppSurfaces/	
└─ LiveSessionReadout.swift	Foundation only, so the widget can link it
└─ LiveSessionReadout+Text.swift	the one struct every surface renders; the throttle
└─ SessionRingMath.swift	the one place a reading becomes text
└─ WorkoutActivityAttributes.swift	ring fills as fractions of today's ceiling
	ContentState = LiveSessionReadout

WHY THE MATHS IS IN TWO PACKAGES

`LiveSessionReadout` and `PushState` live in `AppSurfaces`, which depends on nothing, so the widget extension can decode them without linking HealthKit or SwiftData. The effort model lives in `Integrations`, which added `AppSurfaces` as a dependency. Nothing cycles, and `swift test` runs the whole model on macOS without a simulator.

8.3 Model

THE TIMER

```
// LifeOSKit/Sources/Integrations/ActivitySessionState.swift
/// Wall-clock based: backgrounding never stops the timer and pauses never count.
public struct ActivitySessionState: Codable, Equatable, Sendable {
    public enum Phase: String, Codable, Sendable { case running, paused, finished }
    public let id: UUID
    public let startedAt: Date
    public let activity: String
    public private(set) var phase: Phase
    public private(set) var accumulated: TimeInterval
    public private(set) var runningSince: Date?
    public private(set) var endedAt: Date?

    public func elapsed(at date: Date = .now) -> TimeInterval {
        accumulated + (runningSince.map { max(0, date.timeIntervalSince($0)) } ?? 0)
    }
    public mutating func pause(at date: Date = .now) {
        guard phase == .running else { return }
        accumulated = elapsed(at: date); runningSince = nil; phase = .paused
    }
}
```

A pause banks the time so far into `accumulated` and clears `runningSince`; a resume starts a new run. There is no tick. Every surface derives the elapsed time from two fields and the clock, which is what lets the lock screen use `Text(timerInterval:)` and count on its own without an update per second. The `id` is the Live Activity's key.

ZONES, EFFORT, CAPACITY

All of this is value types and pure functions in one file. Nothing touches HealthKit, Bluetooth, SwiftData or the clock; callers pass values in.

```
// LifeOSKit/Sources/Integrations/LiveEffort.swift
public struct HeartRateZones: Equatable, Sendable {
    public let maxHeartRate: Int
    public init?(birthDate: Date?, on date: Date = .now, calendar: Calendar = .current) {
        guard let birthDate,
              let age = calendar.dateComponents([.year], from: birthDate, to: date).year,
              (10...120).contains(age) else { return nil }
        maxHeartRate = Int((208.0 - 0.7 * Double(age)).rounded()) // Tanaka
    }
    /// 0 below half of maximum, then one zone per ten percent up to 5.
    public func zone(for bpm: Int) -> Int { ... }
}

public struct EffortAccumulator: Equatable, Sendable {
    public static let weights: [Double] = [0, 0.15, 0.28, 0.35, 0.45, 0.63]
    public static let scale = 1500.0
    public static let maxCredit: TimeInterval = 5
    public private(set) var load: Double
    public mutating func add(zone: Int, seconds: TimeInterval) {
        guard seconds > 0, Self.weights.indices.contains(zone) else { return }
        load += Self.weights[zone] * seconds
    }
    public var effort: Double { 21 * (1 - exp(-load / Self.scale)) }
}
```

Max heart rate is Tanaka's estimate, and the zone bands follow Whoop's (zone 1 starts at 50 percent of max, then 60, 70, 80, 90) so the number on the HUD agrees with the number in the app the person already trusts. Each second in a zone adds that zone's weight to a running load; the load maps through a saturating curve so a very long session approaches 21 without reaching it. The six weights and the 1500 are the values that make three calibration points hold at once, and they are pinned by tests: retuning is a deliberate change with a diff, not a drift.

`maxCredit` matters more than it looks. A reading credits at most five seconds. A gap in the stream, a strap that dropped for forty seconds, is not effort that was measured, and crediting the whole gap at the zone of the next reading would invent it.

```
// LifeOSKit/Sources/Integrations/LiveEffort.swift
public struct EffortCeiling: Codable, Equatable, Sendable {
    public let maxZone: Int
    public let targetEffort: ClosedRange<Double>
    public static let green = EffortCeiling(maxZone: 5, targetEffort: 14...18)
    public static let yellow = EffortCeiling(maxZone: 4, targetEffort: 10...14)
    public static let red = EffortCeiling(maxZone: 3, targetEffort: 4...10)
    public static let conservative = yellow
    public static func forCapacity(_ percent: Int) -> EffortCeiling {
        percent >= 67 ? .green : percent >= 34 ? .yellow : .red
    }
}

public struct Capacity: Codable, Equatable, Sendable {
    public enum Source: String, Codable, Sendable { case whoop, health }
    public let percent: Int
    public let source: Source
    public let measuredOn: Date
    public var ceiling: EffortCeiling { .forCapacity(percent) }
}
```

Capacity is today's battery: what the person has to spend and where the number came from. It is computed once at session start and kept in the draft. `CapacityMath.capacity(days:now:)` looks at today's row and yesterday's (yesterday stands in while today has not synced; anything older is stale) and resolves in order:

1. **Whoop.** Recovery percent present and not calibrating. Blended with sleep performance when present: $0.8 * \text{recovery} + 0.2 * \text{sleepPerformance}$, else plain recovery.
2. **Apple Health.** HRV present, with at least three of the previous seven days carrying HRV for a baseline. The ratio to baseline maps piecewise: 0.7 to 1.0 onto 20 to 65 percent, 1.0 to 1.2 onto 65 to 90, clamped. Sleep minutes then adjust by up to plus or minus 10 against a 450-minute reference.
3. **Nothing.** Nil. The UI says "Battery unknown" and the recorder uses the yellow ceiling, cautious without claiming a capacity it does not have.

The ceiling follows Whoop's recovery bands. Battery remaining drains as effort approaches the target's top: $\text{capacity} * (1 - \text{effort} / \text{target.upperBound})$, floored at zero. Zero is a real event, the budget spent, not a missing value. Push state is a four-way word: `overLimit` when the zone exceeds the ceiling's max zone or effort exceeds the target's top, `nearLimit` when the zone is at the max or effort is within 1.5 of the top, `easy` when effort is below the band and the zone is at most 2, `onTrack` otherwise.

THE READOUT

```
// LifeOSKit/Sources/AppSurfaces/LiveSessionReadout.swift
public struct LiveSessionReadout: Codable, Hashable, Sendable {
    public var elapsed: TimeInterval
    public var runningSince: Date?
    public var heartRate: Int?
    public var zone: Int?
    public var effort: Double?           // rounded to a tenth before publishing
    public var calories: Int?
    public var distanceMeters: Int?
    public var batteryPercent: Int?
    public var capacitySource: String? // "whoop" or "health"
    public var ceilingMaxZone: Int?
    public var ceilingTarget: ClosedRange<Double>?
    public var reps: Int?
    public var setIndex: Int?
    public var push: PushState

    /// Anchor for `Text(timerInterval:)`, so the system ticks the timer
    /// without an update per second.
    public var timerAnchor: Date? { runningSince?.addingTimeInterval(-elapsed) }
    public var isPaused: Bool { runningSince == nil }
}
```

This is the one value every live surface renders. Every reading is optional because a missing sensor is not a zero. `WorkoutActivityAttributes.ContentState` is a typealias to it, so the widget and the app can never disagree about what a session looks like. `LiveSessionReadout+Text` is the one place a reading becomes a string, so the HUD, the tiles and the lock screen agree on rounding and on the dash that means no reading.

8.4 ViewModel

`ActivityRecorder` is the largest view model in the app, and it is an `NSObject` because it is the `HKWorkoutSessionDelegate` and `HKLiveWorkoutBuilderDelegate`. It owns the timer, the sensor, the zones, the effort accumulator, the capacity, the draft, the Live Activity controller and the readout.

```
// LifeOS/Features/Activity/ViewModel/ActivityRecorder.swift
@MainActor @Observable
final class ActivityRecorder: NSObject, HKWorkoutSessionDelegate,
HKLiveWorkoutBuilderDelegate {
    var selection: RecordedActivity = .walk
    var saveToHealth = HKHealthStore.isHealthDataAvailable()
    var timer: ActivitySessionState?
    private(set) var busy = false
    var saved = false
    var healthSaved = false
    var energy: Double?
    var distance: Double?
    var heartRate: Double?
    var capacity: Capacity?
    private(set) var readout: LiveSessionReadout?
    var source: SessionSource = .phone
    var zones: HeartRateZones?
    var effort = EffortAccumulator()
    var lastReadingAt: Date?
    let sensor: LiveHeartRateSensor
    /// Injected so previews and checks can fix an age without a profile.
    var birthDate: () -> Date? = { ProfileStore.load().birthDate }
    var whoopConnected: () -> Bool = { false }
    private let liveActivity = WorkoutLiveActivityController()
    private static let draftKey = "activeWorkoutDraft"
```

Three things about its shape. Its dependencies on the world (the birth date, whether Whoop is connected, whether a watch is nearby, Health authorisation) are closures, so the checks harness can fix them without a profile, a strap or a permission sheet. Its defaults are the `UserDefaults.currentAccount` suite, so a draft belongs to one account and an account switch clears it. And it has `active`, flipped off by `deactivate()`, which every async continuation checks before touching state, so a session that finishes after sign-out writes nothing.

START

```
// LIfiOS/Features/Activity/ViewModel/ActivityRecorder.swift
func start(backdatedTo backdatedStart: Date? = nil,
           following video: (id: String, split: String, title: String, channel: String)? =
nil) async {
    guard active, !busy, !hasSession else { return }
    busy = true; error = nil; saved = false; healthSaved = false; timer = nil
    zones = HeartRateZones(birthDate: birthDate())
    capacity = loadCapacity()
    effort = EffortAccumulator(); lastReadingAt = nil
    source = .phone
    recordingHealth = saveToHealth
    defer { busy = false }
    if await handOffToWatch() { return } // Chapter 9
    sensor.prepareForSession(whoopConnected: whoopConnected())
    do {
        if saveToHealth && source == .phone {
            try await healthStore.requestAuthorization(toShare: Self.workoutTypes, read:
Self.workoutTypes)
            guard healthStore.authorizationStatus(for: HKObjectType.workoutType()) ==
.sharingAuthorized else {
                error = "Allow workout saving in Health, or turn off Save to Apple Health to
use the timer."
                return
            }
        }
        let session = try HKWorkoutSession(healthStore: healthStore, configuration:
configuration)
        let builder = session.associatedWorkoutBuilder()
        builder.dataSource = HKLiveWorkoutDataSource(healthStore: healthStore,
workoutConfiguration: configuration)
        let started = backdatedStart ?? .now
        if timer == nil { timer = ActivitySessionState(activity: selection.rawValue, at:
started) }
        session.startActivity(with: started)
        try await builder.beginCollection(at: started)
    } else if timer == nil {
        timer = ActivitySessionState(activity: selection.rawValue, at: backdatedStart ??
.now)
    }
    persist()
} catch { ... }
}
```

The clock is read after Health answers, not before. A permission sheet can sit open for a minute, and a timer started before it would count that minute as exercise. Zones and capacity are computed once here; the capacity is nine days of rows read through `CapacityInputs`, the same helper the workout planner uses, so the battery on the HUD and the battery on the library's Today card are plainly the same number.

A READING ARRIVES

```
// LIfiOS/Features/Activity/ViewModel/ActivityRecorder.swift
func receiveHeartRate(_ bpm: Int, at date: Date) {
    guard active, isRunning else { return }
    heartRate = Double(bpm); heartRateDate = date
    if let zones {
        effort.add(zone: zones.zone(for: bpm),
                    seconds: EffortAccumulator.credit(previous: lastReadingAt, at: date))
    }
    lastReadingAt = date
    persistReading(at: date)
    guard let builder, healthStore.authorizationStatus(for: .quantityType(forIdentifier:
    .heartRate!)) == .sharingAuthorized else { return }
    let sample = HKQuantitySample(type: .quantityType(forIdentifier: .heartRate!),
        quantity: HKQuantity(unit: HKUnit.count().unitDivided(by: .minute()), doubleValue:
    Double(bpm)), start: date, end: date)
    builder.add([sample]) { _, _ in }
}
```

Paused readings accrue nothing because the guard on `isRunning` drops them. The first reading after a resume credits nothing because `togglePause` cleared `lastReadingAt`, so the credit function has no previous date and returns zero. Both are pinned by checks. The strap's beats are also written into the Health builder, so a workout saved to Health carries the heart-rate series the strap measured.

TWO PERSIST PATHS

```
// LIfiOS/Features/Activity/ViewModel/ActivityRecorder.swift
/// State changes: always write the draft and sync.
func persist() {
    refreshReadout()
    writeDraft()
    syncLiveActivity()
}
/// Readings arrive every second: always refresh and sync (the controller
/// throttles the publish), but write the draft at most every ten seconds.
func persistReading(at date: Date) {
    refreshReadout()
    if lastDraftWriteAt.map({ date.timeIntervalSince($0) >= 10 }) ?? true { writeDraft(at:
    date) }
    syncLiveActivity()
}
```

`refreshReadout()` calls `LiveReadoutBuilder.readout(...)`, the pure join of timer, heart rate, zones, effort, capacity, energy, distance, reps and set into one struct. The draft is a `Codable` private struct with every field optional after the first slice, so a draft written before reps or videos existed still decodes.

FINISH AND SAVE

`finish()` marks the timer finished and, for a Health-backed session, calls `session.stopActivity` and returns; the delegate's `.stopped` callback is what calls `saveFinished()`, because collection completes only after HealthKit reaches that state. `saveFinished()` ends collection, finishes the workout, then writes the local `WorkoutRecord` with a stable id:

```
// LIfiOS/Features/Activity/ViewModel/ActivityRecorder.swift
let id = "almanac:\(timer.id.uuidString)"
var fetch = FetchDescriptor<WorkoutRecord>(predicate: #Predicate { $0.externalID == id })
fetch.fetchLimit = 1
if try context.fetch(fetch).isEmpty {
    let row = WorkoutRecord(externalID: id, start: timer.startedAt,
        durationMinutes: Int(timer.elapsed() / 60), activityName: timer.activity, energyKcal:
energy)
    row.distanceMeters = distance
    row.videoID = pendingVideoID; row.split = pendingSplit
    if selection == .strength { row.sets = completedSets + [reps ?? 0] }
    context.insert(row)
}
try context.save()
liveActivity.end()
saved = true; defaults.removeObject(forKey: Self.draftKey)
sensor.stopStreaming()
onSaved?()
```

The stable id is why "Repeated save creates one workout" passes: tapping Finish twice, or a restore after a save that half-completed, finds the row and does not insert a second. A Health failure mid-save keeps the timer and says "Your activity is kept here. Saving failed. Try Save again."

THE SENSOR

`LiveHeartRateSensor` speaks the Bluetooth SIG heart-rate service (0x180D, measurement 0x2A37). It handles 8-bit and 16-bit frames, treats a no-contact flag as no reading, and rejects values outside 20 to 250. Scans stop after 30 seconds. What the live-session slice added is memory: the last connected peripheral's identifier is written to the per-account defaults under `lastHeartRateSensorID` on every successful connection and cleared on an explicit disconnect.

```
// LIfiOS/Features/Activity/ViewModel/LiveHeartRateSensor.swift
/// Called by the recorder at session start. Reconnects the last sensor,
/// or looks for a lone WHOOP when the account is connected to WHOOP and
/// nothing was ever paired here.
func prepareForSession(whoopConnected: Bool) {
    if rememberedPeripheralID != nil { reconnectIfRemembered() }
    else if whoopConnected { autoPairWhoop() }
}
```

`reconnectIfRemembered` uses `retrievePeripherals(withIdentifiers:)` and connects directly; if nothing connects in 20 seconds it falls back to a scan and connects the discovered peripheral whose

identifier matches. `autoPairWhoop` scans and connects only when `WhoopAutoPair.choice(among:)` finds exactly one name containing "WHOOOP". Several stop the scan with "More than one WHOOOP nearby. Choose one under Manage." Nothing auto-pairs a device that is not a WHOOOP, because another person's chest strap in a gym is not a reading this person owns.

8.5 View

`BeginActivityScreen` is a sheet with two states. Before a session: a heading ("Make time to move."), the timer card on an activity pastel, a "Follow a video" link into the library, the six-activity picker (Walk, Run, Cycle, Strength, Yoga, Other), the controls, the connections panel, and a quick-log shortcut. During a session: the whole background becomes a blue-to-canvas gradient 380 points tall, the navigation bar goes dark, and the hero replaces the card.

```
// LIfiOS/Features/Activity/View/BeginActivityScreen.swift
private func liveHero(_ readout: LiveSessionReadout) -> some View {
    VStack(alignment: .leading, spacing: 8) {
        HStack {
            Label(model.selection.rawValue, systemImage:
model.selection.icon).font(LifeOSType.rowTitle)
            Spacer()
            Text(readout.isPaused ? "Paused" : readout.push.headline).font(LifeOSType.label)
                .padding(.horizontal, 10).padding(.vertical, 5)
                .background(.white.opacity(0.22), in: Capsule())
        }
        if let following = model.following {
            Text("Following: \(following.title) · \(following.channel)")
        }
        TimelineView(.periodic(from: .now, by: 1)) { timeline in
            Text(duration(model.timer?.elapsed(at: timeline.date) ?? 0))
                .font(.system(size: 72, weight: .medium, design: .rounded)).monospacedDigit()
        }
        Text("Elapsed time").font(LifeOSType.caption).opacity(0.85)
        SessionRings(readout: readout, activity: model.selection, zonesAvailable:
model.zonesAvailable,
                    countsAutomatically: model.source == .watch,
                    onAddRep: { model.addRep() }, onRemoveRep: { model.removeRep() },
                    onNextSet: { model.nextSet() }, isExpanded: $hudExpanded)
    }
    .foregroundColor(.white)
}
```

The clock lives on its own line at 72 points rather than inside the rings, because a timer plus five readings does not fit in one capsule, and the reading that truncated when it was tried was the battery percentage, the one number that must never be half a number.

THE RINGS

`SessionRings` is a `GlassEffectContainer` with two shapes. Expanded, a rounded rectangle holding three rings (heart, effort, battery) and, for strength, a rep count, with the rep buttons on a second row inside the same glass. Collapsed, a capsule with the clock, a pulsing heart, the bpm and, for strength, the reps. Tapping

the top row toggles between them. It always draws white on glass over a dark wash (`.black.opacity(0.30)`) so the text reads over a bright frame of video or the pale foot of the gradient alike, and the glass is tinted by the push state.

```
// LIfiOS/Features/Activity/View/SessionRings.swift
/// One beat per beat: a scale wave at the live tempo, still when there
/// is no reading to keep time to.
private var pulsingHeart: some View {
    TimelineView(.animation(paused: SessionRingMath.beatPeriod(bpm: readout.heartRate) == nil
|| readout.isPaused)) { timeline in
        let period = SessionRingMath.beatPeriod(bpm: readout.heartRate) ?? 1
        let phase =
    timeline.date.timeIntervalSinceReferenceDate.truncatingRemainder(dividingBy: period) / period
        let beat = max(0, sin(phase * .pi * 2))
        Image(systemName: "heart.fill").scaleEffect(1 + 0.35 * beat)
    }
}
```

The ring fills come from `SessionRingMath`, pure and tested: the heart ring is the zone over today's ceiling zone, the effort ring is effort over the top of the target band, the battery ring is the percent. A missing reading is an empty ring, never a full one; nothing has been earned yet. The effort ring turns red only past the band, not at its top. The animation timeline is paused when there is no reading or the session is paused, so a still heart costs no frames.

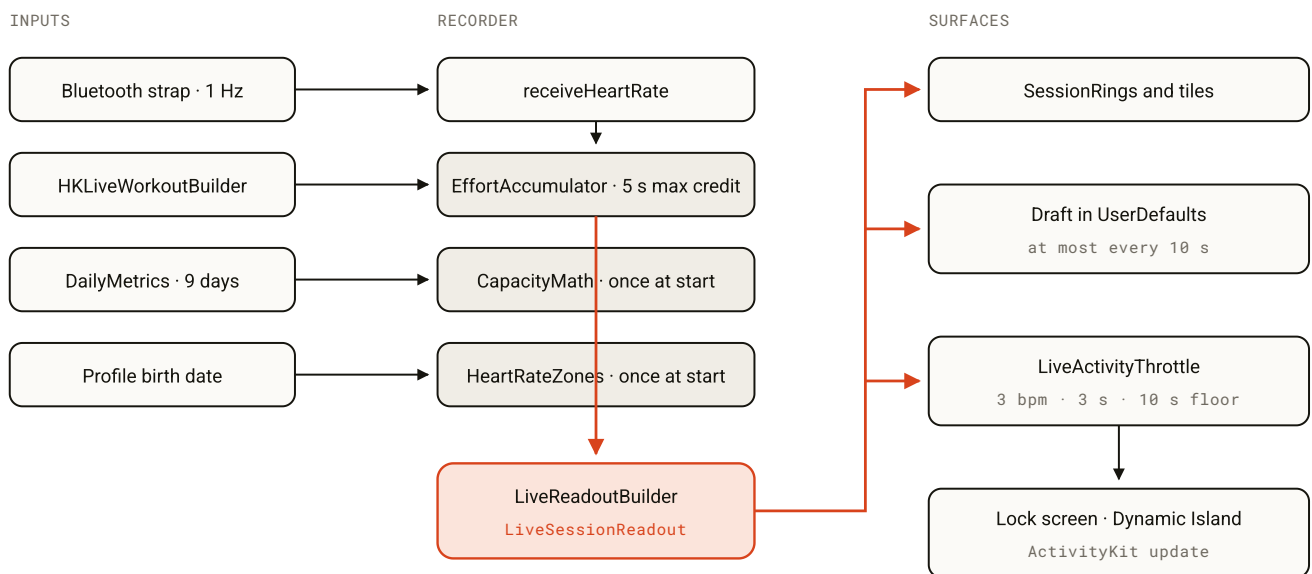
THE TILES

Under the hero, glass tiles in an adaptive grid: heart rate with the zone in its label and a freshness rule (a reading older than 15 seconds shows a dash and the sensor's status as its caption); effort with "of 14" as its unit and the battery and its source as caption; reps for strength; calories; distance for walk, run and cycle only. Yoga hides the effort tile: a zone ceiling has no meaning on a mat.

CONTROLS

`ActivityControls` was extracted from the screen so the video player offers exactly the same buttons in the same order. Two screens that both stop a workout should not disagree about what stopping looks like, and the confirm-discard dialog travels with the button that raises it.

8.6 How it works



One reading a second becomes one readout a second. The screen redraws every time; the draft is written every ten seconds; the lock screen is updated only when the throttle says the change is worth it.

STEP BY STEP: A RUN WITH A STRAP

1. Begin. `start()` builds zones from the birth date and capacity from nine days of rows, asks Health, creates the session and builder, starts the timer at the moment Health answered, and persists. `sensor.prepareForSession` reconnects the remembered strap.
2. A frame arrives on characteristic 0x2A37. `HeartRateMeasurement.beatsPerMinute` decodes it; the sensor calls `onReading`.
3. `receiveHeartRate` credits at most five seconds of the reading's zone to the accumulator, records the time, and calls `persistReading`.
4. `refreshReadout` rebuilds the readout: zone from the bpm, effort rounded to a tenth, battery from capacity and effort, push state from zone and effort against the ceiling.
5. The rings and tiles observe `readout` and redraw. The draft is written if ten seconds have passed. The Live Activity controller is asked to sync.
6. `LiveActivityThrottle.shouldPublish` decides. First state always; a pause, a push-state change or a reps change always; a heart-rate move of 3 bpm or more only if 3 seconds have passed since the last publish; anything else only after a 10-second floor; an identical readout never.
7. The builder's own statistics callback delivers energy and distance sums, and the watch's heart rate when there is no strap. It updates the same fields and calls `persist()`.
8. Pause: the timer banks, the Health session pauses, `lastReadingAt` clears. Readings still arrive and are dropped.
9. Finish: stop the Health activity, wait for `.stopped`, end collection, finish the workout, write the record, end the Live Activity, clear the draft, stop the strap.

THE LIVE ACTIVITY

Widget extensions cannot render materials, so the lock screen's "glass" tiles are white at 18 percent over a linear gradient from `LifeOSTokens.liveGradientTop` (the recovery hue) to `liveGradientBottom`, with a radial white highlight and a half-point white stroke. Row one: the activity icon in a circle, the name, the push headline (or "Paused"), and the timer as `Text(timerInterval:)`. Row two: four tiles, heart with the zone as eyebrow, effort with "EST" and "of 14", calories (or reps and set for strength), and battery with a small ring. A tile with no value shows the dash in the same weight, never a zero.

The Dynamic Island's compact leading is a heart and the bpm tinted by push state; compact trailing is the timer, or reps for strength; minimal is the zone number in a tinted circle. The keyline tint follows push state. Every surface deep-links to `SurfaceRoute.activity.url`.

8.7 Performance and cost

- **No tick.** The timer is two fields and the clock. The lock screen counts on its own through `Text(timerInterval:)`; the in-app clock is a one-second `TimelineView`, which SwiftUI runs only while the view is on screen.
- **Draft writes are rate-limited.** A reading a second would be a `UserDefaults` write a second. `persistReading` writes at most every ten seconds; state changes (pause, resume, finish) always write.
- **ActivityKit updates are throttled by a pure function.** The sensor streams every second; the lock screen does not need to. The 3-second minimum on heart-rate-driven updates and the 10-second floor on everything else keep the activity inside ActivityKit's budget, and the rule is tested rather than tuned by eye. Updates are chained through a `Task` so they land in order.
- **Effort is O(1) per reading.** One multiply-add into a load; the curve is evaluated on read. Nothing keeps a series.
- **Capacity is computed once.** Nine rows at session start, then kept in the draft. A restored session does not re-read the store.
- **The pulse is free when still.** The heart's `TimelineView(.animation)` is paused whenever there is no bpm or the session is paused.
- **Scans end.** Thirty seconds, then stop, so a sheet left open is not draining the radio. The reconnect fallback times out at twenty.
- **Nothing leaves the device.** A workout is a local `WorkoutRecord` and, optionally, an `HKWorkout`. The Live Activity uses `pushType: nil`. There is no server in this chapter.
- **Every continuation checks** `active`. A delegate callback, a recovered session, a finish that lands after sign-out: all guard on it, so a stale session cannot write into the next account's store.

OPEN ON HARDWARE

The spec flags one background-execution question: whether iOS 26 keeps the app running during a phone-side `HKWorkoutSession` or whether workout-processing must join `UIBackgroundModes`. The plist today carries `remote-notification` and `bluetooth-central`. The wall-clock timer means the elapsed time is right either way; what a suspended app would lose is effort accrual for the suspended minutes.

8.8 Edge cases and tests

CONDITION	BEHAVIOUR
No birth date	No zones, effort or battery; heart rate and calories only; one hint line under the tiles
Age outside 10 to 120	Treated as no birth date
Whoop recovery is calibrating	Falls through to the Health path; a calibration score does not support a target
Today has not synced	Yesterday's recovery stands in; two days ago does not
Fewer than 3 baseline HRV days	No Health capacity; "Battery unknown", yellow ceiling
Strap drops for 40 seconds	The next reading credits 5 seconds, not 40
Readings arrive while paused	Dropped; effort and heart rate unchanged
First reading after resume	Credits nothing; the second accrues
Reading older than 15 seconds	The tile shows a dash and the sensor status, not a stale number
Health permission refused	Error line; the toggle can be turned off to use the timer alone
Relaunch mid-session, Health recording	Draft restores the timer; <code>recoverActiveWorkoutSession</code> re-attaches the session and builder, or the notice says the Health session is gone
Relaunch mid-session, timer only	Draft restores without touching Health at all
Health session fails mid-workout	Timer pauses; "Health recording stopped. Your timer is kept."
Finish tapped twice	One <code>WorkoutRecord</code> , found by its stable id
Save fails	Timer kept, draft kept, "Try Save again"
Two WHOOPs nearby	Scan stops; "Choose one under Manage"
Bluetooth off or unauthorised	Status text per state; no loop, no retry storm
Account switches mid-session	<code>deactivate()</code> discards; late starts are rejected; the other account never sees the draft
Yoga selected	No effort tile or ring; a zone ceiling has no meaning on a mat
A card left on screen by another session	<code>endStrayLiveActivities</code> ends every activity whose id is not this timer's

TESTS

- `LiveEffortTests` (18 tests): a 30-year-old has max 187; 94 bpm is zone 1, 131 is zone 3, 169 is zone 5; no birth date yields nil; the calibration points (an hour in zone 3 is about 12, thirty minutes in zone 2 about 6, ninety hard minutes about 18) within 1.0; effort is bounded below 21 and never decreases; credit caps

gaps at 5 and starts at 0; Whoop 80 with sleep 90 gives 82 and green; yesterday stands in; calibrating falls through; Health capacity needs a baseline and moves with sleep; battery drains against the target top; push state follows zone and effort; the builder keeps absence and uses the conservative ceiling without capacity; auto-pair picks exactly one WHOOP; `todayCapacity` reads the store.

- `LiveSessionReadoutTests` (9): round trip keeps absence; a paused readout has no anchor; headlines are words; the throttle publishes first and on meaningful change, spaces heart-rate updates, floors small changes at ten seconds, publishes on a reps change; text formatters keep absence and rounding.
- `SessionRingMathTests` (5) and `ActivitySessionStateTests` (4): fills, over-band, beat period; background time and pauses; identity survives a draft round trip; BLE frames decode.
- `ActivityRecorderChecks`, 34 checks run in the simulator against an in-memory store and disposable defaults suites, launched with `--design-preview --page=checks`. They cover what Swift Testing cannot reach: the timer starting without Health, capacity at start, readings yielding zone and effort, pause and resume accrual, draft restore with and without Health, cross-account isolation, idempotent save, the remembered sensor, and the video stamping cases of Chapter 10.

The design QA report records 1231 package tests passing after this slice, the in-session screen captured on iPhone and iPad in both schemes, and the lock screen and Dynamic Island captured from the simulator with a fixture that requests a real Live Activity behind the `--live-activity` argument.

8.9 Study notes

READ IN THIS ORDER

1. `LifeOSKit/Sources/Integrations/ActivitySessionState.swift`, the timer with no tick.
2. `LifeOSKit/Sources/Integrations/LiveEffort.swift`, top to bottom: zones, accumulator, ceiling, capacity, the maths, the builder.
3. `LifeOSKit/Sources/AppSurfaces/LiveSessionReadout.swift`, the struct and the throttle.
4. `LifeOS/Features/Activity/ViewModel/ActivityRecorder.swift`: `start`, `receiveHeartRate`, the two persist paths, `saveFinished`.
5. `LifeOS/Features/Activity/ViewModel/LiveHeartRateSensor.swift`, the remember-and-reconnect path.
6. `LifeOS/App/Surfaces/WorkoutLiveActivityController.swift` and `AlmanacWidgets/WorkoutLiveActivity.swift`.
7. `LifeOS/Features/Activity/View/SessionRings.swift`, then `ActivityRecorderChecks.swift` as the executable spec.

CHECK YOURSELF

- **Why is the timer started after Health answers rather than before?** A permission sheet can sit open for a minute, and a timer started first would count it as exercise.
- **Why does one reading credit at most five seconds?** A gap in the stream is not effort that was measured; crediting the gap would invent it.
- **What makes "Repeated save creates one workout" true?** The record's external id is derived from the timer's UUID and looked up before insert.

- **Why does the lock screen not need an update every second?** It renders `Text(timerInterval:)` from an anchor date; only readings, not time, need an update, and the throttle spaces those.
- **Why do zones live in Integrations but the readout in AppSurfaces?** The widget must decode the readout without linking HealthKit or SwiftData; AppSurfaces depends on nothing.
- **When does the effort ring turn red?** Only past the top of the target band, not at it.

9 The Watch companion

When an Apple Watch is paired, the watch owns the workout and the phone shows it. Heart rate, energy, reps and sets cross the mirrored HealthKit session once a second; pause, resume, end and the rep buttons cross back. A pure rep counter on wrist acceleration fills the slot the HUD reserved, labelled "auto" because its accuracy on real movement is still unmeasured.

9.1 Intent

Before this slice the watch app was a read-only dashboard: steps, sleep and movement mirrored from the phone, with no way to start, see or end a workout. A person wearing a watch and no strap saw a timer on the phone and no beats, because the phone only ever received heart rate from a Bluetooth strap. And strength sessions showed a "reps" slot that never filled.

Tap Begin on the phone with a watch on your wrist and the phone says "Asking your Apple Watch...". Within a few seconds Almanac opens on the watch with the workout running, and the phone's rings show the watch's heart rate. Or start from the watch: choose Strength, and the phone's lock screen shows the session before you have touched the phone. Curl, and the rep count on your wrist climbs on its own; tap +1 if it missed one, tap -1 if it counted an extra. Pause on either device and both pause. End on either and exactly one workout lands in Health, saved by the watch.

Who owns the session?

Whichever device created the `HKWorkoutSession`. When a watch is paired, installed and Health saving is on, the watch does, and it alone writes the workout to Health. The phone never creates a session of its own for a watch-owned workout, so nothing is saved twice. When no watch answers within ten seconds, the phone path from Chapter 8 runs unchanged.

Why is the rep count labelled "auto" on the watch and not on the phone?

Only a watch session counts reps by itself. On a phone-only strength session every rep is a tap, and calling that "auto" would invert the honesty label.

What has actually been verified on a real watch?

Nothing yet. The rep counter is pinned by tests on synthetic signals, the wire format and the recorder's watch path are pinned by tests and simulator checks, and both targets build. The watch simulator cannot run a real workout session, so the watch screen is verified by compilation only. Five hardware checks and two recovery-behaviour checks are recorded as pending in the QA report, to be run by the author on his own phone and watch.

9.2 Files and folders

AlmanacWatch/ ├─ AlmanacWatchApp.swift ├─ WatchAppDelegate.swift ├─ WatchWorkoutController.swift └─ WatchWorkoutScreen.swift	the watchOS target, added to the pbxproj by hand the scene: dashboard or workout; WatchBridge for th handle(configuration) and handleActiveWorkoutRecover the session, the builder, the counter, the packets timer, heart rate with zone chip, reps, pause, end
LifeOS/App/Surfaces/ └─ WatchSessionBridge.swift	one per app: the mirrored session and its delegate
LifeOS/Features/Activity/ViewModel/ └─ ActivityRecorder+Watch.swift	hand-off, adopt, packets, commands
LifeOSKit/Sources/Motion/ └─ RepCounter.swift	Foundation only; links on watchOS samples in, a count out

LifeOSKit/Sources/AppSurfaces/ └─ WatchWire.swift	WatchPacket, PhoneCommand, the version gate
LifeOSKit/Tests/ └─ MotionTests/RepCounterTests.swift	six calibration cases on synthetic sine waves
└─ AppSurfacesTests/WatchWireTests.swift	round trips, unknown version, kind gate

WHY MOTION IS ITS OWN PACKAGE TARGET

The watch target links two package products, `AppSurfaces` and `Motion`, and neither depends on anything. `CoreMotion` stays in the watch target: the controller feeds `CMDeviceMotion.userAcceleration` samples at 50 Hz into the pure counter. That is what lets the algorithm be tuned against synthetic signals on macOS, and later against a recording from hardware, without a watch in the loop.

9.3 Model

THE WIRE FORMAT

```
// LifeOSKit/Sources/AppSurfaces/WatchWire.swift
/// What the watch sends the phone over the mirrored session's data channel.
/// Every reading is optional: a packet without heart rate says "no reading",
/// never zero. `completedSets` holds reps per finished set, oldest first.
public struct WatchPacket: Codable, Equatable, Sendable {
    public var kind: String = WatchWire.packetKind
    public var v: Int = WatchWire.version
    public var sentAt: Date
    public var heartRate: Int?
    public var heartRateAt: Date?
    public var energyKcal: Double?
    public var reps: Int?
    public var setIndex: Int?
    public var completedSets: [Int]?
}

public enum PhoneCommand: String, Codable, Sendable {
    case configure, pause, resume, end, nextSet, addRep, removeRep, discard
}

public struct PhoneCommandEnvelope: Codable, Equatable, Sendable {
    public var kind: String = WatchWire.commandKind
    public var v: Int = WatchWire.version
    public var command: PhoneCommand
    public var sentAt: Date
    public var maxHeartRate: Int? // sent once with .configure
}
```

Both shapes carry a version, currently 2, and a `kind`. The version gate is strict: a packet with an unknown `v` is ignored, never guessed at. The `kind` field exists because the two shapes otherwise share every required

key (`v` and `sentAt`), and a command would decode as an empty packet. A test pins that a command is never read as a packet.

```
// LifeOSKit/Sources/AppSurfaces/WatchWire.swift
public static func packet(from data: Data) -> WatchPacket? {
    guard let packet = try? decoder.decode(WatchPacket.self, from: data),
          packet.kind == packetKind, packet.v == version else { return nil }
    return packet
}
```

Dates are encoded as seconds since 1970. `heartRateAt` is when the reading was measured, not when the packet was sent, so effort accrual on the phone credits the right seconds.

THE TWO COMMANDS THAT END A SESSION

`end` finishes the workout and saves it to Health on the wrist. `discard` throws it away. The distinction is the fix for a critical finding in the final review: the phone originally sent `end` when it refused a mirrored session or the person discarded, which would have written a stub workout into Health for a session the person had thrown away.

THE REP COUNTER

```
// LifeOSKit/Sources/Motion/RepCounter.swift
public struct RepCounter: Sendable {
    public struct Sample: Sendable { public var t: TimeInterval; public var x, y, z: Double }
    public let settleSeconds: Double // 1.0
    public let minPeriod: Double // 0.6
    public let maxPeriod: Double // 4.0
    public let threshold: Double // 0.15
    public private(set) var reps: Int = 0
    private let fastTau = 0.15
    private let slowTau = 2.0
    private var axis: Vec?
    private var riseT: TimeInterval?
    private var refractoryUntil: TimeInterval = -.infinity

    /// Feed one sample; returns true when this sample completed a rep.
    public mutating func add(_ sample: Sample) -> Bool
    public mutating func reset()
}
```

The algorithm, as it stands after two corrections during implementation:

1. Each axis is smoothed twice with exponential moving averages: a fast one (0.15 s) follows the movement, a slow one (2 s) follows posture and gravity drift. Their difference is a vector `d`.
2. The first lift whose magnitude passes the threshold, *and is still growing*, fixes the axis for the set. The rising condition is the second fix: settling can end mid-fall, and locking the axis onto a fading swing spent the first cycle on a half swing and undercounted by exactly one in two of six tests.

3. The signal `s` is `d` projected onto that axis. It is signed: a rep is a swing, not a rectified bump. The first version rectified the signal with `sqrt(x2+y2+z2)`, and a fall crossing could then structurally never follow a rise.
4. A rep is a rise of `s` above `+threshold` followed by a fall below `-threshold`, with the half cycle between them lasting between 0.3 and 2.0 seconds.
5. Nothing counts during the first second after `reset()`, the person picking up the weight, and each counted rep starts a 0.3-second refractory window.

CALIBRATION IS SYNTHETIC

Every constant is pinned by tests on 50 Hz sine waves with white noise. Ten cycles at 1 Hz and 0.4 g count ten; 0.08 g counts nothing; 3 Hz jitter counts nothing; one ten-second push counts nothing; nine cycles whose first second is swallowed by settling count eight. Real-world accuracy is unknown until hardware. The "auto" label and the +1 and -1 buttons are the honesty.

9.4 ViewModel

ON THE WRIST

```
// AlmanacWatch/WatchWorkoutController.swift
@MainActor @Observable
final class WatchWorkoutController: NSObject, HKWorkoutSessionDelegate,
HKLiveWorkoutBuilderDelegate {
    enum State { case idle, starting, running, paused, ending }
    private(set) var state: State = .idle
    private(set) var accumulated: TimeInterval = 0
    private(set) var runningSince: Date?
    private(set) var heartRate: Int?
    private(set) var energyKcal: Double?
    private(set) var reps: Int?
    private(set) var setIndex: Int?
    private(set) var completedSets: [Int] = []
    private(set) var maxHeartRate: Int?
    private(set) var mirroringFailed = false
    private var counter = RepCounter()
    /// Taps on "+1", kept apart from the counter so the next automatic rep
    /// adds to them instead of overwriting them.
    private var manualReps = 0
    private var lastPacketAt: Date = .distantPast
```

`start(_:)` flips `state` to `.starting` synchronously before its first await, so a second Start tap during the authorisation prompt cannot open a second session. It requests Health sharing for workouts, heart rate and energy, creates the session and builder, begins collection, then calls `startMirroringToCompanionDevice()`. A mirroring failure is recorded, not fatal: the watch still records and saves; the phone simply does not see it. For strength it starts device motion at 50 Hz.

```
// AlmanacWatch/WatchWorkoutController.swift
motion.startDeviceMotionUpdates(to: .main) { [weak self] data, _ in
    guard let self, let data else { return }
    MainActor.assumeIsolated {
        guard self.state == .running else { return }
        let acceleration = data.userAcceleration
        if self.counter.add(RepCounter.Sample(t: data.timestamp, x: acceleration.x, y:
acceleration.y, z: acceleration.z)) {
            self.reps = self.counter.reps + self.manualReps
            self.sendPacket(force: true)
        }
    }
}
```

`.main` is the main-thread queue, so the handler is already on the main actor; `assumeIsolated` keeps every sample in order rather than scattering fifty hops a second through `Task`. The manual tally is separate from the counter's so that an automatic rep adds to the taps instead of overwriting them, which was the critical bug the review found. `removeRep()` spends the manual tally first and then offsets the automatic one, never below zero, and does nothing while the count is unknown, because a dash must not become a zero.

The clock is kept the way `ActivitySessionState` keeps it on the phone: a pause banks `accumulated`, a resume starts `runningSince`. The screen's timer is anchored at `runningSince - accumulated`, so it reads the workout's elapsed time rather than the time since this run began.

Ending has two paths, and they must never be confused. `end()` stops the activity; the `.stopped` delegate callback then ends collection, calls `finishWorkout()`, and the `.ended` callback resets. `discard()` detaches the delegates first, so no late `.ended` re-enters the teardown, then discards the builder and ends the session without finishing it. `recover()`, wired from `handleActiveWorkoutRecovery()`, adopts a session still active in HealthKit after watchOS relaunched the app; a session already stopped is settled at once, and a recovered strength session restarts motion with an unknown rep count rather than a zero.

ON THE PHONE: THE BRIDGE

```
// LIfеOS/App/Surfaces/WatchSessionBridge.swift
/// Owns the workout session mirrored from the watch. One per app: HealthKit
/// hands the session to whichever handler is installed at launch, so the
/// bridge is installed by the app delegate and the recorder subscribes.
@MainActor
final class WatchSessionBridge: NSObject, HKWorkoutSessionDelegate {
    static let shared = WatchSessionBridge()
    private(set) var session: HKWorkoutSession?
    private(set) var placeholderSessionID: UUID?
    private var dropping = false
    var onSession: ((HKWorkoutSession) -> Void)?
    var onPacket: ((Data) -> Void)?
    var onStateChange: ((HKWorkoutSessionState, Date) -> Void)?
    var hasSession: Bool { session != nil && !dropping }

    static func installMirroringHandler() {
        healthStore.workoutSessionMirroringStartHandler = { session in
            Task { @MainActor in
                let bridge = WatchSessionBridge.shared
                let unattended = bridge.onSession == nil
                bridge.adopt(session)
                if unattended { bridge.requestPlaceholderActivity(for: session) }
            }
        }
    }
}
```

The handler is installed from `PushService`'s app delegate at launch, before any session can arrive. The placeholder matters for a workout started on the wrist: HealthKit launches the phone app in the background with no scene and so no recorder. The Live Activity has to be requested inside that launch, so the bridge asks for it itself, keeps the id, and the recorder's timer adopts that id when a screen finally attaches, so the card already on screen is updated rather than duplicated.

```

// LifeOS/App/Surfaces/WatchSessionBridge.swift
/// Whether a watch could take the session right now.
static var watchAvailable: Bool {
    guard WCSSession.isSupported() else { return false }
    let wc = WCSSession.default
    // Not `isReachable`: on iOS that is true only while the watch app is
    // already in the foreground, which is never the case for a person
    // tapping Begin on the phone.
    return wc.activationState == .activated && wc.isPaired && wc.isWatchAppInstalled
}

/// Sends a last command and drops the session only once it has left, in
/// the send's completion handler: releasing the session in the same
/// run-loop turn can cancel the send before the watch ever hears it.
func end(after command: PhoneCommand) {
    guard let session, let data = try? WatchWire.encode(PhoneCommandEnvelope(command:
command, sentAt: .now)) else {
        end(); return
    }
    dropping = true
    session.sendToRemoteWorkoutSession(data: data) { _, _ in
        Task { @MainActor in self.end() }
    }
}

```

`dropping` exists because `hasSession` stays true through that window, and without it the next Start would refuse the hand-off against a session already on its way out.

ON THE PHONE: THE RECORDER'S WATCH HALF

```
// LifeOS/Features/Activity/ViewModel/ActivityRecorder+Watch.swift
func handOffToWatch() async -> Bool {
    guard saveToHealth, watchAvailable(), let watch, !watch.hasSession else { return false }
    // Ask on the phone first. Authorization is shared with the paired
    // watch, so without this the first prompt a person ever sees is on the
    // wrist, while the phone is already counting down its ten seconds.
    guard await healthAuthorizationForHandoff() else { return false }
    let configuration = HKWorkoutConfiguration()
    configuration.activityType = selection.healthType
    source = .watch
    notice = Self.waitingForWatch
    try? await WatchSessionBridge.startWatchApp(configuration)
    let deadline = Date.now.addingTimeInterval(watchHandoffTimeout)
    while Date.now < deadline, watch.session == nil, active {
        try? await Task.sleep(for: .milliseconds(100))
    }
    guard active else { return true }
    if let mirrored = watch.session {
        if timer == nil { adoptMirroredSession(mirrored) }
        if notice == Self.waitingForWatch { notice = nil }
        return true
    }
    source = .phone
    notice = "Apple Watch did not answer. Recording on iPhone."
    return false
}
```

The hand-off is offered only when the workout is meant for Health, since the watch writes it there; a person who turned Health saving off keeps the phone. No timer starts until the watch answers, so nothing counts a session the watch may never take, and `busy` keeps the button reading "Starting...".

`adoptMirroredSession` handles both arrivals: the answer to a hand-off, and a session the person started on the wrist. If the phone is already recording its own session it refuses with a notice and sends `discard`. Otherwise it builds a fresh timer starting where the watch started, adopting the bridge's placeholder id if there is one, ends any stray Live Activity, stops the strap (the watch streams its own heart rate; two feeds would double every reading into the effort accrual), sets `recordingHealth` false so the draft never claims a Health session of the phone's own, and sends `configure` with the person's max heart rate so the watch can show a zone chip.

```
// LIfeOS/Features/Activity/ViewModel/ActivityRecorder+Watch.swift
func receiveWatchPacket(_ data: Data) {
    guard active, hasSession, source == .watch, let packet = WatchWire.packet(from: data)
    else { return }
    // The counts are applied before that reading, so the one refresh carries
    // this packet's reps rather than the previous packet's.
    let refreshes = packet.heartRate != nil && packet.heartRateAt != nil && isRunning
    if let energy = packet.energyKcal { self.energy = energy }
    if selection == .strength {
        if let value = packet.reps { reps = value }
        if let value = packet.setIndex { setIndex = value }
        if let value = packet.completedSets { completedSets = value }
    }
    if let bpm = packet.heartRate, let at = packet.heartRateAt { receiveHeartRate(bpm, at:
at) }
    if !refreshes { persistReading(at: packet.sentAt) }
}
```

Heart rate goes through the same `receiveHeartRate` as a strap reading, so effort accrual, throttling and the Live Activity stay one path. The ordering comment records a bug the checks found: a packet carrying both a heart rate and a rep change once published the previous rep count, because the single readout refresh ran inside `receiveHeartRate` before the counts were applied.

`addRep`, `removeRep` and `nextSet` each have two paths: on a watch session they send the command and wait for the echo packet; on a phone session they count locally. Pause, resume and finish likewise send commands for `.watch` and let the mirrored session's state change drive the timer, exactly as the phone's own session delegate does.

9.5 View

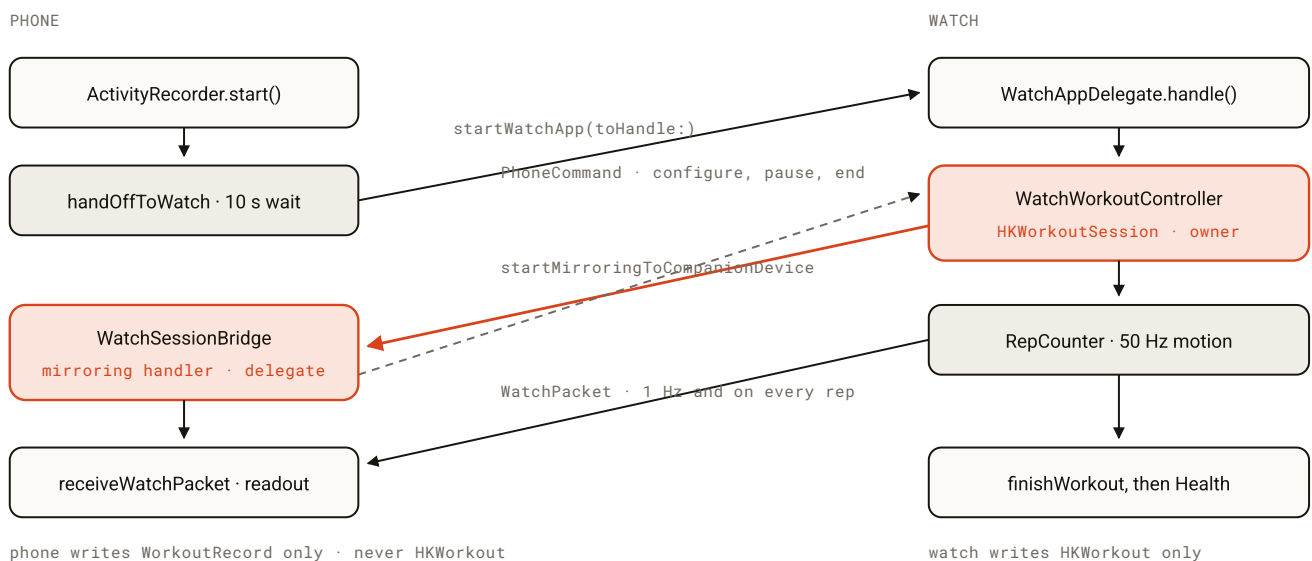
`AlmanacWatchApp` shows `WatchWorkoutScreen` whenever the controller is not idle and the dashboard otherwise. The dashboard gained a Start workout button that presents a `List` of six activities in a sheet, because watchOS has no `Menu`. The scene's `.task` wires both delegate handlers and then drains any configuration or recovery that arrived before the task ran, so either order reaches the controller.

```
// AlmanacWatch/WatchWorkoutScreen.swift
if let runningSince = workout.runningSince, workout.state == .running {
    Text(timerInterval: runningSince.addingTimeInterval(-
workout.accumulated)...Date.distantFuture, countsDown: false)
        .font(.system(size: 34, weight: .semibold, design: .rounded)).monospacedDigit()
} else if workout.state == .paused {
    Text(Duration.seconds(workout.accumulated).formatted(.time(pattern: .minuteSecond)))
}
HStack(spacing: 6) {
    Image(systemName: "heart.fill").foregroundColor(.red)
    Text(workout.heartRate.map(String.init) ?? "\u{2014}").font(.title2.monospacedDigit())
    if let max = workout.maxHeartRate, let bpm = workout.heartRate {
        Text("Z\u{2014}(zone(bpm: bpm, max: max))").font(.caption2).foregroundColor(.secondary)
    }
}
```

The screen is minimal on purpose: the activity name, the timer, heart rate with a zone chip once the phone has sent `configure`, and for strength the rep count over "reps · auto", the set number, and three buttons on two rows (-1 and +1, then Next set full width) because three bordered buttons split a 41 mm screen three ways and "Next set" wraps at about 48 points. Pause or Resume and End sit at the bottom. It uses the system font and the app's orange; the watch target does not link the design system. The zone bands are duplicated here in five lines because the watch cannot see `HeartRateZones` in `Integrations`.

On the phone, the surfaces from Chapter 8 grew a rep slot: the rings show the count and "auto" or "reps"; the tiles gain a Reps tile captioned "Counted from your wrist · auto" or "Tap +1 for each rep" and, after a set, "Earlier sets: 2 reps"; the Dynamic Island's compact trailing shows reps instead of the timer for strength; the lock screen's third tile shows the set and reps instead of calories. The connections panel says "Apple Watch is recording this session", "Apple Watch ready", or "Apple Watch not nearby".

9.6 How it works



The session has one owner. Readings flow one way, commands the other, and each device writes exactly one record: the watch to Health, the phone to its local store.

STEP BY STEP: STARTED ON THE PHONE

1. Begin, with Health saving on and a paired watch. The phone asks for its own Health authorisation first, because authorisation is shared with the paired watch and the first prompt should be on the phone.
2. `HKHealthStore.startWatchApp(toHandle:)` launches Almanac on the watch with the configuration. The phone says "Asking your Apple Watch..." and polls the bridge every 100 ms for up to ten seconds.
3. The watch delegate's `handle(_:)` passes the configuration to the controller (or parks it in `pendingConfiguration` if the scene's task has not run yet). The controller starts the session and begins mirroring.
4. HealthKit calls the phone's `workoutSessionMirroringStartHandler`. The bridge adopts the session and calls `onSession`; the recorder's `adoptMirroredSession` builds the timer from the watch's start date and sends `configure`.
5. Packets arrive once a second and on every rep. Heart rate accrues effort; reps and sets update the readout; the rings, tiles and Live Activity refresh through the Chapter 8 paths.
6. Pause on the phone sends `pause`; the watch pauses its session; the mirrored state change comes back through the bridge and pauses the phone's timer. Pause on the watch takes the same last two steps.
7. Finish on the phone sends `end` through `end(after:)` and calls `saveFinished()`, which writes the local record with the sets and never touches a builder. The watch ends collection, finishes the workout into Health, and resets.

STARTED ON THE WRIST

Steps 4 to 7 are the same. The difference is step 3: there is no phone scene yet. HealthKit launches the phone app in the background, the bridge adopts the session unattended and requests the placeholder Live Activity,

and when the person opens the phone the recorder attaches, finds `watch.session` already held, adopts it, and takes over the card by its id.

OWNERSHIP RULES, STATED ONCE

- The watch is offered the session only when `saveToHealth` is on.
- The phone never creates a Health session for a `.watch` workout, and never writes an `HKWorkout` for one.
- If the phone is already recording when a mirrored session arrives, it keeps its own and discards the mirrored one. It never merges, and it never ends it, since ending means saving.
- A finished, saved timer is the resting Done screen; a workout started on the wrist then is a new one, not a continuation.
- A `.watch` draft restores with Health saving on, asks `recoverActiveWorkoutSession`, hands anything returned to the bridge, and otherwise keeps the timer and waits for the next packet.

9.7 Performance and cost

- **Packets are rate-limited on the watch.** `sendPacket(force: false)` sends at most once a second from the builder's statistics callback; a rep forces one immediately because a rep is what the person is watching.
- **No strap and watch at once.** Adopting a mirrored session stops the Bluetooth stream, so two feeds cannot double the effort accrual, and the radio is not held open for nothing.
- **Motion samples never leave the main actor.** `assumeIsolated` on the `.main` queue keeps 50 samples a second in order without 50 task hops.
- **The counter is O(1) per sample.** Two EMAs, one projection, a few comparisons. It keeps no history.
- **Motion stops on every end.** `end`, `discard`, a failed start and the delegate's failure path all call `stopMotion()`, so the accelerometer is not left running after a session.
- **One placeholder at a time.** A second session arriving before the recorder adopts the first cannot leave two cards on screen.
- **The hand-off costs nothing when there is no watch.** `watchAvailable` is three synchronous reads of `WCSession`; a person with no watch never waits.
- **Nothing leaves either device.** The mirrored session's data channel is HealthKit's own, device to device. No server, no push.

9.8 Edge cases and tests

CONDITION	BEHAVIOUR
Watch paired but does not answer in 10 s	"Apple Watch did not answer. Recording on iPhone." and the phone path runs
Health saving off	No hand-off; the phone records; no notice
Phone refuses Health authorisation before the hand-off	Hand-off skipped; the phone path runs and shows its own error
Phone already recording when a watch session arrives	Watch session discarded with a notice; the phone session continues
Packet with an unknown version	Ignored; nothing changes
Packet without a heart rate	Energy and counts apply; heart rate stays as it was; one refresh
Packet with heart rate and a rep change	Counts applied first, then the reading, so the one publish carries the new count
Watch packet reaches a phone session	Dropped by the <code>source == .watch</code> guard
Automatic rep after a manual +1	Summed, not overwritten
-1 with an unknown count	No change; a dash must not become a zero
-1 at zero	Stays at zero
Mirroring fails on the watch	Watch records and saves alone; screen says "Not connected to iPhone"
A throw after <code>startActivity</code> on the watch	Delegates detached, session ended, builder discarded, state back to idle; no late callback can revive it
Watch app relaunched over an active session	<code>recover()</code> adopts it; a stopped one is settled immediately; a paused one shows elapsed time, not zero
Phone relaunched during a watch session	Draft restores as <code>.watch</code> ; recovery is attempted; otherwise the timer waits for the next packet
Person discards on the phone	<code>discard</code> is sent, not <code>end</code> ; nothing reaches Health
Session ends while the bridge is dropping	<code>dropping</code> keeps <code>hasSession</code> false so the next start can hand off
Watch started while no phone scene exists	Placeholder Live Activity from the bridge; the timer adopts its id later

TESTS

- `RepCounterTests` (6): the five calibration cases plus reset.
- `WatchWireTests` (5): packet round trip keeps absence; unknown version ignored; command round trip; removeRep round trip; a command is never read as a packet.
- `LiveSessionReadoutTests` : reps round trip and format; the throttle publishes on a reps change.
`LiveEffortTests.builderCarriesRepsAndSet` . `WorkoutSetsTests` : sets round trip through `setsData` . `ActivitySessionStateTests.aTimerCanAdoptAnExistingIdentity` .
- `ActivityRecorderChecks` , the watch cases: a phone strength session starts at set 1 with zero reps; remove rep never goes below zero; manual reps and sets count; a watch packet never reaches a phone

session; finishing writes the sets; the wait says so; a watch that does not answer falls back; a seeded `.watch` draft restores as a watch session with Health saving on; an unknown-version packet changes nothing; a current-version packet carries reps and heart rate through; finishing a watch session writes the local record and never claims a Health save.

PENDING ON HARDWARE

Recorded in `docs/design/watch-companion/report.md` as unverified: (1) start from the phone and see the watch launch and the HUD show watch heart rate within 10 seconds; (2) start from the watch and see the phone's Live Activity appear; (3) ten slow bicep curls against the auto count; (4) pause on the watch and see the phone pause; (5) finish and find exactly one workout in Health. Plus two from the draft port: the `-1` correction landing on the number the other device shows, and whether watchOS actually calls `handleActiveWorkoutRecovery()` and the adopted session mirrors and saves. The package test count after this slice was 1247 in 163 suites; the checks page passed 29 of 29.

9.9 Study notes

READ IN THIS ORDER

1. `LifeOSKit/Sources/AppSurfaces/WatchWire.swift`, the two shapes and the gate.
2. `LifeOSKit/Sources/Motion/RepCounter.swift` with its tests open beside it.
3. `AlmanacWatch/WatchWorkoutController.swift`: `start`, `startMotion`, `addRep`, `removeRep`, `end`, `discard`, `recover`.
4. `LifeOS/App/Surfaces/WatchSessionBridge.swift`, particularly `watchAvailable`, `end(after:)` and the placeholder.
5. `LifeOS/Features/Activity/ViewModel/ActivityRecorder+Watch.swift`, top to bottom.
6. The `source == .watch` branch of `ActivityRecorder.attach`.
7. `AlmanacWatch/AlmanacWatchApp.swift` and `WatchAppDelegate.swift` for the before-or-after wiring.

CHECK YOURSELF

- **Why does `watchAvailable` not read `isReachable`?** On iOS it is true only while the watch app is in the foreground, which is never so when the person taps Begin on the phone; `startWatchApp` exists to launch it.
- **Why is the session dropped inside the send's completion handler?** Releasing it in the same run-loop turn can cancel the send before the watch hears it.
- **Why does the phone ask for Health authorisation before offering the watch?** Authorisation is shared; without it the first prompt lands on the wrist while the phone is counting down.
- **What does `kind` prevent?** A command decoding as an empty packet, since the two shapes share every required key.
- **Why are manual and automatic reps tracked separately?** So an automatic rep adds to the taps instead of overwriting them.

- **Why is `discard` a separate command from `end` ?** On the wrist `end` means finish and save; sending it for a refused session would write a stub into Health.

10 The workout library

A curated table of 122 verified YouTube workouts, a pure planner that turns preferences, today's battery and the week's splits into one session, a library that filters the catalog to it and says every time it had to widen, and a player that puts the glass rings over the video wherever the person drags them.

10.1 Intent

Almanac could show what the body did and how recovered it was, but never suggested a session and had no way to follow one. The request was plain: pull up workouts from YouTube matching what the person wants to do (strength, then push, pull, legs, arms, chest), coordinated with how much they can push today, with the live HUD on the video. Three constraints shaped everything that follows.

- **The app ships free.** No runtime AI calls, no per-user YouTube API quota. The catalog is a table the service role writes; the app reads it with the person's own token and caches it.
- **YouTube's terms.** Videos play in YouTube's own player and nothing may obscure its controls or branding. The rings default to the top leading corner, the region farthest from the control bar, and a drop onto that bar is moved off it.
- **A video the catalog cannot verify never ships.** Every id is checked against YouTube's keyless oEmbed endpoint at seed time, and the seed migration is generated, never hand-edited.

Open Health, Fitness, "Plan today's workout". The Today card already knows: "Pull day. You did push on Saturday, so today is pull. Battery 83% · up to zone 5. 30 min." Below it a rail of chips (Saved, then the splits with Pull already lit, then three lengths, then the equipment you said you own) and a list of videos with real thumbnails, the channel, the minutes, three intensity dots and the split. Search by title, channel or muscle. Tap the heart to keep one; hold a row to schedule it for a day, and a "Tue 15" chip appears on it. Tap a row and the video opens in YouTube's player; tap Start and the rings float over the picture. Drag them wherever you like, tap the corner button or turn the phone for full screen, and the same video keeps playing.

Why not let the AI plan the workout?

Cost and honesty. A pure function from four inputs is free, deterministic, and every rule in it is a test. The reason it gives ("You did push on Saturday, so today is pull.") is the rule itself, in words.

What happens offline?

The cache is the source the screen renders; the network only ever adds to it. A refresh that fails leaves the last catalog on screen and says "Showing the last catalog we downloaded." An empty cache with no network says "The library needs a connection the first time." and never a fake list. Thumbnails come from YouTube's image host and fall back to a deliberate tile.

Why does the Saved list ignore today's plan?

A heart is a decision the person already made. Intersecting saves with the plan hid a push video saved today from tomorrow's pull-day Saved list, while its heart was still filled in the full list. The Saved chip is a different list, not a narrower one: the saved rows, newest mark first, narrowed only by the search.

10.2 Files and folders

```

Features/Workouts/
├── Model/
│   └── VideoWorkoutLayout.swift           overlay inset, control strip, embed URL, id patter
├── View/
│   ├── WorkoutLibraryScreen.swift       Today card, chip rail, rows, schedule sheet
│   ├── WorkoutVideoRow.swift            thumbnail, title, channel · minutes, dots, chips,
│   ├── VideoWorkoutScreen.swift         portrait and full screen, rings over the player
│   ├── DraggableHUD.swift               drag, settle, remember per placement
│   ├── YouTubePlayerView.swift          WKWebView host and the wrapper page
│   └── TrainingPreferencesSheet.swift    goal, equipment, minutes
└── ViewModel/
    └── WorkoutLibraryViewModel.swift     cache, refresh, plan, filters, bookmarks

App/

```

└─ OrientationLock.swift	portrait by default; this screen asks for more
LifeOSKit/Sources/Persistence/	
└─ CatalogVideo.swift	the wire row, the cached @Model, CatalogStore.upse
└─ WorkoutBookmark.swift	saved · scheduledFor, one row per video
└─ UserGoals.swift	trainingGoal, equipmentRaw, sessionMinutes
└─ SourceRecords.swift	WorkoutRecord.split and .videoID
LifeOSKit/Sources/Sectors/	
└─ WorkoutPlanner.swift	the four rules and the rotation
└─ WorkoutLibraryFilter.swift	the plan's filter, the widening, the saved list
└─ CatalogRefresh.swift	when an empty answer may replace the cache
LifeOSKit/Sources/Integrations/	
└─ WorkoutCatalogClient.swift	one PostgREST select, read only
└─ CapacityInputs.swift	shared with the recorder (Chapter 8)
LifeOSKit/Sources/AppSurfaces/	
└─ HUDPlacement.swift	anchors, origins, settle
supabase/migrations/	
└─ 20260915090000_workout_videos.sql	the table, RLS, revoke then grant select
└─ 20260915090100_workout_videos_seed.sql	generated: 122 upserts and one trailing delete
scripts/catalog/	
└─ catalog.json	the curated rows; the only file a human edits
└─ verify.ts	oEmbed check per id, writes the seed
└─ verify_lib.ts	parse, match, SQL
└─ verify_lib_test.ts	7 Deno tests

10.3 Model

THE CATALOG

```
-- supabase/migrations/20260915090000_workout_videos.sql
create table public.workout_videos (
  youtube_id text primary key,
  title      text not null,
  channel    text not null,
  duration_s integer not null check (duration_s > 0),
  goal       text[] not null,      -- strength | hypertrophy | endurance | mobility
  split      text not null,      -- push | pull | legs | upper | lower | full | core |
  arms | chest | back | shoulders | mobility | cardio
  muscles    text[] not null default '{}',
  equipment  text[] not null default '{}', -- none | dumbbells | barbell | bands | machine
  kettlebell
  intensity  smallint not null check (intensity between 1 and 3),
  verified_at timestamptz not null,
  updated_at timestamptz not null default now()
);
alter table public.workout_videos enable row level security;
revoke all on public.workout_videos from anon, authenticated;
grant select on public.workout_videos to authenticated;
create policy "catalog is readable when signed in" on public.workout_videos
  for select to authenticated using (true);
```

Enums are text, as the notes table does, so a new split lands as data. The `revoke all` is not decoration: Supabase grants `all` on a new public table to `anon` and `authenticated` by default, so without it the sentence "signed-in users get read-only access" would be untrue at the grant layer. It also takes away `TRUNCATE`, which bypasses row-level security. The final review added it, matching the social groups table.

The seed today holds 122 rows: 22 cardio, 16 upper, 16 mobility, 15 legs, 8 each of push, pull and full, 7 lower, 5 core, 5 arms, 4 each of chest, back and shoulders. Seventy-five come from FitnessBlender, thirteen each from Sydney Cummings and Heather Robertson, five from Caroline Girvan, and the rest from a handful of others. Intensity skews moderate: 71 at 2, 40 at 1, 11 at 3. Thumbnails are not stored; the app derives `https://i.ytimg.com/vi/<id>/hqdefault.jpg`.

THE CACHE

```
// LifeOSKit/Sources/Persistence/CatalogVideo.swift
/// A cached catalog row. The server is the source of truth; a refresh
/// replaces the set, so a video pulled from the catalog disappears here.
@Model
public final class CatalogVideo {
    #Unique<CatalogVideo>([\.youtubeID])
    public var youtubeID: String
    public var title: String
    public var channel: String
    public var durationS: Int
    public var goal: [String]
    public var split: String
    public var muscles: [String]
    public var equipment: [String]
    public var intensity: Int
    public var verifiedAt: Date
    public var cachedAt: Date
    public var durationMinutes: Int { Int((Double(durationS) / 60).rounded()) }
}
```

`CatalogStore.upsert` updates matches, inserts new rows and deletes absent ones, so the cache mirrors the server after every successful refresh. It builds its lookup with `uniquingKeysWith` rather than `uniqueKeysWithValues`: the primary key makes a duplicate id impossible from the real table, and a trap on a malformed response is a crash where keeping the last row is not.

THE BOOKMARK

```
// LifeOSKit/Sources/Persistence/WorkoutBookmark.swift
/// A person's mark on a catalog video: saved for later, scheduled for a day,
/// or both. One row per video id; absent means neither.
@Model
public final class WorkoutBookmark {
    #Unique<WorkoutBookmark>([\.youtubeID])
    public var youtubeID: String
    public var saved: Bool
    /// A calendar day start, never a timestamp.
    public var scheduledFor: Date?
    public var updatedAt: Date
}
```

`BookmarkStore.toggleSaved` and `schedule` delete the row when it ends up neither saved nor scheduled, so the table never accumulates empty marks. Both read the row's fields before a possible delete, because a deleted model is not a thing to ask questions of. `schedule` normalises to `startOfDay` with the caller's calendar so a picker's time of day never decides which day a workout lands on.

PREFERENCES AND THE RECORD

`UserGoals` gained three columns: `trainingGoal: String?`, `equipmentRaw: [String] = []`, `sessionMinutes: Int?`. `WorkoutRecord` gained `split: String?` and `videoID: String?`, written when a session started from a video and `nil` for freeform sessions, and `setsData` from Chapter 9. Every one is optional or defaulted, for the reason in 10.8.

10.4 ViewModel

THE PLANNER

```
// LifeOSKit/Sources/Sectors/WorkoutPlanner.swift
public static func plan(goal: String?, sessionMinutes: Int?, batteryPercent: Int?,
    recentSplits: [(date: Date, split: String)], now: Date,
    calendar: Calendar = .current) -> TrainingPlan {
    let minutes = sessionMinutes ?? 30
    let recent = recentSplits.filter { $0.date >= weekAgo && $0.date <= now }.sorted {
        $0.date > $1.date }
    let cap: Int = { switch batteryPercent {
        case .some(let b) where b >= 67: return 3
        case .some(let b) where b >= 34: return 2
        case .some: return 1
        case .none: return 2 } }()

    if (batteryPercent ?? 100) <= 33 || (batteryPercent == nil && hardYesterday) {
        return TrainingPlan(split: "mobility", minutes: minutes, maxIntensity: 1,
            reason: "Recovery is low today, so this is a mobility day.")
    }
    switch goal {
    case "mobility": ...
    case "endurance": return TrainingPlan(split: "cardio", ..., reason: "Endurance today,
        paced by your battery.")
    case "strength", "hypertrophy":
        if let last = recent.first(where: { rotation.contains($0.split) }) {
            let next = rotation[(rotation.firstIndex(of: last.split)! + 1) % rotation.count]
            return TrainingPlan(split: next, ..., reason: "You did \(last.split) on \(
                weekday), so today is \(next).")
        }
        return TrainingPlan(split: "push", ..., reason: "First session this week: push.")
    default:
        return TrainingPlan(split: "full", ..., reason: "Set a goal for a plan built around
            you.")
    }
}
```

Four rules in order: a red or unknown-after-a-hard-day battery forces mobility at intensity 1; a mobility or endurance goal picks its own split, capped by battery; strength and hypertrophy rotate push, pull, legs off the most recent of those three within seven days; no goal means full body. The weekday in the reason is looked up on an `en_US_POSIX` copy of the calendar, because some process environments resolve the locale to abbreviated names and the reason string should be deterministic. The battery cap on intensity is the same

band table as the effort ceiling in Chapter 8, so "up to zone 4" on the Today card and the yellow ring on the HUD are the same fact.

THE FILTER

```
// LifeOSKit/Sources/Sectors/WorkoutLibraryFilter.swift
public static func apply(videos: [LibraryVideo], plan: TrainingPlan, goal: String?, split:
String?,
                        band: DurationBand?, equipment: Set<String>, query: String = "") ->
(rows: [LibraryVideo], note: String?) {
    let chosenSplit = split ?? plan.split
    // The intensity cap and the equipment the person owns are never
    // widened away: a battery ceiling and a kit list are facts, not
    // preferences to relax when the list comes back short.
    let base = videos.filter { video in
        video.intensity <= plan.maxIntensity
        && (equipment.isEmpty || !Set(video.equipment).isDisjoint(with: equipment))
        && (normalizedQuery.isEmpty || matches(video, query: normalizedQuery))
    }
    var note: String?
    var result = base.filter { $0.split == chosenSplit && inBand($0) }
    if result.isEmpty {
        result = base.filter { $0.split == chosenSplit }
        if !result.isEmpty { note = "Widened to any length." }
    }
    // Only when the split is the plan's own. A chip the person tapped is
    // an instruction, and answering it with another split while the chip
    // still reads selected would be the screen contradicting itself.
    if result.isEmpty, split == nil {
        result = base.filter { video in goal.map { video.goal.contains($0) } ?? true }
        if !result.isEmpty { note = "Widened to any split for your goal." }
    }
    let rows = result.sorted { closeness to plan.minutes, then title }
    return (rows, note)
}
```

The plan first: its split, its intensity cap, its length give or take fifteen minutes. An empty result widens one step at a time and says which step it took, so a short list is never a mystery. Three things never widen: the intensity cap, the equipment, and the search query. The rows are ordered by closeness to the requested length, then title, so a 30-minute plan lists the 28-minute video before the 45.

THE VIEW MODEL

```
// LIfеOS/Features/Workouts/ViewModel/WorkoutLibraryViewModel.swift
func load() {
    guard let context else { return }
    let store = MetricsStore(context: context, calendar: calendar)
    let today = now()
    goals = try? store.goals()
    videos = (try? CatalogStore.all(context: context)) ?? []
    loadBookmarks()
    capacity = CapacityInputs.todayCapacity(store: store, now: today, calendar: calendar)

    let weekAgo = calendar.date(byAdding: .day, value: -7, to: today) ?? today
    let recent: [(date: Date, split: String)] = ((try? store.workouts(from: weekAgo, to:
today)) ?? [])
        .compactMap { record in record.split.map { (date: record.start, split: $0) } }

    plan = WorkoutPlanner.plan(goal: goals?.trainingGoal, sessionMinutes:
goals?.sessionMinutes,
                                batteryPercent: capacity?.percent, recentSplits: recent,
                                now: today, calendar: calendar)

    applyFilters()
}
```

Everything the screen needs is read in one pass: preferences, the cache, today's battery, and the week of splits the rotation turns on. The clock and the calendar are injected so tests can fix a day. Every filter property (`splitFilter` , `durationBand` , `equipmentFilter` , `query` , `savedOnly`) re-runs `applyFilters()` on set, which maps the cached models into `LibraryVideo` values, calls the pure filter, and maps the chosen ids back.


```
// LIfeOS/Features/Workouts/ViewModel/WorkoutLibraryViewModel.swift
func refreshIfDue(force: Bool = false) async {
    guard let context, !isRefreshing else { return }
    let newest = videos.map(\.cachedAt).max()
    let due = force || newest.map { now().timeIntervalSince($0) > Self.refreshInterval } ??
true
    guard due else { return }
    guard let baseURL = AppConfig.supabaseURL, let anonKey = AppConfig.supabaseAnonKey else {
noteOffline(); return }
    guard let token = sessions.load()?.accessToken else { noteSignedOut(); return }
    isRefreshing = true
    defer { isRefreshing = false }
    do {
        let rows = try await WorkoutCatalogClient(baseURL: baseURL, anonKey:
anonKey).videos(accessToken: token)
        guard CatalogRefresh.shouldReplace(cacheCount: videos.count, incoming: rows.count)
    else {
        refreshNotice = "The catalog came back empty. Showing the last one we
downloaded."
        applyFilters()
        return
    }
    try CatalogStore.upsert(rows, context: context, now: now())
    refreshNotice = nil
    load()
} catch {
    noteOffline()
}
}
```

The refresh runs when the newest cached row is more than a day old, or on pull to refresh. A signed-out person is told to sign in rather than to find signal, because the catalog is read with their own token. And a 200 with no rows is refused by `CatalogRefresh.shouldReplace`: an unseeded table, a policy that stopped matching, or a range quirk all look like an empty array at the client, and replacing a full cache with that would leave a person who had the whole library a second ago with nothing.

Bookmarks are refetched after every change rather than mutated in place, because `toggleSaved` and `schedule` can delete a row, and a deleted model left in the dictionary would be read by the next row that asks. `pruneStaleSchedules` runs on every load and clears any day that has passed, keeping the save, so a row never carries a stale chip and the schedule sheet never opens on a day its own picker will not accept.

10.5 View

THE LIBRARY

`WorkoutLibraryScreen` is pushed on whichever stack the person is already on (the Fitness segment's training card, or Begin Activity's "Follow a video"), so the way back is the way they came. It sits on a `GradientCanvas` in the activity hue with the Today card, a status line when the list is non-empty, the chip rail, then the rows or an empty state. The search field is in the navigation bar drawer, always visible, covering no

row. The preferences sheet is raised once, decided in `attach` rather than in every `load`, because deciding it in `load` put the sheet back up the moment a Health sync ran.

Rows are cards in a `LazyVStack`, so scheduling is a context menu rather than a swipe: only a `List` row can be swiped. The heart is a sibling of the opening button, never inside it, because SwiftUI gives a button nested in a button no defined semantics and saving a video would also open it. The channel and the minutes are two `Text`s, not one: a long channel name used to push the minutes off the end of the line, and the minutes are the number a person scans for. The channel truncates; the minutes never do. A scheduled row shows its day chip in place of the equipment chips, because four chips beside a thumbnail truncate to "T..." and "Dum...".

THE PLAYER

```
// LifeOS/Features/Workouts/View/YouTubePlayerView.swift
/// The web view behind the player, owned by the screen rather than the view.
///
/// Portrait, full screen and landscape are different layouts of the same
/// player, and SwiftUI gives a view in a different branch a different
/// identity, which would mean a new web view and a video starting over every
/// time the phone turned or the full screen button was tapped.
@MainActor
final class YouTubePlayerHost {
    let webView: WKWebView
    fileprivate var loaded: String?
    init() {
        let configuration = WKWebViewConfiguration()
        configuration.allowsInlineMediaPlayback = true
        configuration.mediaTypesRequiringUserActionForPlayback = []
        webView = WKWebView(frame: .zero, configuration: configuration)
    }
}
```

`VideoWorkoutScreen` holds one `YouTubePlayerHost` in its state and shows the same web view in three layouts. Portrait: the player at 16:9 with the rings floating over it and a full-screen button in the corner; "Open in YouTube" (always present, not only after a failure, because a video the owner barred from embedding says so inside the player where nothing this screen can see reaches); the title, channel, minutes and chips; then either "Start this workout" or the elapsed clock, the rep buttons and `ActivityControls`. Full screen, entered by the button or by turning the phone: black, the player letterboxed or filling, and one overlay holding the rings, a Start pill before the session, and two glass buttons (session controls in a sheet, and exit).

```
// LifeOS/Features/Workouts/Model/VideoWorkoutLayout.swift
static let embedOrigin = "https://www.youtube-nocookie.com"
static let idPattern = /^[A-Za-z0-9_-]{11}$/

static func embedURL(youtubeID: String) -> URL? {
    guard youtubeID.wholeMatch(of: idPattern) != nil else { return nil }
    // `fs=0` removes YouTube's own full screen button. That button hands
    // the video to a system layer above the whole app, where no HUD can
    // be drawn; the screen offers its own full screen instead.
    return URL(string: "https://www.youtube-nocookie.com/embed/\(youtubeID)"
        + "?origin=\(embedOrigin)&enablejsapi=1&playsinline=1&rel=0&modestbranding=1&fs=0")
}
```

The id pattern is checked before anything is interpolated into a URL or the wrapper page, and a row that fails shows "This video will not play here." There is no JavaScript bridge: the person taps YouTube's play button, the one gesture the embed needs, and the recorder times the session rather than trying to follow the video's clock.

THE DRAGGABLE HUD

```
// LifeOSKit/Sources/AppSurfaces/HUDPlacement.swift
/// (0, 0) is the top leading corner inside the inset; (1, 1) the bottom
/// trailing one.
public struct Anchor: Equatable, Sendable {
    public var x: Double
    public var y: Double
    public static let topLeading = Anchor(x: 0, y: 0)
    public var stored: String { "\(x), \(y)" }
}

/// Where a dropped HUD settles: inside the container, and off the strip
/// it is not allowed to cover, on whichever side of it is nearer and fits.
public static func settle(origin: CGPoint, hud: CGSize, container: CGSize, inset: CGFloat,
    avoiding strip: CGRect?) -> CGPoint
```

A position is kept as an anchor, a fraction of the free travel in each axis, rather than a point. The same anchor puts the rings in the same relative place over a small player in portrait, a letterboxed one in full screen and the whole screen in landscape, and it stays meaningful when the HUD collapses to the bar. `DraggableHUD` stores one anchor per placement ("compact", "fullscreen", "landscape") in `AppStorage`. The rings follow the finger freely while dragging; the settle rule is applied once, on release, so it reads as a snap rather than a wall. The strip it avoids is the bottom 56 points of the player, where YouTube draws its control bar and logo. Four accessibility actions move it to each corner. It renders at zero opacity until measured, because a HUD drawn at the inset for one frame and then jumping to its saved spot reads as a glitch on every appearance.

ORIENTATION

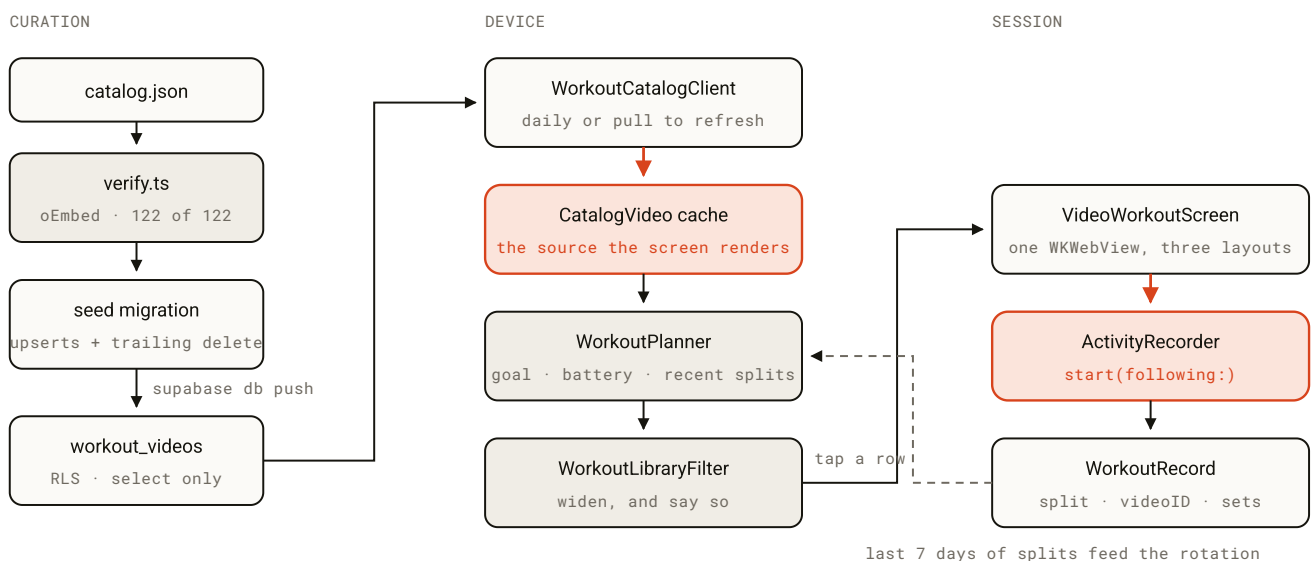
```
// LIfiOS/App/OrientationLock.swift
@MainActor
enum OrientationLock {
    static let standard: UIInterfaceOrientationMask =
        UIDevice.current.userInterfaceIdiom == .pad ? .all : .portrait
    private(set) static var mask = standard

    static func allow(_ allowed: UIInterfaceOrientationMask) { ... }

    /// Back to the default, and back upright: a phone still held sideways
    /// would otherwise stay in a landscape the next screen cannot draw.
    static func release() {
        allow(standard)
        guard standard == .portrait else { return }
        for scene in UIApplication.shared.connectedScenes.compactMap({ $0 as? UIWindowScene
    }) {
            scene.requestGeometryUpdate(.iOS(interfaceOrientations: .portrait))
        }
    }
}
```

The iPhone target now lists landscape in its build settings so the player can turn, but almost nothing else in the app is laid out for it. So the app delegate answers every orientation query from `OrientationLock.mask`, which defaults to portrait on a phone; the player screen asks for `.allButUpsideDown` in `onAppear` and hands it back in `onDisappear`, forcing the scene upright so the next screen is not asked to draw a landscape it has no layout for. iPad keeps every orientation, as it always has.

10.6 How it works



Curation is a script and a migration; the device caches, plans and filters; the session is the Chapter 8 recorder with a video stamped on it. The saved record feeds tomorrow's plan.

STEP BY STEP: FROM CURATION TO A SAVED SESSION

1. A human edits `scripts/catalog/catalog.json`. `verify.ts` parses it (ids match the eleven-character pattern, goals and splits and equipment are from the fixed vocabularies, intensity is 1 to 3, no duplicate ids), then calls YouTube's oEmbed endpoint for every id and fails the run if any returns an error, a title that does not start with the row's title, a channel that differs beyond case and spacing, or a duration more than 10 percent off a length stated in the title.
2. It writes the seed: one upsert per row and one trailing `delete ... where youtube_id not in (...)` naming every verified id, so a seed run is a full sync and a row dropped from the JSON leaves the server on the next push. An empty catalog is refused before a single network call, because the trailing delete would make it a migration that wipes the table. After the first push the file name is frozen, so later passes use `--out` to write a new timestamped migration.
3. The library opens. `attach` loads the cache, the preferences, today's capacity and the week's splits, plans, filters, and raises the preferences sheet once if no goal is set. `.task` calls `refreshIfDue()`.
4. If the newest cached row is older than a day, one PostgREST select runs with the person's token. Rows replace the cache unless the answer is empty and the cache is not.
5. Tapping a row pushes `VideoWorkoutScreen`. The web view loads a wrapper page, based at the privacy-enhanced host, containing an iframe of the embed URL.
6. Start maps the split to an activity (cardio to `.other`, mobility to `.yoga`, everything else to `.strength`) and calls `recorder.start(following:)` with the video's id, split, title and channel. The video belongs to *this* start: a start that bails leaves nothing behind for the next freeform walk.
7. The rings appear over the player through `DraggableHUD`. The session runs exactly as in Chapters 8 and 9.
8. Finish writes the record with `videoID`, `split` and `sets`. The Fitness segment's row shows the split chip; tomorrow's planner reads it for the rotation.

THE EMBED THAT WOULD NOT PLAY

On the first spot-check zero of six seed videos rendered; every one gave YouTube's "Error 152" card. Loading the embed URL straight into the web view gave the harder error 153; wrapping it in an iframe on a page of our own moved every id to 152; the fix was matching the query's `origin` parameter to the page's own `baseUrl` and pointing both at `youtube-nocookie.com`. Six of six then rendered. The report records the elimination.

10.7 Performance and cost

- **Zero per-user API cost.** The catalog is one table read with the anon key and the person's JWT. No YouTube Data API, no quota, no key. Thumbnails are static images from YouTube's CDN.
- **One fetch a day.** `refreshInterval` is 86,400 seconds; a refresh is otherwise only on pull. The library works entirely from the cache after the first download.
- **Curation is free at runtime.** Verification runs on a laptop before a migration; the app never verifies anything.
- **The planner and filter are pure and cheap.** One pass over at most 122 rows per filter change, with the cached models mapped to value types once per pass. No index is needed at this size.

- **One web view.** The player host lives in the screen's state, so rotation and full screen never reload the video or lose the person's place.
- **Bookmarks are one dictionary.** A row asks a dictionary, not the store; the dictionary is rebuilt after a write, when the store is the truth.
- **Capacity is shared, not recomputed differently.** `CapacityInputs.todayCapacity` serves the recorder and the planner, so the battery is read once the same way.
- **The HUD's settle is pure.** Corner cases (a HUD wider than its player, a drop onto the control strip) are tests rather than thumbs on a device.
- **What leaves the device.** One authenticated read of the catalog; image requests to `i.ytimg.com`; the embed to `youtube-nocookie.com`, the privacy-enhanced host. Bookmarks, schedules and preferences never leave.

10.8 Edge cases and tests

A FIELD ADDED TO AN EXISTING @MODEL MUST BE OPTIONAL OR DEFAULTED

Commit 2d01bd8 on 15 September: the goals row gained `equipmentRaw: [String]`, a mandatory array with no default. SwiftData refused to migrate any store written before it ("Validation error missing attribute values on mandatory destination attribute"). The container never opened, the shell had no account to show, and the app sat on the signed-in spinner. The fix is `= []`; the guard is `SchemaMigrationTests.fieldsAddedToTheGoalsRowHaveDefaults`, which walks the entity's attributes and asserts each added one is optional or has a default. Every column added since (bookmarks, `split`, `videoID`, `setsData`, the extras and sources bags) follows the rule.

CONDITION	BEHAVIOUR
First open, no network	"The library needs a connection the first time." with a wifi.slash glyph; never a fake list
Later open, no network	Last catalog shown; "Showing the last catalog we downloaded."
Signed out	"Sign in to download the library." rather than a connection message
Server answers 200 with zero rows	Cache kept; "The catalog came back empty. Showing the last one we downloaded."
Server answers zero rows to an empty cache	Replaced, because there is nothing to lose
No goal set	Preferences sheet once; plan is "full" with "Set a goal for a plan built around you."
Battery 0 to 33	Mobility at intensity 1 regardless of goal
Battery unknown after a hard day	Mobility; unknown otherwise caps intensity at 2
No video at the plan's length	Widened to any length, and the list says so
No video at the plan's split	Widened to any split for the goal, only when no split chip is tapped
A split chip tapped and nothing matches	Empty state; the chip is an instruction and is not widened away
Widened list while offline	Both lines shown, one under the other
Saved chip with nothing saved	"Nothing saved yet. Tap the heart on a workout to keep it here."
A schedule for a day that has passed	Cleared on load; the save is kept
Schedule sheet seeded with a past day	Clamped to today, the picker's floor
Unsave a scheduled row	Row kept for its schedule; unschedule an unsaved row deletes it
A row whose id is not a YouTube id	Nothing interpolated; "This video will not play here."
Owner disabled embedding	YouTube's own card inside the player, with its own link; the app's "Open in YouTube" is always present as a precaution
Wrapper page fails to load	<code>onLoadFailure</code> shows the fallback line
Start bails (Health refused, watch took the session)	The video is not left stamped on the next session
Relaunch mid-video	The draft carries the video; the record the restored session writes still names it
Session started on the wrist while the player is idle	<code>clearPendingVideoIfIdle</code> drops the video; a wrist workout is not the queued video
HUD dropped onto the control strip	Moved to whichever side is nearer and fits
HUD wider than its player	Sits at the inset
Phone turned on any other screen	Stays portrait; only the player asks for landscape
Leaving the player sideways	<code>release()</code> forces the scene upright

TESTS

- `WorkoutPlannerTests` (6): red battery is mobility; strength rotates after the most recent split; the first session of the week is push; endurance and mobility goals; unknown goal is full body; unknown battery after a hard day is mobility.
- `WorkoutLibraryFilterTests` (9): widens to any length then any split; keeps the intensity cap while widening; orders by closeness then title; an explicit split chip is not widened away; equipment and band narrow without widening; search matches title, channel or muscle and survives widening; the saved list ignores the plan but not the query; an empty query changes nothing.
- `CatalogRefreshTests` (3), `CatalogVideoTests` (2: upsert is idempotent and removes missing rows; goals and workout columns default to absent), `WorkoutBookmarkTests` (4), `WorkoutCatalogClientTests` (2: the request carries both keys and selects everything; arrays decode and an unknown split is kept as text), `HUDPlacementTests` (9), `SchemaMigrationTests` (1).
- `scripts/catalog/verify_lib_test.ts` : 7 Deno tests for parsing, title and channel matching, duration plausibility, the SQL, and the empty-catalog refusal.
- `ActivityRecorderChecks` : finishing stamps the video and split; a failed start does not stamp the next session; a wrist-started session carries no stale video; a restored draft keeps its video.

The QA report records 1266 package tests in 168 suites after this slice, 34 of 34 recorder checks, the library captured on iPhone in both schemes and on iPad, the empty state, the schedule sheet, and the player before and during a session. The landscape capture is recorded as pending: at the time of the report the iPhone was still portrait-only, and the rotation the player now asks for arrived with the full-screen commit the following day.

10.9 Study notes

READ IN THIS ORDER

1. `scripts/catalog/verify_lib.ts` , then `verify.ts` : what a row must be before it can ship.
2. `supabase/migrations/20260915090000_workout_videos.sql` : the table and why the revoke comes before the grant.
3. `LifeOSKit/Sources/Sectors/WorkoutPlanner.swift` with its tests open.
4. `LifeOSKit/Sources/Sectors/WorkoutLibraryFilter.swift` , the widening and the saved list.
5. `LifeOS/Features/Workouts/ViewModel/WorkoutLibraryViewModel.swift` : `load` , `refreshIfDue` , `applyFilters` , `loadBookmarks` .
6. `LifeOS/Features/Workouts/View/YouTubePlayerView.swift` and `VideoWorkoutLayout.swift` , the origin fix.
7. `LifeOSKit/Sources/AppSurfaces/HUDPlacement.swift` , then `DraggableHUD.swift` , then `OrientationLock.swift` .

CHECK YOURSELF

- **Why does the seed end with a delete?** So a seed run is a full sync; a row removed from the JSON leaves the server on the next push instead of living forever.

- **Why is an empty 200 not believed?** An unseeded table or a policy that stopped matching looks the same as an empty catalog, and replacing a full cache with it strands the person.
- **Which three filters are never widened, and why?** Intensity cap, equipment, search: a battery ceiling and a kit list are facts, and a query is what the person is typing.
- **Why is the web view held in the screen's state?** Each layout branch would otherwise get a fresh view identity and restart the video on every rotation.
- **Why is the HUD position an anchor and not a point?** A fraction of free travel survives a size change: the small player, full screen, landscape, and the collapse to a bar.
- **What did commit 2d01bd8 teach?** A mandatory field added to a shipped model with no default stops the store from opening at all, and now a test guards it.

11 Money

Real transactions from a real bank, exact to the cent, on six pastel bands. The device holds the ledger, the server holds only the credential, and every figure on the tab is derived from rows that actually happened.

11.1 Intent

The Money tab answers six questions, one per side tab: what came in and went out, where it went, what repeats every month, what is being saved towards, what hurts, and the ledger itself. It reads from a bank connected through Plaid, or from transactions typed in by hand, and it never invents a row. The sample month that used to stand in before a bank was linked is gone. An empty month shows the empty state.

WORKING BACKWARDS

Connect a bank once and Almanac keeps the last two years of transactions on the phone. The net for the month is the first thing on the screen with a one-word verdict under it. Budgets are buckets that claim categories, so a renamed category never silently stops counting. Tap any category, any merchant, or any row to see six months of it on a page of its own.

Why is the savings rate sometimes blank rather than 0%?

A savings rate against no income is undefined, not zero. Rendering 0% says you saved nothing when what happened is that nothing was earned. The whole app draws that distinction and this is its sharpest case.

Where do my transactions live?

On the device, in SwiftData. The Edge Functions hand each sync delta straight back to the phone and store nothing but the bank credential. A compromise of the database exposes a token, not a ledger of where you shop.

Why can Plaid never be called from the app?

Every Plaid endpoint needs a client id and a secret on the request, and an .ipa is a zip file. Any secret compiled into the binary is public the moment it ships, so the four Plaid calls run in Edge Functions and the app talks only to those, authenticated with its own Supabase session.

What happens to a pending charge?

Stored and shown, greyed, but excluded from every rollup. Plaid re-issues a pending transaction under a new id when it settles and sends the old id in `removed`, so counting it twice is the default failure and the rollup refusing to see pending rows is what prevents it.

11.2 Files and folders

Features/Money/	
└─ Model/	
└─┬─ MoneySnapshot.swift	the month: totals, bands, rows, charts, verdict
└─└─ MoneyDetailSnapshot.swift	one category or merchant across six months
└─ ViewModel/	
└─┬─ MoneyViewModel.swift	month fetch, rollup, buckets, logos, series
└─└─ MoneyDetailViewModel.swift	the drill-down page's own six-month fetch
└─ View/	
└─┬─ MoneyScreen.swift	the tab: rail plus one section, empty and fetching states
└─└─ MoneySectionRail.swift	six sections; a vertical rail on phone, chips on iPad
└─┬─ MoneySections.swift	MoneyBand, MoneyCard, MoneyBandStack, the six panels
└─└─ MoneyCharts.swift	week bars, donut, six-month bars (Swift Charts)
└─┬─ MoneyDetailScreen.swift	the drill-down page
└─└─ MerchantTile.swift	logo circle, transaction row, day cards
└─┬─ BucketEditorSheet.swift	budgets list and one bucket's form
└─└─ MoneyPalette.swift	paper, ink, six pastels, MoneyFigure, Pinstripes
Features/Settings/	
└─ ViewModel/PlaidConnectionViewModel.swift	connect, sync, replay, disconnect
└─ View/PlaidLinkPresenter.swift	the only file that imports LinkKit
LifeOSKit/Sources/Persistence/	
└─ MoneyEntry.swift	MoneyEntry, MoneyAccount, MoneySummary, summarise()
└─ MoneyStore.swift	queries, batched upsert ingest, bucket claims
└─ MoneyParts.swift	whole and cents, rounded once
└─ SpendBucket.swift	a budget and the claim keys it holds
LifeOSKit/Sources/Sectors/	
└─ BudgetPeriod.swift	the pure budget assessment (shared with the close)
LifeOSKit/Sources/Insights/	
└─ SpendSeries.swift	week, months, fold for the donut
└─ RecurringSpend.swift	subscriptions detected from history
LifeOSKit/Sources/Integrations/	
└─ PlaidWireFormat.swift	the sync response, decoded
└─ PlaidMapping.swift	Plaid vocabulary to ours, including the sign flip
└─ PlaidCategory.swift	primary codes to readable labels
└─ PlaidClient.swift	the four Edge Function calls
└─ PlaidItemStore.swift	connected banks and their cursors, in UserDefaults
└─ PlaidSync.swift	store the data, then move the cursor
supabase/	
└─ functions/_shared/plaid.ts	callPlaid, error classification
└─ functions/plaid-link-token	/link/token/create
└─ functions/plaid-exchange	/item/public_token/exchange, duplicate guard
└─ functions/plaid-sync	/transactions/sync, three pages a call
└─ functions/plaid-disconnect	/item/remove
└─ migrations/20260826120000_plaid_items.sql	
web/	
└─ plaid-oauth.html	where a bank's OAuth login lands
└─ .well-known/apple-app-site-association	

The arithmetic is in the package, not in the feature. `summarise`, `BudgetPeriod.assess`, `SpendSeries`, `RecurringSpend` and `MoneyParts` are all free functions or value types with no store and no context, which is why the money rules are among the best-tested code in the app. The app target still has no test target of its own, so anything that decides something is pushed down to where it can be tested.

11.3 Model

MONEYENTRY, SHAPED FOR PLAID FROM THE FIRST LINE

```
// LifeOSKit/Sources/Persistence/MoneyEntry.swift
@Model
public final class MoneyEntry {
    public var id: UUID
    public var externalID: String? // Plaid transaction_id, nil for manual
    public var sourceRaw: String
    public var date: Date // always startOfDay
    public var amount: Double // positive = in, negative = out
    public var currencyCode: String
    public var merchant: String
    public var category: String? // display text, never keys arithmetic
    public var categoryCode: String? // Plaid personal_finance_category.detailed
    public var merchantID: String? // Plaid merchant_entity_id
    public var logoURL: String?
    public var accountID: String?
    public var accountName: String?
    public var pending: Bool
    ...
    public var isSpending: Bool {
        amount < 0 && !pending && !MoneyCategoryRule.isTransferLike(categoryCode)
    }
}
```

Three pairs of fields look redundant and are not. `category` is what the row shows; `categoryCode` is what the rollup and the buckets key on, so renaming a label can never change a savings rate. `merchant` is the descriptor the bank sent; `merchantID` is Plaid's stable entity, so `NETFLIX.COM` and `Netflix` are one merchant where the id exists and two strings where it does not. And `MoneyAccount` keeps both `type` and `subtype`, because "is this money owed" reads the first and "is this a credit card" reads the second, and a mortgage is a loan with a balance and no meaningful utilisation.

Every field added since the first edition (`merchantID`, `logoURL`, `subtype`, `mask`, `availableBalance`, `creditLimit`) is optional. That is what makes each one a lightweight SwiftData migration: a row written before the field existed reads back nil, and the app never hangs on the launch spinner the way it does when a new field lacks a default.

SIGN CONVENTION: POSITIVE IS MONEY IN

Plaid's `amount` is positive when money leaves the account. This app's is the opposite.

`PlaidMapping` negates on the way in, and the rule is written on the type, on the mapping, and on the scorer, because a silent flip turns a salary into the month's largest expense and nothing crashes.

THE ROLLUP

```
// LifeOSKit/Sources/Persistence/MoneyEntry.swift
public func summarise(entries: [MoneyEntry], accounts: [MoneyAccount] = []) -> MoneySummary {
    var income = 0.0
    var expenses = 0.0
    for entry in entries where !entry.pending
        && !MoneyCategoryRule.isTransferLike(entry.categoryCode) {
        if entry.amount > 0 { income += entry.amount } else if entry.isSpending { expenses +=
-entry.amount }
    }
    return MoneySummary(
        income: income,
        expenses: expenses,
        netWorth: accounts.isEmpty ? nil : accounts.reduce(0) { $0 + $1.netWorthContribution
    }
    )
}
```

Transfers between your own accounts and credit card payments appear as a real debit and a real credit. Counting them leaves net correct but overstates both income and expenses, and the savings rate is computed from those two. `MoneyCategoryRule.isTransferLike` matches `TRANSFER_IN_` and `TRANSFER_OUT_` as underscore-delimited prefixes plus the card-payment code, and nothing else. The underscore matters: without it a code that merely starts with the same letters would be excluded as if it were a transfer, which is a test (`aCategoryThatMerelyStartsLikeATransferIsStillSpending`) rather than a hope.

MONEYPARTS, OR WHY CENTS ARE SPLIT IN THE KIT

```
// LifeOSKit/Sources/Persistence/MoneyParts.swift
public init(_ amount: Double) {
    // Round in the unit that has to carry: cents, as an integer.
    let totalCents = Int((abs(amount) * 100).rounded())
    self.isNegative = amount < 0 && totalCents > 0
    self.whole = totalCents / 100
    self.cents = totalCents % 100
}
```

The one rule is that rounding happens to the cent before anything is split. Splitting a `Double` first and rounding each half lets 19.995 render as "\$19.100", and lets the float noise in `0.1 + 0.2` leak into the cents. Money that displays as the wrong number is the worst bug a finance screen can have, so the split lives where it is tested against 19.995, 999.999, thirty 19.99s, negatives and zero, and `MoneyFigure` in the view only draws what it is handed.

SPENDBUCKET AND THE CLAIM KEY

```
// LifeOSKit/Sources/Persistence/SpendBucket.swift
@Model
public final class SpendBucket {
    public var name: String
    public var monthlyLimit: Double    // always positive; the store rejects anything else
    public var claimedRaw: [String]    // MoneyEntry.claimKey values
    public var sortOrder: Int
    ...
}

extension MoneyEntry {
    public var claimKey: String? { categoryCode ?? category }
}
```

A budget is a bucket that claims one or more raw categories. This survives Plaid renaming a category, works for manual entries (which have no code and fall back to their free-form category), and lets one budget cover what Plaid calls three different things. The rejected alternatives, raw strings and merchant rules, both fail silently: a renamed category simply stops counting and the budget quietly reads as under. A budget that silently stops counting is worse than no budget.

THE SNAPSHOT

`MoneySnapshot` carries everything the six sections render: the summary figures, the budget and unclaimed bands, categories with a precomputed `share`, recurring charges, the seven-day `week`, the donut `slices`, a `goal`, and four connection flags. Two of those flags exist because a bank linked seconds ago has no transactions yet: `hasConnectedBank` keeps the screen out of the empty state, and `isFetchingHistory` keeps it out of "you spent nothing this month" while Plaid is still assembling the history.

11.4 ViewModel

`MoneyViewModel.load(connection:)` fetches the calendar month to date plus the accounts, calls `summarise`, assesses the buckets through `BudgetPeriod`, and then derives every list and series from the same entries. It is called from `RootView.reloadAll()` on every store save in the app, and again after the Plaid sync that `reloadAll` kicks off.

```
// LifeOS/Features/Money/ViewModel/MoneyViewModel.swift
let logos = Self.logoMap(from: entries)
let rows = Self.rows(from: entries, logos: logos)
let categories = Self.categories(from: entries, expenses: summary.expenses)

snapshot = MoneySnapshot(
    income: summary.income, expenses: summary.expenses, net: summary.net,
    savingsRate: summary.savingsRate, netWorth: summary.netWorth,
    recent: rows, budgets: budgetRows, unclaimed: unclaimedRows,
    categories: categories,
    recurring: Self.recurring(from: entries, calendar: calendar, logos: logos),
    week: Self.week(from: entries, now: .now, calendar: calendar),
    slices: Self.slices(from: categories),
    spendCount: entries.filter(\.isSpending).count,
    goal: Self.goal(saved: summary.net),
    monthLabel: Date.now.formatted(.dateTime.month(.wide).year()),
    isConnected: !entries.isEmpty || !accounts.isEmpty || !bankNames.isEmpty,
    hasConnectedBank: !bankNames.isEmpty,
    isFetchingHistory: connection?.isFetchingHistory ?? false,
    reconnectPrompt: reconnect,
    lastSyncedAt: connection?.lastSyncedAt
)
```

Four derivations are worth reading closely.

- **Categories** are grouped from `isSpending` rows only, the same predicate `summarise` uses, so the list sums to the "Spent" figure above it. Uncategorised spend is kept rather than dropped, because the gap is exactly the spend nobody labelled.
- **Logos** are mapped twice, by `merchantID` where Plaid gave one and by lowercased merchant name always. A row Plaid sent without a logo borrows from any sibling that has one, and a manual "Netflix" picks up the mark from the Plaid rows. A borrowed logo never changes grouping. It is a picture beside a name, not an identity.
- **Slices** fold the categories to the five largest plus "Other" through `SpendSeries.fold`, then assign each an opacity step. Shares are of the full month, so the slices still sum to one.
- **The goal** reads a target and a name from the account's `UserDefaults` suite and reports this month's net as progress, clamped at zero. There is no account history to total, so a lifetime figure would be invented. As of this revision nothing in the app writes those two keys: the "Set a goal" button opens the bucket editor, so the goal band always shows "No goal set" until a setter is built.

Bucket editing goes through the store. `saveBucket` creates or updates, then reconciles claims by unclaiming what was dropped and claiming what was added. The store owns the single-holder rule: `claim` removes the key from whichever bucket held it and returns that bucket's name so the sheet can say "Held by Groceries, saving moves it here". Nothing resolves collisions at read time, because by then the same transaction has already counted twice.

`MoneyDetailViewModel` is a second view model rather than more fields on the first. Six months of rows for a page nobody is looking at, re-run on every save in the app, is the kind of work that makes a tab feel slow. `RootView` reloads it only while its page is up.

11.5 View

The Money tab is the one screen in the app that is not cards on the warm canvas. It is full-bleed pastel bands on pure white, because money is mostly figures and a figure wants a block of colour behind it. `MoneyPalette` is therefore its own palette, not `LifeOSTokens`: paper, one near-black ink, and six pastels (sage, butter, clay, mint, mist, stone) stepped a little darker and more saturated than the reference screens because the paler versions washed out to grey on a real phone at real brightness.

THE RAIL, TWO WAYS

On a phone the app's own nav bar is a floating pill along the bottom, so the side is free and `MoneySectionRail` is a 62-point vertical rail with a glyph and a short word per section. On iPad the nav bar stands on end against the left edge, and a second vertical rail beside it would be two parallel bars of chrome doing unrelated jobs. So on a regular width the rail lies down into a row of chips above the content, carrying the full titles ("Where it went" rather than "Spend") because a horizontal chip has the room. One `matchedGeometryEffect` id makes the selection block slide between sections instead of cross-fading.

BANDS, CARDS AND THE GRID

```
// LifeOS/Features/Money/View/MoneySections.swift
struct MoneyBandStack<Content: View>: View {
    @ViewBuilder let content: Content
    @Environment(\.layout) private var layout

    var body: some View {
        if layout.isRegular {
            LazyVGrid(
                columns: [GridItem(.adaptive(minimum: 340, maximum: 620), spacing:
Space.x1)],
                alignment: .leading, spacing: Space.x1
            ) { content }
        } else {
            VStack(spacing: 0) { content }
        }
    }
}
```

Every section is a `MoneyBandStack` of `MoneyBand` s. Stacked on a phone, the section is clipped as one rounded block and the bands have square corners; in the iPad grid each band is its own card with its own corners, because a single 340-point column down the middle of an iPad is most of the screen saying nothing.

A band that holds a chart or a list is a `MoneyCard` . Its header keeps the eyebrow and the figure, so a collapsed card still says its number; what folds is the chart or the list under it. Nothing is dropped by collapsing, and every card starts open. The handful of chart cards with a stable identity remember their state in `UserDefaults` under `money.card.collapsed.<id>` ; a day of transactions uses transient state, since a key per calendar day forever is not a preference anyone set.

THE SIX SECTIONS

SECTION	BANDS	WHAT IT DECIDES
In and out	Net with verdict, Earned, Spent with pinstripes of income consumed, This week (card), Kept %, Net worth	Kept and Net worth are omitted, not zeroed, when undefined
Where it went	Spent by category (donut card), one band per category with swatch, count, share bar	A category folded into "Other" keeps its band but gets no swatch
Every month	Recurring total (card with rows), "A year of these" at twelve times the monthly sum	The annual figure is the argument for cancelling anything
Saving for	Goal progress, Still to find with months at this month's pace, or "No goal set"	A month that kept nothing gets no arrival date
What hurts	Total, one band per pressure point, Edit budgets	Pressure points are assembled on the snapshot, not in the view
Recent	Count with Add, a collapsible card per day	The day header shows settled net and counts pending beside it

`pressurePoints` is an extension on the snapshot, because deciding what belongs on the "what hurts" list is a judgement about money, not about layout. Three rules feed it: a bucket over its limit, unclaimed spend (money leaving with no bucket watching it), and recurring charges when they are more than a fifth of the month's spending. Below that threshold a subscription pile is a fact, and it already has its own tab.

ROWS THAT NEVER WRAP

Every row is one line. The amount is `fixedSize` and drawn first; the name takes what is left, scales to 0.8, then trails off. This replaced `layoutPriority(1)` on the name column, which starved the amount into "+... 00" and stacked "PENDING" letter by letter. A word cut in half reads as a different category, so names scale before they truncate.

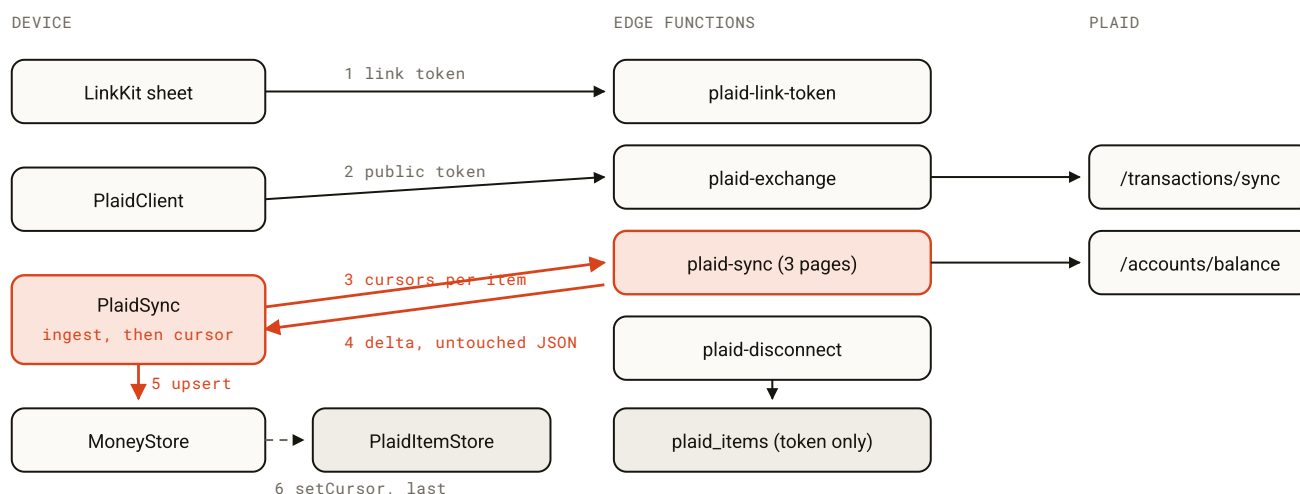
CHARTS

Three charts, all Swift Charts, all one ink at stepped opacity with no y-axis and no grid. The figure above each chart is the number; the marks are the shape. The week chart draws an empty track for every day so a day still to come is visibly a slot and not a gap, and labels only today. The donut uses `SectorMark` with a 1.5-point angular inset so two adjacent steps of the same ink read as two slices rather than a gradient, and `chartAngleSelection` turns a tap into a category page. The six-month chart adds a dashed `RuleMark` at the average of the completed months. The money pastels were run through the `dataviz` validator as a categorical palette and failed every check, which is the long way of saying what the one-ink rule already said.

THE MERCHANT TILE

Plaid's logos come as 100 by 100 PNGs on whatever background the merchant chose. Bare, a row of them is a row of mixed squares that breaks the band. `MerchantTile` puts the mark in a paper-white circle with a hairline, and gives the category-glyph fallback the same footprint, so a list with and without logos lines up. `AsyncImage` loads it; a failed load shows the glyph.

11.6 How it works



Every Plaid call is server-side. The delta comes back to the phone as Plaid's own JSON plus an institution name, and the cursor for each bank moves only after that bank's rows are saved.

CONNECT

- `PlaidConnectionViewModel.connect()` asks `plaid-link-token` for a token. The function calls `/link/token/create` with the Supabase user id as `client_user_id` and returns only the link token.
- `PlaidLinkPresenter.present` opens Plaid's LinkKit sheet. The presenter is the only file in the app that imports LinkKit, and it holds the live `Handler` statically because a bank using OAuth sends the user out to Safari; a handler released while they are away has nothing to hand the redirect back to, and the connection dies silently at the last step.
- LinkKit returns a `public_token`. The app posts it to `plaid-exchange`, which refuses a second connection to the same institution (two Items with overlapping transactions under different ids cannot be deduplicated on the device), calls `/item/public_token/exchange`, and writes the access token into `plaid_items`. The response carries the item id and the institution name, never the token.
- The view model records the item with a nil cursor, sets `isFetchingHistory`, and runs a sync straight away.

The OAuth banks are the reason `web/` exists. Plaid sends the browser to `almanac.shivvyas.com/plaid-oauth` when the bank is done. The `apple-app-site-association` file claims exactly that path, so iOS hands the URL to the app and the HTML page is never rendered. It renders only when the handoff fails, and its whole job is to tell a stranded person to open the app without claiming the connection failed, because by then Plaid has usually already succeeded. The link token that session was built with is kept in `UserDefaults` rather than memory, so a process killed during the round trip can resume the same Link session on relaunch, and the view model re-registers the completion closure on attach because the original one died with the old process.

SYNC

```
// LifeOSKit/Sources/Integrations/PlaidSync.swift
let rows = try PlaidMapping.ingestRows(from: delta.added + delta.modified,
                                       accounts: delta.accounts)

try money.ingest(rows)
try money.remove(externalIDs: delta.removed.map(\.transaction_id))
try money.upsertAccounts(PlaidMapping.accountRows(from: delta.accounts))

// Only now. Every throw above leaves the cursor where it was, and
// the next run replays this page into an idempotent upsert.
if let cursor = delta.next_cursor {
    items.setCursor(cursor, for: delta.item_id)
}
```

This ordering is the whole design in three lines. A cursor advanced before a successful ingest is a page of transactions Plaid will never send again. That is also why the cursor lives on the device, per Item, in `UserDefaults` rather than on the server or in the Keychain: a server-advanced cursor would permanently lose a page to an app that crashed mid-ingest, and a Keychain cursor would survive a reinstall that the SwiftData database does not, leaving Plaid replaying nothing into an empty app. The cursor must die with the data it describes.

The function fetches at most three pages per invocation to stay inside its wall clock and reports `has_more`; the view model loops up to twenty times, so an initial pull of years of history finishes across several runs rather than stopping part-way and looking complete. Balances are fetched opportunistically after the transactions, and a failure there travels separately as `balance_error`: `/transactions/sync` serves cached data and succeeds even with an expired bank login, while `/accounts/balance/get` does a live fetch and is the call that actually throws `ITEM_LOGIN_REQUIRED`. That is what raises the reconnect banner without discarding the transaction pages already fetched.

Sync is triggered by `reloadAll` through `syncIfDue`, which applies the shared `SyncStalenessPolicy`: stale after an hour, or across a day boundary, or when the recorded time is in the future because the clock moved backwards. Before it checks staleness it runs `replayHistoryOnce`, which clears every cursor exactly once under a versioned flag so rows synced before the logo field existed gain their logo on the next pass. A replay on every launch would pull years of rows daily; a replay once is free, because Plaid bills per connected Item, not per call.

DISCONNECT

`plaid-disconnect` calls `/item/remove` and deletes the row. A connected Item bills monthly, so this is not optional polish. The client discards the response body and succeeds on the status alone, because reading a field that carries no further information only creates a way for a disconnect that worked to be reported as a failure. Local history is kept by default; deleting that Item's entries is a separate, explicit choice.

11.7 Performance and cost

- **Batched ingest.** `externalID` has no index and ingest runs on the main actor. A per-row fetch inside the loop would be an unindexed table scan per row, thousands of times on an initial pull. `MoneyStore.ingest` fetches every id the page touches once, builds a dictionary, keeps it current as rows

are inserted so a duplicate id within the page still upserts, and saves once for the whole page. `remove` and `upsertAccounts` do the same.

- **Three pages per function call, twenty calls per sync.** Bounded work on both sides, with `has_more` carrying the continuation.
- **Sync only when stale.** One rule, shared with Whoop and Fitbit, keeps the sync off every save and every foreground.
- **One month loaded per save; six months only while the detail page is up.** The tab view model is cheap because it reads the calendar month to date. The detail view model is separate so its six-month fetch runs only when its page is open.
- **No per-row lookups in the view.** Category shares, logo URLs, slice opacities and day groupings are computed once on the snapshot. Views divide nothing and look nothing up.
- **Reads and writes serialised.** `UserDefaultsPlaidItemStore` is `Sendable` and wraps every read-modify-write in an `NSLock`, so a cursor set from one task cannot lose an update from another. The test `concurrentCursorSetsDoNotLoseUpdates` pins it.
- **Private by construction.** The server holds one row per bank containing the access token and an institution name. Transactions, merchants, categories and logos never touch Postgres. Nothing about spending is sent to a logo service either: the logo URL is Plaid's own, in a response already being paid for.
- **Cost.** No new Plaid products since the first connection: Transactions and Accounts only, no `/transactions/recurring` (a separately billed product that `RecurringSpend` replaces for free from history). Plaid bills per connected Item per month, which scales with the number of banks a person links. The app targets one user on the free tier today; the pricing question is deferred, not answered, and must be answered before a second person connects a bank. See Chapter 24.
- **Offline.** Every band renders from `SwiftData`. A sync failure leaves the previous snapshot on screen with its own timestamp, which is honest, rather than replacing it with zeros. A network outage never disconnects a saved bank.

11.8 Edge cases and tests

CONDITION	BEHAVIOUR
No entries, no accounts, no bank	Empty state with Connect and Add, not a screen of zeros
Bank linked, first sync returns nothing	"Fetching your transactions" with a Check again button. Never rendered as an empty month
Expenses but no income	Savings rate omitted; verdict reads "No income logged"
Pending transaction	Shown at 45% opacity with a Pending label; excluded from income, expenses, net, categories, the donut and the day's settled net
Pending charge settles under a new id	The new id upserts, the old id arrives in <code>removed</code> and is deleted; no duplicate
Transfer between own accounts, card payment	Kept in the ledger, excluded from income and spending by category code
Credit card with a balance	Subtracts from net worth by <code>type</code> , never by <code>subtype</code>
Bank reports no credit limit or no available balance	Stored as nil, never zero. Utilisation against a zero limit is 100% by arithmetic and unknowable in fact
Plaid sends an unreadable date	<code>PlaidMapping</code> throws, the sync fails, the last known-good snapshot stays. A silently dropped row is a wrong total with no error anywhere
Device west of Greenwich	Dates parse in the store's own calendar, so a transaction Plaid dates the 1st is not stored as the 31st and moved into last month
Ingest throws mid-page	Cursor stays where it was; next sync replays the page into the idempotent upsert
One bank's login expires	That item is reported in <code>needsReconnect</code> ; every other bank still syncs; the reconnect banner shows and the figures below it are labelled as from the last sync
One bank fails for another reason	Reported in <code>failedItems</code> ; the "last synced" timestamp is not stamped on a sync that partly failed
Bank removed on another device	The sync response omits it, so the local item cache drops it too
Same bank connected twice	Refused by <code>plaid-exchange</code> before the exchange is spent, backstopped by a unique constraint on (user, institution)
Item store fails to decode	Item list resets to empty and self-heals from the next sync, with every cursor reset; logged so a mystery full resync has a cause
Bucket limit of zero or less	Rejected by the store with <code>limitNotPositive</code> ; the form's Save is disabled
Key claimed by two buckets	Impossible: <code>claim</code> moves it and the sheet says where from
Refund in a budgeted category	Nets within its bucket instead of inflating income
Two coffees in one week	Not a subscription. Recurrence needs two distinct months at an amount within 25% of its own median; income is never a subscription
Account switch	The view model deactivates, the presenter resets, and the next account reads its own <code>UserDefaults</code> suite and store

TESTS

All in `LifeOSKit` , all passing, all without a network or a Plaid account:

- `PersistenceTests/MoneyTests` : the sign convention, the undefined savings rate, pending exclusion, transfer and card-payment exclusion, the underscore-prefix rule, upsert on the provider id, removal of a settled pending charge, merchant entity and logo round trips, and that `isSpending` matches what the summary counts.
- `PersistenceTests/MoneyPartsTests` : the carry at 19.995 and 999.999, float noise, zero-padded cents, a zero that is never negative, grouping of large balances.
- `PersistenceTests/SpendBucketTests` : a key moves rather than being held twice, deleting a bucket frees its keys, the claim key prefers the code over the label.
- `SectorsTests/BudgetPeriodTests` : refunds net within a bucket, income never appears as unclaimed, adherence is 1.0 at the limit and decays linearly to 0 at double it.
- `InsightsTests/SpendSeriesTests` and `RecurringSpendTests` : week boundaries, an empty month at zero in the middle of a run, fold leaving a list at or under keep plus one alone, the median resisting a single price rise.
- `IntegrationsTests/Plaid*` : the client's error mapping through a stub `URLProtocol` , a missing session never reaching the network, disconnect succeeding on status alone, the mapping's negation and time-zone behaviour, the wire format's optional fields, and `PlaidSyncTests` pinning that a failed ingest leaves the cursor where it was and a balance login failure raises reconnect without losing the transactions.

11.9 Study notes

READ IN THIS ORDER

1. `LifeOSKit/Sources/Persistence/MoneyEntry.swift` , for the sign rule and `summarise` .
2. `LifeOSKit/Sources/Integrations/PlaidMapping.swift` , the negation and the date parsing.
3. `LifeOSKit/Sources/Integrations/PlaidSync.swift` , the ingest-then-cursor ordering.
4. `LifeOSKit/Sources/Persistence/MoneyStore.swift` , the batched upsert and the claim rule.
5. `LifeOSKit/Sources/Sectors/BudgetPeriod.swift` , the pure budget assessment.
6. `LIfeOS/Features/Money/ViewModel/MoneyViewModel.swift` , how the snapshot is derived.
7. `LIfeOS/Features/Money/View/MoneySections.swift` , bands, cards and the grid.
8. `supabase/functions/plaid-sync/index.ts` , the server half of a sync.

CHECK YOURSELF

- **Why does the cursor live in `UserDefaults` and not the `Keychain`?** The `Keychain` survives a reinstall and the database does not; a cursor that outlived its data would make Plaid replay nothing into an empty app.
- **What would go wrong if the cursor advanced before `ingest` ?** A page that failed to save would never be sent again, and the total would be silently short.
- **Why is a card payment excluded from spending?** The purchases on the card were already spending; the payment is the same money moving, and counting it inflates expenses and lowers the savings rate.

- **Why does a budget claim a category code rather than a merchant or a label?** A label can be renamed and a merchant can be spelled three ways; a silently non-counting budget is worse than none.
- **Why is a missing credit limit stored as nil rather than zero?** Zero renders as 100% utilised and looks like a fact; nil renders as nothing, which is the truth.
- **Why is the donut one ink at stepped opacity?** The six pastels fail as a categorical palette; a sequential encoding of one hue passes, and identity is carried by the legend and the matching swatches on the bands, never by colour alone.

12 Notes and the plan

A block editor over a PARA library, synced to Supabase, with a derived index underneath that turns a pile of pages into something you can ask questions of. Habits are the one thing from the old Plan tab that did not become a page, and this chapter explains why.

Revision D described a Plan tab with four lists behind a segmented pill. That tab is gone. Goals, content items, loose notes and journal entries all became pages on 26 August 2026, migrated once and never rewritten, and the tab slot they occupied now belongs to Notes. What survived is the one plan kind that is not a document: a habit, which is a daily tick with a streak behind it. 12.9 covers what is left of the plan; everything before it is the notes tab as it is on disk today.

12.1 Intent

WORKING BACKWARDS

Notes is a library with four shelves and a page you can write on. Every page is a list of blocks: paragraphs, headings, to-dos, quotes, code, a sketch you draw in line with the text. Type a slash to change what a line is, type two brackets to link to another page, and the page you linked to lists you at the bottom. Pick up a Pencil and it writes, draws or annotates depending on where it is. Everything saves on the device first and reaches the server afterwards.

Why PARA rather than folders you name yourself?

Because PARA sorts by actionability, not by subject. A note about running belongs under a marathon *project* while the race is ahead and under the *area* that outlives it once the race is run. Encoding that as the top of the hierarchy is the method's whole point, so `NoteBucket` is an enum with four cases rather than four folders a person could rename into meaninglessness. The app calls the third shelf Research, because PARA's own word, Resources, is vaguer than what the shelf holds.

Why blocks instead of one text field with formatting?

A block editor's trick is that the document is a list of small typed pieces rather than one string with ranges hung off it. Splitting, merging, reordering and turning a paragraph into a to-do become list operations, and none of them can corrupt the others. The operations themselves are pure functions in `NoteBlockEditor`, tested against literal arrays rather than a screen.

If blocks are JSON inside the page, how does the app find every open to-do?

It does not search the JSON. On every save, `NoteIndexer` mirrors the page's to-do blocks and wiki links into two derived SwiftData models, `NoteTask` and `NoteLink`. They are never pushed to the server, and every row can be rebuilt from the documents, so the worst an indexer bug can do is show a stale list. The repair is one function.

Does a note leave the device?

Only when someone is signed in, and only after the write has landed locally. `NoteSync` pushes dirty rows to two Postgres tables guarded by row level security, then pulls what other devices wrote. Sync can fail without anyone noticing and without losing an edit: push runs before pull, the newer `updatedAt` wins, and a row is marked synced only after the server confirms it.

What Notes deliberately does not do, yet: collections that cut across the shelves, goals as their own model, a mindmap of the link graph, voice capture, a share extension. Those are phases three to six of the notes redesign spec and none of them has code. Phase one, the index layer, shipped. Phase two, the phone's Inbox stream and its pinned composer, was built on 28 August and then set aside in the 13 September restyle: the store still knows what an Inbox is and the screen still compiles, but nothing routes to it. 12.5 says exactly what the phone shows instead.

12.2 Files and folders

Features/Notes/

- └─ Model/
 - └─ NoteSelection.swift
 - └─ NotesCommands.swift
- └─ ViewModel/
 - └─ NotesViewModel.swift
 - └─ NoteEditorViewModel.swift
 - └─ NoteInboxViewModel.swift
- └─ View/
 - └─ NotesHubScreen.swift
 - └─ NotesSidebar.swift
 - └─ NotesLibraryList.swift
 - └─ NoteShelfScreen.swift
 - └─ NoteInboxScreen.swift
 - └─ NoteCard.swift
 - └─ NoteEditorScreen.swift
 - └─ NoteBlockRow.swift
 - └─ BlockTextView.swift
 - └─ NoteAccessoryBar.swift
 - └─ SketchBlockView.swift
 - └─ InkCanvasView.swift
 - └─ NoteToolPicker.swift
 - └─ NoteAccentPalette.swift
 - └─ NotesDesignPreview.swift

what the rail points at: recent, favourites, a shelf, a folder, the Notes menu and its hardware keyboard shortcuts

the library: snapshot, cards, selection, filter, sort, search, one open page: blocks in memory, debounced save, links, the Inbox stream and composer (built, not routed to today)

the two shells: phone stack, iPad columns
the iPad rail: shelves, folders, drag targets
the phone library: glass tiles and cards, one orange
one shelf or folder: header, search, filter, sort, cards
the Inbox stream (built, not routed to today)
one page in a shelf, with its move menu
the page: header, blocks, ink layer, backlinks
one block: gutter mark in SwiftUI, text in UIKit
the UITextView that owns Return, Backspace and link paint
slash menu, link picker, formatting bar, taking turns
a drawing that takes its own space in the flow
the page-wide annotation layer and the Pencil rule
the one PKToolPicker every canvas shares
colour for the eight accent names
in-memory fixtures for captures, never an account

LifeOSKit/Sources/Persistence/

- └─ NoteDocument.swift
- └─ NoteBlock.swift
- └─ NoteBlockEditor.swift
- └─ NoteBucket.swift
- └─ NoteAccent.swift
- └─ NoteStream.swift
- └─ NoteSnapshot.swift
- └─ NoteIndex.swift
- └─ NoteIndexer.swift
- └─ NoteIndexMigration.swift
- └─ NoteEvidence.swift
- └─ NotesStore.swift
- └─ PlanEntry.swift
- └─ PlanStore.swift
- └─ PlanNoteMigration.swift

NoteKind, NoteFolder, NoteDocument (the page)
NoteBlockKind, NoteBlock, NoteBlockParser, NoteLinkScanner
Return, Backspace, indent, transform, links: pure functions
the four PARA shelves
eight accent names, no colour
the chip row and what a chip resolves to
NoteCardSnapshot, NoteFolderSnapshot, NotesSnapshot, NoteBlockSnapshot, NoteTask and NoteLink, the derived rows
the only writer of the derived rows
stamp an existing library as filed, once; build the index
what the monthly close reads off notes
every read and every write, with the index kept current
PlanKind, PlanStatus, PlanEntry, HabitTick
habit ticks and streaks
the one-time move of plan rows into pages

LifeOSKit/Sources/Integrations/

- └─ NoteSync.swift
- └─ NoteWireFormat.swift

push then pull against Supabase
NoteDocumentRow and NoteFolderRow, the shape a page travels

Features/Plan/

- └─ Model/PlanSnapshot.swift
- └─ ViewModel/PlanViewModel.swift
- └─ View/
 - └─ PlanScreen.swift
 - └─ ContentCalendar.swift

the four sections, still declared; only habits are shown
load, add, advance, toggle, delete

hosts the habits list inside the notes tab
the content month grid, unreachable since the migration

WHERE THE BOUNDARIES FALL

`Persistence` owns the shape of a note and every rule about it. `Integrations` owns how it travels. The app target owns how it looks and what a keystroke means for the caret.

`NoteAccentPalette` is the one deliberate join: accent *names* are a persistence concern, colours are a design concern, and `DesignSystem` depends on nothing, so the two meet in a 66-line file in the app rather than in either package.

12.3 Model

THE PAGE

A page is a `NoteDocument`. Every field carries a default, because `SwiftData` adds a property to an existing store without a migration plan only when it can fill it in, and this model has grown four times since it was created.

```
// LifeOSKit/Sources/Persistence/NoteDocument.swift
@Model
public final class NoteDocument {
    public var id: UUID = UUID()
    public var title: String = ""
    public var icon: String = ""
    public var kindRaw: String = NoteKind.note.rawValue // note | journal | task
    public var bucketRaw: String = NoteBucket.projects.rawValue
    public var accentRaw: String = NoteAccent.sage.rawValue
    public var folderID: UUID? // nil: loose on the shelf
    public var blocksData: Data = Data() // the blocks, JSON encoded
    public var drawingData: Data? // a PKDrawing, when inked
    public var entryDate: Date? // the day a journal entry is
    about
    public var dueDate: Date?
    public var statusRaw: String = PlanStatus.todo.rawValue
    public var isFavorite: Bool = false
    public var openedAt: Date? // drives Recent; written on open
    public var archivedAt: Date? // Archive is this flag, not a
    bucket
    public var filedAt: Date? // nil means still in the Inbox
    public var syncedAt: Date?
    public var deletedAt: Date? // a tombstone, reaped after push
    public var originPlanEntryID: UUID? // the plan row this came from
}
```

Four of those fields carry the whole design, and each earns a sentence.

- `blocksData` is JSON, not a relationship. Blocks are only ever read and written as a document, no query asks for one block, and a relationship would buy one migration hazard per block field in exchange for nothing. When the JSON cannot be decoded the getter hands back a blank page rather than throwing: the bytes stay on disk untouched until someone saves over them, and the person at least has somewhere to type.

- `archivedAt` is a flag, not a fifth shelf. An archived page keeps the bucket it was filed on, so restoring it needs no decision.
- `filedAt` is the whole Inbox concept. A bucket cannot answer "has anyone decided where this goes", because every page has a bucket from the moment it is created. The stamp is written by filing and never by editing, so a page you keep writing in stays unfiled until you choose a home for it.
- `syncedAt` **and** `deletedAt` are the sync contract. `updatedAt > syncedAt` is the entire dirty test. A delete writes a tombstone the server can pull as a fact, rather than an absence it cannot observe.

Three enums sit behind the raw strings. `NoteKind` decides how a page is listed, not how it is edited: a `note`, a `journal` entry titled by its day, or a `task` page that exists to hold to-dos. `NoteBucket` is the four shelves. `PlanStatus` is reused from the old plan rather than declared again, which is what lets the monthly close in Chapter 13 read goal progress off notes without learning a second vocabulary. Raw strings with typed accessors, for the reason Revision D gave: a new case is a code change, not a migration, and an unrecognised value reads as the default instead of failing the fetch.

A `NoteFolder` is the same shape with fewer fields: a name, an emoji or an accent dot, a shelf, an optional `parentID`, and the same `syncedAt` and `deletedAt` pair. Folders nest arbitrarily in the model and one level in practice.

THE BLOCK

```
// LifeOSKit/Sources/Persistence/NoteBlock.swift
public struct NoteBlock: Codable, Sendable, Identifiable, Equatable {
    public var id: UUID
    public var kind: NoteBlockKind           // paragraph, three headings, three lists, quote,
                                           // callout, code, divider, sketch

    public var text: String
    public var isChecked: Bool               // kept on every block so a tick survives a transform
    public var indent: Int                   // 0 through 4, purely visual
    public var drawing: Data?                // sketch blocks only
    public var sketchHeight: Double?         // dragged height, so ink crops the same on every
device
    public var dueDate: Date?                // to-do blocks only; authored, so it lives here
    public var goalID: UUID?                 // to-do blocks only; authored, for the same reason
}
```

Two details matter. Indent is a number on a flat array, not a tree: a nested list item is still a sibling, which is what keeps every editing operation a list operation. And `dueDate` and `goalID` live on the authored block rather than on the derived `NoteTask` row, because the row is rebuilt on every save and would lose them.

`NoteBlockParser` owns the Notion-style typed prefixes. `candidates` is every prefix sorted longest first, and that order is the whole correctness argument: `- [ ]` starts with `-`, so checking kinds in declaration order would turn every to-do typed by hand into a bullet reading "[] whatever". `---` and the code fence are whole-line markers rather than prefixes, so a line reading `"--- and then some"` stays a paragraph. The same type renders blocks as Markdown for the share sheet and for the coach's context window, and parses Markdown back into blocks for paste and for the plan migration. Round-tripping is not a goal; a sketch becomes `_[sketch]_` on the way out, because saying a drawing was here beats dropping it silently.

`NoteLinkScanner` finds `[[wiki links]]`. Case is preserved in what was typed and folded at lookup, so `[[Marathon]]` and `[[marathon]]` reach the same page without rewriting either.

THE DERIVED ROWS

```
// LifeOSKit/Sources/Persistence/NoteIndex.swift
@Model
public final class NoteTask {
    public var id: UUID = UUID()           // the block's own id: stable across reindexes
    public var documentID: UUID = UUID()   // scoped by page, so a pasted block cannot collide
    public var text: String = ""
    public var isChecked: Bool = false
    public var indent: Int = 0
    public var dueDate: Date?
    public var goalID: UUID?
    public var sortOrder: Int = 0          // position in its page
    public var documentUpdatedAt: Date = Date.distantPast // copied down, so a cross-page
                                                    // list can order pages
}

@Model
public final class NoteLink {
    public var sourceID: UUID = UUID()
    public var targetTitleFolded: String = "" // lowercased: one edge per title
    public var targetID: UUID?                // nil: the page does not exist yet
}
```

Two rules make these safe. They are never pushed, so two devices cannot disagree about them. And every row is reconstructible from the documents, so any inconsistency is repaired by `NoteIndexer.rebuildAll`. A `NoteTask` keeps the block's own id so the row is stable across reindexes rather than churning a new identity on every keystroke; it is scoped by `documentID` because a block copied and pasted into a second page keeps its id.

A RESOLVED EDGE CAN DANGLE

`NoteLink.targetID` being non-nil is not proof the page is still there. `reindex` only rewrites the edges out of the document being saved, so renaming or deleting a page leaves every edge that pointed at it holding an id that now names something else or nothing at all. The next launch's `rebuildAll` repairs them. Until then a consumer must treat the lookup as a miss it can survive, not as a page it is owed. This is written on the model so it is not rediscovered as a mystery bug.

WHAT VIEWS SEE

Views never hold a `NoteDocument`. `NoteCardSnapshot` is a page flattened for display: title, icon, excerpt, accent, folder name, task progress, whether it has ink, how many links it carries. `NoteFolderSnapshot` is a folder row with its children already attached and a count that rolls up through them. `NotesSnapshot` is everything the sidebar needs in one read. `NoteBacklink` is a page that links here, with the line the link appears on, so the backlinks pane shows context rather than a bare list of titles.

12.4 ViewModel

Three view models, one per concern, and the store does the deciding for all of them. `NotesStore` lives in the package because the app target has no test target, so any rule worth pinning (what "All" includes, when a page is filed, what order a shelf sorts in) is put where `swift test` can reach it.

THE LIBRARY

```
// LIfiOS/Features/Notes/ViewModel/NotesViewModel.swift
@Observable @MainActor
final class NotesViewModel {
    private(set) var snapshot: NotesSnapshot = .empty
    private(set) var cards: [NoteCardSnapshot] = []
    private(set) var isSearching = false

    var selection: NoteSelection = .recent { didSet { if selection != oldValue { load() } } }
    var filter: NoteShelfFilter = .all { didSet { if filter != oldValue { load() } } }
    var sort: NoteSort = .recentlyEdited { didSet { if sort != oldValue { load() } } }
    var query: String = "" { didSet { if query != oldValue { load() } } }

    func attach(_ context: ModelContext, sync: NoteSyncing? = nil)
    func load()
    func createNote(kind: NoteKind = .note, title: String = "") -> UUID?
    func openToday'sJournal() -> UUID?
    func createFolder(named:in:icon:) func renameFolder(_:to:) func deleteFolder(_:)
    func toggleFavorite(_:) func toggleArchive(_:) func delete(_:) func
    move(_:to:folderID:)
}
```

`NoteSelection` is one enum rather than a pair of optionals, because "a folder is selected" and "a shelf is selected" are mutually exclusive and two nullable properties make a fourth, meaningless state representable. `load()` reads the snapshot once, then either a search or the cards for the selection. Every mutating method funnels through `mutate(id:)`, which fetches the live object, applies the change through the store, reloads, and fires a sync. Nothing in the UI waits on the server; the write has already landed locally by the time `requestSync()` runs, and a failure there only means the next pass carries the row.

`NoteSyncing` is a one-method protocol the view model asks for, so a preview and a design capture can hand it nothing. The real `NoteSync` conforms in one line at the top of `RootView`, which is the only place that knows both types.

THE PAGE

`NoteEditorViewModel` holds the blocks in memory and writes them back on a delay. Saving on every keystroke would mean a `SwiftData` write and a `didSave` notification per character, and every screen in the app reloads on that notification.


```
// LifeOS/Features/Notes/ViewModel/NoteEditorViewModel.swift
private func scheduleSave() {
    guard !isLoading else { return }
    saveMessage = "Saving..."
    saveTask?.cancel()
    saveTask = Task { [weak self] in
        try? await Task.sleep(for: .milliseconds(600))
        guard !Task.isCancelled else { return }
        self?.saveNow()
    }
}

/// Called on the way out, so a page closed inside the debounce window still lands.
func flush() {
    saveTask?.cancel()
    saveTask = nil
    saveNow()
}
```

Ticking a checkbox is the exception: it is a decision, not a keystroke, and `toggleCheck` saves at once so it survives the app being closed a second later. The screen calls `flush()` on disappear and whenever the scene leaves the active phase.

The keystroke rules are not here. `splitBlock`, `backspaceAtStart`, `indent`, `transform` and `completeLink` each call the matching pure function on `NoteBlockEditor`, apply the returned blocks, move the caret to the returned focus, and schedule a save. The view model also tracks the two queries the accessory bar renders: `slashQuery`, the text after a leading slash, and `linkQuery`, the partial title inside an unclosed `[]`. `linkSuggestions` filters a title index the model refreshes after every save, and `resolveLink` creates the page when nothing carries that title, the way Obsidian does.

`setDrawing` compares bytes before writing, because PencilKit reports a change on every stroke and on every tool switch, and rewriting identical bytes would mark the page dirty for a sync that carries nothing new.

THE INBOX

`NoteInboxViewModel` is 78 lines: a chip, a draft, a sticky to-do toggle, and a `capture()` that hands the draft to `NotesStore.capture`, clears it on success and leaves it alone on failure so a thought is never silently dropped. It is thin by the same rule as `LifeBoardViewModel`: what a chip means lives in the store. It compiles, its store methods are tested, and no screen constructs it today.

12.5 View

TWO SHELLS OVER ONE LIBRARY

`NotesHubScreen` reads `layout.isRegular` from the environment and builds one of two arrangements. Both drive the same `NotesViewModel` and the same `NoteShelfScreen`; only the chrome around the shelf differs.

The phone is a `NavigationStack` with the shelf at its root. The leading toolbar button presents the library as a full-height sheet titled "Your library", and choosing anything in it sets the selection, clears the path and closes the sheet in one binding, so the rail and the sheet cannot drift apart about what a tap does. A page is pushed onto the path. Two small pieces of state, `pendingNewFolder` and `pendingRename`, exist because a sheet cannot present a second sheet: the library records what was asked for, dismisses, and the hub presents the folder sheet in `onDismiss`.

The iPad is an `HStack` of up to three columns: the rail, the shelf, and the open page. Whether the third column exists is measured, not derived from the size class.

```
// LifeOS/Features/Notes/View/NotesHubScreen.swift
private static let libraryWidth: CGFloat = 268
private static let minShelfWidth: CGFloat = 340
private static let minPageWidth: CGFloat = 520 // below this the page stops being somewhere
you would write

private var isThreeColumn: Bool {
    guard layout.isRegular else { return false }
    let spokenFor = layout.railInset + (isLibraryVisible ? Self.libraryWidth : 0)
    return paneWidth - spokenFor >= Self.minShelfWidth + Self.minPageWidth
}
```

A single overall threshold was wrong: at 1024 points a pane read as wide enough, but the library and the tab rail take 380 of that before the shelf and the page have divided anything, leaving a 310-point column to write in. Subtracting what is already spoken for is what lets collapsing the library buy a third column on a screen that could not otherwise hold one. Rotating an iPad or resizing a Stage Manager window flips `isThreeColumn`, and the open page moves between the detail column and the navigation path with it, or it would vanish. Exactly one of `openPage` and a `.page` route on the path is ever non-empty, which is what stops them disagreeing.

The detail column rebuilds its editor per page with `.id(openPage)`, so the editor never shows the previous page's blocks for a frame while the new ones load. `NoteEditorHost` exists because `@State` cannot be initialised from a navigation destination's argument; it creates the editor view model in a `.task` keyed on the document id and keeps it alive while the page is on screen.

THE RAIL AND THE LIBRARY

The iPad rail, `NotesSidebar`, is text rows on the canvas rather than a `List`, which brings its own background, separators and insets, all of which would have to be undone. Recent and Favourites lead, Habits pushes its own screen, then the four shelves, each expanded on first launch because four collapsed rows tell a new user nothing about what PARA is. Every shelf and folder row is a drop destination for `NoteDragPayload`; a drop on a shelf files the page loose on it, and a drop on Archive archives rather than refiles, because Archive is a state, not a place. The drop highlight is a different colour from the selection highlight: "this is where it will land" and "this is what you are looking at" are different statements.

The phone library, `NotesLibraryList`, is the same inputs and callbacks in the app's own card vocabulary: Recent, Favourites and Habits as a row of tiles, then each shelf as one soft card with its folders under a hairline and a quiet row to add one. The whole list sits inside one `GlassEffectContainer`, so neighbouring

glass tiles merge and separate as they move rather than each rendering its own lens. Colour is Apple's: system grays for every icon square and fill, and the accent orange for exactly one thing, whatever is selected.

THE SHELF

`NoteShelfScreen` is a header, a search field, a count, a kind filter (shown only when a shelf or folder is selected, since Recent and Favourites already cut across kinds), a sort menu with the four orders `NoteSort` allows, and a `LazyVStack` of `NoteCard`. Each card carries its move targets as a menu as well as being draggable, because drag and drop is a pointer gesture and the same move has to exist for a finger.

THE PAGE

`NoteEditorScreen` caps its measure at 760 points even on an iPad, because prose at 1000 points across is unreadable and this is the one screen in the app that is genuinely prose. Above 1040 points of editor width, and only when there are backlinks to show, the backlinks list moves out from under the text into a 280-point inspector column beside it.

The body is a `ZStack`: a column of `NoteBlockRow` underneath, and an `InkCanvasView` sized to the measured height of that column on top. A canvas sized to the screen instead would clip strokes the moment the page scrolled. Under the last block sits 160 points of clear, tappable space that focuses the last block, because without it the bottom half of a short page is dead.

Each block row draws its gutter mark in SwiftUI and its text in UIKit. Putting the bullet inside the text view as a literal character was the alternative, and it makes the bullet deletable, which means a list item can silently stop being one. `BlockTextView` wraps a `UITextView` for three things SwiftUI's text field cannot do here: intercept Return so it splits the block instead of inserting a newline, intercept Backspace at offset zero so it merges with the block above, and paint `[[links]]` as the person types. Apple Pencil Scribble arrives free with `UITextView`, which is the fourth. Every font comes from `Life0Style.UikitScale`, so a heading in a note is the same size as the same heading anywhere else in the app.

`NoteAccessoryBar` is one strip above the keyboard with three states that take turns: a slash menu while `slashQuery` is set, a link picker while `linkQuery` is set, otherwise a formatting bar with indent, ink and dismiss. Three floating panels competing for the same corner of a phone screen was the alternative. Escape backs out one layer at a time: the block menu, then the link picker, then the keyboard. Closing the page on the first press would lose someone mid-sentence.

THE PENCIL RULE

There are two kinds of drawing on a page and they are deliberately different. `SketchBlockView` is a block in the flow: it pushes the text below it down and moves with the paragraph it belongs to, which is what a diagram between two paragraphs wants. It is Pencil-only by default on iPad so a finger still scrolls past it, and finger-drawable by default on a phone. `InkCanvasView` floats over the whole page for annotation.

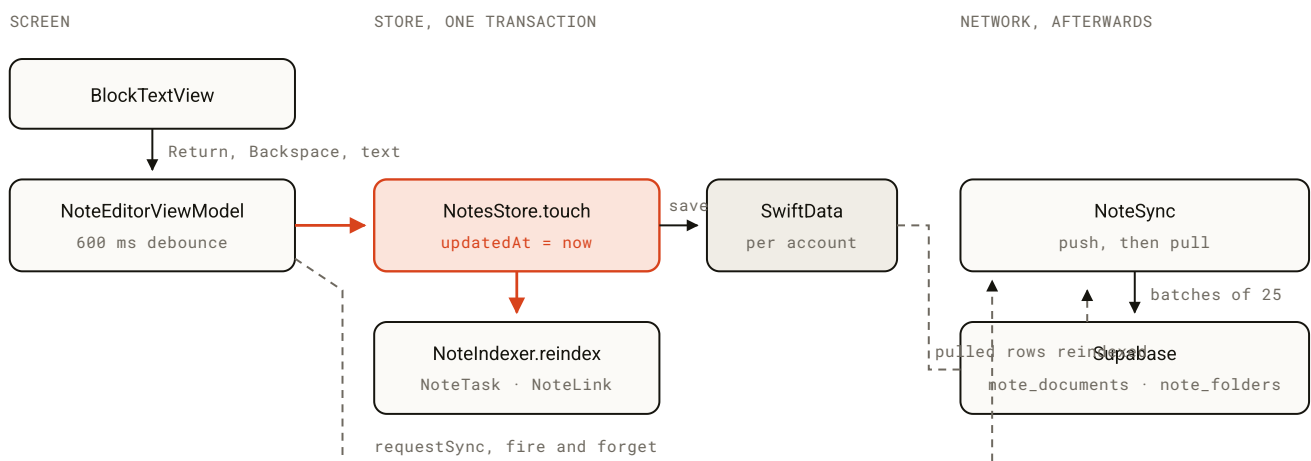
The ink layer used to claim every Pencil touch, on the theory that picking up the Pencil should mean drawing. The cost was hidden and larger than the benefit: a layer that swallowed the touch first meant Scribble could never fire anywhere, and sketch blocks were unreachable for the same reason. So the rule is stated by where the Pencil is, not by what it is. Over a text block it writes. Over a sketch block it draws in that block. With ink mode on, it annotates the page, and a double-tap or squeeze of the Pencil turns ink mode on without a trip to the toolbar.

`NoteToolPicker` is one `PKToolPicker` shared by the ink layer and every sketch block. Two of them fight: whichever became visible last wins, and the loser leaves a stale observer behind that keeps receiving tool changes for a canvas nobody is drawing in. It also tracks which canvases already observe, because PencilKit keeps duplicate observers rather than ignoring them and a view that updates twenty times would otherwise register twenty.

THE HARDWARE KEYBOARD

`NotesCommands` is a `CommandMenu` installed in the scene, so its shortcuts work wherever focus is and appear in the overlay an iPad shows when Command is held. It reads a `NotesCommandTarget` from `focusedSceneValue`: a struct of closures, one per action, because opening a page means pushing on a phone and filling the detail column on an iPad and the scene must not have to know which. `closePage` is optional and nil when nothing is open, which is what greys out Command-W rather than letting it fire into an empty column. Command-Option-0 is Recent and Command-Option-1 to 4 are the shelves, matching how editors number their tabs.

12.6 How it works



The write lands locally before anything touches the network. The index is rewritten inside the same save as the page, so a page and its rows can never be half written with respect to each other.

STEP BY STEP, FROM A KEYSTROKE

1. The `UITextView` reports text, or a Return, or a Backspace at offset zero. `NoteBlockRow` forwards it as a closure call.
2. The editor view model applies the matching `NoteBlockEditor` function to its block array, moves the caret, and schedules a save 600 milliseconds out. A second keystroke inside that window cancels the first task.
3. `saveNow()` fetches the live document and calls `NotesStore.update`, which returns early if the blocks are equal and otherwise assigns them and calls `touch`.
4. `touch` stamps `updatedAt`, calls `NoteIndexer.reindex`, and saves once. Every document mutation in the store funnels through this one function, which is why the index is rewritten here and nowhere else.

5. `reindex` fetches and deletes the page's existing rows, inserts one `NoteTask` per to-do block, and, only if the page carries any links, builds the folded title index and inserts one `NoteLink` per link.
6. The view model refreshes its backlinks and title index, updates the "Saved on this device" label, and asks `NoteSync` to run.

THE INDEXER'S TWO DELETES

`reindex` and `rebuildAll` delete rows differently, on purpose, and the code says not to "fix" either to match the other.

```
// LifeOSKit/Sources/Persistence/NoteIndexer.swift
// Inside reindex: fetch and delete each, NOT `context.delete(model:where:)` .
// The batch form runs against the persistent store and cannot see rows
// inserted earlier in this same unsaved transaction, so a second reindex
// would duplicate rows instead of replacing them.
let existing = try context.fetch(
    FetchDescriptor<NoteTask>(predicate: #Predicate { $0.documentID == documentID })
)
for row in existing { context.delete(row) }

// Inside rebuildAll: the batch form is correct HERE. This deletes everything
// before anything is inserted, and the rows it clears are already saved.
try context.delete(model: NoteTask.self)
try context.delete(model: NoteLink.self)
```

`rebuildAll` also starts from empty rather than reindexing page by page, because a per-document pass never visits a page that is no longer there, and so could never clear the rows a deleted page left behind.

FILING

Where a page lands, and when it counts as filed, is decided in the store and pinned by tests.

- `createDocument` stamps `filedAt` only when a folder was given. Choosing a folder is choosing a home. A page created loose on a shelf is not filed, because the shelf was the default, not a decision.
- `capture` writes one block into Areas with no title and no `filedAt`. The title is left empty on purpose: `displayTitle` already falls back to the first textual block, so storing the sentence twice would show it twice on the card.
- `move` stamps `filedAt` if it is nil. Editing never does.
- Deleting a folder empties it onto its parent rather than taking the pages with it. Losing a folder is an organising decision; losing what was in it is a different decision that must never be made by implication. Those pages change outside `touch`, so the store reindexes them by hand in the same loop.

SYNC

`NoteSync` coalesces overlapping calls into one in-flight task, reads the access token fresh from the keychain on every pass rather than capturing it (the app shell refreshes the session hourly, so a captured token would be stale within the hour), and returns quietly for a guest. Then:

1. **Push folders first.** A page whose folder has never been pushed would arrive pointing at a row the server does not have, and a pull on another device would file it at the root until the folder caught up.
2. **Push documents in batches of 25.** A page carrying ink is large, and a person offline for a week can have many. One oversized body is a 413 that fails every page in it, including the small ones. Each batch is marked synced as soon as the server confirms it, and the mark does not touch `updatedAt`, so an edit made mid-flight stays dirty.
3. **Pull rows newer than the cursor,** rewind one second so a row written in the same instant as the cursor is not stranded on the far side of it. For each row: a local copy newer than the server's wins and stays dirty; a tombstone deletes the local row after clearing its index; an update reindexes; an insert stamps `filedAt` from `createdAt`, because a page from another device was filed on the device that made it and is not new capture here. Ink that was too large to push comes back null, and null means "not carried", not "erased", so the local drawing is kept.
4. **Advance the cursor** to the newest `updatedAt` seen, in the per-account defaults suite rather than the shared one, so two people on one device cannot read each other's position.

The one trigger in the app today is a write: every mutating path in the two view models ends in `requestSync()`. There is no launch or foreground pull, so on a second device the first pull happens as the tail of the first push.

THE ONE-TIME MIGRATIONS

`RootView.attachAll` runs three idempotent steps on every launch. `PlanNoteMigration.seedIfEmpty` creates one Journal folder, and only when the tree is completely empty, because pre-filling shelves with folders nobody asked for is how a fresh install feels like someone else's mess. `PlanNoteMigration.run` turns goals, content, notes and journal entries into pages (12.9). `NoteIndexMigration.run` stamps every page that predates the Inbox as filed, then rebuilds the index. That stamp is keyed by a marker in the per-account defaults, and the marker is spent only when there was history to interpret: on a fresh install the first launch runs before the first pull, and consuming the one-shot on an empty store would leave every page that arrives afterwards stranded in the Inbox.

12.7 Performance and cost

- **One write per pause, not per character.** The 600 millisecond debounce in the editor is the single most consequential number in this chapter: without it every keystroke would fire a `SwiftData` save and a `didSave` notification that reloads every view model in the app.
- **The title index is built once per rebuild, not once per page.** Letting each `reindex` build its own made `rebuildAll` quadratic in the size of the library: one fetch of every document per document, decoding the blocks of each untitled page every time, on the main actor before the UI was live. `rebuildAll` now builds the index once and passes it down.
- **A page with no links never fetches the library.** `reindex` scans for links first and returns before building a title index when there are none, which is most pages. Otherwise the editor would pay a fetch of every document on every debounced keystroke.
- **Delete and reinsert beats a diff.** A page holds tens of blocks, the rewrite is cheap, and a diff would need its own correctness argument for every field.
- **The sidebar is one read.** `NotesStore.snapshot` walks the folders and documents once to build the tree, the per-shelf counts, the recent list and the favourites. Four separate reads would each walk the same

rows.

- **Search has no index and does not need one.** It is a substring match over titles and block text, case and diacritic insensitive, capped at 40 results. That is fine until a person has thousands of notes, and the derived rows are where an index would go when it is needed.
- **Ink stays on the device past 1.5 megabytes.** A dense PencilKit page is a few hundred kilobytes, a pathological one is megabytes, and base64 adds a third. One runaway page must not stall every other page's sync behind it, so `NoteDocumentRow` leaves oversized ink out of the payload and the page syncs its text.
- **Opening a page does not dirty it.** `markOpened` advances `openedAt` without touching `updatedAt`, so reading on one device cannot fabricate an edit that wins a sync conflict against a real one on another.
- **Blocks travel as real JSON.** The `blocks` column is `jsonb`, so a malformed document is refused by Postgres at write time rather than surfacing on another device later, and a future server-side reader can query into it.
- **Cost is zero beyond Postgres rows.** No AI, no third-party service, no per-user fee. Two tables, five indexes, one policy each, all scoped to `auth.uid()`. `user_id` defaults to `auth.uid()` on the server and the client never sends it, so ownership is decided from the token rather than asserted by the device.

12.8 Edge cases and tests

CONDITION	BEHAVIOUR
Return on an empty list item	Outdents one level if it can, otherwise the item becomes a paragraph. That is how every editor gets out of a list.
Backspace at the start of a list item	Strips the indent first, then the kind, then merges into the block above. A plain paragraph merges its whole tail, not just the first word.
Backspace in the first block	Not the editor's to consume; UIKit does what it would normally do.
Backspace onto a drawn sketch	Refused. An untouched sketch is removed; one with ink is not swallowed.
Typing <code>---</code> then more text	Stays a paragraph. Dividers and code fences fire only on their own line.
A to-do turned into a paragraph and back	Keeps its tick, because <code>isChecked</code> is kept on every block.
Two pages share a title	The older one wins the link, every launch, because the title index is built in <code>createdAt</code> order.
A link to a page that does not exist	The edge is kept unresolved; following it creates the page in the current shelf.
Same block id on two pages (copy and paste)	Two task rows, scoped by document. Neither collides.
Page archived	Leaves every stream; its task rows stay in the index but the To-dos chip filters them by live page.
Page deleted locally	Tombstoned, index cleared, row reaped only after the server confirms the push.
Tombstone pulled for a page this device never had	Already satisfied; nothing is inserted.
Local edit newer than the server's copy	Local wins and stays dirty; the next push carries it up.
Ink larger than 1.5 MB	Text syncs, drawing stays local, and a pulled null never erases it.
Blocks JSON cannot be decoded	The editor gets a blank page; the original bytes stay on disk until saved over.
A row from the server with an unknown block kind	That block is dropped, not guessed at. A row with no id or timestamp is refused.
Fresh install, empty store	The index migration does not spend its one-shot, so pulled pages are not swept into the Inbox.
Signed out	Everything works locally; sync returns without an error. The first pass after sign-in carries it all up.
Page closed inside the debounce window	<code>flush()</code> on disappear and on leaving the active scene phase saves it anyway.

TESTS

The notes layer is the most heavily tested feature in the package: 151 tests across eleven suites, all runnable from the terminal with no simulator.

- `NoteBlockEditorTests` (35): every Return and Backspace rule above, indent bounds, numbered runs restarting, slash and link query detection, sketch insertion and protection.
- `NoteBlockTests` (15): prefix resolution and longest-match ordering, Markdown round trips, link scanning, and that a block written before `dueDate` existed still decodes.
- `NotesStoreTests` (34): filing rules, folder deletion rehoming, tombstones, dirty tracking, journal-once-per-day, backlinks, search reaching archived pages, capture, and that the index is updated by every write path including emptying a folder.
- `NoteIndexerTests` (16): rows in and out, block identity kept, deleted pages keep no rows, edges resolve or stay unresolved, rebuild is idempotent and clears orphans.
- `NoteIndexMigrationTests` (4): existing pages are stamped and indexed, a later capture is not swept up by a second run, an empty store does not spend the one-shot.
- `NoteStreamTests` (8): what each chip shows, newest first, archived pages leave every stream, only a page given a folder is born filed.
- `NoteWireFormatTests` (16): the round trip, blocks as JSON not a string, journal dates on the same calendar day, oversized ink left behind, the client never claiming ownership, unknown kinds dropped.
- `NoteSyncIndexTests` (4): a pulled page is indexed, an updated one reindexed, a tombstoned one loses its rows, a pulled page is not in the Inbox. `apply` is internal rather than private so these run without a network.
- `PlanNoteMigrationTests` (8) and `PlanStoreTests` (10): see 12.9.

12.9 The plan that remains

`PlanEntry` still exists with all five kinds, and `PlanScreen` still renders four sections. Only one of them is reachable. The notes tab's library offers a Habits row that pushes `PlanScreen` with `section: .constant(.habits)` and `showsSections: false`, and the add sheet in `RootView` sets `plan.section = .habits` before it opens. Goals, notes and content are pages now.

WHY HABITS STAYED BEHIND

A habit is a daily tick with a streak behind it. Today toggles those ticks live from its own screen, and turning a habit into a document would either break Today or leave two rows claiming to be the same habit. So `HabitTick` stays a flat table keyed by `entryID` and day, `PlanStore.tickedHabitIDs(on:)` answers a whole day in one fetch for Today, and the streak rule from Revision D is unchanged:

```
// LifeOSKit/Sources/Persistence/PlanStore.swift
public func streak(for entry: PlanEntry, endingOn end: Date = .now) throws -> Int {
    let ticks = try recentTicks(for: entry, days: 365, endingOn: end)
    var count = 0
    for (index, done) in ticks.reversed().enumerated() {
        if done { count += 1 }
        else if index == 0 { continue } // today still open
        else { break }
    }
    return count
}
```

Walk backwards from today, counting until a gap. An unticked today is skipped rather than treated as a break, because the day is not over; yesterday unticked still ends the streak.

HOW THE REST MOVED

`PlanNoteMigration.run` is idempotent through `originPlanEntryID` on each page rather than through a flag in `UserDefaults`, because a flag says "this device has migrated", which is the wrong question once a second device syncs the same pages down. PARA's own rule decides where each kind lands:

PLAN KIND	SHELF	FOLDER	PAGE KIND	NOTE
goal	Projects	Goals	task	Detail becomes blocks; status survives. Milestone targets were dropped, which the spec's Goals phase is meant to restore.
content	Projects	Content	task	Due date carried over.
note	Research	Notes	note	
journal	Areas	Journal	journal	The old title field was the whole text, so the day becomes the title and the text becomes the first block.
habit	stays			Not migrated.

Folders are made only when there is something to put in them, and the original plan rows are left in place. They are what the months already closed were scored against, and rewriting history to match a new filing system is the kind of thing an app should never do behind someone's back. `NoteEvidence` is the bridge the monthly close uses: journal dates come off journal pages by `entryDate`, and a project page's status is what its to-do blocks say when it has any, because ticking the last box is a clearer statement that a thing is done than remembering to flip a menu afterwards.

12.10 Study notes

READ IN THIS ORDER

1. `LifeOSKit/Sources/Persistence/NoteBlock.swift` : the block, the parser, the link scanner. Everything else is built on this shape.
2. `LifeOSKit/Sources/Persistence/NoteBlockEditor.swift` : what Return and Backspace mean, as pure functions. Read the tests beside it.
3. `LifeOSKit/Sources/Persistence/NoteDocument.swift` : the page and its four load-bearing nullable dates.
4. `LifeOSKit/Sources/Persistence/NotesStore.swift` : every read and write. Notice that `touch` is the only place the index is rewritten.
5. `LifeOSKit/Sources/Persistence/NoteIndexer.swift` : the two different deletes and the once-per-rebuild title index.
6. `LifeOSKit/Sources/Integrations/NoteSync.swift` : push before pull, and the four rules in its header comment.
7. `LifeOS/Features/Notes/View/NotesHubScreen.swift` : the measured three-column rule and the page that moves between column and stack.

8. `LIfеOS/Features/Notes/View/InkCanvasView.swift` : the Pencil rule, and why it used to be wrong.

CHECK YOURSELF

- **Why are blocks JSON inside the page rather than SwiftData rows?** Blocks are only ever read and written as a document; a relationship would add a migration hazard per field for no query that needs it. The derived rows exist for the one query that does.
- **What single field is the Inbox?** `filedAt` . Nil means nobody has chosen where the page belongs. Filing stamps it; editing never does.
- **Why does `reindex` fetch-and-delete while `rebuildAll` uses the batch delete?** The batch form cannot see rows inserted earlier in the same unsaved transaction, so inside `reindex` it would duplicate rows. `rebuildAll` clears already-saved rows before inserting anything, so the batch form is correct there.
- **What made a rebuild quadratic, and what fixed it?** Each page's reindex built its own title index by fetching every document. `rebuildAll` now builds it once and passes it down; a single save still builds it on demand, and only when the page has links.
- **Why push folders before documents, and why batches of 25?** A page arriving before its folder would be filed at the root on another device until the folder caught up. One oversized body is a 413 that fails every page in it, including the small ones.
- **Why did habits not become pages?** Today ticks them live from `HabitTick` ; a document version would either break Today or duplicate the habit.

13 The Life board and the nine sectors

Nine sectors of a life, scored once a month by a rule that shows its evidence and a person who has the last word. The same arithmetic that closes a month also projects the month still being lived, so the board can say where a sector will land and which lever moves it most.

13.1 Intent

Shiv had been scoring himself by hand every month across nine sectors: Family, Romance, Soul, Friends, Growth, Money, Mission, Body, Mind. The practice worked. What it lacked was evidence and memory: a feeling recorded on a page, nothing underneath it, and no way to see a year at a glance. The app already held much of the evidence and had no concept of a sector at all. The Life tab is that concept.

Five decisions settled the shape, and the reasoning is what a future reader needs.

- **The app proposes, the user decides.** Every month a rule proposes a score per sector with the rows behind it; the person confirms or overrides, and the person's number is what gets stored. Full automation was rejected because Romance and Soul have no honest data source, and one invented number costs trust in all nine.
- **Monthly close, as a ritual.** Scores change once a month. Between closes the board holds steady, and Today remains the daily surface. A continuously drifting score duplicates Today's job and invites gaming the number.
- **Rules score, the model explains.** A deterministic rule computes the number; the on-device model turns the evidence rows into one sentence. A model-judged score could differ on two runs with no way to see why, and a score you cannot explain is a score you stop trusting.
- **The close-time check-in fills the cold start.** A sector with no tracked data asks two or three short questions during the close. The answers become its evidence, stored and comparable next month, so every sector is scorable in month one.
- **No overall life score.** An average of nine sectors sits near the middle forever and moves by fractions, which reads as nothing happening in a month where one sector collapsed and another soared. The header carries the two facts that prompt action instead: the lowest sector and the biggest mover.

Two later projects extended the spine. The sector screens project gave every sector the same history screen and amended the original "board as hub" decision: Body, Money and Mission now open a history like the other six, with an extra row through to their full tab. The dry run project added the in-flight mode, where a sector shows a range it can still close in rather than a number, and a ranking of what still moves it.

13.2 Files and folders

```
Features/Life/
├── ViewModel/
│   ├── LifeBoardViewModel.swift    cards, header summary, close banner, board mode
│   └── MonthlyCloseViewModel.swift one pass through the nine sectors
```

		InFlightSectorViewModel.swift	band, levers, early answers for one sector
		SectorDetailViewModel.swift	fetch, map to records, one pure call
		View/	
		LifeBoardScreen.swift	banner, mode picker, header, the deck
		SectorStack.swift	the nine cards as an overlapping wallet deck
		CloseBanner.swift	the dashed "Close August" prompt
		MonthlyCloseScreen.swift	one sector per screen, Skip and Next
		SectorDetailScreen.swift	history in four bands
		InFlightSectorScreen.swift	range, levers, questions, both evidences
		CheckInQuestionView.swift	one question, shared by close and in-flight
		SectorPalette.swift	hue, icon and nine deck tones per sector
LifeOSKit/Sources/Persistence/			
		LifeSector.swift	the nine cases and their board order
		SectorScore.swift	SectorScore and CheckInAnswer, keyed by month start
		SectorStore.swift	record, commit, history, scoredCounts, answers
		Evidence.swift	EvidenceRow, Evidence, the one scoring rule
LifeOSKit/Sources/Sectors/			
		SectorScorer.swift	the protocol: values in, Evidence out
		BodyScorer.swift	days on target, sleep and exercise vs goal
		MoneyScorer.swift	budgets kept, kept of what came in, spend vs last month
		MissionScorer.swift	goals moved, habits kept; progressFraction
		GrowthScorer.swift	targets met, goals completed
		JournalScorer.swift	days written, vs last month (Mind and Soul)
		CheckInScorer.swift	answers to a 0...1 scale
		CheckInQuestion.swift	the question set per sector, ids append-only
		SectorEvidenceFactory.swift	which scorer serves which sector; Soul and Mind merge
		MonthWindow.swift	the instants one month spans
		MonthProgress.swift	elapsed and remaining days as of now
		MonthInputs.swift	everything the scorers need, already fetched
		MonthInputsLoader.swift	the store reads, in the package so they are tested
		ProjectedInputs.swift	coasting, perfect, one lever perfected
		Lever.swift	the seven dimensions a projection can perfect
		Leverage.swift	which lever moves a sector most
		SectorBand.swift	floor, ceiling, decided, rangeText
		ChosenScore.swift	seeding the stepper without overwriting a choice
		CloseSchedule.swift	which month the banner offers
		CloseProgress.swift	which sector is next
		AnswerPersistence.swift	immediate for a choice, debounced for text
		NoteRequest.swift	when to ask for a sentence, when to apply one
		SectorHistory.swift	months, question tracks, notes, as plain records
		SectorObservations.swift	two rules, three consecutive months each
		BoardSummary.swift	lowest and biggest mover
		BudgetPeriod.swift	budget assessment (shared with Money)
		LifeSectorTab.swift	which three sectors own a tab
LifeOSKit/Sources/Insights/Tasks/			
		SectorNote.swift	the on-device one-sentence task

The view models are deliberately thin. There is no test target for the app, so nothing put in a view model can be unit tested; every decision about a score, a band, a schedule or a history lives in the `Sectors` package, where 222 tests in `SectorsTests` and another 34 in `PersistenceTests` cover it.

13.3 Model

THE NINE SECTORS

```
// LifeOSKit/Sources/Persistence/LifeSector.swift
public enum LifeSector: String, Codable, Sendable, CaseIterable {
    case family, romance, soul, friends, growth, money, mission, body, mind

    /// Reading order on the board. Kept separate from `allCases` so that
    /// adding a sector later cannot silently reshuffle the grid.
    public static let boardOrder: [LifeSector] = [
        .family, .romance, .soul,
        .friends, .growth, .money,
        .mission, .body, .mind,
    ]
}
```

Raw values are written to the store, so a case may be added but never renamed: a rename orphans every score already recorded against it. The same append-only rule applies to check-in question ids.

A SCORED MONTH

```
// LifeOSKit/Sources/Persistence/SectorScore.swift
@Model
public final class SectorScore {
    public var sectorRaw: String
    public private(set) var month: Date           // always the first day of the month
    public var proposedScore: Int?                // what the rule said, nil with no evidence
    public var userScore: Int?                    // what the person said, nil until decided
    public var closedAt: Date?
    public var evidenceData: Data?               // the rows, frozen at close time
    ...
    public var isScored: Bool { userScore != nil }
}
```

`proposedScore` and `userScore` are both kept, permanently. The distance between what the rule computed and what the person felt is the most interesting number in the system, and averaging them would erase it. The evidence is archived rather than recomputed: change a goal in March and February's reasoning must still read the way it did in February. Every row is keyed by `Date.startOfMonth`, so closing August on 3 September files the score under 1 August.

THE ONE SCORING RULE

```
// LifeOSKit/Sources/Persistence/Evidence.swift
public struct EvidenceRow: Codable, Sendable, Equatable {
    public let label: String
    public let value: String          // already formatted, rendered verbatim
    public let normalised: Double     // clamped into 0...1 on the way in
    public let weight: Double         // never negative; zero means shown, not counted
}

public struct Evidence: Codable, Sendable, Equatable {
    public let rows: [EvidenceRow]

    public var proposedValue: Double? {
        guard !rows.isEmpty else { return nil }
        let totalWeight = rows.reduce(0) { $0 + $1.weight }
        guard totalWeight > 0 else { return nil }
        let weighted = rows.reduce(0) { $0 + $1.normalised * $1.weight }
        return (weighted / totalWeight) * 10
    }

    public var proposedScore: Int? { proposedValue.map { Int($0.rounded()) } }
}
```

Identical for all nine sectors: each row normalises to 0...1, rows are weighted, the weighted mean scales to 0...10 and rounds. One arithmetic path means one thing to test and one thing to explain. A sector with no evidence proposes nothing, not zero: zero out of ten for Romance in month one is both wrong and demoralising. Weights live in code, not in settings, because user-editable weights turn every score into a dial to twist until the number is agreeable. A zero-weight row is how a scorer shows a figure (earned, spent, unclaimed) without letting it argue.

THE SCORERS

SECTOR	SCORER	ROWS (WEIGHT)	PROPOSES NOTHING WHEN
Body	BodyScorer	days on target (2), sleep vs goal (1), exercise vs goal (1)	no day carries any data
Money	MoneyScorer	earned (0), spent (0), budgets kept (3, when buckets exist), kept of what came in (2 with buckets, 3 without), spend vs last month (1), unclaimed (0)	no transactions
Mission	MissionScorer	goals moved (2), habits kept (1)	no plan entries moved, or only blocked ones
Growth	GrowthScorer	targets met (2), goals completed (1)	no targets tracked and no goals
Mind	JournalScorer + CheckInScorer	days written (2), vs last month (1), how clear was your head (2)	nothing written and nothing answered
Soul	JournalScorer + CheckInScorer	days written (2), vs last month (1), how settled (2), a note (0)	nothing written and nothing answered
Family, Romance, Friends	CheckInScorer	two scored questions (2 each) and one free-text note (0)	no answers

Every scorer is built from values already fetched, never from a store, which is why `evidence()` neither throws nor awaits and why every scoring test runs without a container. Two details recur. Days with no data are excluded rather than scored as misses, so a charging night does not argue against the person. And "blocked" plan work returns nil from `progressFraction` rather than zero: waiting on someone else is not a failure of the person being scored.

Money's rule changed when budgets arrived. With buckets, adherence takes the top weight and the saving rate steps down, because a budget is a plan the person set and beating it measures what they decided mattered, where a saving rate measures what happened to be left over. Without buckets the evidence is exactly what it was before budgets existed, which `withNoBucketsTheEvidenceIsUnchangedFromToday` pins.

CHECK-IN QUESTIONS

```
// LifeOSKit/Sources/Sectors/CheckInQuestion.swift
case .friends:
    [
        CheckInQuestion(id: "friends.seen", prompt: "How often did you see friends?",
options: scale, weight: 2),
        CheckInQuestion(id: "friends.depth", prompt: "Did any of it go beyond small talk?",
options: [
            CheckInOption("no", 0.0), CheckInOption("once", 0.5), CheckInOption("more than
once", 1.0),
        ], weight: 2),
        CheckInQuestion(id: "friends.note", prompt: "Who is worth calling next month?",
options: []),
    ]
```

Each option carries its own place on the shared scale. A question with no options is free text: recorded as context, shown on the close, never scored. An answer that matches no option is stale data from a question that has since changed, and it is dropped rather than guessed at.

13.4 ViewModel

THE BOARD

`LifeBoardViewModel.load()` builds nine `Card`s in board order, each with a six-month history for the sparkline. In closed mode a card carries last month's `userScore`; in flight it carries a `SectorBand` instead. The header is `BoardSummary` (lowest and biggest mover, comparing the last two closed months) or, in flight, the days left and the mean share of the month that is decided. The close banner is decided by `CloseSchedule`.

```
// LifeOSKit/Sources/Sectors/CloseSchedule.swift
let oldestGap = scoredCounts
    .filter { month, scored in
        scored < totalSectors && month >= earliestAllowed && month <= previousMonth
    }
    .keys
    .min()

if let oldestGap { return oldestGap }
return scoredSectorsInPreviousMonth < totalSectors ? previousMonth : nil
```

The oldest month with a gap inside a twelve-month look-back is offered first, so a gap two months back surfaces before a more recent one and is never buried. The fallback to the previous month exists to break a deadlock in the naive version: a query over `SectorScore` rows only sees months that already have a row, rows are created only by `record()`, and `record()` runs only during a close. On a fresh install there are no rows, so the query returns nothing, no banner shows, and no close can ever start.

THE CLOSE

`MonthlyCloseViewModel` drives one pass. `advance()` asks `CloseProgress` for the first sector in board order that is neither committed in the store nor skipped this session, loads that sector's stored answers, loads the month's inputs once through `MonthInputsLoader`, and recomputes. `recompute()` is the single call into the factory, and it runs on every answer, including one keystroke at a time into a free-text field.

```
// LifeOS/Features/Life/ViewModel/MonthlyCloseViewModel.swift
func recompute() {
    guard let sector else { return }
    evidence = SectorEvidenceFactory.evidence(
        for: sector, inputs: monthInputs, answers: answers, calendar: calendar
    )
    proposed = evidence.proposedScore
    chosenScore = ChosenScore.seeded(
        current: chosenScore, hasChosen: hasChosenScore,
        proposed: proposed, previousUserScore: previousUserScore
    )

    let isEmptyNow = evidence.isEmpty
    if NoteRequest.shouldRequest(wasEmpty: evidenceWasEmpty, isEmptyNow: isEmptyNow) {
        scheduleNoteRequest(for: sector)
    }
    evidenceWasEmpty = isEmptyNow
}
```

Three pure policies sit behind that method, each a few lines and each tested.

- `ChosenScore.seeded` . Before the person touches the stepper, the number shown tracks the proposal, falling back to last month's score, then to 5. The moment they touch it, their pick is authoritative and no later recompute may replace it, however small the change that triggered the recompute.
- `AnswerPersistence.isImmediate` . A choice button is one discrete event and saves at once. Free text is a stream of keystrokes, and every `context.save()` makes `RootView` reload nine view models, so text is debounced 600 milliseconds and flushed synchronously before Skip or Next clears the answers.
- `NoteRequest` . The sentence is requested only on the transition from no evidence to some, after a 400 millisecond debounce, and applied only if the sector on screen is still the one it was asked about. Asking on appearance would ask about nothing for a check-in sector, and a request in flight when the close advances must not write its sentence over the next sector's.

`commit()` records the proposal and its evidence, then commits the chosen score. `SectorStore.record` returns an existing decided row untouched and `SectorStore.commit` refuses to overwrite a decided score, so the store does not depend on the view model getting the "never re-offer a committed sector" rule right. `skip()` moves on without writing anything; a skipped sector stays genuinely unscored and is offered again next time, because a score nobody looked at is noise in the history.

IN FLIGHT

`InFlightSectorViewModel.load()` reads the current month's inputs and answers, builds the `SectorBand` and the `Leverage` ranking, and notes whether the ceiling came from `GoalTargets.default` so the screen can say whose numbers they are. Answers given here are written against the current month through the same `saveAnswer` upsert the close uses, so an answer given on the 12th is visible, scored, and revisable at close with no new model.

13.5 View

THE DECK

`SectorStack` lays the nine cards out at their full height (190 points) and pulls each one up over the one before it, so all that shows is a 78-point band carrying the sector and its score. Tapping a card lifts it clear of the deck: its z-index jumps, the cards below slide down by the open height less the peek, and the space that appears carries the trend line, a six-month `RoundedBarChart`, and an Open button. Only one card is ever open, because a deck with several cards open is a list with gaps in it. An open card is sized to what it has to show: 320 points with a chart, 226 without, so a sector with one month does not spend space on a void.

Nothing renders below the band while a card is closed. A covered card has only the peek of room and anything taller gets sliced by the card in front; the trend line used to show as a clipped strip of letter-tops under every title. VoiceOver reads every card whole, because the visual truncation must never truncate what is spoken.

Nine cards that physically overlap need nine distinct colours, and the app's shared `ModuleHue` has six.

`SectorPalette.tone` is therefore a second, feature-local palette: one muted earth family in light and dark, with a single shared `cardInk` chosen so it clears contrast on all nine. The six-case `hue` mapping still exists for the close and the detail screen, which use shared components that take a `ModuleHue`.

THE BANNER

`CloseBanner` is deliberately not `AlertBanner`. That component is red on pink with an ECG glyph and it is the app's anomaly surface: it means something is wrong with your body. An unclosed month is a piece of admin, and borrowing the alarm styling for it spends the alarm. The banner is a dashed outline, collapsed by default, because the deck is the reason anyone opens this screen.

THE CLOSE SCREEN

One sector per screen, titled "4 of 9". Questions first where the sector has them, then a `PastelFillCard` with the proposal (its own dash when there is no evidence: not scored badly, not scored at all), the evidence rows label left and value right, the model's sentence if one arrived, "Last month you said 7", and a `HeroNumeral` with a 0...10 stepper. Skip and Next in the toolbar. The sheet owns its view model, and the board reloads on dismissal rather than on completion, because scoring three sectors and swiping the sheet away used to leave the deck showing dashes for scores that were already saved.

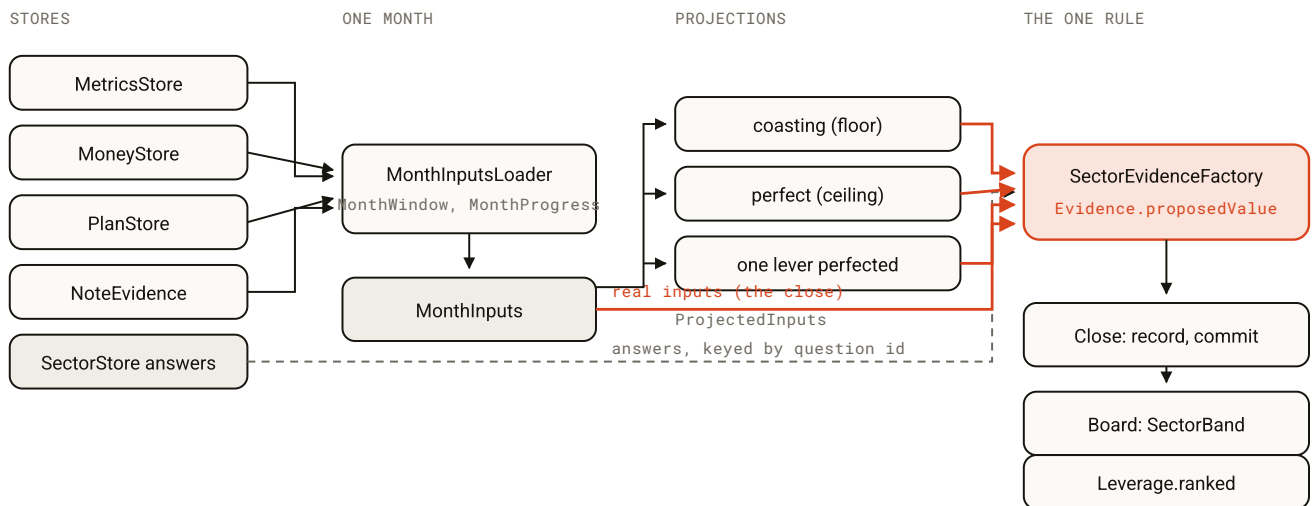
THE HISTORY SCREEN

Four bands, each omitted rather than rendered empty: the trend as bars from the window minimum (a score is a rating, not a count building from nought, so a flat year of 4s and a flat year of 9s must not both draw as full bars), the reasoning frozen at the latest close, every question's answers laid across the last six months with a dash for a month not answered, and the free-text notes newest first. Observations sit above them when there are any. Body, Money and Mission get one extra row, "Open Body", and `LifeSector.ownsTab` is the one place that decides which three.

THE IN-FLIGHT SCREEN

The range at display size, the days left, then "What moves it most" as a ranked list of levers with their worth in points, the check-in questions for a sector that has them, and the floor and ceiling evidence under "If nothing changes" and "If you finish at your targets". Money with no buckets says so in a line instead of inventing a ceiling. A ceiling built from default targets is disclosed in one line and points at Health.

13.6 How it works



There is one piece of arithmetic. The close hands the factory the real month; the in-flight board hands it three projected versions of the same month. A band therefore cannot disagree with the close that follows it.

LOADING A MONTH

`MonthInputsLoader.load` is the only place store reads happen for scoring. It builds a `MonthWindow` (the exact instants a month spans, with a half-open `[start, end)` for filtering and an inclusive `lastDay` for the stores whose `to:` bound is by day), reads metrics, goals, transactions, buckets, plan entries, project pages and journal dates, filters the undated stores by `updatedAt` or `createdAt`, and assembles one `MonthInputs`. It moved out of the close view model when the in-flight board needed the same thing for the current month, and the move fixed a bug the in-flight case exposed: `PlanStore.recentTicks` reads a day with no tick as a miss, so running the window to the end of a month still being lived scored every day still to come as a habit already broken. Habits are now judged only across the days already lived.

Notes changed what the loader reads without changing what old months mean. Journaling and goals moved to note pages, so `NoteEvidence` bridges them in, and the legacy plan entries are still read and summed alongside, because a month already closed must keep scoring the way it did.

PROJECTING THE REST OF IT

```
// LifeOSKit/Sources/Sectors/ProjectedInputs.swift
public static func coasting(
    _ inputs: MonthInputs, progress: MonthProgress, calendar: Calendar = .current
) -> MonthInputs {
    guard progress.remainingDays > 0, progress.elapsedDays > 0 else { return inputs }
    let remaining = Double(progress.remainingDays)
    let elapsed = Double(progress.elapsedDays)
    var projected = inputs

    if let average = averageReading(inputs.readings) {
        projected.readings += Array(repeating: average, count: progress.remainingDays)
    }
    let earned = inputs.amounts.filter { $0 > 0 }.reduce(0, +)
    let spent = -inputs.amounts.filter { $0 < 0 }.reduce(0, +)
    if earned > 0 { projected.amounts.append(earned / elapsed * remaining) }
    if spent > 0 { projected.amounts.append(-(spent / elapsed * remaining)) }
    projected.budget = inputs.budget.map { scaled($0, by: (elapsed + remaining) / elapsed) }
    ...
    return projected
}
```

Coast means continuing at the current pace, not stopping. This is the subtle one. For Body the two are nearly the same, since `BodyScorer` counts only days with data. For Money they are opposites: spending nothing for the rest of the month is the best case, so an empty tail would put the floor above the ceiling and invert the card. One definition serves every sector, and it is the one that reads correctly for spend.

Perfect fills the remaining days at the person's own targets. Not a theoretical maximum, which produces ceilings nobody reaches, and not a personal best, which is undefined in month one. A ceiling built from `GoalTargets` is reachable by definition, and it makes setting a target consequential for the first time. Only a tracked metric is raised, and always with `max` against the running average: a person already averaging 20,000 steps against an 8,000 target must not be "perfected" down to 8,000. Perfecting spend spends up to each bucket's remaining allowance and no further, and cannot repair a bucket already blown. Income is not a lever and keeps coasting. Goal statuses are never touched, because a ceiling must never assume a goal closes.

One lever perfected is built on the coast and then overwrites only the dimension named, so a lever's delta is never contaminated by a second lever quietly improving beside it. `Leverage.ranked` runs the factory once per tracked lever, subtracts the coasting value, clamps at zero (perfecting a lever cannot make a month worse, and a negative row would read as advice to stop), and sorts. A sector scored only from answers has no levers.

THE BAND

```
// LifeOSKit/Sources/Sectors/SectorBand.swift
public var decided: Double? {
    guard let floor, let ceiling else { return nil }
    return min(max(1 - Double(ceiling - floor) / 10, 0), 1)
}
```

Decided is literally the share of the score that can no longer move. The board header shows the mean across only the sectors that have a band, not all nine, because a sector nobody has evidence for has not been judged settled or unsettled; folding it in would let silence read as certainty. Money with no buckets has no ceiling: nothing in the data says what a good remaining spend would be, zero is a fantasy and any other figure is invented, so the card shows a floor and points at bucket setup. A ceiling below its floor is always a projection bug, never a fact, and is clamped up.

HISTORY AND OBSERVATIONS

`SectorDetailViewModel` fetches twelve months of scores and their answers, maps them to plain records, and makes one pure call: `SectorHistory.build`. Only decided months appear. Question tracks follow the order the questions are asked in and align by index to the months, so a missing answer leaves a gap rather than shifting later answers left. Free text becomes a note, not a track.

Observations are two deterministic rules, both needing three calendar-adjacent months so a single unusual month never fires one: the same answer three months running, and the person scoring below the proposal three months running. The second is the signal the spine kept `proposedScore` and `userScore` in separate columns to preserve: the rule is measuring something the person does not feel. "Consecutive" is checked on the calendar, not on array position, because a partial close can leave a sector unscored in the middle month and three entries spread across a longer stretch are not three months running. No model is called from this screen at all, which also keeps it free and instant.

THE SENTENCE

`SectorNoteTask` is an on-device `CoachTask` with no remote name. Its prompt is only the rows the rule used, so the sentence cannot cite a figure that played no part in the number, and it deliberately never carries last month's score, because the instructions forbid mentioning a score and feeding the model a number on the same scale works against that. The close runs it through the ordinary `CoachRouter` so availability caching and refusal handling behave as they do everywhere else; an unavailable model leaves the close fully usable with rows and no sentence.

13.7 Performance and cost

- **Inputs loaded once per sector, not per keystroke.** The close rebuilds `MonthInputs` in `advance()`; `recompute()` only re-runs the factory against values already in memory. Scoring is a handful of reductions over small arrays and runs on every keystroke without cost.
- **Free text debounced, choices immediate.** One `context.save()` per pause rather than per character, because every save reloads nine view models. The pending value is flushed synchronously on Skip, Next and screen dismissal so no keystrokes are lost.
- **One model call per sector, at most.** The sentence is requested on the transition into having evidence, debounced 400 milliseconds, on device, and discarded if the sector has moved on. Nothing about a sector leaves the phone.
- **No model on the board or the history.** Observations are rules. The board is nine store reads and a summary.
- **In-flight is eight factory runs per sector at most.** Two for the band and one per tracked lever (up to four for Body and Growth, one elsewhere). It is recomputed when the board mode switches and when a screen loads, not continuously.

- **The archive is a JSON blob per row.** Evidence is encoded once at close and decoded on read; nothing is recomputed against changed goals, and the history screen renders the stored strings verbatim.
- **Everything is local.** Sector scores, answers and evidence are SwiftData rows in the account's own store. There is no sync for them and no server table. Cost is zero.

13.8 Edge cases and tests

CONDITION	BEHAVIOUR
Fresh install, no rows at all	The banner offers the previous month; without the fallback no close could ever start
August never closed, September ends	August is offered first; the oldest gap within twelve months wins
A month older than the look-back is partial	Never offered; one ancient partial month must not pin the banner forever
Sector has no evidence	Proposes nothing; the card shows a dash and the stepper seeds from last month or 5
Person adjusts the stepper, then edits a note	The recompute runs but <code>ChosenScore</code> keeps their number
Leaving the close midway	Committed sectors stay committed; reopening resumes at the first unscored one
Skip	Nothing is written; the sector is offered again next time
Re-recording a decided sector	The store returns the row untouched; the archive never lies
Committing twice	The first decision stands
Note arrives after the close advanced	Rejected by <code>NoteRequest.shouldApply</code>
Model unavailable or refuses	Rows shown, no sentence, close fully usable
Closing August on 3 September	Filed under 1 August by <code>startOfMonth</code> , in the store's injected calendar
Twelve journal entries on one afternoon	One day written; distinct days are counted, not entries
Blocked plan item	Excluded from judgement rather than scored zero
Metric never logged this month	Absent from the Body rows and from Growth's target count, and not offered as a lever
Money in flight without buckets	Floor only, no ceiling, no decided share; the card explains why
Last day of the month	The band collapses to a single number; a finished month has nothing left to move
Already exceeding a target	The ceiling keeps the running average; it never asks for less
Bucket already blown	A perfect finish cannot unspend money; that row stays where it is
Skipped a sector in the middle month	Three entries are not three months running; both observation rules stay silent
December to January	Counts as consecutive across the year boundary
Answer matches no current option	Dropped; a changed question does not score stale text
Question last answered seven months ago	Its track is hidden rather than rendered as a prompt over dashes

TESTS

- `PersistenceTests/EvidenceTests` : the weighted mean scales to ten, no rows and zero total weight propose nothing, normalised values clamp, `proposedScore` is always the rounded `proposedValue` .
- `PersistenceTests/SectorStoreTests` and `SectorScoreTests` : idempotent record, commit closes once, re-recording leaves a closed score alone, `scoredCounts` reflects a genuine partial close, month normalisation under an injected calendar.
- `SectorsTests` , one suite per file: every scorer, the factory's routing and Soul and Mind's merge without double counting, `CloseSchedule` , `CloseProgress` , `ChosenScore` , `AnswerPersistence` , `NoteRequest` , `MonthWindow` (leap February), `MonthProgress` (today counts as elapsed), `MonthInputsLoader` against an in-memory container (habits judged only on days lived, a closed month still judging every day), `ProjectedInputs` (perfect scores at least as well as coasting, spend up to the allowance and no more, goals untouched), `Leverage` (a lever at target is worth nothing, never less than nothing), `SectorBand` (ceiling never below floor, collapses on the last day, Money without buckets), `SectorHistory` , `SectorObservations` , `BoardSummary` (ties go to the drop), and `LifeSectorTab` .
- `InsightsTests/SectorNoteTests` : the prompt carries every row, the previous score never reaches it, the task runs on device.

13.9 Study notes

READ IN THIS ORDER

1. `LifeOSKit/Sources/Persistence/Evidence.swift` , the one rule.
2. `LifeOSKit/Sources/Sectors/BodyScorer.swift` , then `MoneyScorer.swift` , to see two scorers shape rows.
3. `LifeOSKit/Sources/Sectors/SectorEvidenceFactory.swift` , the routing and the merges.
4. `LifeOSKit/Sources/Sectors/MonthInputsLoader.swift` , where the stores are read.
5. `LifeOS/Features/Life/ViewModel/MonthlyCloseViewModel.swift` , the close pass and its three policies.
6. `LifeOSKit/Sources/Sectors/ProjectedInputs.swift` , coasting and perfect.
7. `LifeOSKit/Sources/Sectors/SectorBand.swift` and `Leverage.swift` .
8. `LifeOS/Features/Life/View/SectorStack.swift` , the deck.

CHECK YOURSELF

- **Why does a sector with no evidence propose nil rather than zero?** It has not been judged badly; it has not been judged. Zero for Romance in month one is wrong and demoralising.
- **Why are `proposedScore` and `userScore` both stored?** The gap between them is the signal; the standing-gap observation is the first place it is read.
- **Why is the evidence archived rather than recomputed?** A goal changed in March must not rewrite what February's reasoning said.
- **What does coasting mean, and why not an empty tail?** The rest of the month looks like the part already lived; an empty tail would put Money's floor above its ceiling.

- **Why does the in-flight board never compute a score of its own?** The moment a projection scores by another path, the band and the close can disagree, and a band that contradicts the close is worse than none.
- **Why does `CloseSchedule` fall back to the previous month?** Rows exist only after a close, so a query over rows can never start the first close.

14 LIFO, the coach

LIFO is the one screen in Almanac that talks back. It reads a fortnight of your own numbers, your month's money and your nine sector scores, answers in cards rather than paragraphs, can listen and speak, remembers the conversation it is in, and, once a day at most, speaks first. This chapter is how all of that is built, where the words go, and what it costs.

14.1 Intent

WORKING BACKWARDS

Ask LIFO how you slept this week and it answers with a table of your actual nights, on the cloud when the cloud can answer and on the phone when it cannot. Say it out loud and it stops listening when you stop talking, reads one warm sentence back while the cards are still arriving, and never reads a table aloud. Ask a follow-up and it knows what it said. Some mornings it sends a line first: three short nights against your own baseline, a streak that today would break, a strong week worth naming. Tap it and the conversation opens on that sentence.

Which model answers?

The cloud first, the device second. The cloud tier is OpenAI's `gpt-5-mini` behind the `lifo-agent` Edge Function; the device tier is Apple's Foundation Models. It used to be the other way round on the grounds that the free engine should be tried first, and the result was a worse common answer with the better one appearing only when the weaker model failed outright. Quality picks the tier; a metered daily cap on the server is what keeps that affordable.

What leaves the phone?

On a cloud turn, one rendered text block: a fortnight of derived daily numbers, the month's money summary with up to thirty recent transactions, the sector scores, and the last twenty turns of the conversation. Never the raw Whoop series (HRV, SpO2, skin temperature, respiratory rate), which exist only in the on-device render. Every answer carries a disclosure you can open that shows the verbatim text that was sent and which tier answered.

How does it avoid inventing numbers?

Two ways. The prompt is the data: the model is told to cite only figures that appear in the text it was given, and the render omits a missing field rather than writing zero. The reply is then parsed by `CoachResponse` into paragraphs, tables and lists, and a table cell displays exactly the string the answer supplied; nothing is recalculated on the way to the screen.

Why cards instead of a paragraph?

A paragraph about six metrics is six numbers hidden in prose. The coach is instructed with a small presentation grammar (a marker string, then rules: short paragraph for a simple question, a two-column table for metrics, up to three bullets for actions, at most three sections) and the screen renders those shapes natively. A short answer stays short; nothing is forced into a dashboard.

What the coach deliberately does not do: it has no tools. The calendar assistant in Chapter 15 carries six and runs a tool loop; LIFO carries none, which is exactly what lets its whole reply be streamed. It does not diagnose, it does not pick investments, and it declines requests that are not about coaching this person over their own data, because the guardrail that says so lives on the server where a client cannot strip it.

14.2 Files and folders

Features/Coach/	
└─ Model/	
└─ AssistantVoice.swift	three ElevenLabs stock voices, off by default
└─ ElevenLabsSpeechClient.swift	Scribe speech to text, multipart upload
└─ ElevenLabsVoiceClient.swift	Flash text to speech, plus VoicePlayer
└─ SpeechListener.swift	mic capture, endpointing, transcription
└─ ViewModel/	
└─ CoachViewModel.swift	the session: phases, send, hold and reveal, seed
└─ View/	
└─ LifoCoachScreen.swift	text screen and voice screen over one model
└─ CoachResponseView.swift	paragraphs, metric cells, comparison tables, actions
└─ CoachVoiceOrb.swift	SceneKit sphere driven by audio level
└─ LifoAura.swift	the blue or green gradient behind everything
└─ CoachDesignPreview.swift	DEBUG fixture, simulated audio, no network
App/ChatTier.swift	builds the one RemoteChatEngine both screens share
App/PushService.swift	APNs registration, inbox, the tap-through
LifeOSKit/Sources/Insights/	the inference layer; reads Persistence only
└─ CoachTask.swift	Tier, CoachTask, BriefTask, AnswerTask
└─ CoachTool.swift	the tool contract, shared by both tiers
└─ ToolGate.swift	PendingWrite and ConfirmationBroker
└─ ToolInvoker.swift	per-turn cap, gating, error shaping
└─ CoachRouter.swift	one-shot routing and CoachResult
└─ EscalationPolicy.swift	what each on-device error means
└─ TierPreference.swift	Automatic, Cloud, On device
└─ ModelAvailability.swift	Apple's availability, collapsed to three
└─ AssistantTurn.swift	the chat router: cloud first, device after
└─ CoachResponse.swift	the presentation grammar and its parser
└─ ResponseStyle.swift	house style, the cleaner, the conversation style
└─ SpokenReply.swift	splits the SAY: line from the shown answer
└─ SpeechCapturePlan.swift	which transcriber owns the sentence
└─ SpeechEndpointer.swift	when a spoken turn is over
└─ AudioEnvelope.swift	attack and release smoothing for the visuals
└─ BriefCache.swift	one brief per day (not wired to a screen yet)
└─ Engines/	
└─ Engine.swift	one method, both tiers
└─ OnDeviceEngine.swift	LanguageModelSession per call
└─ RemoteEngine.swift	RemoteWire and the one-shot cloud call
└─ SessionTool.swift	binds a CoachTool into a session
└─ Chat/	
└─ ChatEngine.swift	ChatTurnMessage, ChatInstructions, the protocol
└─ ChatWire.swift	request body, stream frames, reply parsing
└─ OnDeviceChatEngine.swift	Foundation Models driving a streamed turn
└─ RemoteChatEngine.swift	the tool loop and the SSE reader
└─ ToolSchema.swift	CoachTool to an OpenAI function declaration
└─ Context/	
└─ MetricsDigest.swift	the privacy boundary; the render per audience
└─ ContextBundle.swift	digest plus money plus sectors, under one budget
└─ Tasks/	CoachAnswer · DailyBrief · SectorNote
LifeOSKit/Sources/Persistence/ChatMessage.swift	conversations, local only
supabase/functions/lifo-agent/index.ts	the door: auth, budget, fetch, stream relay

supabase/functions/lifo-nudge/index.ts	the hourly job: triggers, claim, phrase, send
supabase/functions/_shared/lifo.ts	SCOPE, task table, body builders, parsers
supabase/functions/_shared/nudge.ts	the four triggers and the volume rules
supabase/functions/_shared/apns.ts	ES256 provider token and the send
supabase/migrations/20260827090000_lifo_usage.sql	
supabase/migrations/20260830090000_proactive_nudges.sql	

WHY THE BRAIN LIVES IN THE PACKAGE AND THE MOUTH IN THE FEATURE

Everything that decides is in `Insights` : which tier, what the model may see, when a sentence ends, how a reply splits between voice and screen. Everything that touches hardware is in `Features/Coach` : the microphone, the audio session, SceneKit, the network calls to ElevenLabs. The line is drawn so that every decision is a value type with a test, and the feature folder is left holding only the parts that need a device to run.

14.3 Model

A TASK NAMES A FLOOR, NOT A MODEL

```
// LifeOSKit/Sources/Insights/CoachTask.swift
public enum Tier: Sendable, Equatable {
    case onDevice
    case cloud(Role)
    public enum Role: String, Sendable, Codable { case reasoning, vision }
}

public protocol CoachTask: Sendable {
    associatedtype Output: Generable & Decodable
    associatedtype Context: Sendable
    var floor: Tier { get }
    var remoteName: String? { get }
    var instructions: String { get }
    func prompt(_ context: Context, for audience: MetricsDigest.Audience) -> String
}
```

Three things in this declaration carry most of the design. `Output` is both `Generable` and `Decodable` , so one struct yields a schema for Apple's model and decodes the JSON that comes back from the cloud; that constraint is what lets `RemoteEngine.run` stay generic. `remoteName` is the server-side name of the task, or nil for one that never leaves the phone: the model behind a name lives in the Edge Function, so changing which model answers is a deploy, not an App Store release. And `prompt` takes an audience, which is the whole privacy mechanism in one parameter: a remote engine can only ever ask for the `.offDevice` render.

Three tasks exist. `AnswerTask` is named `answer` on the server and takes a `ContextBundle` . `BriefTask` and `SectorNoteTask` are on-device only; the sector note is what the monthly close in Chapter 13 asks for, and the brief has a cache but no screen yet.

A CHAT TURN IS A THREAD, NOT A PROMPT

```
// LifeOSKit/Sources/Insights/Chat/ChatEngine.swift
public struct ChatTurnMessage: Sendable, Equatable {
    public enum Role: String, Sendable { case user, assistant }
    public let role: Role
    public let text: String
}

public struct ChatInstructions: Sendable, Equatable {
    public let onDevice: String // may name data that must not leave the phone
    public let cloud: String    // this one travels
}

public protocol ChatEngine: Sendable {
    func reply(
        to thread: [ChatTurnMessage],
        instructions: String,
        tools: [any CoachTool],
        invoker: ToolInvoker,
        onPartial: @escaping @Sendable (String) -> Void
    ) async throws -> AssistantTurn.Reply
}
```

`ChatInstructions` is two strings rather than one because the tiers are not owed the same render. A single string would have to choose, and whichever it chose would be wrong on the other tier. `onPartial` is handed the answer **so far**, cumulative, not the piece that just arrived. Foundation Models yields snapshots and the cloud yields deltas, and a screen that assigns the whole text it is given cannot lose a fragment the way one that appends can.

WHAT THE SCREEN HOLDS

```
// LIfiOS/Features/Coach/ViewModel/CoachViewModel.swift
struct LifoTurn: Identifiable, Equatable {
    let id: UUID
    let question: String // empty when LIFO spoke first
    let answer: String
    let sent: SentContext? // nil for a seeded opening line
}

struct SentContext: Equatable {
    let text: String // the verbatim render, not a description of it
    let tier: AssistantTurn.Tier
    var headline: String {
        let place = tier == .cloud ? "the cloud" : "this device"
        let facts = text.split(separator: "\n")
            .filter { !$0.trimmingCharacters(in: .whitespaces).isEmpty }.count
        return "\(facts) lines of your data, answered on \(place)"
    }
}
```

`SentContext.text` is the thing itself. A summary of what was sent is a second thing that can be wrong; the verbatim string cannot. The headline counts lines rather than characters because the render is one fact per line and "1,847 characters" is a number nobody has a feel for.

THE CONVERSATION ON DISK

`ChatMessage` is a `SwiftData` model with a conversation id, a role stored as a raw string (so the enum can gain cases without a migration), the text, tool summaries, and an optional JSON blob of calendar event ids. `ChatStore.recent(conversationID:limit:)` returns the last twenty messages oldest first, which is the order both a transcript and a prompt want. The store is local only; chat history never leaves the device. The coach and the calendar assistant write into the same store under different conversation ids, each kept in the account's own `UserDefaults` suite so two people sharing a phone cannot resume into each other's transcript.

14.4 ViewModel

`CoachViewModel` is `@MainActor @Observable` and owns one session. Its state is a phase (`idle`, `listening`, `thinking`, `answered`), the live transcript while listening, the answer as it streams, a status line, a history of turns, and the question in flight.

```
// LIfiOS/Features/Coach/ViewModel/CoachViewModel.swift
var pendingQuestion = ""           // on screen the instant it is asked
var pendingSent: SentContext?
private var holdingAnswer = false
private var latestShown = ""
private var heldTurn: LifoTurn?
private var revealTimeout: Task<Void, Never>?
private var spokeThisTurn = false
private var turnID = UUID()

private let chatRemote: (any ChatEngine)? = ChatTier.remote()
private let speech = SpeechListener()
let voicePlayer = VoicePlayer()
```

`pendingQuestion` exists because a turn used to be appended to history only once there was an answer to append it with, which meant the one moment you most want to see what you said, while it is being thought about, was the one moment nothing showed it. It is pinned before any store read in `send`, so whatever the rest of the method does or fails to do, the question is on screen from the moment it was asked.

The `send` method does the following, in order: stops any speech in or out, pins the question, mints a new `turnID`, decides once whether the answer will be held for the voice, builds the context bundle from the last fourteen days of `MetricsStore` rows plus whatever `bundleExtras` returns, appends the question to `ChatStore`, reads the thread back, renders one instruction string per audience, and hands everything to `AssistantTurn.run`. The `bundleExtras` closure is set by `RootView` and pulls the money snapshot and the sector cards out of the view models that own them, so the coach never holds screens it does not render.

```
// LIfeOS/Features/Coach/ViewModel/CoachViewModel.swift
let reply = try await AssistantTurn.run(
    instructions: ChatInstructions(onDevice: onDevice, cloud: cloud),
    thread: thread,
    tools: [],
    broker: ConfirmationBroker(),
    remote: chatRemote,
    onPartial: { [weak self] text in
        Task { @MainActor in
            guard let self, self.phase == .thinking else { return }
            self.received(text)
        }
    }
)
```

Every failure is mapped to a sentence the screen can act on. A refusal shows the model's own words. A spent budget says "That is today's thinking budget used up. It resets tomorrow." An on-device context overflow says the question covered too much at once, not that the network is down. A missing session says to sign in again, which used to read as a connectivity problem and sent people to check their wifi. Anything else is logged with the real error and, depending on whether a session exists, either asks for a retry or offers the Apple Intelligence setting through `needsAppleIntelligence`. On every failure the half-streamed answer is cleared: a fragment under a failure line reads as LIFO contradicting itself.

A ROUTER THAT IS BUILT AND NOT USED

`CoachViewModel` still constructs a `CoachRouter` with a `RemoteEngine`, but `send` routes through `AssistantTurn.run` and never calls it. The router is the one-shot path and its live caller today is the monthly close's sector note. The unused property is a leftover from before the conversational core landed, and it is safe to delete.

14.5 View

`LifoCoachScreen` is one view with two presentations, chosen by `CoachScreenStyle`: `.text`, a cobalt gradient fading into ink with a composer along the bottom, and `.voice`, a sage gradient fading into forest black with the orb in the middle and a microphone button below. Switching never recreates the model, so an answer already being generated survives the switch. The screen forces the dark scheme and caps its content at 800 points wide, which is how it reads on an iPad without becoming a wall of text.

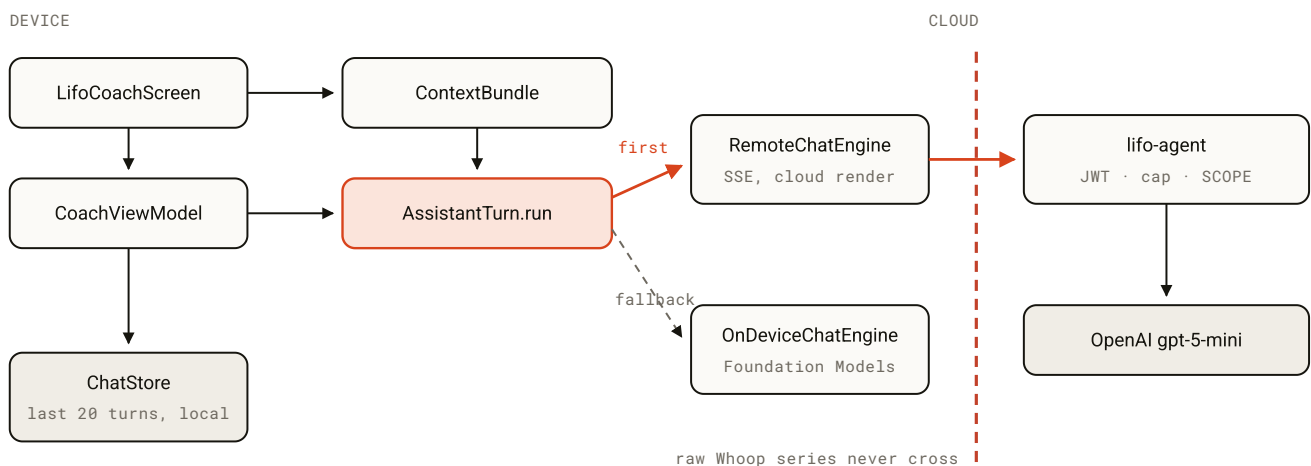
The text screen shows an opening when the transcript is empty (a two-line headline, three suggestion cards for sleep, spending and today's focus) and the transcript otherwise. Each turn is a white bubble for the question, right-aligned and black on white because your own words are the brightest thing on the screen, then a `CoachResponseView` card, then a closed `DisclosureGroup` labelled with the sent headline and a cloud or iPhone glyph. The turn in flight draws from `pendingQuestion` with a "Thinking" row until the first chunk lands, and then the answer as it is written, with no cursor and no per-character animation because the text arrives fast enough that animating it would slow it down.

`CoachResponseView` parses the reply and renders each block: a paragraph as text with inline markdown attributes, a two-column table of up to six rows as a grid of metric cells (one column at accessibility type

sizes), a wider table as a horizontally scrolling `Grid` with alternating row tints, a list as numbered action rows with dividers. Each block fades and lifts in a beat after the one above, with state on the block so a block already on screen is never re-animated when the text under it streams on.

The voice screen shows `CoachVoice0rb`, a SceneKit sphere with a physically based emerald shell, a geometry shader that swells the surface with layered sine currents, a surface shader that paints jade bands and a faint shimmer, and an additive halo plane behind it. A `CADisplayLink` at 30 to 60 frames per second feeds the shader a phase and an energy uniform; energy comes from `AudioEnvelope`, which smooths the microphone or playback level with a fast attack and a slow release so the sphere breathes rather than twitches. Drag tilts it, tap gives it an impulse, Reduce Motion freezes it, Reduce Transparency hides the halo, and leaving the screen invalidates the display link. Under the orb sit the live transcript while listening, the pending question and streaming answer, or the latest turn with its disclosure.

14.6 How a turn is answered



One question, two engines. The cloud is tried first because it answers better; the device is what still works when the cloud will not. The thread lives on the phone and travels with every turn.

THE ROUTER

`AssistantTurn.run` restates `CoachRouter`'s rules for the chat path. With no remote engine configured, the device answers. Otherwise the tier preference is read now, not captured, because the switch in Settings can be flipped while the screen is open. `onDevice` means the device and nothing else. `cloud` means the cloud with no fallback, because someone who chose the stronger model wants to be told when it did not answer rather than quietly handed a weaker one. `automatic` tries the cloud and falls back to the device on any error except a refusal.


```
// LifeOSKit/Sources/Insights/AssistantTurn.swift
} catch RemoteEngineError.refused(let reason) {
    throw RemoteEngineError.refused(reason) // never re-asked of a second model
} catch {
    guard await invoker.didExecuteGatedWrite == false else { throw error }
    guard availability() == .available else { throw error }
    let retry = ToolInvoker(tools: tools, broker: broker)
    onPartial("") // clear the cloud's half sentence
    return try await device.reply(
        to: thread, instructions: instructions.onDevice,
        tools: tools, invoker: retry, onPartial: onPartial
    )
}
```

Two guards matter. A refusal is never retried on the other tier, because a second model is not a second opinion on a refusal, it is a way around one. And a turn in which the user already confirmed a gated write is never retried, because the write has already happened and a fresh invoker would let a second model ask for the same create, raise a second card, and get a second yes from someone with no way to see that the first landed. That guard is set the moment the gate opens, not after the call returns, since a write that threw may still have reached the calendar.

THE CLOUD ENGINE

`RemoteChatEngine.reply` has two paths. With no tools, which is the coach's case, it sends one request with `stream: true` and reads the server's event stream line by line through `ChatWire.payload(inLine:)` and `ChatWire.streamEvent(payload:)`. The text is accumulated in the engine, cleaned, and handed to `onPartial` whole on every frame. A refusal or a spent budget arrives as an ordinary JSON body with a status, never inside the stream, so the failure is read the same way as on the unstreamed path. A stream that carried no text at all throws `unavailable` rather than returning an empty bubble.

With tools, which is the calendar assistant's case, it runs the loop: send the thread and the function declarations, and if the reply is tool calls, run each one on the device through the shared `ToolInvoker`, append an assistant message carrying the calls and a tool message per result, and send again. The loop runs at most `ToolInvoker.invocationLimit + 1` rounds, seven, bounded by the invoker's own cap rather than a second number of the engine's invention. A call whose arguments will not parse is still answered, with a message telling the model to explain rather than retry, because a provider left waiting on a result simply asks again and spends the remaining rounds on it.

```
// LifeOSKit/Sources/Insights/Chat/ChatWire.swift
public static func request(baseUrl: URL, anonKey: String, accessToken: String,
                           messages: [Message], tools: [any CoachTool],
                           context: String = "", stream: Bool = false) throws -> URLRequest {
    var request = URLRequest(url: baseUrl.appendingPathComponent("functions/v1/lifo-agent"))
    request.httpMethod = "POST"
    request.setValue(anonKey, forHTTPHeaderField: "apikey")
    request.setValue("Bearer \(accessToken)", forHTTPHeaderField: "Authorization")
    var body: [String: Any] = [
        "task": "chat",
        "messages": messages.map(\.payload),
        "tools": try tools.map { try ToolSchema.function(for: $0) },
    ]
    if !context.isEmpty { body["context"] = context }
    if stream { body["stream"] = true }
    request.httpBody = try JSONSerialization.data(withJSONObject: body)
    return request
}
```

The user's data travels in a `context` field of its own, never as a system message the client wrote. The server wraps it in a system message of its own placed behind `SCOPE`. A client-authored system message would land after the guardrail, providers weight the later one heavily, and that is a way around the guardrail rather than a way to carry data; the server refuses any message whose role is not user, assistant or tool.

THE DEVICE ENGINE

`OnDeviceChatEngine` creates a `LanguageModelSession` per turn, binds any tools through `SessionTool`, flattens the thread into one prompt with `User:` and `Assistant:` lines, and streams the response. Foundation Models yields snapshots of the whole answer so far, which is exactly the shape `onPartial` is defined in, so there is nothing to accumulate. The style appended is `ResponseStyle.conversation` unless the instructions carry the structured coach marker, in which case `CoachPresentation.instruction` is used instead, so the device writes cards too.

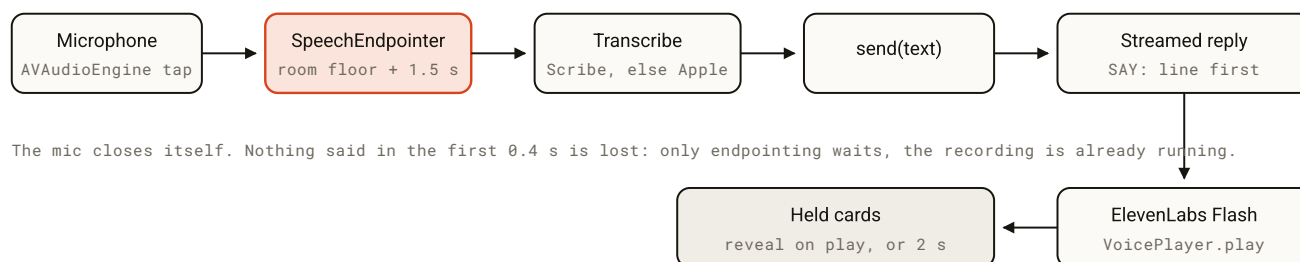
THE SERVER'S SIDE OF THE CONTRACT

`lifo-agent` is the door. It rejects anything but POST, resolves the caller from the bearer token through `resolveUser` (the anon key admits the app to the project; the token says which user is asking and is what gets billed), validates the body with `parseRequest`, reads today's `lifo_usage` row for the `chat` kind and returns 429 `exhausted` at 150,000 tokens, builds the OpenAI body, calls the provider once with `reasoning_effort` and `verbosity` set to low and once more without them if the provider returns 400, and either relays the stream or parses the reply. It debits what the provider billed on every path, success, refusal and parse failure alike, through the atomic `lifo_debit` function, because a refusal still consumed real tokens and leaving those unmetered would let a user burn unlimited budget by asking off-topic questions in a loop. The bundle in the request is used for one completion and discarded; log lines carry ids, task names, token counts and latency, never content. Chapter 20 covers the function internals; what matters here is the shape the device relies on.

```
// supabase/functions/_shared/lifo.ts
const SCOPE = `You are LIFO, a personal life coach inside the LifeOS app.
You coach one person using only their own data, which arrives in the prompt:
health metrics, sleep, workouts, money, and life-sector scores.
Stay on that scope. Decline requests unrelated to coaching this person over
their own data (essays, code, general world knowledge) and redirect to what
their data shows. Never give medical diagnoses; describe patterns and suggest
talking to a professional where it matters. Never recommend specific
investments or securities; keep money talk at budgeting and pattern level.
Cite only numbers that appear in the data given to you; never invent one.`;
```

SCOPE is first in every system prompt and always. The conversational style behind it is a verbatim copy of `ResponseStyle.conversation`, and both a Swift test and a Deno test pin the same prohibitions and permissions, because the copy had already drifted once at birth: it dropped the quotation-mark rule and the "if you can answer without asking, answer" clause, and the cloud model was allowed two habits the device model was not. Two prompts that are meant to agree and are written twice do not stay agreeing unless something checks.

14.7 Speech in, speech out



The spoken turn. Two transcribers can run at once but only one owns the sentence; the reply is asked for with a spoken first line so the voice can start before the last table row exists.

CLOSING THE MIC WHEN THE SENTENCE ENDS

SpeechEndpointer is the whole of the "stop when I stop talking" rule, as a value type with no audio types in it. The listener feeds it one RMS reading per buffer, timed by audio frames over the sample rate rather than the wall clock, so a busy thread cannot put time into a silence the speaker never left. A single fixed level cannot work: what means speech in a bedroom means silence in a café. So the room measures itself. The first 0.4 seconds learn the noise floor from the quietest frame; after that the floor learns only from non-voice frames, drops the moment the room does and climbs slowly, and is clamped to 0.004 to 0.12 so a calibration that lands on speech cannot put the bar above every later voice.

```
// LifeOSKit/Sources/Insights/SpeechEndpointer.swift
public var openThreshold: Double { max(noiseFloor * 3 + 0.020, 0.055) }
public var continueThreshold: Double { max(noiseFloor * 2 + 0.012, 0.035) }

guard elapsed - lastVoiceAt > tuning.trailingSilence else { return .keepListening }
guard spokenFor >= tuning.minSpeech else {
    hasHeardSpeech = false; speechStartedAt = 0; lastVoiceAt = 0    // a blip, not a sentence
    return .keepListening
}
decided = true
return .finished
```

Two thresholds, because a frame has to be clearly louder than the room to begin a sentence but only a little louder to continue one; that is what keeps the soft tail of a word from reading as the end of a thought. The tuning is named rather than left as numbers in a callback: 1.5 seconds of trailing silence ends the turn, voice under 0.35 seconds is a cough, seven seconds of nothing closes the mic, and sixty seconds is the backstop for a room so loud every frame reads as voice. A verdict is latched once so buffers still in flight cannot finish the same turn twice.

TWO TRANSCRIBERS, ONE SENTENCE

ElevenLabs Scribe transcribes a clip uploaded once the turn is over, which is accurate but cannot show a word while someone is still speaking. Apple's recognizer reports partial results as they are heard.

`SpeechCapturePlan` decides the split: with a Scribe key Apple runs in the `liveTextOnly` role, pinned to on-device recognition where the hardware allows so the clip does not go to a second vendor merely to draw the words, and its final result is dropped so the two transcripts cannot race. Without a key Apple is the transcriber and is never pinned, since that transcript is the sentence and accuracy outranks the preference. A recognizer failure ends the turn only when Apple owned the sentence; on the Scribe path it has cost the words under the waveform and nothing else.

`SpeechListener` is deliberately not on the main actor: the audio engine's tap runs on the I/O thread, and every `AVAudioSession` activation runs on a private serial queue because `setActive` on the main thread stalls the UI. The clip is always written to a temporary WAV, on both paths, so a second pass over the file is possible when the live partials came back empty. When the endpointer says the sentence is over, `onEndOfSpeech` fires before any transcript exists, and the screen switches to "Hearing that back" at that moment rather than claiming to listen through the whole upload. Level callbacks are throttled to one every 80 milliseconds.

SAY ONE LINE FIRST

When the reply will be spoken, the model is asked for one extra line, prefixed `SAY:` , before the written answer: one or two warm sentences, at most one figure, no markdown, not a reading of the details.

`SpokenReply` parses this from partial text as well as whole text, because a stream spends most of its life half arrived, and reports `isSpokenLinePending` while the text so far could still turn out to begin with the prefix.

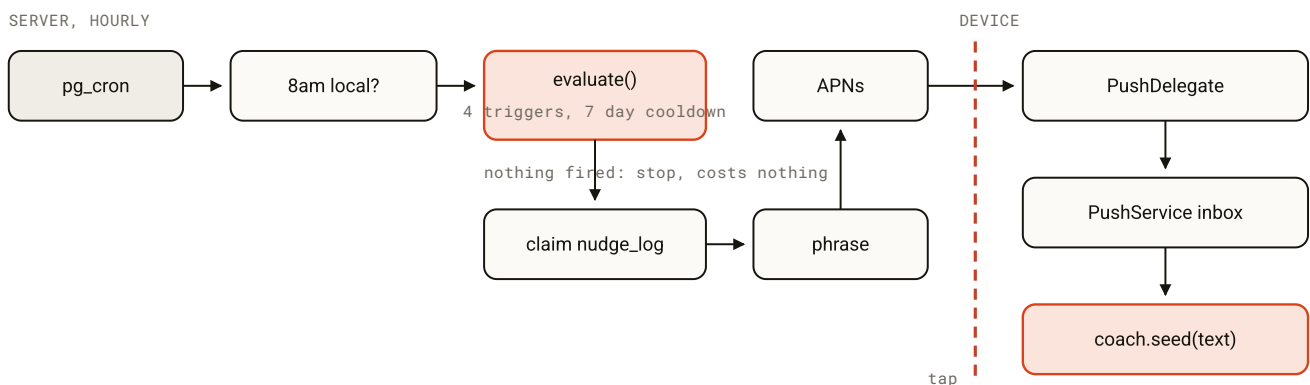
```
// LifeOSKit/Sources/Insights/SpokenReply.swift
public static let prefix = "SAY:"
public let spoken: String? // nil until the line is complete, nil when there is none
public let shown: String // empty while the spoken line is still arriving
public let isSpokenLinePending: Bool
```

The view model holds the cards while the voice fetches. `received(_:)` parses each chunk, keeps `latestShown`, and the first time a spoken line is complete calls `speak`, which fetches audio from `ElevenLabsVoiceClient` using the `eleven_flash_v2_5` model for latency and plays it through `VoicePlayer`. Whatever happens to the audio, a failure, a cancellation, or a voice switched off between asking and answering, `reveal` runs on the way out, because silence is acceptable and a blank screen is not. A two-second timeout reveals regardless, so a slow synthesis is a reason to hear first, not a reason to see nothing. The `turnID` keeps a late exit from an old reply from revealing a new one early, and `spokeThisTurn` keeps a line from being read once per chunk. The spoken line is for the voice only: it is not rendered, not kept in history, and not written to the store the next prompt is built from. If the model was asked for a line and did not write one, the first plain paragraph of the answer is read instead, never a table.

Speech is billed per character, so `ElevenLabsVoiceClient` caps the text at 1,200 characters, cut on a sentence boundary because a voice stopping in the middle of a number sounds like a fault rather than a limit. `VoicePlayer` holds its `AVAudioPlayer` because one that is deallocated stops playing, sets the session to spoken audio that ducks other apps, meters the level every 33 milliseconds for the orb, and hands the session back on stop so a podcast returns to full volume. Voice is off by default, since a coach that starts talking in a room the phone knows nothing about is a worse default than silence, and the three voices (Rachel, Adam, Sarah) are ElevenLabs stock voices with stable public ids, offered only once speech is on.

14.8 LIFO speaks first

Every sentence LIFO produced before September was a reply. The proactive half is a scheduled job that reads rows the server already has, decides by rule whether there is anything worth saying, phrases it with one small model call, and sends it through APNs. The design decision that shapes everything is **rules decide, the model writes**. Letting a model read a nightly digest and decide for itself was rejected on two grounds: it bills for every user every night whether or not anything happened, and models calibrate "is this worth interrupting someone" poorly, usually toward yes.



The proactive path. Silence is the common outcome and a valid one; only a fired trigger reaches a model, and only to choose the words.

THE FOUR TRIGGERS

`_shared/nudge.ts` holds four pure functions from rows to an optional fired result, each carrying a name, the numbers that made it fire, and a deterministic fallback sentence that is sent when the model is unreachable so an outage never swallows an observation.

- **short_sleep**: three nights each more than an hour under the person's own median over the previous four weeks, requiring at least fourteen baseline nights. Not a fixed seven hours: a fixed number tells a habitual six-hour sleeper they are failing every day.
- **steps_collapsing**: the last three days average under 60 percent of the trailing fortnight, with a baseline of at least 2,000 steps so someone who does not carry their phone is not nudged about steps.
- **streak_at_risk**: a workout streak of three or more days counted back from yesterday, since at eight in the morning nobody has trained yet.
- **strong_week**: a week averaging 25 percent above the previous three, or a personal best in the last three days against at least 21 days of history.

The positive is evaluated first. A genuinely strong week is rare, the negatives are still true tomorrow, and a channel that only ever carries bad news gets switched off, after which none of the rest matters. A missing day is dropped, never treated as zero, so a phone that has not synced does not fire a sleep nudge.

VOLUME AND DELIVERY

The job runs hourly because the send hour, eight in the morning, is local, and each run picks up whichever users' `device_tokens` row carries a timezone that has just reached it. A row whose timezone the runtime does not recognise never sends, on the rule that a nudge at the wrong hour is worse than no nudge. Nothing sends outside seven to nine local whatever the send hour is set to. One push per user per day is enforced by the primary key on `nudge_log (user_id, day)`: the job inserts before it phrases, and a second overlapping run conflicts on 23505 and stops. The same trigger cannot fire again for seven days. The job makes one query per table for the whole batch rather than three per user, then loops sequentially because the expensive step is a model call per firing user and a burst of parallel APNs connections is how a sender gets throttled. A dead token (APNs says unregistered) is deleted so the job does not push at it forever.

On the phone, `PushDelegator` is the only reason the app has a `UIKit` delegate at all, because the device token arrives through `didRegisterForRemoteNotificationsWithDeviceToken` and nowhere else. `PushService` keys the registration to the active account, upserts the token and current timezone on every foreground (which is what keeps the hour right for someone who has flown somewhere), and deletes the row on sign out before the session is thrown away. The table makes the token its own primary key, so registering under a second account replaces the first account's row in one upsert and the wrong-account push is unrepresentable rather than merely unlikely. Authorization is asked for from the notification inbox, never at launch, because a permission prompt before the app has shown anything worth being interrupted about is the fastest route to a permanent no.

A tapped notification lands in the inbox and sets `PushService.pending`; `RootView` opens the coach on it through `coach.seed(_)`. The sentence is written to `ChatStore` as an assistant turn as well as appended to the transcript, because the store is what `send` renders into the thread, and a nudge that skipped it would be on screen and invisible to the model answering the reply. The payload carries the sentence, the trigger name and the day, deliberately not the figures: a phone that has been off for days would otherwise open on numbers two days stale, so whatever the conversation goes on to say is worked out against the store as it

stands now. A seeded turn has an empty question and draws no bubble, since an empty bubble reads as a message you sent and deleted.

WHAT IS STILL NOT BUILT

Device-side triggers (a sector ceiling slipping out of reach, a spend spike), the `sector_snapshots` upload and the open-commitment rows from the proactive spec are slice 3 and have no code. The server path is complete to the APNs request; the QA report notes that no real push had been delivered when it was written, and that the APNs key, the App ID capability and the `schedule.sql` cron job are one-time setup steps rather than code. Until `APNS_KEY` is set the function returns `apns_not_configured` and sends nothing, on purpose.

14.9 The privacy boundary

`MetricsDigest` is the only thing an engine is ever given about health. `DailyMetrics` is a `SwiftData` class, not `Sendable`, full of per-source detail; the digest is a value type of derived numbers. Because `Engine.run` takes a digest, the compiler rather than a reviewer stops a raw row reaching a model. The render then differs by audience:

```
// LifeOSKit/Sources/Insights/Context/MetricsDigest.swift
if audience == .onDevice {
    if let v = day.hrvMs { parts.append("hrv \(number(v))ms\(\delta(v, averages.hrvMs))") }
    if let v = day.spo2Pct { parts.append("spo2 \(number(v))%") }
    if let v = day.skinTempCelsius { parts.append("skin \(number(v))C") }
    if let v = day.respiratoryRate { parts.append("resp \(number(v))") }
}
```

HRV, SpO2, skin temperature and respiratory rate are the raw Whoop physiology, and they appear only in the on-device render. Everything else in a day block (recovery, sleep and its stages, debt, strain, resting heart rate, steps, exercise, workouts with their high-zone minutes) is a derived daily figure and travels. A calibrating recovery is marked rather than dropped. A delta against the fortnight baseline appears only where a baseline exists, so a comparison is never implied that was not made.

`ContextBundle` adds the month's money and the sector scores. On device it renders the digest plus one sector line, because the local window is small and a transaction list would push the health data out of it. Off device it renders everything under one 8,000 character budget: the user, sector and money-summary lines are built first and never trimmed, the digest takes what is left and drops whole days oldest first to fit, and the recent transactions get only what remains, which is why they are the first thing to disappear under a tight budget.

WHAT	ON-DEVICE TURN	CLOUD TURN	STORED ON THE SERVER?
Derived daily metrics, 14 days	rendered	rendered	no
HRV, SpO2, skin temp, respiratory rate	rendered	never	no
Money summary and up to 30 transactions	summary only	rendered	no
Sector scores and deltas	rendered	rendered	no
The last 20 turns of the thread	rendered	sent	no; ChatStore is local
The spoken clip	Apple, on device where possible	ElevenLabs Scribe when a key is set	no
Token counts per day	none	lifo_usage	yes, numbers only
Nudge sent, which trigger, which day	none	nudge_log	yes, never the sentence

The disclosure under every answer is how this is made honest rather than asserted. The row shows which tier answered, reported by the engine rather than inferred from the preference, because an automatic turn falls back to the device exactly when it matters and a screen that worked out the tier from Settings would name the wrong one. Opening the row shows the verbatim render.

THE ELEVENLABS KEY

Speech to text and text to speech call ElevenLabs directly from the app with a key in `Secrets.xconfig`. The agent spec's second slice moves both behind a `lifo-speak` Edge Function so the key leaves the bundle; that function does not exist yet. Until it does, the voice path is the one place a vendor key ships in the binary.

14.10 Performance and cost

- **The daily cap is enforced where it can be.** 150,000 tokens per user per day for chat, 4,000 for nudges, read before the call and debited after it through `lifo-debit`, an upsert-and-add in one SQL statement so two concurrent turns cannot both read the same balance and each write their own. The two kinds are separate rows so a heavy chat evening cannot eat the next morning's nudge. When the cap is hit the router falls back to the device rather than to a bill or an error.
- **Low reasoning effort.** The chat body asks for `reasoning_effort: low` and `verbosity: low`. The default is medium and medium is most of the wait; a coaching question over a fortnight of numbers is not a reasoning problem, the thinking already happened on the device when the digest was built. Low also spends fewer tokens, the same lever as the cap pulled from the other end.
- **Streaming, with usage.** The wait for a first word is the whole of how fast this feels, and it used to be the wait for the last one. The server re-emits the provider's stream in its own `{"delta"}` shape so the device never learns a provider's chunk format, asks for `stream_options.include_usage` so a streamed turn is still metered, and debits in `flush`, which a client that walks away still reaches.

- **Character budgets on the prompt.** The digest fits 6,000 characters on device and the whole bundle 8,000 off device, pinned by tests, dropping whole days oldest first so a day that renders at all renders completely and the newest day, the one a question is about, is always present.
- **Missing fields cost nothing.** A blank field is omitted rather than written as nil; the same information for zero tokens.
- **One request per turn for the coach.** No tools means no rounds. The calendar assistant's loop is bounded at seven rounds and the invoker refuses the seventh call.
- **Speech is capped and cleaned.** 1,200 characters at most, with markdown stripped first, because paying a per-character meter to narrate asterisks is paying twice for a formatting bug. The reply is only asked for a spoken line when the voice screen is up and speech is on, so a typed question costs no extra tokens for a voice nobody will hear.
- **The orb is cheap.** One sphere with 96 segments, built once; the motion is a GPU shader driven by two uniforms, with no per-frame mesh allocation, and the display link stops when the screen leaves or Reduce Motion is on.
- **Brief cache.** `BriefCache` is an actor with an in-flight map so two callers arriving for the same day generate once, and only a real answer is kept, since caching a refusal would strand the user without a brief until midnight. It is ready for the day the brief gets a screen.
- **The nudge job batches.** One query per table across every user due this hour, a 94-day lookback stated in one constant so a new trigger cannot silently be starved of rows, and roughly 300 tokens in and 60 out per fired user.

The agent spec's cost check assumed twenty questions a day at about 3,000 tokens in and 500 out on the mini model, one brief a day, and speech on half the replies, and landed around \$0.60 for mini traffic, \$0.40 for escalations and \$0.50 for Flash speech per user per month, inside the ceiling in Chapter 24 with the ledger capping the tail. Those are modelled, not measured.

14.11 Edge cases and tests

CONDITION	BEHAVIOUR
Cloud refuses	The model's words are shown. Never retried on the device; a refusal is surfaced, not laundered.
Daily cap spent	Automatic falls back to the device. On the cloud-only preference the screen says the budget resets tomorrow and offers no retry.
Offline, DNS, timeout, 500	<code>unavailable</code> ; automatic falls back to the device, otherwise "try again in a moment".
Session lapsed	<code>notSignedIn</code> before any network call; the message says to sign in, not to check wifi.
On-device context overflow	Its own sentence: the question covered too much at once. Not the cloud message, not the Apple Intelligence prompt.
Apple Intelligence off	Re-read on every request, never cached; the error offers the Settings deep link.
Device not eligible	The one availability the router remembers for the life of the process.
Stream drops mid sentence	<code>unavailable</code> ; half a sentence is never returned as the whole one, and the fragment is cleared from the screen.
Stream carries no text	<code>unavailable</code> , never an empty bubble.
Model emits markdown on the plain path	<code>ResponseStyle.clean</code> strips it on every partial, so asterisks never show for a beat.
Model writes a malformed table	Rows stay visible as text; no cells are invented.
Nothing said into the mic	Closes after seven seconds with "I did not hear anything"; no clip uploaded.
Cough or chair scrape	Under 0.35 seconds of voice is forgotten and the turn goes back to waiting.
Scribe fails	Apple recognises the recorded file; if that fails too, the user is asked to type.
Speech permission refused with a Scribe key	The turn proceeds without live words. Without a key it cannot proceed.
Voice fetch fails or is cancelled	The held answer is revealed on the way out; two seconds is the ceiling either way.
New question while the last is being read	<code>stopSpeaking</code> first; the turn id prevents the old voice task revealing the new answer.
Same notification tapped twice	<code>seed</code> sees the sentence already first in history and reopens rather than repeats.
Two cron runs overlap	The second insert into <code>nudge_log</code> conflicts and that run stops before phrasing.
Provider rejects the tuning parameters	One retry without them; a slower answer rather than no answer.
Provider omits usage	A zero debit is skipped, but the reply is still delivered; noted as the one path the cap cannot see.

TESTS

The Insights target is the most heavily tested part of the package, because almost everything in it is a pure decision. The suites and what they pin:

- `AssistantTurnTests` , 15: cloud first, device fallback, spent budget falls back, signed-out is served by the device, refusal never crosses tiers, each tier gets its own render, a confirmed write is never retried, the preference cannot conjure a cloud that is not configured.
- `CoachRouterTests` , 18, and `EscalationPolicyTests` , 9: every branch of the one-shot router and every disposition of the escalation table, including that contention retries locally and is never billed, and that availability is re-read rather than trusted forever.
- `RemoteChatEngineTests` , 14: the streamed path yields the whole answer so far each time, only a toolless turn asks for a stream, a spent budget is still the budget on the streamed path, a tool call runs on the device and its result is sent back, unparseable arguments are answered rather than stranded, a loop that will not converge is bounded, the context is repeated on every round.
- `ChatWireTests` , 12, and `RemoteEngineTests` , 11: request shape, both credentials, the context in its own field, the 429 and 403 mappings, a wrong-shaped 200 is `unavailable` , a signed-out user never reaches the network.
- `MetricsDigestTests` , 16, and `ContextBundleTests` , 7: the off-device render carries no raw Whoop series, nil never becomes zero, naps stay separate from nights, whole days are dropped oldest first, a fully populated fortnight fits the budget, transactions never push the render past budget, a collapsed budget still emits one day.
- `SpeechEndpointerTests` , 10, `SpeechCapturePlanTests` , 8, `SpokenReplyTests` , 8, `AudioEnvelopeTests` , 3: the made-up rooms, the transcriber split, the SAY line from partial text, and frame-rate independence.
- `ResponseStyleTests` , 18, `CoachResponseTests` , 8: the cleaner, the instruction forbids what the cleaner strips, conversation keeps every typography prohibition and grants exactly three permissions, tables keep units and missing values, escaped pipes do not add columns, the voice reads table values without markdown.
- `ToolInvokerTests` , 6, `ToolSchemaTests` , 4, `CoachToolTests` , 2, `SchemaEncodingTests` , 3, `BriefCacheTests` , 5.
- `ChatStoreTests` , 8, in Persistence: oldest first and capped, conversations do not bleed, event ids survive a reload.
- On the server, `lifo_test.ts` has 44 Deno tests (a client system message is refused, the thread, tool list and context are bounded, the chat prompt keeps every prohibition and the three permissions, the tuning can be lifted off for the retry, a stream is only granted to a turn with no tools), `nudge_test.ts` 25 (a habitual short sleeper is not told they are failing, a phone that has not synced does not fire, the positive wins, every fired trigger carries a sendable sentence), and `apns_test.ts` 9.

What is not tested by a suite: the microphone path end to end, real ElevenLabs round trips, a real push delivery, the orb's frame rate on hardware, and whether a given nudge was worth interrupting someone for. The QA reports say so, and the last of these needs a week of real days on a real device.

14.12 Study notes

READ IN THIS ORDER

1. `Insights/CoachTask.swift` , for the floor, the audience and the three tasks.

2. `Insights/Context/MetricsDigest.swift` , then `ContextBundle.swift` , to see exactly what a model is shown and how the budget trims it.
3. `Insights/AssistantTurn.swift` , the chat router, with `CoachRouter.swift` beside it for the one-shot rules it restates.
4. `Insights/Chat/RemoteChatEngine.swift` and `ChatWire.swift` , the stream reader and the tool loop.
5. `supabase/functions/_shared/lifo.ts` and `lifo-agent/index.ts` , the other end of the wire.
6. `Features/Coach/ViewModel/CoachViewModel.swift` , following `send` from top to bottom, then `received` , `speak` and `reveal` .
7. `Insights/SpeechEndpointer.swift` with its test file open next to it.
8. `supabase/functions/_shared/nudge.ts` , then `lifo-nudge/index.ts` .

CHECK YOURSELF

- **Why does the cloud go first when the device is free?** Quality picks the tier; the device answered worse on the questions this app gets, and the daily cap makes the cloud affordable.
- **Why is a refusal never retried on the other tier?** A second model is not a second opinion on a refusal; it is a way around one.
- **What stops a raw Whoop row reaching OpenAI?** The type system: `Engine.run` takes a `MetricsDigest` , and the remote engine can only request the `.offDevice` render, which omits the four physiology fields.
- **Why is `onPartial` cumulative?** Because both tiers can honestly produce it and a screen that assigns cannot lose a fragment the way one that appends can.
- **Why does the context travel in its own field and not as a system message?** So the server can place it behind `SCOPE` ; a client system message would land after the guardrail and outweigh it.
- **Why two thresholds in the endpointer?** Beginning a sentence must clear the room clearly; continuing one need only stay a little above it, so the soft tail of a word is not the end of a thought.
- **Why does the nudge job insert into `nudge_log` before it phrases?** The primary key on `(user_id, day)` is the one-per-day rule, and inserting first makes two overlapping cron runs safe without any locking logic.
- **What is the one place a vendor key still ships in the app?** ElevenLabs, until `lifo-speak` exists.

15 The calendar assistant

The assistant is LIFO's sibling with hands. It reads your calendar through six tools, creates events, and asks before it changes or deletes one. The gate that asks is a property of the code, not a request in the prompt, and the reply is drawn as a day you can move around in rather than described in a sentence.

15.1 Intent

Ask "what's on tomorrow?" and the answer needs no tool call: today and tomorrow are inlined in the prompt. Ask "find me an hour on Thursday" and the model calls `find_free_time` against the local event cache. Say "move the dentist to four" and it calls `get_events` to learn the id, then `update_event`, which does not run until a card on screen shows the event as it is now, what it would become, and whether a location is being dropped, and you tap Confirm. Every reply carries an agenda card: a week strip with a dot on the days the reply touched, and the whole of the selected day's agenda underneath, read live from the store rather than from the model's prose.

The assistant is separate from the coach on purpose. The coach's context is metrics and it has no tools; the assistant's context is the calendar and it has six. Neither task sees the other's data. They share the engines, the router, the invoker, the confirmation gate and the chat store, which is why the assistant honours the same tier preference the coach does and drifts nowhere on its own.

THE WEAKER PRIVACY GUARANTEE, STATED

Every other cloud workload sends derived numbers. On a cloud turn the assistant sends event titles, times and locations, which are content and cannot be aggregated without destroying their use. The control is the tier preference in Settings: "On device" means nothing you ask ever leaves the phone, for the assistant as for the coach. The design spec also asked for a one-time disclosure before first use; that copy does not exist in the app today and is an open item.

15.2 Files and folders

Features/Assistant/	
└─ ViewModel/	
└─ AssistantViewModel.swift	one conversation, pending cards, events per reply
└─ View/	
└─ AssistantSheet.swift	transcript, confirmation cards, chips, composer
└─ AssistantAgendaCard.swift	the week strip and the day's rows under a reply
LifeOSKit/Sources/Assistant/	
└─ AssistantContext.swift	depends on Insights and Persistence only
└─ CalendarAccess.swift	CalendarAssistant.tools, instructions, contextPrefix
└─ CalendarEventCollector.swift	CalendarReading and CalendarWriting protocols
└─ CalendarTools.swift	the events a turn actually touched
└─ ToolDates.swift	six CoachTool conformers and their argument types
	ISO-8601 in, "Tue Aug 26 09:00" out

Reused from Insights (Chapter 14)

— CoachTool.swift	the contract
— ToolGate.swift	PendingWrite, ConfirmationBroker
— ToolInvoker.swift	the per-turn cap and the gate
— AssistantTurn.swift	cloud first, device after
— Chat/RemoteChatEngine.swift	the tool loop
— Engines/SessionTool.swift	the on-device binding

Reused from Persistence and Integrations

— CalendarStore.swift	the cache; events(from:to:), freeSlots, events(withIDs:)
— CalendarSync.swift	writes go to the provider, then a re-sync
— EventKitSource.swift	the one authorized source today
— ChatMessage.swift	replies remember event ids, not events

15.3 Model

WHAT A TOOL MAY TOUCH

```
// LifeOSKit/Sources/Assistant/CalendarAccess.swift
@MainActor
public protocol CalendarReading: Sendable {
    func events(from: Date, to: Date) throws -> [CalendarEventSnapshot]
    func snapshot(id: UUID) throws -> CalendarEventSnapshot?
    func freeSlots(from: Date, to: Date, durationMinutes: Int) throws -> [DateInterval]
}
extension CalendarStore: CalendarReading {}

public protocol CalendarWriting: Sendable {
    func create(_ draft: CalendarEventDraft) async throws
    func update(id: UUID, with draft: CalendarEventDraft) async throws
    func delete(id: UUID) async throws
}
```

Reads come from the cache and writes go through `CalendarSync`, which the app declares conformant in one line at the top of the view model. That is the whole reason a write made by asking lands in the same place as a write made by tapping: the sync routes the draft to the source that owns the event, then re-syncs so the cache reflects what the provider actually stored, since providers normalise all-day handling and time zones and trusting the local draft would drift.

THE SIX TOOLS

TOOL	ARGUMENTS	NEEDS	GATED
<code>get_events</code>	start, end	reading	no
<code>find_free_time</code>	start, end, durationMinutes	reading	no
<code>analyze_schedule</code>	start, end	reading	no
<code>create_event</code>	title, start, end, isAllDay, location, notes	writing	no
<code>update_event</code>	id plus the full draft	reading and writing	yes
<code>delete_event</code>	id	reading and writing	yes

Each argument set is a `@Generable` struct with `@Guide` descriptions, so the same declaration yields a schema for Apple's session and, through `ToolSchema`, an OpenAI function declaration with `additionalProperties: false`. Dates are ISO-8601 on the way in and rendered as `EEE MMM d HH:mm` in the user's zone on the way out. A garbled date throws a `ToolDateError` whose description tells the model the format to use, and the invoker turns that into a result string the model can recover from rather than a crash.

```
// LifeOSKit/Sources/Assistant/CalendarTools.swift
func eventLine(_ event: CalendarEventSnapshot, timeZone: TimeZone = .current) -> String {
    let start = ToolDates.render(event.startDate, timeZone: timeZone)
    let end = ToolDates.render(event.endDate, timeZone: timeZone)
    let span = event.isAllDay ? "all day" : "\(start) to \(end)"
    return "\(event.id.uuidString) | \(span) | \(event.title)"
}
```

Every tool result and the inlined context render an event with its id first, and the instructions say never to invent one. The model can therefore only reference ids it has actually seen, and an update or delete with an unknown id returns "No event with that id" rather than touching the wrong row. Creates are not gated, per the design decision; a created event is announced and appears on the card, where it can be edited or deleted by hand.

WHAT A TURN TOUCHED

Tools hand the model text, because that is what a model consumes. The same events arrive in `CalendarEventCollector` as values, because that is what a card needs. The alternative was parsing the reply back into events, which means trusting a model to restate a date correctly in order to draw it correctly, and a card that disagrees with the sentence above it is worse than no card. The collector is an actor, deduplicates by id (a turn asking about today and then this week surfaces the same lunch twice) and returns chronologically whatever order the tools ran in.

A reply persists the touched event ids on `ChatMessage.eventIDsData` as JSON, optional so an existing store migrates without a plan. Ids rather than events, because an event can be moved or deleted after the sentence about it was written, and a card showing what was true last Tuesday is worse than no card. They are re-

resolved against the store whenever the conversation is reopened, and one that has since been deleted simply stops appearing.

15.4 ViewModel

`AssistantViewModel` takes its `ModelContext` in `init`, unlike every other view model in the app, because it builds a `ChatStore`, a `CalendarStore`, an `EventKitSource` and a `CalendarSync` from it. That is why `RootView` creates it once inside `attachAll()` rather than at property declaration. Its state is the messages, the pending writes, the events per reply, whether it is thinking, whether the calendar is authorized, and the draft.

```
// LIfiOS/Features/Assistant/ViewModel/AssistantViewModel.swift
func send() async {
    let text = draft.trimmingCharacters(in: .whitespacesAndNewlines)
    guard !text.isEmpty, !isThinking else { return }
    draft = ""
    isThinking = true
    defer { isThinking = false; pending = [] }

    try? chat.append(conversationID: conversationID, role: .user, text: text)
    reloadMessages()

    let broker = ConfirmationBroker { [weak self] write in
        Task { @MainActor in self?.pending.append(write) }
    }
    self.broker = broker
    let collector = CalendarEventCollector()
    let tools: [any CoachTool] = isAuthorized
        ? CalendarAssistant.tools(reading: store, writing: sync, collector: collector)
        : []
    ...
    let reply = try await AssistantTurn.run(
        instructions: ChatInstructions(
            CalendarAssistant.instructions(authorized: isAuthorized) + "\n\n" +
calendarContext()
        ),
        thread: thread, tools: tools, broker: broker, remote: remote
    )
    let events = await collector.collected()
    try? chat.append(conversationID: conversationID, role: .assistant,
        text: reply.text, toolSummaries: reply.toolSummaries,
        eventIDs: events.map(\.id))
}
```

Three things are per turn: the broker, the collector, and (inside `AssistantTurn`) the invoker. The broker is rebuilt so a card from an old turn cannot resolve a new one; the collector so the cards under a reply are the events that reply was about rather than everything the conversation has ever looked at; the invoker so the six-call cap and the activity chips are turn state. With no authorized calendar the tool list is empty and the instructions say so, so the model explains what to connect rather than calling something that would fail.

The instructions are one string for both tiers, unlike the coach's two, because nothing in them is device-only and the current date and time zone the context prefix carries are exactly what the cloud has no other way to know; without them an update gets built against no notion of today. The conversation id lives in the account's own defaults suite under its own key, distinct from the coach's, because the two screens write into one store and "whichever conversation was written to last" is no longer necessarily this screen's own.

`confirm(_:)` and `cancel(_:)` remove the card from `pending` immediately and then resolve the broker. The broker prunes its own copy on resolve; without the first line the view's copy lingered with live-looking buttons until the whole turn finished. `events(on:)` reads one day straight from the store on every call rather than caching, because a person moving across the strip is asking about seven different days in a few seconds, a one-day fetch is a single indexed range query, and the alternative is a cache invalidated by every write the assistant makes. `save(_:editing:)` and `delete(id:)` from the card's own sheet go through the same `CalendarSync` the tools write through.

15.5 View

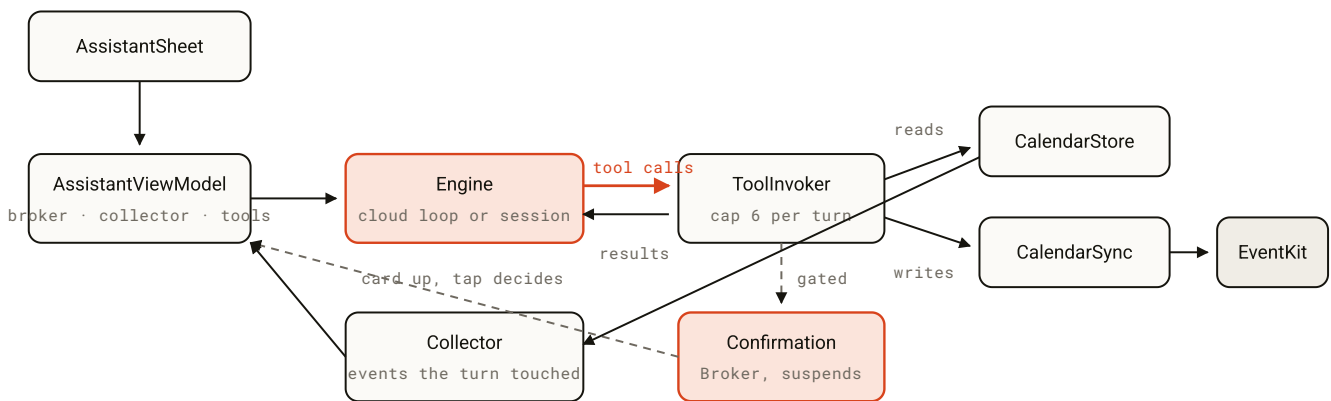
`AssistantSheet` is presented from the quick actions bar on every tab through the `calendar.badge.clock` action (not a bare calendar glyph, because the month button in Today's top bar is already that). It is a `NavigationStack` with a Schedule link to `MonthScreen` on the left and Done on the right. If Apple Intelligence is unavailable on the device the whole sheet is one sentence saying so. Otherwise it is the conversation, a reconnect row when the calendar is not authorized but history exists (the empty state carries the connect button, and a conversation with history hides that state, so someone who revoked access later still needs a way back in), and the composer.

The conversation is a `LazyVStack` of user bubbles, assistant replies, then the pending confirmation cards, then a "Thinking" row while a turn is in flight and no card is up. The list scrolls to the newest message or the newest card as either count changes. An assistant reply is an `InsightCallout` of the text, an `AssistantAgendaCard` when the reply touched events or is the latest reply (a reply about the schedule that draws no schedule is the text-only answer this screen was changed to stop producing; older replies keep a card only if they touched events, so scrolling back through twenty turns is not twenty week strips), and a row of activity chips ("Checked your calendar", "Created an event") from the invoker's summaries.

A confirmation card lists the preview lines the tool produced and two buttons. For an update the lines are the event now, what it becomes, and, when the arguments carry no location and the event has one, "Removes the location", because the update is a full replacement and the card must surface what a glance would miss. For a delete it is the one line of the event being deleted.

`AssistantAgendaCard` opens on the day the reply was about, not today: a question about next Tuesday that answers itself with this morning is not an answer. The header is the weekday large with an accent dot, filled on today and hollow otherwise, and the date stacked small on the right. The strip is the seven days of that week; days the reply touched carry a three-point dot that is always laid out and only coloured when set, because hiding it would shift every other column as the selection moved. The rows are the selected day's whole agenda, all-day events first, each a day-part glyph, a title and a time with a dotted rule between, past events at 42 percent opacity so the day is still the day but what is left of it is what you are being asked about. A plus under the list creates on the day being looked at. Taps go out through `onTapEvent` and `onAddEvent` to the same `EventSheet` Today uses; the card owns no store and no calendar, so it previews like any other view.

15.6 How it works



The loop runs on the device even when the model is in the cloud: every tool reads local state.

One turn. The model asks, the invoker executes, the broker holds a write until the person answers the card, and the collector turns tool results into the agenda card without going through prose.

THE GATE IS STRUCTURAL

```
// LifeOSKit/Sources/Insights/ToolGate.swift
public actor ConfirmationBroker {
    func decision(for write: PendingWrite) async -> Bool {
        pending.append(write)
        onPending(write)
        return await withCheckedContinuation { continuation in
            continuations[write.id] = continuation
        }
    }
}

public func confirm(_ id: UUID) { resolve(id, allowed: true) }
public func cancel(_ id: UUID) { resolve(id, allowed: false) }
}
```

`ToolInvoker.invoke` checks `requiresConfirmation` and, for a gated tool, awaits `broker.decision(for:)` before `tool.call`. The continuation suspends the tool's execution path until the UI resolves it; there is no other way a gated call proceeds. That is what makes the gate a property of the code rather than a request in the prompt, and it is the substantive improvement over the app this was ported from, which asked the model in the system prompt to confirm first and therefore depended on the model choosing to. A cancelled write returns "User declined this change" as the tool result so the model can acknowledge it. The invoker records `didExecuteGatedWrite` the moment the gate opens, and `AssistantTurn` reads it to refuse a device retry after a cloud failure, so a confirmed create can never happen twice.

THE LOOP

On the cloud path `RemoteChatEngine` sends the thread, the six function declarations and the context, and loops on tool-call replies as described in 14.6. The context is repeated on every round because a stateless server has no memory of the last one. On the device path `OnDeviceChatEngine` binds each tool through

`SessionTool` and Apple's session drives the calls itself; the invoker is the same object in both cases, so the cap, the gate and the error shaping are identical whichever model is answering. The seventh call in a turn is refused with a message telling the model to say the request could not be completed.

THE CONTEXT PREFIX

```
// LifeOSKit/Sources/Assistant/AssistantContext.swift
return ""
Current time: \(formatter.string(from: now)) (\(label))
\(\todayLines)
\(\tomorrowLines)
""
```

Today and tomorrow are inlined so "what's on today?" needs no tool call: the cheapest possible path and the most reliable one on a small model. The time zone label is printed as the caller reached for it (Foundation relabels UTC as GMT, and the test pins that it comes back as UTC). Relative dates are resolved against the stated current time, times are stated in the user's zone, replies are a sentence or two, and the model is told to ask before assuming a duration.

FREE TIME

`CalendarStore.freeSlots` filters all-day events out of the busy set, because "Anniversary" is context, not a meeting, and treating it as busy would zero out the whole day. The pure core merges overlapping busy intervals, subtracts them from the window, and drops gaps shorter than the requested duration; it is `nonisolated static` so six tests exercise it without a container.

THE SPEC'S FLOOR WAS SUPERSEDED

The design spec floored the assistant on the device and escalated through the router's table, and costed it at a rounding error on that basis. Since the conversational core landed, the assistant runs through `AssistantTurn.run` exactly like the coach: cloud first under Automatic, device under On device, no fallback under Cloud. The on-device path is the fallback, not the floor.

15.7 Performance and cost

- **Two days inline.** The most common question costs zero tool rounds.
- **Everything reads the cache.** No tool touches EventKit; reads are indexed range queries on `CalendarEvent`, and writes go through the provider once and re-sync a 30-day-back, 90-day-forward window.
- **One fetch for all cards.** `resolveEventCards` gathers every id across the conversation into one `events(withIDs:)` query and a dictionary lookup, rather than one round trip per reply.
- **Bounded rounds.** Six tool calls per turn, seven rounds at most, and the thread is capped at twenty messages by `ChatStore.recent`. The server independently bounds the request at 64 messages, 16 tools and 32,000 characters of context, so the endpoint cannot be used as a general proxy on the project's key.

- **Not streamed, on purpose.** A tool call arrives in a stream as fragments of a JSON argument string spread across chunks; the server grants a stream only to a turn with no tools, and the assistant is not the screen someone watches word by word.
- **Per-turn objects, no caches.** The broker, collector and invoker are cheap value-sized actors made per turn; `events(on:)` is a fresh one-day query per strip tap. Nothing has to be invalidated.
- **Cost.** Per the spec's model, a cloud turn is roughly 1,000 tokens of tool declarations, 600 of instructions and inlined events, and up to 2,000 of history per round, with about 300 out. On the mini model that is well under a cent per turn and shares the coach's 150,000-token daily cap rather than needing one of its own.

15.8 Edge cases and tests

CONDITION	BEHAVIOUR
Calendar not authorized	No tools are bound, the instructions say so, the empty state offers Connect, and a conversation with history shows a reconnect row.
Apple Intelligence unavailable	The sheet shows one sentence and no composer.
Model invents an event id	<code>update_event</code> and <code>delete_event</code> return "No event with that id" and the wrong row is never touched.
Event is one occurrence of a series	Writes are refused with an explanation to change the series in the calendar app.
Model emits a bad date	The thrown description names the format; the invoker returns it as the tool result.
Update drops a location	The card says "Removes the location" before Confirm is enabled.
Cloud fails after a confirmed write	The failure stands; no device retry, no second card.
Cloud refuses	Surfaced; never re-asked of the device.
Any other failure	The reply reads "I couldn't answer that. Try again." and is written to the store like any reply.
Event deleted after the reply	Its id no longer resolves; the card drops it silently.
Reply touches no events	The latest reply still draws a card; older ones do not.
Seven tool calls in one turn	The seventh is refused and the model is told to say the request could not be completed.
A second account on the device	Its own conversation id in its own defaults suite; no shared transcript.

TESTS

- `CalendarToolsTests`, 10, against a fake calendar: there are six tools and only the writes are gated, `get_events` returns ids and titles in range, a garbled date throws rather than crashes, free slots are reported, a create maps onto the draft, an update rejects an unknown id, writes to a recurring event are refused with an explanation, a delete dispatches for a known one-off, the confirmation preview shows the current event, and the update preview calls out a dropped location.

- `AssistantContextTests` , 2: unauthorized instructions say there are no calendar tools; the context prefix inlines today and tomorrow.
- `ToolInvokerTests` , 6: a gated tool does not execute until confirmed, cancelling yields the declined result, the seventh invocation is refused, an unknown name is an error result not a crash.
- `RemoteChatEngineTests` , 14, and `AssistantTurnTests` , 15: the loop, the cap, the confirmed-write guard, and the tier rules, all shared with the coach.
- `CalendarFreeSlotTests` , 6, and `ChatStoreTests` , 8: the slot maths and that a reply remembers the events it was about across a reload.

Untested by a suite: the card's strip and rows (previews only), and the real EventKit write round trip, which needs a device with calendar access.

15.9 Study notes

READ IN THIS ORDER

1. `Assistant/CalendarAccess.swift` , the two protocols, then the one-line conformance at the top of `AssistantViewModel.swift` .
2. `Assistant/CalendarTools.swift` , top to bottom, watching where `eventLine` is used.
3. `Insights/ToolGate.swift` and `ToolInvoker.swift` , to see how a continuation becomes a gate.
4. `Assistant/AssistantContext.swift` , for the instructions and the prefix.
5. `Features/Assistant/ViewModel/AssistantViewModel.swift` , following `send` and then `resolveEventCards` .
6. `Features/Assistant/View/AssistantAgendaCard.swift` , then the sheet.
7. `Persistence/CalendarStore.swift` , the free-slot core.

CHECK YOURSELF

- **Why can the model not route around the confirmation?** The gated call is suspended on a continuation inside the invoker; nothing the model emits resumes it.
- **Why store event ids on the reply rather than the events?** Events change; ids re-resolve to the current truth and a deleted one vanishes.
- **Why do writes go through `CalendarSync` and not the store?** So the provider is the source of truth and the cache reflects what it actually stored.
- **Why is the loop on the device when the model is in the cloud?** Every tool reads local state; a server-driven loop would have to call back into the phone.
- **Why are today and tomorrow inlined?** The most common question then needs no tool call, which is both the cheapest and the most reliable path.
- **Why does an all-day event not block free time?** It is context, not a meeting; treating it as busy would zero out the day.

16 Social

Find people, keep the ones you found, message them, and compare a day's effort in a private group. Built on the Supabase project the app already has, with row level security doing every bit of the guarding and no chat service in between.

Revision D described a personal app with no accounts. Since then Almanac gained sign-in, and with it the first feature that is about someone other than the person holding the phone. The design constraint was cost: the app ships free, so a friends list, a mailbox and a leaderboard had to fit on Postgres rows behind PostgREST, with nothing per user to pay for. They do. This chapter describes the three tables that arrived on 27 August 2026, the shared-figures table that followed, and the groups and daily health comparisons that landed on 14 September.

16.1 Intent

WORKING BACKWARDS

From your profile, tap Together. Search a name, send a request, and once it is accepted you can message that person, see the three figures they chose to share, and invite them into a group. A group is a private chat with an optional leaderboard that ranks Effort, Recharge and Rest for today or yesterday, on the app's own 0 to 100 scale, with the device each score came from printed beside it. Health scores are shared per group, off by default, and stop the moment you say so.

Why is authorisation in the database and not in the app?

Because the client can only filter for display, never for privacy. Every table carries a policy written against `auth.uid()`, and every write that has to be atomic goes through a `security definer` function that takes the caller's identity from the token, never from an argument. A leaked or mistaken query returns nothing it should not, and the Swift side is free to be a thin request shaper.

Why no chat SDK?

A message is a row. `SocialAPI` is PostgREST over `URLSession`, the same idiom `SupabaseAuth` and `SupabaseREST` established, and the whole social layer adds no dependency. Polling replaces realtime: five seconds while a chat is open, eight in a group, and nothing at all otherwise.

Are the leaderboard numbers WHOOP's or Apple's?

No, and the screen says so. Effort, Recharge and Rest are app-defined comparisons computed on the device from `DailyMetrics`, each against the person's own goal or their own 28-day baseline. Only the derived score, its source and its day leave the phone. Raw heart rate variability, resting heart rate and sleep minutes never reach the leaderboard endpoint.

What does a friend request let someone see?

A display name and whatever profile fields the person filled in. Nothing else until it is accepted. Pending is not enough to read shared figures: asking to be someone's friend must not be a way to read their numbers while they decide, or the request itself becomes the attack.

What Social does not do: read receipts, edits, deletes or push for messages. The `messages` table carries no read state, so the inbox never claims a message is unread; it only says whether the last word was theirs. A message is a fact once sent.

16.2 Files and folders

```
Features/Social/
├── ViewModel/
│   ├── FriendsViewModel.swift    friends, requests, outgoing, search with a generation count
│   ├── ChatViewModel.swift       one conversation: messages, pending rows, a send that cannot
│   ├── InboxViewModel.swift      requests to answer, conversations to return to
│   └── GroupViewModel.swift      SocialSession, GroupsViewModel, GroupViewModel
└── View/
    ├── SocialHubScreen.swift     "Together": People, Groups, Rankings; CreateGroupScreen
    ├── FriendsScreen.swift       search field, requests above friends, one of three trailing
    ├── UserProfileScreen.swift   a person: photo, relationship actions, shared figures
    ├── FriendChatScreen.swift    a direct conversation, polled every five seconds
    ├── InboxScreen.swift         requests, then threads; InitialBubble
    ├── GroupDetailScreen.swift   chat, leaderboard, members; GroupInviteScreen; bubble; comp
    ├── SocialLeaderboard.swift   podium and lineup, ties kept intact
    ├── ProfileActivitySharingCard.swift the friends-only sharing toggle
    ├── SocialStyle.swift         SocialTheme, SocialCanvas, SocialPanel, SocialAvatar, notific
    └── SocialDesignPreview.swift sample fixtures, driven by launch arguments

LifeOSKit/Sources/Integrations/
├── SocialAPI.swift              profiles, friendships, messages over PostgREST
├── GroupAPI.swift               groups, members, group messages, leaderboards, via RPC
├── SharedStatsClient.swift      profile_stats: the three figures shared with friends
├── SocialWellness.swift         Effort, Recharge and Rest, computed on the device
└── InboxDigest.swift           a flat mailbox folded into one row per person

supabase/migrations/
├── 20260827130000_friends_and_messages.sql  profiles, friendships, messages
├── 20260827140000_friendship_column_grants.sql update may touch status and nothing else
├── 20260829120000_shared_stats.sql          profile_stats, readable by accepted friends
├── 20260914090000_social_groups.sql         social_groups, group_members, group_messages, RPCs
└── 20260914110000_group_wellness_leaderboards.sql shares_wellness, group_wellness_daily, ranki

supabase/tests/
├── social_groups.test.sql    43 pgTAP assertions, run in a rolled-back transaction
└── group_wellness.test.sql  23 more
```

Chapter 20 owns the backend in full. This chapter describes the tables only as far as the Swift code depends on their shape.

16.3 Model

Every type the screens render is a `Codable` value decoded straight from PostgREST. There is no `SwiftData` model in this feature: nothing social is cached on the device, and a screen that loses the network shows what it last fetched plus one quiet sentence.


```
// LifeOSKit/Sources/Integrations/SocialAPI.swift
public struct SocialProfile: Codable, Sendable, Equatable, Identifiable {
    public let userID: UUID
    public let displayName: String
    public let firstName: String?    // every field past the name is optional:
    public let country: String?      // a question someone may decline to answer
    public let birthDate: Date?
    public let heightCM: Double?
    public let avatarPath: String?   // the object's path inside the avatars bucket
}

public enum FriendshipStatus: String, Codable, Sendable { case pending, accepted }

public struct Friendship: Codable, Sendable, Equatable, Identifiable {
    public let id: Int
    public let requester: UUID
    public let addressee: UUID
    public let status: FriendshipStatus
}

public struct SocialMessage: Codable, Sendable, Equatable, Identifiable {
    public let id: Int                // negative while provisional, see ChatViewModel
    public let sender: UUID
    public let recipient: UUID
    public let body: String
    public let createdAt: Date
}
```

A friendship is one row whoever asked first, `pending` until the addressee accepts. The unique constraint is on the ordered pair, and the API guards the reverse direction before inserting. A profile that pads itself with placeholders for the fields somebody skipped looks broken rather than private, which is why the optionals stay optional all the way to the screen.

The decoder is the one piece of the model worth a paragraph. PostgREST emits timestamps with fractional seconds, which the default date strategy does not parse, and a bare `date` column such as `birth_date` arrives with no clock at all, which neither ISO 8601 formatter accepts. Before the third branch was added, a profile carrying a birthday failed to decode entirely, taking the person's name down with it.

`SocialAPI.decoder` tries fractional, then plain, then the day format, and is internal rather than private so a test can exercise each branch.

GROUPS

```
// LifeOSKit/Sources/Integrations/GroupAPI.swift
public struct SocialGroup: Codable, Sendable, Equatable, Identifiable {
    public let id: UUID
    public let name: String
    public let description: String
    public let ownerID: UUID
    public let members: [GroupMember]           // decoded from the embedded group_members
    public var isInvitation: Bool { members.first?.status == "invited" }
}

public struct GroupMember: Codable, Sendable, Equatable, Identifiable {
    public let groupID: UUID
    public let userID: UUID
    public let status: String                    // invited | accepted
    public let sharesActivity: Bool
    public let sharesWellness: Bool?            // optional: added by a later migration
}

public struct GroupMessage: Codable, Sendable, Equatable, Identifiable {
    public let id: Int
    public let groupID: UUID
    public let sender: UUID
    public let body: String
    public let clientID: UUID                   // the idempotency key a retry reuses
    public let createdAt: Date
}

public struct LeaderboardEntry: Codable, Sendable, Equatable, Identifiable {
    public let userID: UUID
    public let displayName: String
    public let avatarPath: String?
    public let score: Int                       // 0 to 100
    public let position: Int                    // the server's rank, ties included
    public let updatedAt: Date
    public let source: String?                  // appleHealth | whoop | fitbit
    public let day: String?
}
```

`groups(token:)` asks for `social_groups` with the caller's own `group_members` row embedded and filtered, so one request returns the groups and, for each, whether this person is a member or merely invited. `sharesWellness` is optional because the column arrived in a second migration and an older row shape must still decode.

SHARED FIGURES AND WELLNESS DAYS

```
// LifeOSKit/Sources/Integrations/SharedStatsClient.swift
public struct SharedStats: Codable, Sendable, Equatable, Identifiable {
    public let userID: UUID
    public let shares: Bool           // the owner's standing answer
    public let streak: Int?           // nil is "not sent", not zero
    public let daysTracked: Int?
    public let workouts: Int?
    public var hasFigures: Bool { shares && (streak != nil || daysTracked != nil || workouts
!= nil) }
}
```

`shares` is separate from the figures because they say different things. A person who turned sharing on this morning and has not opened the app since is sharing, with nothing to show, which is not the same as having declined. The profile screen draws the panel only when `hasFigures` is true: a row of dashes is not a fact.

`SocialWellnessDay` is one day's three scores with a source per score and an `observedAt`. Its shape is enforced twice, once in Swift and once by the table's check constraints: a score and its source are both null or both present, and every score is between 0 and 100.

16.4 ViewModel

Five plain `@Observable` classes, none with retry logic of its own. Each reads the session from the keychain on every call rather than capturing the token once, because `AppShell` refreshes the session hourly and on foregrounding, and a screen left open across that refresh would otherwise strand itself on a rotated token with endless 401s. Identity does not rotate, so `myUserID` is the one thing still safe to capture.

A NEWER SEARCH NEVER LOSES TO A SLOWER OLDER ONE

Search re-runs on every keystroke, so an older request can still be waiting on the network when a newer one is issued. Without tracking which query is in flight, a slow response for "al" could land after the fast response for "alex" and silently replace the right results with the wrong ones.

```
// LIfеOS/Features/Social/ViewModel/FriendsViewModel.swift
private var searchTask: Task<Void, Never>?
private var searchGeneration = 0

func search() {
    searchTask?.cancel()
    searchGeneration += 1
    let generation = searchGeneration
    // ... guards that name why nothing would happen: no config, signed out, no id yet
    let issuedQuery = query
    searching = true
    searchResults = []
    searchTask = Task {
        defer { if generation == searchGeneration { searching = false } }
        do {
            try await Task.sleep(for: .milliseconds(250))
            let results = try await api.search(trimmed, accessToken: accessToken)
            guard !Task.isCancelled, issuedQuery == self.query else { return }
            searchResults = results.filter { $0.userID != myUserID }
        } catch {
            guard !Task.isCancelled, issuedQuery == self.query else { return }
            errorMessage = Self.reason("Search failed", error)
        }
    }
}
```

Three guards together: the previous task is cancelled, a 250 millisecond sleep debounces the keystrokes, and the response is applied only if the query it was issued for is still the query on screen. The generation counter is what lets the `defer` clear the spinner for the latest search alone. The signed-in person is filtered out of results, because finding yourself in your own friends search is noise.

The three early guards each set a sentence instead of returning silently, because saying nothing is what "it just spins" looks like from the outside. `reason(_:_:)` carries the server's own message through: an expired token, a missing grant and a dropped connection all arrive as "it did not work" otherwise, and PostgREST always says which.

A SEND CAN NEVER LOSE ITS ROW

```
// LIfiOS/Features/Social/ViewModel/ChatViewModel.swift
private(set) var messages: [SocialMessage] = [] // refresh() only ever assigns this
private(set) var pending: [SocialMessage] = [] // provisional rows, kept apart
private var nextProvisionalID = -1 // server ids are always positive

func send() async {
    guard !sending else { return }
    let text = draft.trimmingCharacters(in: .whitespacesAndNewlines)
    guard !text.isEmpty, text.unicodeScalars.count <= 2000, let api, let accessToken, let
myUserID else { return }

    let provisionalID = nextProvisionalID
    nextProvisionalID -= 1
    pending.append(SocialMessage(id: provisionalID, sender: myUserID, recipient:
friend.userID,
                                body: text, createdAt: .now))

    sending = true
    do {
        try await api.send(text, to: friend.userID, accessToken: accessToken)
        pending.removeAll { $0.id == provisionalID }
        if draft.trimmingCharacters(in: .whitespacesAndNewlines) == text { draft = "" }
        await refresh()
    } catch {
        pending.removeAll { $0.id == provisionalID }
        errorMessage = "Message wasn't confirmed. Your draft is still here; check the
conversation before trying again."
    }
    sending = false
}
```

The optimistic row goes into `pending`, not `messages`, so a poll landing mid-send can never wholesale-replace it. The screen renders `messages + pending`. A failed post removes the row it added rather than leaving a message on screen that never sent, and keeps the draft. Reentry is guarded so a fast double-tap before the disabled state applies cannot post twice. The draft is cleared only if it still equals what was sent, so a sentence typed during the round trip is not thrown away.

Group chat goes one step further. `GroupViewModel.send` generates a `clientID` per attempt and reuses it on retry; the server's `send_group_message` inserts with `on conflict (sender, client_id) do nothing` and returns the existing id, so a retry after a timeout that actually landed cannot post a duplicate. A failed send is held in `failedSend` with a Retry button, the composer is disabled until it is resolved, and a later refresh that finds the client id in the timeline clears it.

THE REST

- `InboxViewModel` loads three things, not one per friend: the friendships, the profiles behind them, and the whole mailbox in one 200-row request. The grouping into one thread per counterpart is `InboxDigest.threads`, in the package, because it is the only real logic the inbox has and it should be testable without a network. The badge counts requests only: a conversation is not a task.

- `GroupsViewModel` loads the groups, then publishes the day's wellness scores if any group has sharing on, then reads and, if sharing is on, pushes the three profile figures. A refresh is skipped while a change is busy, and its results are dropped if the signed-in user changed while it was in flight.
- `GroupViewModel` refreshes under a generation counter too, and checks the roster before anything else: if the caller is no longer an accepted member, it sets `accessLost`, clears the messages and board, and the screen says so instead of rendering a stale chat. Messages merge by id rather than replace, so a page loaded through `older()` survives a poll. The leaderboard has its own generation so a metric or day switch cannot be overwritten by the previous request landing late.

16.5 View

Social is reached from the profile, not from a tab. `ProfileOverview` offers a Together shortcut, and `ProfileScreen` pushes `SocialHubScreen` for it. This is a place you go, not a home you live in, and the bar is the scarcest surface in the app.

TOGETHER

`SocialHubScreen` is an `UnderlinePicker` with three sections. People embeds `FriendsScreen`. Groups lists invitations first, each as a panel with Decline and Join, then the joined groups as an adaptive grid of coloured cards. Rankings is the same grid with a trophy and "View leaderboard", and opening a group from it lands on the leaderboard section instead of the chat. The toolbar carries the inbox tray and a Create button; `CreateGroupScreen` takes a name up to 60 characters, a description up to 240, and invitees chosen from the friends list, capped at 49 so the group never exceeds 50. It generates its group id up front, which is what makes a retried create idempotent on the server.

PEOPLE

`FriendsScreen` is one list with three shapes. A search field that, once typed in, replaces the screen's own sections with results. Otherwise requests above friends, in that order, because a decision waiting on you outranks a list you already know. A search result's trailing control is always exactly one of Add, Requested or Friends, never a fourth "request me back" state; accepting still happens from the requests section, which already owns that decision. Tapping a friend opens `UserProfileScreen`, which shows the photo, the relationship's actions (message and remove, accept and decline, cancel a sent request, or add), and the shared figures panel when there are figures to draw.

CONVERSATIONS

`FriendChatScreen` and the group chat share `SocialMessageBubble` and `SocialComposer`. Mine is ink on the right with a small bottom-trailing radius; theirs is paper on the left. Day separators appear where the calendar day changes. The composer shows a counter past 1800 characters and disables send past 2000, the same limit the table's check constraint enforces. `followLatest` is read from scroll geometry: the list follows a new message only if the reader was already within 140 points of the bottom or the message is their own, so someone reading history is not yanked down by a reply. Polling runs in a `.task(id: scenePhase)` loop and stops the moment the scene leaves the active phase.

The chat's first version on 27 August rode a night aurora. The 14 September restyle replaced that with `SocialCanvas`, the cream gradient every social screen now shares, with a dark counterpart, so a conversation and the friends list read as one place.

THE LEADERBOARD

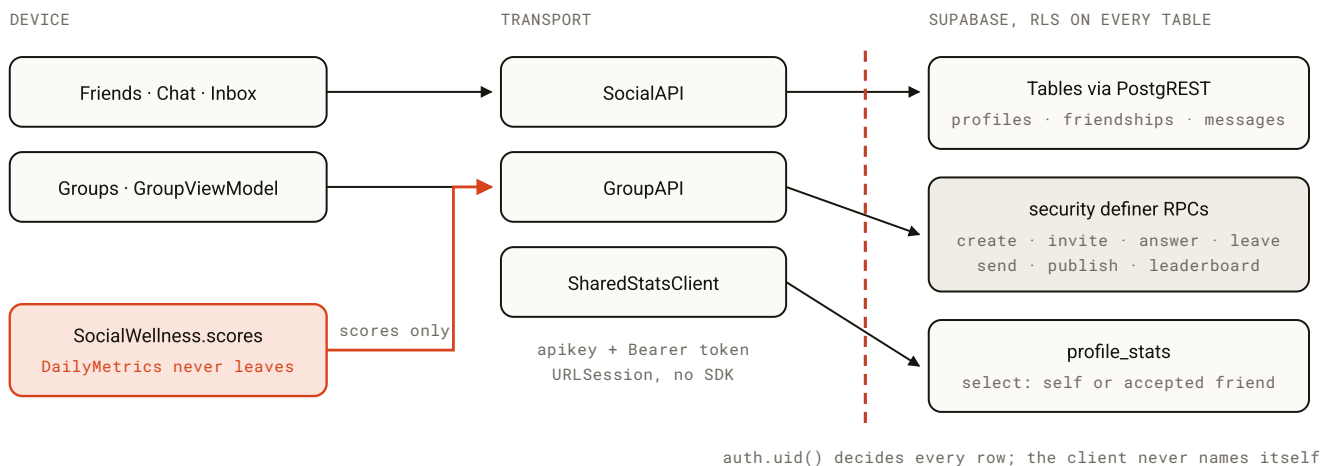
`SocialLeaderboard` keeps the server's ranks intact, ties included. A podium of three is drawn only when it would not arbitrarily feature some people from a tie that crosses the podium boundary, and the winner's avatar is larger only when all three ranks are distinct. Accessibility sizes skip the podium for a single readable list. Each row prints the score, the source device and how long ago it synced, and the footer says the one thing a reader needs: equal scores share a rank; compare the same day; devices and goals differ. A

`DisclosureGroup` explains the selected metric's formula in plain words, and a date picker allows today or yesterday and nothing else, matching the server's window.

STYLE

`SocialTheme` is four pastels (coral, mint, lilac, yellow) chosen by a stable hash of a uuid, so a person or a group is the same colour on every screen and every launch. `SocialAvatar` shows the profile photo when there is one and initials on that pastel when there is not. `InitialBubble` in the inbox hashes the name by hand for the same reason: `String.hashValue` is randomised per launch, and a friend who changed colour every time the app opened would be a small, constant lie.

16.6 How it works



The Swift side shapes requests and decodes rows. The database decides who may see or write what, and the atomic operations live in functions the anon role cannot call.

FRIENDSHIP, END TO END

1. On appear, `FriendsViewModel` upserts the caller's own `profiles` row from the local profile name. This is what makes the person findable; a failure here is the difference between having friends and not, and it used to be discarded. It is now a `publishWarning` that outlives a successful list fetch.
2. Search is an `ilike` filter on `display_name`, limited to 20, with `%`, `*` and backslash escaped first so a search for a literal "50%" does not match everything.
3. Add fetches the caller's own friendships and, if the pair is already linked in either direction, returns without inserting. Idempotent, not an error: asking someone who already asked you is not a mistake worth surfacing. The button flips to Requested before the round trip so it does not flicker.

4. Accept sends a `PATCH` carrying exactly one column, `status`. The database grants `update` on `friendships` to that column alone, and Postgres enforces column grants before policies even run, so the same statement that accepts a request can never smuggle in a new requester or a new timestamp.
5. Remove is a `DELETE` scoped by id, the same call for declining a pending request and ending an accepted friendship, since either way the row stops existing.
6. Sending a message requires the `sender` column in the body, and RLS insists it equal `auth.uid()`. The only place that identity exists on the device is the access token's `sub` claim, so `SocialAPI.subject(ofAccessToken:)` reads it without verifying the signature. Verification is the server's job; this only reads a claim already trusted to authenticate every other call.

GROUPS, END TO END

Reads are plain PostgREST selects gated by `is_group_member`, a `security definer` helper that exists to avoid recursive membership policies and that takes the caller only from `auth.uid()`. Every write is an RPC, because each has an invariant a bare insert cannot hold: `create_social_group` checks every invitee is an accepted friend and rolls the whole thing back if one is not; `invite_group_member` locks the group row before counting to 50; `leave_social_group` hands ownership to the longest-standing member or deletes the group when the owner was the last one; `send_group_message` locks the group so a concurrent removal cannot race a send in. The anon role is revoked from all of them, and `truncate`, which bypasses row level security, is revoked from everyone.

WELLNESS SCORES, END TO END

`RootView.socialActivity` assembles a `SocialActivitySnapshot` from the local store: the streak, a count of tracked days, a count of workouts, and the last 30 days of `DailyMetrics` run through `SocialWellness.scores` against the person's exercise and sleep goals. The scoring rules are strict on purpose.

- **Effort** is exercise minutes over the goal, capped at 100. Apple Health uses exercise time, WHOOP tracked workout time, Fitbit fairly plus very active minutes. A duplicate reading is never summed across providers; `MetricArbiter` already chose one source per metric and the score uses that one.
- **Rest** is sleep minutes over the goal, capped at 100. Extra sleep earns no extra points.
- **Recharge** is $50 + 100 \times (0.6 \times (\text{HRV} \div \text{usual HRV} - 1) + 0.4 \times (1 - \text{RHR} \div \text{usual RHR}))$, clamped to 0 to 100, where "usual" is the median of at least seven distinct earlier days within 28, all from the same provider, never including today. A WHOOP reading still calibrating, a non-finite number, an HRV outside 0 to 500 or a resting rate outside 20 to 150 yields nil, not zero. Fifty means near your own baseline, and the screen says this is an estimate, not a recovery percentage.
- Only today and yesterday are published. The server refuses a day outside `current_date - 2` to `current_date + 1`, an observation more than five minutes in the future or four days in the past, and an older observation can never overwrite a fresher row.
- The day key is a Gregorian local date even when the device uses another calendar, so two people in different locales rank on the same day.

Publishing happens from `RootView` after provider state changes, from the hub on refresh, and when sharing is switched on, and only if some group has `shares_wellness` set. The server-side ranking function joins

members with sharing on to their daily row and `rank()` s by the chosen metric, so ties share a position and a null score is simply absent from the list rather than ranked last.

16.7 Performance and cost

- **Three requests to build the inbox, not one per friend.** Friendships, profiles by an `in.()` list, and the whole mailbox in one 200-row request ordered newest first. The limit is on messages, not conversations, so a very busy thread can crowd a quiet one off the end. That is the right failure: what falls off is what was least recent.
- **Polling is bounded to an open screen.** Five seconds in a direct chat, eight in a group, both inside `.task(id: scenePhase)` so backgrounding ends the loop. No screen polls while not on screen, and nothing polls the friends list at all.
- **Search is debounced and cancelled.** 250 milliseconds, one in-flight request, late responses dropped. A person typing a six-letter name costs at most one or two round trips rather than six.
- **Optimistic writes, no waiting.** Add flips to Requested and a message appears in `pending` before the round trip; both are reverted on failure rather than left lying.
- **Group messages page by id.** Fifty newest on open, `id.lt` for older, merged into the existing dictionary so a poll cannot discard a loaded page.
- **Avatars are downloaded per row with a token and cached only by the task.** `SocialAvatar` keys its `.task` on `avatarPath`, so a list that re-renders does not refetch faces unless the path changed.
- **Nothing raw leaves the phone.** The leaderboard endpoint receives three integers, three source names, a day and a timestamp. HRV, heart rate and sleep minutes stay in `DailyMetrics`. The profile figures are three counts. There is no AI in this feature and no token budget to protect.
- **Cost is Postgres rows and PostgREST calls.** No realtime channel, no storage beyond the avatars bucket Chapter 17 owns, no third-party chat. The gin index on `display_name`, the two `friendships` indexes, the `(least, greatest, created_at)` conversation index and the `(group_id, id desc)` timeline index are the whole query plan.

16.8 Edge cases and tests

CONDITION	BEHAVIOUR
Signed out	Every screen shows a one-sentence guest state, never a request that 401s into a blank screen.
No display name yet	The friends screen says to add a name in Edit profile; no row is published and nobody can find you.
Publishing the profile fails	A warning that survives a later successful friends fetch, because "nobody can find you" outlives "the list loaded".
Slow search response arrives after a newer one	Dropped. Only the query still on screen may write results.
Search for "50%"	Wildcards escaped; it matches the literal text.
Both people request each other	The second request is a no-op; one row links the pair.
Accept sends more than <code>status</code>	Rejected by the column grant before RLS runs.
Message to someone who is not an accepted friend	Refused by the insert policy on <code>messages</code> .
Send times out but landed	Direct: the pending row is removed and the draft kept, with a sentence asking to check the conversation first. Group: retry reuses the client id and the server returns the existing row.
Double-tap send	Reentry guard; one post.
Message from a person no longer in <code>profiles</code>	Dropped from the inbox rather than shown nameless.
Removed from a group while it is open	The next refresh sets <code>accessLost</code> ; chat, board and members are cleared and the screen says so.
Owner leaves	Ownership passes to the longest-standing member; if none, the group and its chat are deleted.
Invite a non-friend	The RPC raises and, on create, the whole group creation rolls back.
Group at 50 members	Invite refused under a row lock, so two concurrent invites cannot both succeed.
Sharing switched off	<code>SharedStatsClient.push</code> sends explicit nulls rather than omitting the figures, because an upsert leaves omitted columns standing.
Sharing on, nothing pushed yet	<code>hasFigures</code> is false; the panel is omitted.
Pending friend reads shared figures	Refused by the <code>profile_stats</code> policy, which requires <code>accepted</code> .
Fewer than 7 baseline days, or mixed providers	Recharge is nil and the person is unranked for that metric.
Score day out of the server's window	The publish raises; an older observation never overwrites a fresher row.

CONDITION	BEHAVIOUR
Two equal scores	Same rank from <code>rank()</code> ; the podium is skipped if the tie crosses its boundary.
Accessibility text sizes	No podium; one readable list. Friend request rows stack vertically.

TESTS

Sixty Swift tests in five suites, all pure: request builders are static and free of `self` , so a test asserts on the `URLRequest` a call would produce without a network or a stub `URLProtocol` .

- `SocialAPITests` (22): every decode branch including the bare birthday, wildcard escaping, the accept sending only `status` , the friendship request being a no-op in both directions, the sender derived from the token's subject, and a refusal carrying the server's reason.
- `GroupAPITests` (9): groups scoped to the caller, no request without a valid session, message pages read newest first and shown chronologically, retry preserving the idempotency key, the wellness upload carrying day and source and no raw signals, and a membership denial arriving as an error rather than an empty success.
- `SharedStatsClientTests` (8): sharing off nulls the figures rather than omitting them, an empty id list never leaves the device, the two `hasFigures` cases.
- `SocialWellnessTests` (15): missing and recorded zero remain different, goal progress capped, manual and unattributed data excluded, the baseline cannot mix providers or use today, duplicate days cannot complete calibration, non-finite signals yield no score, the Gregorian day key in other calendar locales, naps cannot replace the main night.
- `InboxDigestTests` (6): one row per person carrying their newest message, the newest conversation leads, a message from someone with no profile is dropped.

The database has its own suite: 43 pgTAP assertions for groups and chat and 23 for the wellness leaderboard, each file inserting disposable identities inside a transaction that is rolled back, switching to the `authenticated` role and setting the JWT claim by hand. They pin the things Swift cannot: that creation is idempotent, that a blank name is a constraint error, that an invalid invitee rejects creation atomically, that `authenticated` cannot truncate, that nonmembers and invitees cannot read rankings, and that a stale device write does not overwrite a fresher row. The wellness suite ran with the CLI's pgTAP runner unavailable, through the management API instead; the validation notes record that live two-account verification and three-provider accounts were not exercised.

16.9 Study notes

READ IN THIS ORDER

1. `supabase/migrations/20260827130000_friends_and_messages.sql` : three tables and seven policies. Everything in Swift is shaped by these.
2. `LifeOSKit/Sources/Integrations/SocialAPI.swift` : the request builders, the decoder, and `subject(ofAccessToken:)` .
3. `LIfeOS/Features/Social/ViewModel/FriendsViewModel.swift` : the search generation counter and the three named guards.

4. `LIfе0S/Features/Social/ViewModel/ChatViewModel.swift` : `pending` kept apart from `messages` .
5. `supabase/migrations/20260914090000_social_groups.sql` : why every write is an RPC, and the row locks inside them.
6. `LIfе0SKit/Sources/Integrations/SocialWellness.swift` : the three formulas and every reason a score is nil.
7. `supabase/migrations/20260829120000_shared_stats.sql` : why the figures are a separate table from `profiles` .
8. `LIfе0S/Features/Social/View/SocialLeaderboard.swift` : the podium rule for ties.

CHECK YOURSELF

- **Why are the shared figures not columns on `profiles` ?** The select policy on `profiles` is using `(true)` for every signed-in user, because search needs it, and row level security is row level. A column there would be world-readable. A table of its own gets a policy that can name friendship.
- **How does a slow search response lose to a faster newer one?** The task is cancelled, and the response is applied only if the query it was issued for still equals the query on screen. A generation counter clears the spinner for the latest search alone.
- **Why can a poll never erase a message being sent?** Provisional rows live in `pending` ; `refresh()` only ever assigns `messages` ; the screen renders the sum.
- **Why does accept send only one column?** The database grants `update` on `friendships` to `status` alone. Any other key fails the whole request before RLS runs.
- **Where does the sender id in a message come from?** The access token's `sub` claim, read without verification, because the server already verified it when it issued the token and verifies it again on the request.
- **What makes a group retry safe?** A client-generated `clientID` reused on retry, and a unique `(sender, client_id)` on the server with `on conflict do nothing` returning the existing id.
- **Why is Recharge nil for a new WHOOP user?** It needs seven earlier distinct days from the same provider within 28, none still calibrating, and today never counts toward its own baseline.

17 Profile and settings

A face in the corner of Today opens everything that used to be behind a gear: who you are, what your tracking adds up to, your friends, every connection with its own live state, the goals that decide a good day, and the account switcher. Settings is a page inside the profile, not a tab.

17.1 Intent

WORKING BACKWARDS

Tap your photo on Today and the screen fills with it, fading into a gradient sampled from the picture itself. Under your name sit three figures that describe who you have been: day streak, days tracked, workouts. Then today's living numbers, then all-time totals, then three doors: Together, Connections, Settings. Edit the profile from a glass sheet that saves locally first and publishes after. Settings holds appearance, the five daily goals, the accounts signed in on this device, which engine the coach uses, whether it speaks, and a summary of every connection with a green dot for on and a hollow one for off.

Why a face instead of a gear?

A gear says "settings"; a face says "you", and everything that was behind the gear is about you. The button falls back to a person glyph rather than initials, because a single letter in a circle reads as an avatar that failed to load.

Why is the profile on the server now?

Because signing in on a second phone, or as a second account on this one, used to produce a profile with no name and no picture: both lived only on the device that typed them. The `profiles` row and the `avatars` bucket are the source of truth; the device keeps a per-account cache so the screen draws instantly and draws offline.

What does Settings do when Health access was denied?

It cannot know. HealthKit read authorisation is opaque by design, so the row says "No data yet. Check Almanac in Health > Sharing" rather than asserting a state it cannot verify.

Why can I edit the goals?

Because the dot grid is the app's central claim, and a claim you cannot calibrate is one you stop believing. Defaults are 8,000 steps, 7h sleep, 30 min exercise, 2,500 ml water, 3 of 4 required.

17.2 Files and folders

```
Features/Settings/
├─ Model/
│   ├── AppConfig.swift          build configuration read from the bundle; what is publi
│   └─ GoalsDraft.swift         the editable copy of the goals row
├─ ViewModel/
│   ├── SettingsViewModel.swift  load and save the goals
│   ├── HealthConnectionViewModel.swift  permission, sync, cycle default, the foreground watch
│   ├── WhoopConnectionViewModel.swift  PKCE in Safari, manual code, stalled sign-in, sync
│   ├── FitbitConnectionViewModel.swift  ASWebAuthenticationSession; the server holds the tokens
│   └─ PlaidConnectionViewModel.swift  Link, exchange, cursor sync, disconnect
└─ View/
    ├── ProfileScreen.swift      navigation: edit sheet, four destinations; ProfileStat
    ├── ProfileOverview.swift    the photo-led page, phone and wide layouts
    ├── ProfilePhotoBackdrop.swift  the gradient sampled from the photo
    ├── ProfileEditSheet.swift    local first, publish after, stays open on failure
    ├── SettingsScreen.swift     appearance, goals, accounts, coach tier, voice, at a gl
    ├── ConnectionsSettingsScreen.swift  one card per source, one chip vocabulary
    ├── PlaidLinkPresenter.swift  the only file that imports LinkKit
    ├── AccountPanel.swift       AccountPanel, AccountPageHeading
    └─ ProfileDesignPreview.swift  DEBUG fixture

Components/View/ProfileAvatar.swift          the toolbar button
Components/Model/ProfilePhotoDownsizing.swift  512pt, scale 1, JPEG under the bucket limit
Features/Onboarding/Model/ProfilePhotoStore.swift  per-account caches and ProfileSync (shared w
App/AccountSession.swift                    switch, sign out, the store per account
App/Surfaces/SurfaceSettingsScreen.swift    the Widgets & Watch page (Chapter 18)

LifeOSKit/Sources/Integrations/ProfileClient.swift  RemoteProfile, AvatarStamp, the bucket call
LifeOSKit/Sources/DesignSystem/ActionFan.swift      QuickAction, the row and the fan
LifeOSKit/Sources/DesignSystem/QuickActionsToolbar.swift  the environment value and the modifie
```

17.3 Model

THREE SHAPES OF A PROFILE

```
// LifeOSKit/Sources/Integrations/ProfileClient.swift
public struct RemoteProfile: Codable, Sendable, Equatable {
    public var firstName: String
    public var lastName: String
    public var country: String
    public var birthDate: Date?
    public var heightCM: Double?
    public var gender: String?      // nil is "prefer not to say"
    public var avatarPath: String?  // a path in the bucket, never a URL

    public var displayName: String { fullName.isEmpty ? "Someone" : fullName }

    public func payload(userID: String) -> [String: Any] {
        var row: [String: Any] = [ "user_id": userID, "display_name": displayName, ... ]
        row["birth_date"] = birthDate.map { SupabaseREST.day($0) } ?? NSNull()
        row["height_cm"] = heightCM ?? NSNull()
        row["gender"] = gender ?? NSNull()
        row["avatar_path"] = avatarPath ?? NSNull()
        return row
    }
}
```

`RemoteProfile` is the row as the server holds it. Two decisions are written into it. The avatar is stored as a path, `{user_id}/avatar.jpg`, never a URL, because a URL bakes in the project host and the signing scheme, both of which outlive their correctness. And cleared fields are sent as explicit nulls rather than omitted, because the save is an upsert of the whole row: an omitted key would leave the old value standing and make deleting a birthday impossible. `displayName` is never empty because the column is not nullable and a profile with no name is still a profile that friend search has to match.

`LocalProfile`, in `ProfilePhotoStore.swift`, is the per-account display cache, and `ProfileStore.save(_remote:)` only ever writes it from what the server confirmed, so the two cannot drift. The profile used to live in `auth.users.user_metadata` with a copy in a single `UserDefaults` key. That payload rides inside the JWT on every authenticated request, which is the wrong place for anything that grows, and the copy was not keyed by account, so with two accounts on one device the second saw the first's name.

```
// LifeOSKit/Sources/Integrations/ProfileClient.swift
public enum AvatarStamp: Sendable, Equatable {
    case missing          // no such object in the bucket
    case unknown          // the object exists, but the listing did not say when it changed
    case at(String)       // the object exists and last changed at this stamp
}
```

Three cases rather than an optional string, because "there is no object" and "the object is there but undated" want opposite answers. Collapsed into a single nil, the safe reading of nil is "do not download", and then a

device with nothing cached downloads nothing, ever, and shows an empty circle for a profile that has a picture.

THE GOALS DRAFT AND THE STATS

```
// LIfeOS/Features/Settings/Model/GoalsDraft.swift
struct GoalsDraft: Equatable {
    var steps: Int = 8000
    var sleepMinutes: Int = 420
    var exerciseMinutes: Int = 30
    var waterML: Double = 2500
    var requiredCount: Int = 3
}

// LIfeOS/Features/Settings/View/ProfileScreen.swift
struct ProfileStat: Identifiable, Equatable {
    let id: String
    let value: String
    let label: String
}
```

The draft is a separate type from `UserGoals` on purpose. Editing writes to the draft; committing validates it into the row. Binding steppers straight to the persisted model would write to the database on every increment and re-evaluate a month of history mid-drag. `ProfileStat` is a label and a pre-formatted string: the profile shows facts and never computes one.

APPCONFIG: WHAT IS PUBLIC AND WHY

`AppConfig` reads the bundle's Info.plist for the Supabase URL and anon key, the WHOOP, Fitbit and Plaid redirect URIs, the ElevenLabs key, and the function endpoints derived from the project URL. Everything here is public by design except the ElevenLabs key, which is gitignored but still extractable from a shipped `.ipa` and therefore never logged. Client secrets for WHOOP, Fitbit and Plaid are deliberately absent: they live only in Edge Function environments (Chapter 23). An unresolved build setting comes through as a literal `$(...)`, so `string(_:)` treats that prefix as absent. The app answers to two URL schemes, `almanac` and the older `lifeos`, so the WHOOP redirect bridge can point at either without the app and the server having to ship together.

17.4 ViewModel

SETTINGSVIEWMODEL

`draft` is the one mutable published property on this view model, and it is mutable because SwiftUI's `@Bindable` needs it to be. `load()` copies the goals row into it; `save()` writes the row back in one `context.save()`, which is what makes the dot grid re-evaluate. The rule is the user's, not hard-coded. `SettingsScreen` calls `save()` from `onChange(of: model.draft)`, so each stepper commits once per change rather than once per dismissal.

FOUR CONNECTIONS, FOUR SHAPES

Each provider has its own view model because each has a different relationship with credentials, and the differences are the point.

PROVIDER	WHERE THE TOKEN LIVES	SIGN-IN SURFACE	WHAT THE VIEW MODEL OWNS
Apple Health	Nowhere; it is a permission	The system sheet, once	Whether it was asked, the sync cursor, the cycle switch, the foreground watch
WHOOOP	Keychain on the device, plus a server copy for restore	Safari, by <code>UIApplication.open</code>	PKCE attempts, the manual desktop code, stalled sign-ins, sync with a staleness policy
Fitbit	A row on the server only	<code>ASWebAuthenticationSession</code> in-app	PKCE attempts and a "connected" flag per account; the server is the authority
Bank accounts (Plaid)	Access tokens on the server behind service-role RLS; item ids and cursors on the device	Plaid Link, sometimes via a bank's OAuth in Safari	Link presentation, exchange, paged sync, reconnect state, disconnect with or without history

Health is deliberately the thinnest, because it has no OAuth, no tokens and no server, and the one thing it does have is a permission model that refuses to tell us anything. `connect()` asks once, records that it asked, decides the cycle-tracking default, syncs, and starts the watch. After that "Connect" becomes "Sync", because iOS shows the sheet exactly once and offering to connect again would be a button that visibly does nothing. The cycle-tracking switch is offered to everyone and defaulted from Health's own sex characteristic, not from the gender asked at signup: that field says how someone wants to be addressed, and using it here would get the answer wrong in both directions and strand everyone who chose "Prefer not to say". Turning the switch on asks for the extra permission there and then, so the prompt arrives with the tap that caused it.

WHOOOP opens sign-in in Safari rather than an in-app web session because WHOOP's login sits behind a Cloudflare bot challenge that never clears inside `ASWebAuthenticationSession`. That decision creates two problems the view model then has to solve. Leaving for Safari looks identical whether or not the person comes back, so `resolveStalledConnect()` waits two seconds after the app becomes active and, if the card is still connecting with no token, returns it to disconnected. Two seconds is the whole subtlety: the redirect usually lands a moment after activation, so resolving immediately would cancel sign-ins about to succeed. And a challenge behind a VPN or Private Relay can hang forever, so `beginManual()` offers a desktop URL and a code field; the PKCE verifier never left this phone, so a transcribed code alone is not enough to impersonate it. Pending attempts are persisted, not held in memory, because iOS may terminate the app while Safari is up, and the callback is matched to the attempt whose `state` it carries because tapping Connect twice starts two valid authorisations.

Fitbit is thinner than WHOOP, and the missing half is the point. Fitbit rotates its refresh token on every use, so exactly one writer may hold it, and that writer is a row in the database. The view model hands a code to the `fitbit-token` function and asks `fitbit-sync` what the state is. A "connected" flag per account is remembered only so the card does not flash "Not connected" on every launch while the server is asked.

Plaid keeps LinkKit behind one seam. `PlaidLinkPresenter` is the only file that imports it, which is what lets the connection view model be tested against the `PlaidClient` alone. The presenter holds the live handler

itself, because a bank using OAuth sends the person out to Safari and a handler released while they are away has nothing to hand the redirect back to. It also keeps the link token in the account's defaults, because the case that matters is the process not surviving: `resume(from:)` rebuilds a handler on the original token and calls `resumeAfterTermination`. The completion closure died with the old process, so `PlaidConnectionViewModel.attach` re-registers `onResumedConnect` every time. Sync runs the cursor loop up to twenty pages per pass and sets `isFetchingHistory` between connecting and the first transaction arriving, because Plaid fetches history asynchronously and an empty first sync must not render as "you spent nothing this month" (Chapter 11).

All four share three habits. A launch and a foreground arriving together coalesce into one sync through a held `Task` or an `isSyncing` flag. `isConnected` is false while connecting, so the summary dot never claims success while Safari is still open. And every failure is a sentence: "Whoop needs new permissions. Disconnect, then connect again" is a different problem from "Whoop sign-in expired", and collapsing them once sent people around the reconnect loop with no idea why it kept failing.

17.5 View

THE PROFILE PAGE

`ProfileScreen` is presented as a `fullScreenCover` from the avatar on Today. It owns a `NavigationStack`, the local profile and photo as state, the edit sheet, and a single `destination` enum for its four pushes: notifications, friends, connections, settings. Every push is a value, so the destination is a function of it. `ProfileOverview` is the presentation, shared with the DEBUG fixture, and is forced into the dark colour scheme with a hidden navigation bar because it is drawn over the photo.

```
// LIfiOS/Features/Settings/View/ProfilePhotoBackdrop.swift
static func extract(from data: Data?) -> Self {
    guard let data, let image = UIImage(data: data) else { return fallback }
    let thumbnail = UIGraphicsImageRenderer(size: CGSize(width: 48, height: 48), format:
format)
        .image { _ in image.draw(in: CGRect(x: 0, y: 0, width: 48, height: 48)) }
    guard let source = CIImage(image: thumbnail) else { return fallback }
    func average(_ y: CGFloat) -> Color {
        let bounds = CGRect(x: 0, y: y, width: 48, height: 16)
        let result = source.applyingFilter("CIAreaAverage",
            parameters: [kCIInputExtentKey: CIVector(cgRect: bounds)])
        ...
    }
    return Self(top: average(32), middle: average(16), bottom: average(0))
}
```

The gradient behind the profile is sampled from the photo: a 48 pixel thumbnail, three `CIAreaAverage` passes over its top, middle and bottom thirds, and the three colours become a diagonal gradient. It is recomputed only when the photo changes, through `.task(id: photo)`. Over that sit a blurred copy of the whole photo at 35 percent, the portrait cropped 1.3 to 1 from the top and masked to fade out, and a dark wash that keeps white text readable. With no photo the backdrop is the fallback palette and a soft orange ellipse.

On a phone the portrait sits at the top and the page scrolls. At 760 points or wider the layout becomes a 340 point identity column beside a 540 point details column, the portrait becomes a rounded card, and the backdrop drops the portrait layer. Under the name are up to three parts of a subtitle: country, height, age. Then, in order, "Your rhythm" (three cumulative figures), "Today" (only the readings that exist), "Over time" (only totals above zero), and the three doors, all on clear Liquid Glass panels. An empty profile at accessibility sizes collapses every grid to one column.

THE EDIT SHEET

`ProfileEditSheet` is the same fields as signup, reachable at any time, because signup asks once and never again, which is fine for a name and wrong for a photo. It saves locally first and publishes after: the local copy is what the app displays, so writing it first means the change is visible immediately and survives a failed request. Dismissal, though, waits for the publish. The row is what friend search reads, so a save that only ever reached the device would leave someone invisible to everyone looking for them while their own screen said it worked. On failure the sheet stays open with the reason, Save becomes "Try again", and Cancel becomes "Close anyway", because the local copy is already written and trapping someone in a sheet they cannot satisfy is worse than letting them out informed. Removing a photo is a delete on the bucket rather than a null in the row; leaving the object behind would keep serving a face the person took down.

SETTINGS AND CONNECTIONS

`SettingsScreen` is a page pushed from the profile with no inner stack and no Done: the back chevron is the way out, which is what makes it feel like a place rather than an overlay. Its cards, top to bottom: appearance (system, light, dark), the five goal steppers, the accounts signed in on this device with a radio to switch, the coach tier (automatic, cloud, on-device), the voice toggle and its three voices, "At a glance" with Notifications and Widgets & Watch, the connections summary, and Log out. Switching accounts does not sign anybody out: each account keeps its own store, cursors and connections, so moving between them is opening a different file rather than tearing one down (Chapter 4).

The connections summary used to be one row with a chain-link icon reading "Whoop, Health, banks" over WHOOP's status line, which said nothing about the other two. It now shows every source, one line each, with a green dot for on and a hollow one for off. Never red: not being connected is a choice someone is allowed to make, and an alarm dot next to WHOOP would nag every user who does not own one. While a sync runs the dot is replaced by a spinner, because the honest answer to "is it on" is "ask me in a second".

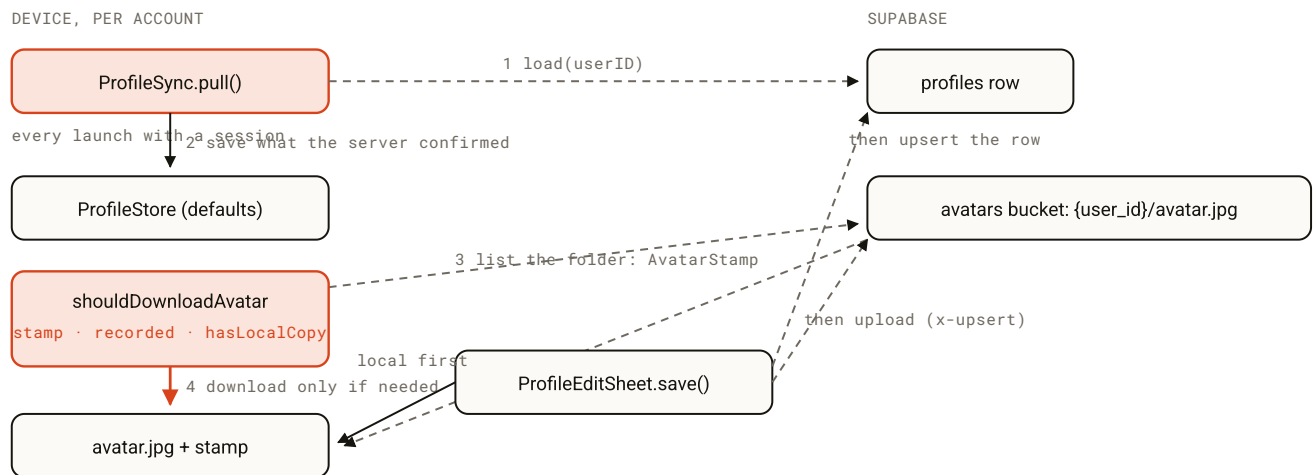
`ConnectionsSettingsScreen` is the page it opens, one card per source with a chip whose colour is its meaning: orange asks for a tap, green reports one that already happened. The Fitbit card's "Sync" calls the forced refresh rather than reopening OAuth, and the Health card's "Sync" replaces "Connect" once the sheet has been shown.

WHERE THE ACTIONS LIVE

The calendar assistant, LIFO and Begin Activity are the app's three free-floating actions. They used to be laid flat on home and collapsed into a bottom-corner fan on every other tab, which put the control people reach for most in a different place depending on where they were and hid it behind a trigger on four of the five. They are now a `[QuickAction]` defined once in `RootView`, injected through the `quickActions` environment value, and drawn by `.quickActionsToolbar()` across the top of every tab as an `ActionFan` in its row arrangement: 34 point bare glyphs sized for a 44 point bar. The environment value is the mechanism because Notes and Life own their own `NavigationStack` s several levels below the root, and a toolbar has to be

attached inside the stack it belongs to. The fan arrangement still exists in `ActionFan`, with its arc from 90 to 170 degrees and its 106 point radius chosen so three 52 point circles never overlap, but nothing in the app uses it today. The plus becomes a timer glyph labelled "Current activity" while a session is running.

17.6 How it works



The remote path never changes, so "have I got a copy" is the wrong question; the bucket listing's `updated_at` is the only thing that tells two devices apart. A failed lookup reads as unknown, so a flaky network costs at most a stale face and never a blank one.

THE AVATAR STAMP, STEP BY STEP

1. `AppShell`'s launch task restores the session and, if the person is signed in with a store open, calls `ProfileSync.pull()`.
2. The `profiles row` is loaded and mirrored into the per-account defaults. If the row says there is no picture, the local file is deleted, so a photo removed on one phone does not survive on another forever.
3. The account's folder in the bucket is listed (one object, never the whole bucket) and the object's `updated_at` becomes an `AvatarStamp`.
4. `shouldDownloadAvatar` decides: missing never downloads, unknown downloads only when there is no local copy, a dated stamp downloads when there is no copy or the stamp differs from the recorded one.
5. After a download the stamp is recorded, and only when the listing gave one. Saving a photo clears the stamp first, because the stamp describes the bytes on disk; a photo taken on this device has no server stamp yet, and keeping the old one would make the next sync believe the copy it holds is the one already up there.

Every step re-checks that the signed-in account is still the one the pull started for, so an account switch mid-pull cannot write one person's face into another person's folder.

THE PHOTO MATHS

```
// LIfeOS/Components/Model/ProfilePhotoDownsizing.swift
static let side: CGFloat = 512
static let sizeLimit = 5_242_880           // the bucket's file_size_limit, kept in step by
hand

static func jpeg(from image: UIImage) -> Data? {
    let ratio = min(side / image.size.width, side / image.size.height, 1)
    let size = CGSize(width: image.size.width * ratio, height: image.size.height * ratio)
    let format = UIGraphicsImageRendererFormat.default()
    format.scale = 1
    let resized = UIGraphicsImageRenderer(size: size, format: format).image { _ in
        image.draw(in: CGRect(origin: .zero, size: size))
    }
    var quality: CGFloat = 0.8
    var data = resized.jpegData(compressionQuality: quality)
    while let current = data, current.count > sizeLimit, quality > 0.3 {
        quality -= 0.1
        data = resized.jpegData(compressionQuality: quality)
    }
    return data
}
```

Scale 1 is the line that matters. The renderer otherwise draws at the device's scale, so a 512 point box becomes a 1536 pixel bitmap on a 3x phone: nine times the pixels asked for, which undoes most of the downsizing. The quality loop is a backstop against a pathological image, not the primary way the output is kept small. The limit is deliberately the bucket's real ceiling rather than something tighter, because a client-side constant below the server's would start rejecting uploads from installs running older code with nothing on screen to explain it.

THE STATS UNDER THE NAME

`RootView` computes the three panels from the store at the moment the profile opens. "Your rhythm" is the streak from Today's snapshot plus two `fetchCount` calls, which never materialise a row. "Today" lists only the readings that exist: a missing one is left out rather than dashed, so the panel always reads as facts, and money is deliberately absent because a sample-data default would put invented dollars on a person's face. "Over time" sums steps and sleep from one fetch of the daily rows, a few hundred small objects at most, and counts habit ticks, calendar events and scored sectors with `fetchCount`; zero-valued figures are left out.

17.7 Performance and cost

- **Two small requests per launch, and the third only when needed.** The profile pull is a row and a one-object listing; the picture downloads only on a changed stamp.
- **Reads are signed-in-only, not signed-URL-per-face.** The bucket is private, but any signed-in user may read any avatar, which is what keeps a friend list from needing a signed URL per face on every refresh (Chapter 16).
- **Uploads replace in place.** `x-upsert: true` rather than a delete and an insert, so a person changing their picture is never briefly without one.

- **The gradient is three 48-pixel averages, recomputed only when the photo changes.** The `CImageContext` is shared and built with intermediate caching off.
- **Goals commit once per change, not once per frame.** The draft absorbs the stepper; the row is written on `onChange` .
- **Syncs coalesce.** Every connection view model folds a launch and a foreground into one pass, and WHOOP, Fitbit and Plaid consult `SyncStalenessPolicy` before spending a request.
- **Provider secrets never reach the device.** Fitbit tokens never even pass through it; Plaid access tokens sit behind service-role-only RLS.
- **Per-account defaults, per-account files.** Cursors, "was Health asked", the cycle switch, the avatar and the profile cache all live in the account's own suite or directory, so an account switch is a lookup change, not a purge.

17.8 Edge cases and tests

CONDITION	BEHAVIOUR
Goal set out of range	Unreachable: the steppers bound each value (sleep 240 to 660 in 15s, steps 1,000 to 30,000 in 500s, exercise 5 to 180, water 500 to 6,000 in 250s, required 1 to 4)
Health denied	Undetectable. The row reads "No data yet" and points at Health > Sharing; "Connect" becomes "Sync"
Health on an iPad	<code>.unavailable</code> ; the row says so and its button is disabled
WHOOOP sign-in abandoned in Safari	Card returns to disconnected two seconds after the app becomes active, unless a manual code is pending
WHOOOP code expired or reused	"Code expired or already used. Get a fresh one" (a 400), distinct from "Server not configured" (a 500)
WHOOOP token lacks a scope	"Whoop needs new permissions. Disconnect, then connect again"; not "expired"
WHOOOP linked on another device	Restored from the server copy on the first <code>syncIfStale</code> ; a 404 means never linked
Fitbit refresh token refused	<code>.needsReauth</code> ; the chip reads "Reconnect", an action rather than an error
Fitbit sign-in cancelled	Not a failure: the card reads as it did before
Bank OAuth outlives the process	Link resumed on relaunch from the stored token; the completion is re-registered on attach
Bank already connected	"That bank is already connected"
Bank login invalidated	<code>.needsReconnect(name)</code> ; the summary dot is hollow because a stale login is not working
Disconnect a bank	History kept unless asked; revoking a credential is not a request to erase the past
Profile publish fails	Local copy saved; sheet stays open with the reason and offers Try again or Close anyway
Offline when editing the profile	Same, with a sentence that says the profile is saved on this device but not published yet
Photo removed on another device	The next launch's pull deletes the local copy
Bucket listing fails	Read as <code>.unknown</code> : a device with a copy keeps it, a device without one downloads
Account switched mid-pull	Every write step re-checks the signed-in id and bails
Sign out	Push row deleted with the outgoing token first, then connections deactivated, pending OAuth cleared, calendar and Health opt-ins removed, the account removed from the roster, the store closed

TESTS

- `RemoteProfileTests` : a profile survives the round trip, a birthday comes back on the same calendar day, cleared fields are sent as nulls, the display name is never empty, the avatar path is the account's own folder.
- `AvatarTransferTests` (nine tests): the authenticated download route with a bearer token, a listing with and without the object, unknown versus missing, and every branch of `shouldDownloadAvatar` including the one that gets forgotten: an unknown stamp with nothing cached still downloads.
- `AccountStoreTests` and `UserScopeTests` : two accounts get different store files, an id cannot climb out of the accounts directory, rows do not cross between accounts, removing an account leaves its store on disk.
- `PlaidClientTests` , `PlaidItemStoreTests` , `PlaidSyncTests` : the four functions, the per-item cursor, and the paged sync (Chapter 11).
- `FitbitOAuthTests` , `FitbitSyncTests` , `WhoopTokenTests` , `WhoopArbitrationTests` : the PKCE URLs, the state check, the sync outcomes.
- `SyncStalenessPolicyTests` : the rule every automatic sync consults.

The screens are verified by build and by capture: the profile QA of 14 September 2026 checked the phone and iPad layouts, the glass editor, and the empty profile at accessibility text size, using a generated teal illustration to exercise the photo-colour path. Remote saving and photo library selection were not run against a live account in that pass.

17.9 Study notes

READ IN THIS ORDER

1. `LifeOSKit/Sources/Integrations/ProfileClient.swift` : `AvatarStamp` and `shouldDownloadAvatar` .
2. `Features/Onboarding/Model/ProfilePhotoStore.swift` : `ProfileSync.pull()` , then the stamp accessors.
3. `Features/Settings/View/ProfileEditSheet.swift` : `save()` .
4. `Features/Settings/ViewModel/HealthConnectionViewModel.swift` : `connect` , `decideCycleDefault` , `startWatching` .
5. `Features/Settings/ViewModel/WhoopConnectionViewModel.swift` : `resolveStalledConnect` and `finish(callbackURL:)` .
6. `Features/Settings/View/PlaidLinkPresenter.swift` : `resume(from:)` .
7. `App/RootView.swift` : `quickActions` , `profileStats` , `profileAllTime` .
8. `LifeOSKit/Sources/DesignSystem/ActionFan.swift` : why the row and the fan are one component.

CHECK YOURSELF

- **Why does `AvatarStamp` have three cases and not two?** "Missing" must never download and "unknown" must download when nothing is cached; one nil cannot say both.

- **Why is the avatar stored as a path and not a URL?** A URL bakes in the host and the signing scheme; both can change while the object stays where it is.
- **Why does `ProfileEditSheet` save locally first but wait for the publish before dismissing?** Local first makes the change visible and durable; waiting makes a publish failure visible, because an unpublished profile is one nobody can find.
- **Why does the WHOOP stall check wait two seconds?** The redirect usually lands just after the app activates; resolving at once would cancel sign-ins about to succeed.
- **Why is the cycle-tracking default read from Health rather than from the gender field?** The gender field is a form of address, its own comment says so, and using it would be wrong both ways and strand the default answer.
- **Why is `sizeLimit` exactly the bucket's ceiling?** A tighter client constant would reject uploads from older installs one release early, silently.

18 Widgets, Live Activities and notifications

Three surfaces outside the app: a Home and Lock Screen widget with a Watch complication behind it, a Live Activity and Dynamic Island for a running session, and a push channel that lets LIFO speak first. All three share one rule: the surface is handed a small, credential-free value and never a store, and every value is owned by exactly one account.

18.1 Intent

WORKING BACKWARDS

Put "Your day" on the Home Screen and it shows steps against the goal, sleep and movement, refreshed after Almanac syncs and blanked after six hours or at midnight, whichever comes first. Add the circular complication to a watch face and the step gauge is there. Begin an activity and the Lock Screen carries a blue card with the timer, heart rate zone, estimated effort, calories and battery; the Dynamic Island shows the heart rate and the timer, and the zone number when minimal. Once a day, at eight in the morning your time, LIFO may send one sentence about your sleep, your steps or a streak; tap it and the conversation opens on that sentence. A notification inbox in the profile keeps the last hundred.

What does a widget actually get?

A `SurfaceSnapshot` : an owner id, when it was generated, when it expires, and four numbers. No token, no store, no rows. It is written as one encoded value into the App Group, so a reader can never join fields from two accounts.

Why does it expire?

Because a widget that shows yesterday's steps as today's is worse than one that says "Open Almanac to sync". Six hours is long enough to be useful between launches and short enough that a stale number does not survive a night. The timeline carries an explicit expiry entry so the blanking happens even when WidgetKit delays a refresh.

Why is the Live Activity throttled?

The heart-rate sensor streams every second and ActivityKit has an update budget. The lock screen does not need a new frame per beat; it needs one when something meaningful changed. A pure `LiveActivityThrottle` decides, and the timer ticks itself through `Text(timerInterval:)` without any update at all.

Who decides whether to send a nudge?

The server, as a pure function with tests. The device's whole job is to be reachable, to say which time zone eight in the morning is in, and to open on what arrived. The numbers are not in the payload: a phone that has been offline for days re-evaluates locally rather than opening on figures two days stale.

18.2 Files and folders

```
AlmanacWidgets/  
├─ AlmanacWidgets.swift  
├─ HealthWidget.swift  
└─ WorkoutLiveActivity.swift
```

```
iOS widget extension (WidgetKit)  
the bundle: HealthWidget, BeginActivityWidget, WorkoutL  
timeline, every family, the empty state  
lock screen card, Dynamic Island regions
```

AlmanacWatchWidgets/ └─ AlmanacWatchWidgets.swift	the same HealthWidget, watchOS families only
LifeOSKit/Sources/AppSurfaces/ └─ SurfaceSnapshot.swift └─ LiveSessionReadout.swift └─ LiveSessionReadout+Text.swift └─ WorkoutActivityAttributes.swift └─ InboxEntry.swift └─ SessionRingMath.swift └─ HUDPlacement.swift └─ WatchWire.swift	depends on nothing; both extensions and the watch import the App Group value; SurfaceRoute (almanac://) PushState, the readout, LiveActivityThrottle the one place a reading becomes text ActivityAttributes; ContentState is the readout one notification, deduplicated by day and trigger ring fills for the HUD (Chapter 8) where a dragged HUD settles (Chapter 10) packets and commands (Chapter 9)
LIfiOS/App/Surfaces/ └─ SurfaceCoordinator.swift └─ WorkoutLiveActivityController.swift └─ SurfaceSettingsScreen.swift └─ SurfaceNativeChecks.swift └─ SurfaceDesignPreview.swift └─ WatchSessionBridge.swift LIfiOS/App/PushService.swift LIfiOS/Features/Notifications/NotificationInboxScreen.swift	publishes the snapshot; WCSession to the watch request, update, end; owns the throttle state the Widgets & Watch page nine DEBUG checks over accounts and the snapshot a gallery of every family, behind a build flag mirroring (Chapter 9) registration, the inbox, PushDelegate
LifeOSKit/Sources/Integrations/PushTokenClient.swift supabase/functions/lifo-nudge/ supabase/functions/_shared/apns.ts	PushRegistration, NudgePayload, device_to the hourly job (Chapter 20) provider token, payload, headers

AppSurfaces is its own package target with no dependencies because three processes import it: the app, the widget extension and the watch app. It holds value types and pure functions only, so nothing about a store, a keychain or a network client can leak into an extension that must stay small and must launch cold in a few milliseconds.

18.3 Model

THE SNAPSHOT THE WIDGET READS

```
// LifeOSKit/Sources/AppSurfaces/SurfaceSnapshot.swift
public struct SurfaceSnapshot: Codable, Equatable, Sendable {
    public static let appGroup = "group.com.shivvyas.lifeos"
    public static let storageKey = "surface.snapshot.v1"
    public var ownerId: String?
    public var generatedAt: Date
    public var expiresAt: Date
    public var measuredAt: Date?
    public var steps: Int?
    public var stepGoal: Int
    public var sleepMinutes: Int?
    public var exerciseMinutes: Int?

    public init(ownerID: String? = nil, generatedAt: Date = .now, measuredAt: Date? = nil,
                steps: Int? = nil, stepGoal: Int = 8000, sleepMinutes: Int? = nil,
                exerciseMinutes: Int? = nil, calendar: Calendar = .current) {
        ...
        let midnight = calendar.date(byAdding: .day, value: 1, to: calendar.startOfDay(for:
generatedAt))!
        expiresAt = min(midnight, generatedAt.addingTimeInterval(6 * 3600))
        self.steps = steps.flatMap { $0 >= 0 ? $0 : nil }
        self.stepGoal = max(1, stepGoal)
        ...
    }

    public func isAvailable(at date: Date = .now) -> Bool {
        ownerId != nil && date >= generatedAt.addingTimeInterval(-60) && date < expiresAt
    }
    public func supersedes(_ previous: Self?) -> Bool {
        previous.map { generatedAt > $0.generatedAt } ?? true
    }
}
```

The initialiser is where the rules live. A negative reading becomes nil rather than a lie, a goal of zero becomes one so progress can never divide by nothing, and the expiry is the earlier of midnight and six hours. A snapshot with no owner is a tombstone: it is what sign-out writes, and it is never available. `supersedes` exists because WatchConnectivity can deliver a queued older application context after a newer reply, and the watch must not replace a fresh value with a stale one.

```
// LifeOSKit/Sources/AppSurfaces/SurfaceSnapshot.swift
public enum SurfaceRoute: String, CaseIterable, Sendable {
    case today, health, activity, notifications
    public var url: URL { URL(string: "almanac://\(rawValue)")! }
    public init?(url: URL) {
        guard url.scheme == "almanac", url.user == nil, url.password == nil,
              url.port == nil, url.path.isEmpty, url.query == nil, url.fragment == nil,
              let host = url.host, let route = Self(rawValue: host) else { return nil }
        self = route
    }
}
```

Routes are the four places a widget tap can land. The parser accepts exactly `almanac://host` and nothing else: no path, no query, no credentials, no port. That is deliberate, because the same scheme carries OAuth callbacks, and a widget URL must never be mistaken for one.

THE READOUT THE LIVE ACTIVITY RENDERS

```
// LifeOSKit/Sources/AppSurfaces/LiveSessionReadout.swift
public struct LiveSessionReadout: Codable, Hashable, Sendable {
    public var elapsed: TimeInterval
    public var runningSince: Date? // nil while paused
    public var heartRate: Int?
    public var zone: Int?
    public var effort: Double? // WHOOP's 0 to 21 scale, rounded to a tenth
    public var calories: Int?
    public var distanceMeters: Int?
    public var batteryPercent: Int?
    public var capacitySource: String? // "whoop" or "health"
    public var ceilingMaxZone: Int?
    public var ceilingTarget: ClosedRange<Double>?
    public var reps: Int?
    public var setIndex: Int?
    public var push: PushState

    public var timerAnchor: Date? { runningSince?.addingTimeInterval(-elapsed) }
    public var isPaused: Bool { runningSince == nil }
}
```

This is the one value every live surface renders: the in-app HUD, the lock screen and the Dynamic Island. Every reading is optional because a missing sensor is not a zero. `WorkoutActivityAttributes.ContentState` is this very type, so the app and the extension cannot disagree about its shape. `timerAnchor` is the trick that makes the timer free: `Text(timerInterval: anchor...distantFuture)` lets the system tick the clock itself, so an activity update is needed only when a reading changes, never per second. `PushState` carries its own headline, "Easy going", "On track", "Near your limit", "Over your target", so the words never drift between surfaces.

```
// LifeOSKit/Sources/AppSurfaces/LiveSessionReadout.swift
public enum LiveActivityThrottle {
    public static let heartRateStep = 3
    public static let floor: TimeInterval = 10
    public static let readingInterval: TimeInterval = 3

    public static func shouldPublish(previous: LiveSessionReadout?, next: LiveSessionReadout,
                                     lastPublishedAt: Date?, now: Date) -> Bool {
        guard let previous, let lastPublishedAt else { return true }
        if previous == next { return false }
        if previous.isPaused != next.isPaused { return true }
        if previous.push != next.push { return true }
        if previous.reps != next.reps { return true }
        switch (previous.heartRate, next.heartRate) {
        case let (old?, new?) where abs(old - new) >= heartRateStep:
            return now.timeIntervalSince(lastPublishedAt) >= readingInterval
        case (nil, .some), (.some, nil):
            return now.timeIntervalSince(lastPublishedAt) >= readingInterval
        default: break
        }
        return now.timeIntervalSince(lastPublishedAt) >= floor
    }
}
```

Read it as a ranking of what deserves a frame. A pause, a push-state change or a rep always publishes at once. A heart rate that moved three beats or more, or appeared or vanished, publishes if three seconds have passed. Anything smaller waits ten. An identical readout never publishes. Because it is a pure function it is pinned by tests instead of thumbs on a device.

THE INBOX ENTRY AND THE NUDGE

```
// LifeOSKit/Sources/AppSurfaces/InboxEntry.swift
public struct InboxEntry: Codable, Equatable, Identifiable, Sendable {
    public let id: String // "\(day)|\(trigger)"
    public let ownerId: String
    public let text: String
    public let trigger: String
    public let day: String
    public let receivedAt: Date
    public var isRead: Bool

    public static func inserting(_ entry: Self, into entries: [Self], ownerId: String) -> [Self] {
        guard entry.ownerID == ownerId else { return entries.filter { $0.ownerID == ownerId } }
        var result = entries.filter { $0.ownerID == ownerId }
        if !result.contains(where: { $0.id == entry.id }) { result.append(entry) }
        return Array(result.sorted { $0.receivedAt > $1.receivedAt }.prefix(100))
    }
}
```

The id is the day plus the trigger, which is exactly the uniqueness the server enforces in `nudge_log`, so a notification delivered twice, once as a banner and once as a tap-through, is one row. `inserting` filters to the owner, deduplicates, keeps existing read state, and caps at a hundred. `NudgePayload`, in `Integrations`, is the same three fields parsed out of a notification's `userInfo`, and it returns nil rather than throwing for a notification that is not ours: something to ignore, not something to report.

18.4 ViewModel

SURFACECOORDINATOR: ONE WRITER FOR THE APP GROUP

```
// LIfiOS/App/Surfaces/SurfaceCoordinator.swift
func publish() {
    guard let ownerID, let context, sharingEnabled else { return }
    let store = MetricsStore(context: context)
    let now = Date.now
    let row = try? store.metrics(from: now, to: now).first
    let goals = try? context.fetch(FetchDescriptor<UserGoals>()).first
    let next = SurfaceSnapshot(ownerID: ownerID, measuredAt: row?.updatedAt,
        steps: row?.steps, stepGoal: goals?.stepsGoal ?? 8000,
        sleepMinutes: row?.sleepMinutes, exerciseMinutes: row?.exerciseMinutes)
    // Coalesce saves unrelated to health. Still renew the Watch lease hourly.
    if let previous = snapshot, previous.ownerID == next.ownerID,
        previous.steps == next.steps, previous.sleepMinutes == next.sleepMinutes,
        previous.exerciseMinutes == next.exerciseMinutes, previous.stepGoal == next.stepGoal,
        previous.measuredAt == next.measuredAt,
        next.generatedAt.timeIntervalSince(previous.generatedAt) < 3600 { return }
    write(next)
}
```

`publish()` is called from `RootView.reloadAll()`, which means on every save in the app. Most saves are notes, money or a habit tick, and none of those change the four numbers, so the coalescing guard is what stops a widget reload and a watch transfer per keystroke. The hour limit is the exception: even with nothing changed, the snapshot is rewritten hourly so its expiry keeps moving and the watch's copy stays fresh.

`write` stores the value, asks WidgetKit to reload every timeline, and sends the same bytes to the watch as an application context (queued, survives the watch being asleep) and, if it is reachable, as a message.

`adopt(ownerID:context:)` is called by `AccountSession` whenever a store is opened. If the snapshot on disk belongs to a different owner, or there is no owner, or sharing is off, a tombstone is written before anything else, so the next account starts with missing readings and never with the previous account's values.

`sharingEnabled` is the toggle on the Widgets & Watch page; turning it off writes the tombstone at once.

WORKOUTLIVEACTIVITYCONTROLLER

```
// LifeOS/App/Surfaces/WorkoutLiveActivityController.swift
func sync(_ readout: LiveSessionReadout, timer: ActivitySessionState, icon: String) {
    guard timer.phase != .finished else { end(); return }
    if current == nil {
        current = Activity<WorkoutActivityAttributes>.activities.first {
            $0.attributes.sessionID == timer.id
        }
        let now = Date.now
        guard current == nil || LiveActivityThrottle.shouldPublish(previous: lastState, next:
readout,
                                                                    lastPublishedAt:
lastPublishedAt, now: now) else { return }
        lastState = readout; lastPublishedAt = now
        let content = ActivityContent(state: readout, staleDate: now.addingTimeInterval(8 *
3600))
        if let current {
            let preceding = updates
            updates = Task { await preceding?.value; await current.update(content) }
        } else if ActivityAuthorizationInfo().areActivitiesEnabled {
            current = try? Activity.request(attributes: WorkoutActivityAttributes(sessionID:
timer.id,
                                                                    name: timer.activity, icon: icon), content: content, pushType: nil)
        }
    }
}
```

The recorder calls `sync` on every state change and every reading; the controller decides whether that becomes an ActivityKit call. Three details carry weight. The activity is keyed on the session id, so a relaunch, or a session the watch started whose card was requested before the phone's timer existed, adopts the activity already on screen rather than orphaning it. Updates are chained through a single `Task` so they land in order. And the stale date is eight hours out, which means a phone that dies mid-run leaves a card that the system will eventually grey rather than one that claims a heart rate forever. `endAll()` is called on every account change and sign-out so no session card outlives its owner.

PUSHSERVICE: REGISTRATION AND THE INBOX IN ONE OBJECT

One object rather than two because the two halves share the state that matters: the device token. It arrives asynchronously from the system long after the sign-in that asked for it, and it is needed again at sign-out to delete the row. The service keeps the token in `UserDefaults.standard` (it belongs to the phone), and everything else per account.

- **Asks late, on purpose.** `requestAuthorization()` is called from the inbox screen's button, never at launch. A permission prompt on first run, before the app has shown anyone anything worth being interrupted about, is the fastest way to a permanent no. A denied status short-circuits: registering anyway would silently succeed and produce a token that can never deliver anything.
- **Registers on every foreground.** `syncRegistration()` upserts the token, the account id and the current time zone. The zone is what the send hour is read in, so somebody who has flown somewhere would otherwise keep being nudged at eight in the morning where they used to live. The row is keyed on the token, so a second account on the same device replaces the first account's row in one upsert.

- **Deregisters before the session goes.** `AppShell` 's sign-out captures the outgoing access token by value and deletes the row with it, because the delete needs the token of the account whose row it is, and the keychain is cleared a line later. Left behind, that row would push one person's sleep onto a phone showing somebody else's name.
- **Attaches per account.** `attach(ownerID:)` re-reads the inbox from that account's defaults suite and clears any pending route if the owner changed. `receive` refuses an entry whose owner is not the attached account, and refuses an id the person has already cleared, so a delayed duplicate cannot resurrect a dismissed check-in.
- **Reads back what the system already delivered.** `refreshInbox()` walks `deliveredNotifications()` , so a notification that arrived while the app was dead, and was never tapped, still lands in the inbox on the next open.

18.5 View

THE WIDGET, IN EVERY FAMILY

`HealthTimeline` produces two entries: the snapshot now, and, if it has not already expired, a nil entry at its expiry. The policy asks for a refresh after thirty minutes. The view switches on the family. The inline accessory is a label of steps and sleep. The circular accessory is a `Gauge` of step progress. The rectangular accessory is the wordmark, the two figures and "Updated 7:41". The small home widget is the step numeral at 34 points against the goal with a progress bar; medium adds sleep and a movement line; large and extra large add a movement tile and a "Begin activity" link that deep-links to `almanac://activity` . Every figure is `privacySensitive()` , so a locked phone redacts it. The empty state is one sentence per family and never a zero. `BeginActivityWidget` is a second, accessory-only widget whose whole job is the deep link. The watch bundle ships the same `HealthWidget` with only the accessory families, and the watch canvas is plain black.

THE LIVE ACTIVITY

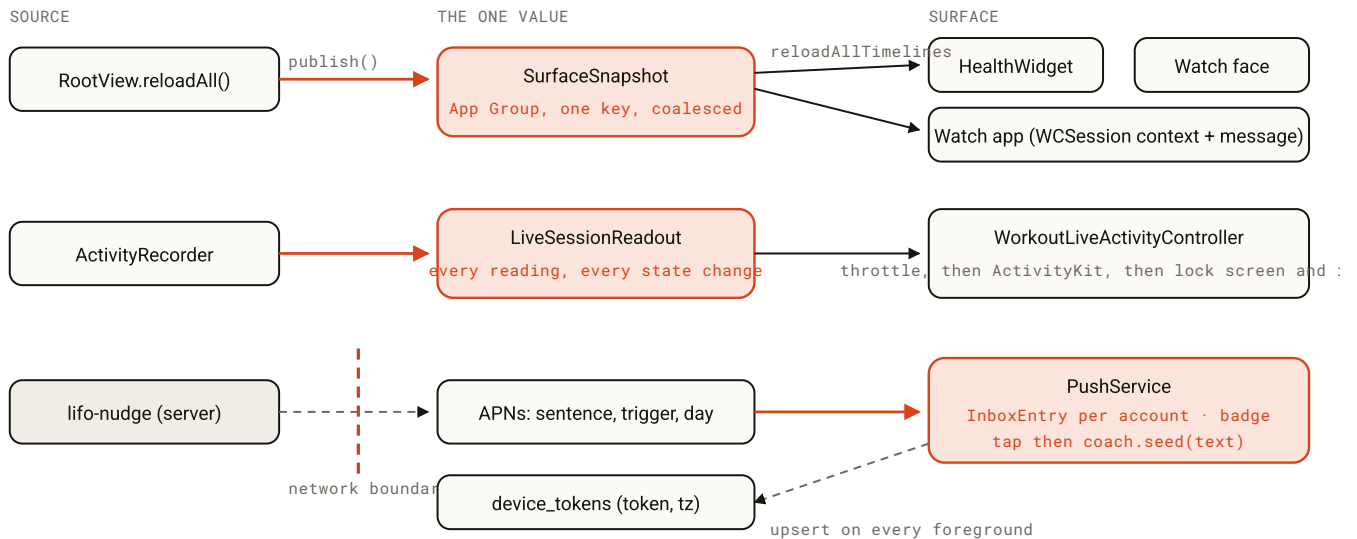
Widget extensions cannot render materials, so the "glass" tiles on the lock screen card are translucent white at 18 percent over the live gradient with a half-point hairline. The card is the activity's icon and name, the push headline in the push tint, the timer at title size, then four tiles: heart rate with the zone as eyebrow, effort with "of 18" as unit once there is a reading, calories or, in a strength session, the set and reps, and battery with a tiny ring. The Dynamic Island's expanded view puts the name leading, the timer trailing, and four readings across the bottom with an "Open activity" link. Compact shows a heart and the beats per minute leading and either the timer or the reps trailing. Minimal is the zone number in a circle of the push tint, or a pause glyph. `keylineTint` follows the push state, so the island's outline goes green, amber or red with the person's effort. Every text field is `privacySensitive()` .

THE INBOX AND THE SETTINGS PAGE

`NotificationInboxScreen` is reached from the bell on the profile and from Settings. It shows the permission card first ("Enable notifications" when never asked, "Notification settings" otherwise), then either "You're all caught up" or the unread count, a "Read all" button, one panel per entry, and "Clear read notifications". Tapping an entry marks it read and opens a full-screen `LifoCoachScreen` seeded with the sentence, so the notification is an opening line rather than a reminder (Chapter 14). The screen refreshes from delivered notifications on appear and on every return to the foreground. `SurfaceSettingsScreen` is the

Widgets & Watch page: the sharing toggle and four short guides for adding the widgets, the complication and the activity timer.

18.6 How it works



Each surface is fed one value type from `AppSurfaces`, and each value is produced by exactly one object in the app. Nothing outside the process ever holds a token, a store or a row.

THE WIDGET PATH

1. A save anywhere fires `ModelContext.didSave`; `RootView.reloadAll()` calls `SurfaceCoordinator.shared.publish()` first.
2. `publish()` reads today's row and the goals, builds a snapshot, and returns early unless one of the four numbers, the goal, or the measured time changed, or an hour has passed.
3. `write()` encodes the snapshot into the App Group defaults under one key, reloads every `WidgetKit` timeline, and sends the bytes to the watch.
4. The widget's timeline provider decodes it on the next reload, renders it if `isAvailable`, and schedules its own blank entry at `expiresAt`.
5. The watch app decodes the context or message, keeps it only if it `supersedes` the copy it holds, writes it to its own App Group and reloads its complication. Its dashboard redraws on a sixty-second `TimelineView`, so the expiry takes effect without a message.

THE LIVE ACTIVITY PATH

1. The recorder rebuilds its readout on every state change (`persist()`) and every sensor reading (`persistReading(at:)`), then calls `syncLiveActivity()`. The draft is written at most every ten seconds; the readout is refreshed every time.
2. The controller either finds the activity for this session id, or asks `LiveActivityThrottle` whether the change deserves an update, or requests a new activity if the person has them enabled.
3. On finish, discard, account change or sign-out the activity is ended immediately. `endStrayLiveActivities()` removes any card that belongs to a different session id, because only one

activity belongs on screen.

THE NUDGE PATH

1. Once an hour, `pg_cron` calls `lifo-nudge`. The job selects the rows in `device_tokens` whose local hour is eight, evaluates the triggers over the account's rows, claims the day in `nudge_log`, phrases one sentence, and sends it through APNs with a collapse id so a phone that was off for three days gets today's nudge rather than three, and a twelve-hour expiry so a morning observation never turns up stale (Chapter 20).
2. `PushDelegate.willPresent` hands the notification to `PushService.receive`. A nudge that arrives while the app is open is still shown as a banner, because suppressing it would drop the message entirely; nothing else surfaces it.
3. `PushDelegate.didReceive` is the tap. The entry is marked read and `pending` is set to the payload.
4. `RootView` watches `pending` and, on the launch task and every foreground, checks it directly too, because a notification tapped from a cold launch lands before the view exists and `onChange` never fires for it. `openNudge` clears it, seeds the coach with the sentence, and presents LIFO, deferring behind any open profile or inbox cover.

18.7 Performance and cost

- **A widget launch reads one defaults key.** No store is opened in the extension; `AppSurfaces` has no dependencies, so the extension links almost nothing.
- **Publishes are coalesced.** Saves unrelated to health cost no widget reload and no watch transfer; only a changed number or the hourly lease renewal does.
- **Timelines refresh themselves.** Two entries and a thirty-minute policy; the blanking at expiry costs no process wake at all.
- **The timer is free.** `Text(timerInterval:)` ticks in the widget process; an ActivityKit update is spent only on a meaningful change, and the throttle's floor of ten seconds keeps a jittery sensor from burning the budget.
- **The watch gets bytes, not rows.** An application context is a few hundred bytes and is delivered whenever the watch next wakes.
- **APNs costs nothing per send, and the job sends at most once a day per person.** The unique key on `nudge_log` is claimed before any model call, so an overlapping cron run cannot double-spend the phrasing.
- **Silence is the common path.** Most hourly runs evaluate triggers and send nothing, which costs a query and no inference.
- **Dead tokens are pruned.** An "unregistered" answer from Apple deletes the row, so the job does not push at a wiped phone forever.
- **The inbox is capped at a hundred, and cleared ids at two hundred.**

18.8 Edge cases and tests

CONDITION	BEHAVIOUR
Widget refresh delayed past expiry	The timeline's own nil entry blanks it at <code>expiresAt</code>
Sign out	A tombstone with no owner replaces the shared value; every widget shows the empty state; the watch adopts it because it supersedes
Account switch	Tombstone first, then the new account's snapshot; the next account starts with missing readings, never the previous values
Sharing turned off	Tombstone written at once; turning it back on republishes
Watch receives an old context after a new message	Rejected by <code>supersedes</code>
Watch away from the phone	Shows its last copy until it expires, then "Open Almanac on your iPhone to sync"
Negative or absurd reading	Stored as nil; a goal of zero becomes one
Live Activities disabled by the person	<code>Activity.request</code> is never called; the HUD in the app still runs
Session card from a previous session still on screen	Ended as a stray the moment a new timer exists
Phone dies mid-session	The card's stale date of eight hours lets the system grey it
Heart rate jitters by one or two beats	No update until ten seconds have passed
Notification for another account arrives	Refused by <code>receive</code> ; no banner, no inbox row
Same nudge delivered twice	One inbox row; read state preserved
Cleared check-in redelivered late	Refused; the dismissed id is remembered
Notification tapped from a cold launch	Picked up from <code>pending</code> in the launch task, not from <code>onChange</code>
Tap arrives while the profile is open	Deferred; the cover closes first, then LIFO opens
Permission denied	No registration; the inbox card offers the system settings
Registration fails	Logged, not retried; the next foreground calls it again
Time zone changes	Re-registered on the next foreground, so the send hour follows the person
Development build against production APNs	400 BadDeviceToken on every send; <code>aps-environment</code> must match the signing and <code>APNS_SANDBOX</code> picks the host
Simulator	APNs registration fails routinely and is logged at info level
Widget URL carrying a path or a query	Not a route; falls through to the OAuth handlers

TESTS

- `AppSurfacesTests` (seven tests): the snapshot is bounded and missing is not zero, it expires at midnight, invalid metrics are rejected and the goal clamped, a tombstone replaces every field and an old watch delivery is rejected, routes accept only exact internal destinations, the inbox deduplicates, preserves read state, rejects another account, and keeps only the hundred most recent.
- `LiveSessionReadoutTests` (nine tests): the readout round-trips and keeps absence, a paused readout has no anchor, the push headlines are words, the throttle publishes first and on meaningful change, spaces heart-rate updates, floors small changes at ten seconds, and publishes on a rep.
- `PushTokenTests` : a registration carries the account and the zone, defaults to this device's zone, a nudge parses into its sentence and what it was about, and a foreign notification is ignored rather than reported.
- `apns_test.ts` (nine tests): the JWT header and claims, base64url, a PEM key that survives a dashboard paste, the payload carries the sentence and never the numbers, the headers collapse the channel and let a stale nudge expire, a dead token is recognised from both answers Apple gives, a provider token is signed once and reused inside its window.
- `nudge_test.ts` : every trigger, the cooldown, the send hour read in the person's zone, and the rule that the positive wins (Chapter 20).
- `SurfaceNativeChecks` (nine checks, run in the simulator through the design preview): current-account notification accepted, repeat delivery deduplicated, a switch clears the visible inbox and pending route, a delayed notification from the previous account is rejected, the account inbox and read status are restored, a cleared check-in cannot be restored by a duplicate, the widget exports today's data and not a historical day, sign-out replaces the shared cache with an empty snapshot, and the next account starts with missing readings.

STILL PENDING ON HARDWARE

The QA note of 14 September records that real simulator Home widget reads and a paired WatchConnectivity transfer were verified with synthetic values, and that physical-device behaviour, App Group provisioning across every target, and a fresh iPad surface pass remain to be done. The push path has been proved end to end with the forced-send route on the nudge function, but only against a development build.

18.9 Study notes

READ IN THIS ORDER

1. `LifeOSKit/Sources/AppSurfaces/SurfaceSnapshot.swift` : the initialiser's rules and `SurfaceRoute` .
2. `App/Surfaces/SurfaceCoordinator.swift` : `adopt` , `publish` , `write` .
3. `AlmanacWidgets/HealthWidget.swift` : the timeline's two entries.
4. `LifeOSKit/Sources/AppSurfaces/LiveSessionReadout.swift` : the throttle.
5. `App/Surfaces/WorkoutLiveActivityController.swift` , then `ActivityRecorder.syncLiveActivity` and `endStrayLiveActivities` .
6. `AlmanacWidgets/WorkoutLiveActivity.swift` : which region shows what.
7. `App/PushService.swift` : `attach` , `receive` , `syncRegistration` , then `PushDelegate` .

8. `App/RootView.swift` : `openNudge` and the three places it is called.

CHECK YOURSELF

- **Why is the snapshot one encoded value under one key rather than four keys?** So a reader can never join fields from two accounts; a tombstone replaces everything at once.
- **What does `publish()` do on a notes save?** Builds a snapshot, sees nothing changed within the hour, and returns without writing, reloading or sending.
- **Why does the Live Activity need no update per second?** The readout carries `timerAnchor`, and `Text(timerInterval:)` ticks in the widget process.
- **What always publishes regardless of timing?** A pause or resume, a push-state change, and a rep.
- **Why are the nudge's numbers absent from the payload?** A phone offline for days would open on stale figures; the tap-through re-evaluates locally.
- **Why is the push row deleted before the keychain is cleared at sign-out?** The delete needs the outgoing account's token; afterwards there is nothing to authenticate it with.

PART THREE

Under the hood

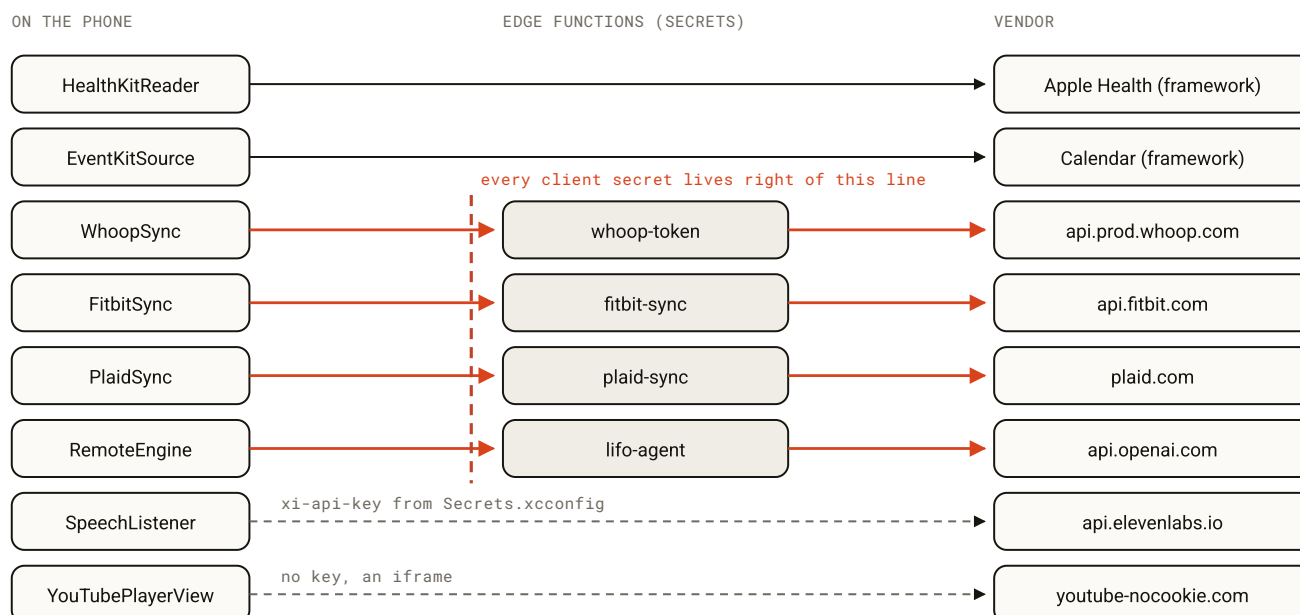
The external systems, the backend, every stored type, the edge cases the app is built to survive, how to set it up and ship it, and what it costs.

- 19. Integrations
- 20. The backend
- 21. Data reference
- 22. Production edge cases
- 23. Operations
- 24. Cost and the allowance

19 Integrations

Eleven external systems feed or serve the app, and one had to be cut. Each is confined to the `Integrations` target or a single feature file, so no screen knows a vendor exists: a `ViewModel` sees rows in `DailyMetrics` and a connection state, nothing more.

Revision D described four integrations. Since then Apple Health went from nine types to forty seven, Fitbit arrived as a second strap, Plaid brought bank sync, EventKit brought the calendar, the coach gained a cloud tier and a voice, YouTube embeds carry the workout library, and APNs delivers the morning nudge. Two straps writing the same row also forced a real merge rule, the arbiter, which sits in the middle of this chapter because every health source now goes through it. The chapter takes each system in turn: what it provides, how it authenticates, the client code, the wire format, how its readings reach the spine, how it fails, and which tests pin it.



Solid arrows are direct framework or REST calls. Highlighted arrows pass through an Edge Function because the vendor requires a client secret, and an `.ipa` is a zip file. Dashed arrows carry no secret worth protecting. Whatever the path, every health reading ends in the same `DailyMetrics` row via the arbiter in 19.4.

19.1 Apple HealthKit

The primary source, and since 27 August the widest. Revision D read nine types with anchored queries. The reader now covers forty seven metrics, and the design decision that made that possible without forty seven schema changes is worth understanding first.

THE VOCABULARY LIVES OUTSIDE HEALTHKIT

`HealthMetric` is a plain Swift enum in `Integrations` with no `HealthKit` import. It carries the vocabulary (a title, a unit label, a decimal count, a display group) and the merge rules (which metrics a strap outranks the

phone on, which a person can type). The translation to `HKQuantityTypeIdentifier` lives in one static function on `HealthKitReader`, behind `#if canImport(HealthKit)`, which is what keeps the package building for macOS and `swift test` running from the terminal.

```
// LifeOSKit/Sources/Integrations/HealthMetric.swift
public enum HealthMetric: String, CaseIterable, Sendable, Identifiable {
    // Movement
    case steps, activeEnergyKcal, restingEnergyKcal, exerciseMinutes,
        standMinutes, distanceKm, flightsClimbed, activeZoneMinutes
    // Heart and fitness
    case restingHR, walkingHR, hrvMs, vo2Max, cardioRecoveryBpm, readinessScore
    // ... body, sleep, vitals, intake, mind, gait, cycle

    /// Measured by a strap worn on the body, so a wearable outranks the phone.
    public var claimedByWearable: Bool { ... }
    /// A person can type this into the app, so a background sync must not
    /// replace what they entered on purpose.
    public var acceptsManualEntry: Bool {
        switch self {
            case .weightKg, .waterML: true
            default: false
        }
    }
    public var healthIsAuthoritative: Bool {
        !claimedByWearable && !acceptsManualEntry
    }
}
```

Adding a metric is four lines in this enum plus one mapping line in the reader. Nothing else changes, because of how the row stores it. `DailyMetrics` keeps its ten original named columns (steps, sleep, weight and so on) because the sector scorers, the charts and the Whoop merge read them by name. Everything else lands in `extrasData`, a JSON dictionary keyed by `HealthMetric.rawValue`. A missing key means no reading; nothing writes a zero to stand in for one. A parallel `sourcesData` bag records who wrote each value, which is what the arbiter reads back.

READING A DAY

Revision D used anchored queries and persisted anchors. That is gone. The reader now asks for one day at a time with `HKStatisticsQuery`, summing counts and averaging measurements, because a day is the unit the spine stores and a statistics query de-duplicates across sources for free. Summing raw samples yourself double-counts steps on a day you carried a phone and a watch.

```
// LifeOSKit/Sources/Integrations/HealthKitReader.swift
/// Counts accumulate across the day; measurements are averaged over it.
/// Summing a heart rate would produce a number in the thousands.
static func option(for metric: HealthMetric) -> HKStatisticsOptions {
    switch metric {
    case .steps, .activeEnergyKcal, .restingEnergyKcal, .exerciseMinutes,
        .standMinutes, .distanceKm, .flightsClimbed, .waterML,
        .dietaryEnergyKcal, .proteinG, .carbsG, .fatG, .caffeineMg,
        .daylightMinutes:
        .cumulativeSum
    default:
        .discreteAverage
    }
}

/// What to multiply HealthKit's number by to get the unit the app stores.
static func scale(for metric: HealthMetric) -> Double {
    unit(for: metric) == .percent() ? 100 : 1
}
```

The `scale` function exists because of a real bug. `HKUnit.percent()` yields 0.98 for a 98 percent blood oxygen reading, and the app stores percentages as percentages everywhere else, Whoop included. A Health-sourced SpO2 landed two orders of magnitude below a Whoop one and rendered as "1.0". It went unnoticed because Whoop owns that field and Health only fills its gaps.

Sleep is a category, not a quantity, so it is summed by hand from `HKCategoryValueSleepAnalysis` samples. One query answers six metrics: total asleep, deep, REM, core, awake and in-bed. `.inBed` is deliberately not counted as sleep, because it is time on a mattress and would inflate the night by however long its owner reads before turning the light off. Mindful minutes sum a category's durations; menstrual flow takes the day's latest ordinal sample, because a value entered later is a correction.

A metric that errors is dropped rather than failing the day. One unreadable type must not cost the user the other forty six.

WHICH DAYS TO RE-READ

`HealthSyncWindow` is a pure function from "when did we last sync" to "which days to read", oldest first. It always includes today, because a step count climbs until midnight. It includes the day of the last sync as well as the days since, because a night's sleep is recorded against the morning it ends and the day already covered can still gain a value. A first sync backfills thirty days, the month the dot grid shows. A stamp from the future means a clock change, not a sync that has not happened, and yields just today.

LIVE CHANGES AND THE FOREGROUND RULE

`changes(for:)` returns an `AsyncStream` fed by one `HKObserverQuery` per type. An observer rather than a timer: a step count changes dozens of times an hour and not at all overnight, so any interval short enough to look live is mostly wasted queries. The stream buffers one event (`bufferingNewest(1)`) because a batch of samples landing together is one reason to re-read the day, not thirty. Every observer handler calls its

completion even on the error path, because HealthKit retries a query whose handler never completed, and a silent leak of those is how an app ends up with a hundred live observers.

BACKGROUND DELIVERY IS DELIBERATELY OFF

`enableBackgroundDelivery` needs an entitlement and wakes the app to write rows nobody is looking at. The launch and foreground syncs already cover the time the app was away, and the observer covers the time it is open. Revision D listed background delivery as a freshness mechanism; the code that shipped chose not to.

AUTHORISATION AND THE CYCLE GATE

The authorisation trap from Revision D still holds: HealthKit never says whether read access was granted, only whether it was asked, so branching on status is always a bug and an empty result is indistinguishable from a refusal. The settings row therefore links out to Health rather than claiming a state.

What is new is a second prompt. The first request asks for `HealthMetric.universal`, everything that applies to everyone, plus the biological sex characteristic. Cycle metrics (`basalBodyTemperatureCelsius`, `menstrualFlowLevel`) are requested in a separate `requestCycleAuthorisation()` and only when cycle tracking is on. Not because they are more private than a heart rate, but because a permission sheet that asks every person for menstrual data tells most of them the app has misread who they are.

`healthSuggestsCycleTracking()` reads Health's own sex field to pick the default, and it is a default only: the setting is offered either way. The display side of the gate is

`HealthDaySummary.groups(for:includingCycle:)`, which withholds cycle rows even for a day that was read while the switch was on.

Two write purpose strings ship. The main app's declares that it saves activities the user chooses to record; the watch's declares that it saves workouts recorded on the wrist. Revision D's "read-only in V1" is no longer true: the live session recorder in Chapter 8 writes workouts, and App Store validation demanded the string.

APPLYING A DAY

`HealthApply.write` is the one place the `HealthMetric` vocabulary meets `DailyMetrics` columns. It is separated from the reader so the mapping can be tested against a real in-memory store without a device: `MetricArbiter` can be perfectly correct and still be wired into the wrong column, and only this layer catches that. Every write goes through `MetricsStore.upsert`, a merge rather than a replace, so the columns Health has nothing to say about are left exactly as Whoop or the user left them.

FAILURE HANDLING

CONDITION	BEHAVIOUR
Device has no Health store (iPad)	<code>HealthKitError.unavailableOnThisDevice</code> ; the connection row reads "Not available on this device"
User refuses the sheet	Indistinguishable from no data. Row reads "No data yet. Check Almanac in Health > Sharing"
One type's query fails	Dropped from that day; the other metrics still write
Observer query errors	Logged, completion still called, stream stays open
Last sync stamp is in the future	Treated as a clock change; only today is re-read

TESTS

`HealthApplyTests` (nine tests) pins that Health fills an empty day across every column, that Whoop values survive a Health sync, that a later sync corrects a step count, that an empty Health day changes nothing, that counts are stored as integers, and that every write records its provenance. `HealthMetricTests` and `HealthSyncWindowTests` pin the vocabulary and the re-read window. The reader itself is verified on a device: it needs a Health store to exist.

19.2 Whoop

Still the only source of Recovery percentage, Strain and Whoop's sleep scoring, and still the integration with the most code: eleven files in `Integrations` and one archive model in `Persistence`. Everything from Revision D holds, so this section states the design briefly and spends its words on what changed: server-held connections, a wider digest, the arbiter, and the two URL schemes.

THE FIVE COLLECTIONS, DEFINED ONCE

`WhoopCollection` pairs each endpoint path with its OAuth scope, and `WhoopOAuth.defaultScopes` is derived from it plus `offline`. This exists because path and scope once drifted: two endpoints were added to the client while the authorize request still asked for three scopes, Whoop 401'd a token it had just issued, and the sync read that as a dead credential and deleted it. Colocating them makes a fetchable-but-unauthorised collection unrepresentable, and `everyCollectionsScopeIsRequestedAtAuthorization` pins it.

COLLECTION	PATH	SCOPE	FEEDS
Recovery	<code>recovery</code>	<code>read:recovery</code>	Recovery %, HRV, resting HR, SpO2, skin temp, calibrating flag
Sleep	<code>activity/sleep</code>	<code>read:sleep</code>	Asleep minutes, stages, performance, efficiency, consistency, need and debt components
Cycle	<code>cycle</code>	<code>read:cycles</code>	Day strain, calories, average and max HR
Workout	<code>activity/workout</code>	<code>read:workout</code>	<code>WorkoutRecord</code> rows with six zone durations, exercise minutes
Body	<code>user/measurement/body</code>	<code>read:body_measurement</code>	Weight, when nothing else has written it

OAuth: PKCE ON THE PHONE, THE SECRET ON THE SERVER

Whoop registers apps as confidential clients, so the token exchange runs in the `whoop-token` Edge Function. The app builds the authorize URL with an S256 challenge over a random 32-byte verifier, hands the returned code and verifier to the function, and the function performs the exchange with `WHOOP_CLIENT_SECRET`. PKCE is kept even though the exchange is server-side because it binds the code to this app instance.

The redirect is two hops. Whoop requires an https redirect URI, so it points at `whoop-callback`, which 302s to a custom scheme. Sign-in opens in real Safari via `UIApplication.open` because Whoop's login sits behind a Cloudflare challenge that never clears inside `ASWebAuthenticationSession`. The pending verifier and state are persisted to the Keychain as a list capped at five and fresh for fifteen minutes, because Safari backgrounds the app and iOS may kill it before the callback returns, and because tapping Connect twice creates two valid in-flight authorisations.

TWO URL SCHEMES, AND WHICH ONE THE BRIDGE SENDS

The app claims both `almanac` and `lifeos`. The bridge sends `lifeos://whoop-callback` because that is the scheme every build already installed knows, including ones that predate the rename. Pointing it at `almanac` stranded anyone who had not updated: Safari was handed a scheme no installed app claimed and refused it as an invalid address. The comment in `whoop-callback/index.ts` records the plan to move to `almanac` once no old build is in use, and why `lifeos` is the weaker claim: a device that had the app before its bundle identifier changed carries the old install as a separate app that also claims it.

THE CONNECTION NOW LIVES ON THE SERVER TOO

Migration `20260913090000_whoop_account_connections.sql` added a `whoop_connections` table, and `whoop-token` now stores every issued pair there under the caller's user id. The phone still keeps its own copy in the Keychain, scoped to the signed-in account, but three things changed:

- **Restore.** A fresh install, or a second device, calls `WhoopTokenExchange.restore()` (`{ "action": "restore" }`). If the server has a live pair it returns it; a 404 means this account never linked Whoop.

`WhoopConnectionViewModel.syncIfStale` tries this once per process before deciding the strap is disconnected.

- **Refresh under a lock.** Whoop rotates the refresh token on every use, so two devices refreshing at once would invalidate each other. The function claims the row through `claim_whoop_refresh`, a 30-second lock, and a second caller gets `refresh_busy` (503) rather than a dead credential. The update is conditioned on the lock still being held, so a late refresh cannot resurrect a removed or reconnected row.
- **Disconnect is a DELETE.** The row goes, then the Keychain is cleared.

The function short-circuits when the stored access token has more than sixty seconds left, returning it without touching Whoop, so a restore or a refresh check costs one database read.

TOKEN LIFECYCLE ON THE DEVICE

`WhoopTokens.isExpired` treats a token as expired sixty seconds early, and `WhoopSync.sync` refreshes proactively before fetching rather than reacting to a 401 mid-sync. Only a refused refresh clears the credential. A 401 from a collection endpoint throws `accessDenied` and keeps the tokens: it usually means the grant predates a scope the app now needs, and the fix is reconnect, which the message says in those words. `aCollectionRejectionKeepsTheConnection` and `aRejectedRefreshDoesClearTheConnection` pin the two branches.

WIRE FORMAT

Whoop's timestamps carry fractional seconds, which Foundation's stock ISO 8601 strategy rejects, so `WhoopClient.decoder` tries fractional first then plain. Pages take `start`, `end` and `limit=25` (Whoop's maximum) and follow `next_token` up to a cap of 20 pages, logging when the cap truncates. Records with a `score_state` other than `SCORED` are archived but not derived.

The DTO to sample mapping lives exactly once, as a computed `sample` property on each DTO, because the live sync and `rederive()` once drifted apart. Three widenings landed on 26 August and all three are in that one mapping:

```
// LifeOSKit/Sources/Integrations/WhoopClient.swift
// An empty object (every member nil) means Whoop reported no zone
// data at all, not that every zone measured zero: `nil` here, not
// `[0, 0, 0, 0, 0, 0]`, which would claim a reading that was never taken.
zoneMinutes: score?.zone_durations.flatMap { z -> [Int]? in
    let raw = [z.zone_zero_milli, z.zone_one_milli, z.zone_two_milli,
               z.zone_three_milli, z.zone_four_milli, z.zone_five_milli]
    guard raw.contains(where: { $0 != nil }) else { return nil }
    return raw.map { WhoopSleepMath.minutes(fromMilliseconds: $0) ?? 0 }
}
```

- **Zone durations.** Six int64 milliseconds per workout, stored as six columns on `WorkoutRecord`. They feed the live effort model in Chapter 8 and the coach's digest. An empty object decodes to nil, not six zeros; `anEmptyZoneDurationsObjectDecodesToNil` pins it.
- **Sleep need components.** Baseline, need from debt, need from strain and need from a recent nap, the last one negative by definition and preserved with its sign.

- **The calibrating flag.** `user_calibrating` lands on `DailyMetrics.whoopRecoveryIsCalibrating`, optional so an existing store migrates. A recovery produced during calibration is not a score that supports a comparison, and presenting it unqualified is the same class of error as a false zero.

ARCHIVE FIRST, DERIVE SECOND

`WhoopSync.sync()` fetches all five collections for a rolling fourteen days, splits every page into one payload per record with `WhoopRawSplit` (recovery is keyed by its `cycle_id` because it carries no id of its own), stores them in `WhoopRawRecord` under `(kind, externalID)` in one batched save, and only then derives. There is no cursor: idempotency comes from upserts, so re-syncing is always safe.

`WhoopDerivation.rederive()` rebuilds every derived row from the archive with no network call. The body measurement carries no date, so its archive row uses `receivedAt` and a stable external id of `self`.

Derivation attributes sleep to the morning it ends, excludes naps from the day roll-up while still storing them as `SleepRecord`s, converts kilojoules to kilocalories with the exact divisor 4.184, and never lets a nil field overwrite a stored value. What changed is that the columns other sources also write (resting HR, HRV, SpO2, respiratory rate, sleep minutes, exercise minutes, weight) now go through the arbiter instead of being assigned directly. The Whoop-only columns (recovery, strain, sleep performance) are still written directly, and `whoopOnlyColumnsAreWrittenDirectly` pins that distinction.

SYNC TRIGGERS

Revision D had three manual triggers. There is now an automatic one: `syncIfStale()` runs on launch and on every return to the foreground, gated by `SyncStalenessPolicy` (19.14). It is silent by design because the screens already show correct, if stale, local data; only a dead credential surfaces. The nightly server-side job and the webhook receiver remain unbuilt.

AUTO-PAIRING THE STRAP'S BROADCAST

A Whoop 4.0 can broadcast heart rate over Bluetooth. `WhoopAutoPair.choice(among:)` decides whether to connect without asking: only a device whose name contains "whoop", and only when there is exactly one. Another person's strap in a gym is not a reading this person owns. The live sensor that consumes it is in Chapter 8.

TESTS

`WhoopTests` (29), `WhoopWireFormatTests` (28, on payloads captured from a live v2 response, so a decode failure means the API moved), `WhoopPaginationTests` (16), `WhoopTokenTests`, `WhoopArbitrationTests` (5) and `WhoopArchiveTests` in `PersistenceTests`. Workout is the one collection with no captured fixture; its tests use a hand-built payload.

19.3 Fitbit

The second wearable, added 27 August. It reads nine range collections through an Edge Function and merges by rank. The interesting part is what it deliberately does not do: the phone never holds a Fitbit token.

WHY THE TOKENS STAY ON THE SERVER

Fitbit rotates its refresh token on every use and the old one stops working. Two devices sharing one credential would sign each other out on every sync. So exactly one writer may hold it, and that writer is a row in `fitbit_connections`. `FitbitAuthStoring` has no token methods at all; it holds only the pending PKCE attempts, in the Keychain, scoped to the signed-in account so a second account cannot complete the first one's authorisation.

```
// LifeOSKit/Sources/Integrations/FitbitAuthStore.swift
/// Deliberately has no token methods.
///
/// This is the whole difference from `WhoopTokenStoring`, and it is not an
/// oversight: Fitbit rotates its refresh token on every use, so exactly one
/// writer may hold it, and that writer is a row in the database.
public protocol FitbitAuthStoring: Sendable {
    func pendingAuths() -> [FitbitPendingAuth]
    func savePending(_ pending: FitbitPendingAuth) throws
    func clearPending()
}
```

OAuth

Fitbit permits a custom scheme for a native app, so the redirect is `lifeos://fitbit-callback` with no bridge function. `FitbitOAuth.session` builds the authorize URL with PKCE and sends the ten scopes as one space-separated parameter. Repeating the parameter makes Fitbit issue a narrower grant without saying so, and the first sign of it is a 403 much later. The scope list is pinned on both sides:

`FitbitOAuth.defaultScopes` in Swift and `FITBIT_SCOPES` in `_shared/fitbit.ts`, each with its own test, and the Deno one checks that no scope the app does not read is requested (no location, no social, no ECG).

`fitbit-token` exchanges the code with `FITBIT_CLIENT_SECRET`, upserts the pair on `user_id`, and returns only `{connected, fitbit_user_id, scopes}`. No token in the response is the whole point.

THE SYNC AND ITS LEASE

`FitbitSync.sync(days:token:)` posts `{days}` to `fitbit-sync` with the Supabase access token. The function reads the connection through `read_fitbit_connection` (a security-definer RPC, because the table has RLS enabled and zero policies), refreshes if the access token is within sixty seconds of expiry, and pulls the nine collections sequentially.

Refresh runs under a lease. `claim_fitbit_refresh` sets `refresh_lease_until` thirty seconds ahead only if no lease is live, and returns the row; a second sync on another device gets nothing back and the function answers `retry: true`, which the app reads as "come back shortly" rather than a fault.

`aRefreshLockDoesNotBecomeAnEmptySuccessfulSync` pins that an empty retry response throws `retryLater` instead of reporting zero days written.

```
// supabase/functions/fitbit-sync/index.ts
/// Stop while there is still headroom rather than at zero. A sync that spends
/// the user's last request leaves nothing for the retry, and Fitbit's quota
/// resets only at the top of the hour.
const QUOTA_FLOOR = 10;

// Sequential, not Promise.all: the quota headroom arrives in each response,
// and it can only stop the loop if the loop is still running.
for (const collection of COLLECTIONS) {
  if (stoppedForQuota) { failures[collection.key] = "rate_limited"; continue; }
  ...
  const remaining = quotaRemaining(response.headers);
  if (remaining !== null && remaining <= QUOTA_FLOOR) stoppedForQuota = true;
}
```

Fitbit allows 150 requests an hour. Range collections cost one request for up to 30, 100 or 365 days depending on the endpoint, which is why per-date collections (the daily activity summary, nutrition) are absent: they cost a request per day. Each collection's cap is declared in both `FitbitCollection.maximumRangeDays` and the function's `COLLECTIONS` table, and exceeding it is a 400 that names nothing useful.

WIRE FORMAT AND DERIVATION

`FitbitWireFormat.swift` declares every numeric field optional. Fitbit omits what a device did not record, and a non-optional field turns one missing number into a whole failed range. SpO2 arrives as a bare array rather than an envelope. Resting heart rate is under a key called `activities-heart`. Skin temperature is a nightly variation from baseline, not an absolute, and is stored as `wristTemperatureCelsius` without pretending to be a body temperature. VO2 max is a string that may be a range like "40-44"; the derivation takes the midpoint.

`FitbitDerivation` is split in two: `days(from:)` is pure and turns payloads into `[FitbitDay]`, and `derive` writes them through the arbiter with `.fitbit` as the source. Dates are parsed with a POSIX locale in the user's own time zone, because an ISO parser treats `yyyy-MM-dd` as UTC midnight and shifts every night by a day for anyone west of Greenwich. Fitbit's "light" sleep is filed as Apple's "core", the same stage under two vendors' names, so the sleep panel's total adds up.

The sync archives the raw response before deriving, under kinds prefixed `fitbit-`, using `WhoopArchive` because that type is already a generic raw-record store and only its name is Whoop-specific. The order of the error checks after the write is deliberate: a declined scope must not discard the six collections that arrived intact, so `partial(failed:)` is thrown after the days are written.

FAILURE HANDLING

CONDITION	SERVER ANSWER	APP BEHAVIOUR
No row for this user	404	<code>notConnected</code> ; the card offers Connect
Refresh refused (<code>invalid_grant</code>)	<code>needs_reauth: true</code> , row flagged	<code>reauthenticationRequired</code> ; the card says Reconnect and never retries
Another device holds the lease	<code>retry: true</code>	<code>retryLater</code> after writing nothing
Quota floor reached mid- loop	Remaining collections marked <code>rate_limited</code> , <code>retry: true</code>	Days that arrived are written, then <code>retryLater</code>
One collection 403s (declined scope)	That key in <code>failures</code>	<code>partial(failed:)</code> after writing the rest

TESTS

`FitbitOAuthTests` , `FitbitWireFormatTests` , `FitbitDerivationTests` (ten, including `lightSleepBecomesCore` , `avO2MaxRangeBecomesItsMidpoint` and `aDateIsReadInTheUsersOwnTimeZone`) and `FitbitSyncTests` (nine, over a stub `URLProtocol`). On the server, `fitbit_test.ts` has ten Deno tests over the pure helpers: failure classification, the quota header, the token form, the scope list and each collection's path.

19.4 The arbiter: who wins a column

Revision D's rule was "Whoop writes its columns, HealthKit fills the gaps", implemented as a boolean called `fillGapsOnly` . That worked with exactly two writers. A second strap broke it: `fillGapsOnly` could not tell a Whoop value from a typed one, and the question "should Fitbit replace this HRV" has a different answer depending on whether Whoop, the phone or the person wrote it. So provenance is now recorded per metric per day, and the merge is a rank.

```
// LifeOSKit/Sources/Integrations/MetricSource.swift
public enum MetricSource: String, Sendable, CaseIterable, Equatable {
    case manual, whoop, fitbit, appleHealth
}

public struct SourceRanking: Sendable, Equatable {
    public let primaryWearable: MetricSource // .whoop unless the user chose Fitbit

    /// 4 typed, 3 the authority, 2 a secondary reading, 1 a fallback, 0 unknown.
    public func rank(_ source: MetricSource?, for metric: HealthMetric) -> Int {
        guard let source else { return 0 }
        switch source {
        case .manual:
            return 4
        case .appleHealth:
            return metric.healthIsAuthoritative ? 3 : 1
        case .whoop, .fitbit:
            guard metric.claimedByWearable else { return 2 }
            return source == primaryWearable ? 3 : 2
        }
    }
}
```

The resolve function is eight lines and pure. A missing incoming reading never changes anything (a source having no reading is not evidence of zero). An empty column takes whatever arrives. Otherwise the incoming source must rank greater than *or equal to* the holder: equal rank means the same source syncing again, which must be allowed to correct itself, and that is what lets an afternoon Health sync raise the morning's step count.

The one subtle case is a value written before provenance existed. `MetricSource.legacy(for:)` reads an unattributed value as rank 0 for almost every metric, so the first sync after upgrade re-attributes it. Weight and water are the exception: an unattributed typed metric is read as `.manual`, because treating it as unknown would let the next Health sync silently replace a figure the person typed. That is the one path that destroys data a person entered by hand, and `anUnattributedTypedValueIsTreatedAsTyped` exists to keep it closed.

WHY THE VOCABULARY AND THE RULE LIVE APART FROM THE SOURCES

`MetricArbiter`, `MetricSource` and `HealthMetric` import nothing from `HealthKit`, `Whoop` or `SwiftData`. Every branch is exercised in `MetricArbiterTests` without a device, a prompt, a network or a store. Each sync (`HealthApply`, `WhoopDerivation.merged`, `FitbitDerivation.merged`) is a loop around this one call, and `WhoopArbitrationTests` and `HealthApplyTests` then check the wiring into real columns.

19.5 Plaid

Bank accounts and transactions, through four Edge Functions and a device-side cursor. Chapter 11 owns the feature; this section is the integration contract.

- **Secret on the server.** Every Plaid endpoint requires `client_id` and `secret` on the request, so the device never calls Plaid. `PlaidClient` calls `plaid-link-token`, `plaid-exchange`, `plaid-sync` and `plaid-disconnect`, all resolving the user from the bearer token.
- **Access tokens never reach the phone.** `plaid_items` holds them behind RLS with no policies, and `plaid-exchange` returns only `item_id` and the institution name.
- **The cursor lives on the device.** `PlaidItemStore` keeps a `/transactions/sync` cursor per item, serialised with an `NSLock`, and only advances it after the rows are stored. The function pulls at most three pages per call and returns `has_more`, so a years-long initial pull is several calls.
- **OAuth banks.** Institutions like Chase hand the browser to the bank, and the bank redirects to `https://almanac.shivvyas.com/plaid-oauth`, a universal link that iOS routes back into the app. Chapter 20 covers the two-file site behind it.
- **Failure kinds.** `classifyPlaidFailure` maps to three kinds: `item_login_required` (reconnect banner, not a spinner), `rate_limited` (retry later), `upstream_failure` (opaque, because Plaid's error bodies echo request parameters). Balance failures travel separately from transaction failures because `/transactions/sync` serves cached data and succeeds with an expired login, while `/accounts/balance/get` is the call that actually throws it.

19.6 EventKit and the calendar

The calendar is read and written through a protocol so that a second provider (Google, planned) slots in without the cache or the assistant changing. Chapter 6 covers the screens and Chapter 15 the tools; this section covers the source.

```
// LifeOSKit/Sources/Integrations/CalendarSource.swift
public protocol CalendarSource: Sendable {
    var source: CalendarEventSource { get }
    var isAuthorized: Bool { get async }
    func requestAccess() async throws -> Bool
    func events(from: Date, to: Date) async throws -> [CalendarEventSnapshot]
    func create(_ draft: CalendarEventDraft) async throws -> CalendarEventSnapshot
    func update(sourceID: String, with draft: CalendarEventDraft) async throws ->
    CalendarEventSnapshot
    func delete(sourceID: String) async throws
}
```

`EventKitSource` is an actor because `EKEventStore` is not `Sendable`. Three details matter:

- **Permission is per account, not per phone.** iOS grants calendar access to the app. `AccountDeviceAccess.allowsCalendar` requires both the system grant and a flag in the account's own defaults suite, so a second person signing in does not inherit the first person's calendar. The source also captures the account id at construction and refuses to answer if the current account has changed since.
- **Recurring occurrences get a qualified id.** Every occurrence of a series shares one `eventIdentifier`, so the natural key is qualified with the occurrence start (`id#timestamp`). A qualified id never resolves through `event(withIdentifier:)`, which makes a write against an occurrence fail with `unknownEvent` instead of silently editing the wrong one; the tool layer refuses recurring writes with a real explanation before that can happen.

- **The prompt is never at launch.** `requestAccess()` is called from the agenda card's empty state, when the person has just asked for the thing that needs it.

`CalendarSync` pulls every authorised source over a window of 30 days back and 90 forward, merges with `CalendarMerge` (EventKit wins a collision on lowercased title plus start date, which is what stops every event appearing twice when a Google account is also added under iOS Settings), and hands the result to `CalendarStore.apply`. A source that throws is skipped: partial data beats an empty screen, and sync is ambient, never a user-visible request. Writes route to the source that owns the event and then re-sync, so the cache reflects what the provider actually stored rather than the local draft.

`CalendarSyncTests`, `CalendarMergeTests` and `EventKitSourceIDTests` cover the merge, the prune-only-what-you-fetched rule and the occurrence key.

19.7 Supabase auth and REST

Still no SDK. `SupabaseAuth` makes five auth calls and `SupabaseREST` makes an upsert, a windowed fetch and a narrow delete. The anon key goes in the `apikey` header; RLS is what protects rows.

TWO CHANNELS, TWO PATHS

STEP	EMAIL	PHONE
Send a code	<code>POST auth/v1/otp</code> with <code>create_user: true</code>	<code>POST functions/v1/otp-start</code> (Twilio Verify)
Verify	<code>POST auth/v1/verify</code> , <code>type: email</code>	<code>POST functions/v1/otp-check</code> , which mints a GoTrue session
Availability	Read from <code>auth/v1/settings</code>	Always offered; delivery is the function's job

Both channels are six-digit codes. The email template puts `{{ .Token }}` on the page instead of the stock magic link, which cannot be typed into an app. Phone goes through Twilio Verify rather than GoTrue's own SMS provider because the function can classify Twilio's failure codes into sentences a person can act on, and rate-limit by phone, prefix and IP (Chapter 20).

`AuthError.fromHTTP` is the one place server codes become copy. A `{"error": "sms_opted_out"}` body becomes "That number opted out of our texts. Text START to our number to opt back in". A bare 502 with no message is channel-neutral, because email is now the default and the old copy assumed SMS.

`AuthErrorMappingTests` pins every mapping.

THE REFRESH URL BUG

```
// LifeOSKit/Sources/Integrations/SupabaseAuth.swift
/// Deliberately not `appendPathComponent` on the whole path. That
/// treats what it is given as a single component and percent-encodes the
/// `?` into `%3F`, which sent the one call that carries a query to
/// POST /auth/v1/token%3Fgrant_type=refresh_token. GoTrue answers 404 to
/// that, so a stored session could never be renewed.
private func authURL(_ path: String) throws -> URL {
    let parts = path.split(separator: "?", maxSplits: 1, omittingEmptySubsequences: false)
    var components = URLComponents(
        url: baseURL.appendPathComponent("auth/v1/\(parts[0])"),
        resolvingAgainstBaseURL: false
    )
    if parts.count > 1 { components?.query = String(parts[1]) }
    guard let url = components?.url else { throw AuthError.transport }
    return url
}
```

This is worth reading because of how it presented: sign-in always worked, and an hour later every authenticated call began failing 401. `AuthRefreshURLTests` now pins the query string.

SESSION LIFECYCLE: THREE PIECES

- `SessionRefresher.restore()` runs on launch and on every foreground. It returns `signedOut`, `active` or `rejected`, and only a definitive refusal (400, 401, 403, 422) clears the store. Offline, a 5xx and a 429 keep the session. A refreshed session carries forward the user id, phone, email and refresh token from the one it replaces, because refresh responses may be sparse. Two guards protect against races: a sign-out during the refresh cannot be resurrected (the store's refresh token is compared before saving), and a refresh cannot change the account's identity.
- `SessionKeepAlive` answers the case the refresher cannot: a person who keeps the app open for over an hour. It schedules a renewal 45 seconds before expiry, inside `isExpired`'s own minute of slack so the wake finds a session the refresher will actually renew, with a five-second floor so a clock that jumps backwards cannot become a spin against the auth endpoint.
- `AccountStore` holds the roster of accounts that have signed in on this device (ids and labels in defaults) and one Keychain session per user id. Signing in as a second person no longer overwrites the first. `currentScope` returns nil rather than the first account it can find, so a current id naming a signed-out account never opens somebody else's store. The pre-accounts session slot is adopted once and cleared on sign out, because a credential that can restore an account is that account's session whatever slot it sits in. Chapter 4 covers what the shell does with this.

`KeychainAuthSessionStore()` defaults its account to whichever id is current. This matters more than it looks: every network client in the app builds one to get a token, and before accounts there was one slot, so after a switch they would all have kept presenting the previous person's token.

REST

`SupabaseREST.upsert` sends `Prefer: resolution=merge-duplicates` so a push that lost its response and retried does not create a second copy of every row. `fetch(table:since:)` orders by `updated_at`

ascending and uses `gte` on the cursor, inclusive, because a row written in the same millisecond as the previous page's last row must not be skipped; re-reading one row is free, missing one is silent data loss. Both take and return bytes rather than dictionaries because a `[String: Any]` is not `Sendable` and the call crosses an isolation boundary. `delete` is deliberately narrow (one column, one value): the one thing the app deletes over REST is its own push registration.

`SessionRefreshTests` (twelve, serialised, over a stub `URLProtocol`), `SessionKeepAliveTests`, `AccountStoreTests` (seventeen) and `AuthSessionProfileTests` pin the above.

19.8 Apple Foundation Models

The on-device tier. Revision D's correction still stands: on iOS 26 there is no public `LanguageModel` protocol, so the app keeps two thin protocols of its own, `Engine` for one-shot tasks and `ChatEngine` for conversations, and the remote engines conform to the same ones. When iOS 27 ships Apple's seam, the remote engines conform to it and the protocols are deleted without a caller changing.

```
// LifeOSKit/Sources/Insights/ModelAvailability.swift
public enum ModelAvailability: Sendable, Equatable {
    case available
    case unavailablePermanently
    case unavailableForNow

    public static func from(_ availability: SystemLanguageModel.Availability) ->
    ModelAvailability {
        switch availability {
        case .available: return .available
        case .unavailable(let reason):
            switch reason {
            case .deviceNotEligible: return .unavailablePermanently
            case .appleIntelligenceNotEnabled, .modelNotReady: return .unavailableForNow
            @unknown default: return .unavailableForNow
            }
        }
    }
}
```

Apple's availability is collapsed to the one distinction the router acts on: can this answer still change. Hardware eligibility never changes while the process is alive, so it is the only answer the router may remember. Everything else is re-read every time, and an unknown reason is assumed transient because re-checking costs a property read while wrongly caching "never" would disable the free tier for the life of the process.

`OnDeviceEngine` creates a `LanguageModelSession` per call. `LanguageModelSession` is a non-`Sendable` final class and `Engine` is `Sendable`, and a one-shot task has no conversation to remember.

`OnDeviceChatEngine` also creates a session per turn, because the conversation's memory lives in the thread the app renders, not in the session; it flattens the thread into one prompt, binds each `CoachTool` as a `SessionTool`, and streams snapshots through `onPartial`, cleaning markdown on the way past so the screen never shows a raw asterisk for a second. Chapter 14 covers the router that chooses between this and the cloud.

The floor is iOS 26 because `FoundationModels` does not exist before it. There is no older-OS cohort that always escalates; unavailability is only the runtime reasons above.

19.9 ElevenLabs: hearing and speaking

LIFO's voice loop uses ElevenLabs twice, both from the app target with the key from `Secrets.xcconfig` under `ELEVENLABS_API_KEY`. This is the one vendor called from the device with a real key. The key is passed into each call, never stored on a type, and never logged. It is a personal key for a single-user build; the cost chapter (Chapter 24) records what it spends.

- **Scribe (speech to text).** `ElevenLabsSpeechClient.transcribe` posts a WAV clip as multipart form data to `v1/speech-to-text` with `model_id: scribe_v2`. A clip under 1,024 bytes is rejected before upload as too short to hear.
- **Text to speech.** `ElevenLabsVoiceClient.speech(for:voice:apiKey:)` posts to `v1/text-to-speech/{voiceID}` with `eleven_flash_v2_5`, the low-latency model, because a coach that speaks three seconds after the text has been read is repeating, not answering. Speech is billed per character, so the text is capped at 1,200 characters (roughly ninety seconds) and truncated on a sentence boundary, because a voice stopping mid-number sounds like a fault.
- **Three voices.** `AssistantVoice` names Rachel, Adam and Sarah by ElevenLabs' public stock voice ids, which are stable across accounts and need no provisioning. Three, deliberately: a list of dozens is a settings screen nobody finishes. Spoken replies are off until asked for, because a coach that starts talking in a room the phone knows nothing about is a worse default than silence.

`VoicePlayer` holds the `AVAudioPlayer` because it stops the moment it is deallocated, sets the session to `.playback` with `.spokenAudio` and `.duckOthers`, meters the level at 30 Hz for the orb in Chapter 14, and hands the session back with `notifyOthersOnDeactivation` so a podcast returns to full volume. A delayed completion from an old player is checked by identity so it cannot stop a newer reply.

19.10 Apple Speech and the microphone

`SpeechListener` in `Features/Coach/Model` is the microphone. One tap opens it and no second tap is needed: it watches the level, decides when the sentence is over, and transcribes what it recorded. It is `nonisolated` because the `AVAudioEngine` tap runs on the I/O thread, and every `AVAudioSession` activation runs on a private serial queue because `setActive` on the main thread can stall the UI.

The layering is: ElevenLabs Scribe is primary, Apple Speech is the backup when there is no key, the upload fails, or the clip is too short. But Apple's recogniser runs alongside Scribe even when Scribe will do the final transcript, for one reason: Scribe transcribes a clip that is only uploaded once the turn is over, so it cannot put a word on screen while someone is still talking. Apple's partials do exactly that and nothing else.

The decision "is the sentence over" lives in `Insights` as `SpeechEndpointer`, a pure state machine fed one RMS reading per audio buffer. Its clock is the audio itself, buffer frames over the sample rate, rather than the wall clock: a thread busy elsewhere would otherwise put time into a silence the speaker never left.

`SpeechCapturePlan` decides, from whether a Scribe key exists and whether Apple's recogniser is authorised and available, whether the app can listen at all. Both are tested in `SpeechEndpointerTests` and `SpeechCapturePlanTests` against a made-up room. `onEndOfSpeech` fires the moment the endpointer

decides, before any transcript exists, because the second or two spent uploading otherwise looks like the mic ignoring you.

19.11 YouTube embeds

The workout library (Chapter 10) plays curated YouTube videos. Two pieces touch the vendor.

`WorkoutCatalogClient` is a read-only PostgREST call against `workout_videos`, ordered by `updated_at`, with the user's bearer token. The table grants `select` to authenticated users and nothing else. The catalog itself is generated by `scripts/catalog/verify.ts` from `catalog.json`, which checks each id against YouTube's oEmbed endpoint before it is allowed into the seed migration.

`YouTubePlayerView` wraps a `WKWebView` owned by the screen, not the view, so rotating the phone or entering full screen does not give SwiftUI a new view identity and restart the video. There is no JavaScript bridge: the person taps YouTube's own play button and the recorder times the session. The load itself had to be learned by elimination:

```
// LIfiOS/Features/Workouts/View/YouTubePlayerView.swift
// Loading the embed URL straight into the web view answers with
// "Video player configuration error, Error 153": the embed needs a
// page with an origin to sit in. So the iframe is wrapped in a
// page of our own, based at the host the embed's `origin` names,
// which is what youtube-ios-player-helper does too.
view.loadHTMLString(Self.page(embedding: url), baseURL: Self.base)
```

The QA report records the numbers: zero of six seed videos rendered on the first pass (error 152), and six of six after the page's `baseURL` was matched to the embed's `origin` parameter and both were pointed at `youtube-nocookie.com`. A video whose owner has barred embedding shows YouTube's own "Watch on YouTube" card, which no delegate can see without the bridge this slice leaves out; the "Open in YouTube" link under the player is the way out.

19.12 APNs and the morning nudge

The proactive half of LIFO sends one notification at eight in the morning local time, when a rule finds something worth saying. The integration has a device half and a server half.

Device. `PushService` asks once for notification permission and registers with APNs.

`PushTokenClient.register` upserts `{token, user_id, platform, timezone}` into `device_tokens`, where the token is the primary key. That key is a correctness requirement: several accounts can share one device, and a token left registered under the account that signed out would push one person's sleep data onto a phone showing somebody else's name. Registering under a second account replaces the row in one upsert, and sign-out deletes it. The time zone rides along because the send hour is local, and a row with no usable zone never sends.

`NudgePayload(userInfo:)` reads the tapped notification. It carries the sentence, the trigger name and the day, and deliberately not the numbers: a phone that has been off for days would otherwise open on figures two days stale, so the tap-through re-evaluates locally.

Server. `_shared/apns.ts` signs one ES256 provider token from the `.p8` in the function environment and reuses it for fifty minutes, because Apple throttles senders that mint a token per push. Every send carries `apns-collapse-id: lifo-nudge` so a phone that has been off for three days gets today's nudge and not three, and `apns-expiration` twelve hours out so a morning observation does not arrive after midnight. A 410, or a 400 with `BadDeviceToken` (the one that actually appears after a reinstall), deletes the row so the job does not push at a dead token forever. The reason is logged; the token never is, because a device token in a log line is a way to push at somebody from outside the function. The triggers, cooldown and cron are in Chapter 20.

19.13 Screen Time: still cut

Unchanged from Revision D and recorded so it is not re-attempted. Apple's `DeviceActivity` API lets a report extension render usage figures and lets nothing read them: not the host app, not `SwiftData`, not the coach. Since the product thesis is the join across domains, a signal that provably cannot enter the join has no home here. The MacBook has no API at any privilege level.

19.14 What the integrations share

THE STALENESS RULE

```
// LifeOSKit/Sources/Integrations/SyncStalenessPolicy.swift
public static func shouldSync(lastSync: Date?, now: Date = .now,
                              calendar: Calendar = .current,
                              staleAfter: TimeInterval = 3_600) -> Bool {
    guard let lastSync else { return true }
    guard lastSync <= now else { return true }           // clock moved backwards
    if lastSync < calendar.startOfDay(for: now) { return true }
    return now.timeIntervalSince(lastSync) > staleAfter
}
```

Two clocks, not one. Staleness handles the common case: data older than an hour is worth refreshing on foreground. The day boundary handles the one that matters most: the first open of the morning must pull the night's sleep, and a sync from 23:50 is only minutes old while still knowing nothing about the night. Whoop, Fitbit and Health all consult it, which is why the three connection cards refresh themselves at the same moments.

PER-ACCOUNT CREDENTIALS

Every Keychain store (`KeychainAuthSessionStore` , `KeychainWhoopTokenStore` , `KeychainFitbitAuthStore`) defaults its account attribute to the id under `accounts.current` . Every per-account preference (sync cursors, "seen the tour", calendar consent) lives in `UserDefaults.currentAccount` , a suite named for the account. Device preferences (appearance, the assistant's voice, which engine answers) stay in `standard` on purpose: they belong to the phone, not to whoever is signed in.

THE SECRET RULE

Whoop, Fitbit, Plaid, Twilio and OpenAI each require a secret, and each has an Edge Function that holds it. The device holds public identifiers (client ids, the anon key), one personal ElevenLabs key, and its own tokens. 23.2 tabulates the tiers.

19.15 Study notes

READ IN THIS ORDER

1. `LifeOSKit/Sources/Integrations/HealthMetric.swift`, the vocabulary and the three merge properties.
2. `MetricSource.swift` and `MetricArbiter.swift`, the rank and the eight-line rule.
3. `HealthApply.swift`, then `HealthKitReader.swift` from `day(_:)` downward.
4. `WhoopCollection.swift`, `WhoopClient.swift` (the DTOs and their `sample` properties), `WhoopSync.swift`, `WhoopDerivation.swift`.
5. `FitbitAuthStore.swift` for what is absent, then `supabase/functions/fitbit-sync/index.ts`.
6. `SupabaseAuth.swift`, `SessionRefresh.swift`, `AccountStore.swift`.
7. `SyncStalenessPolicy.swift`, then find its three callers with a `grep`.

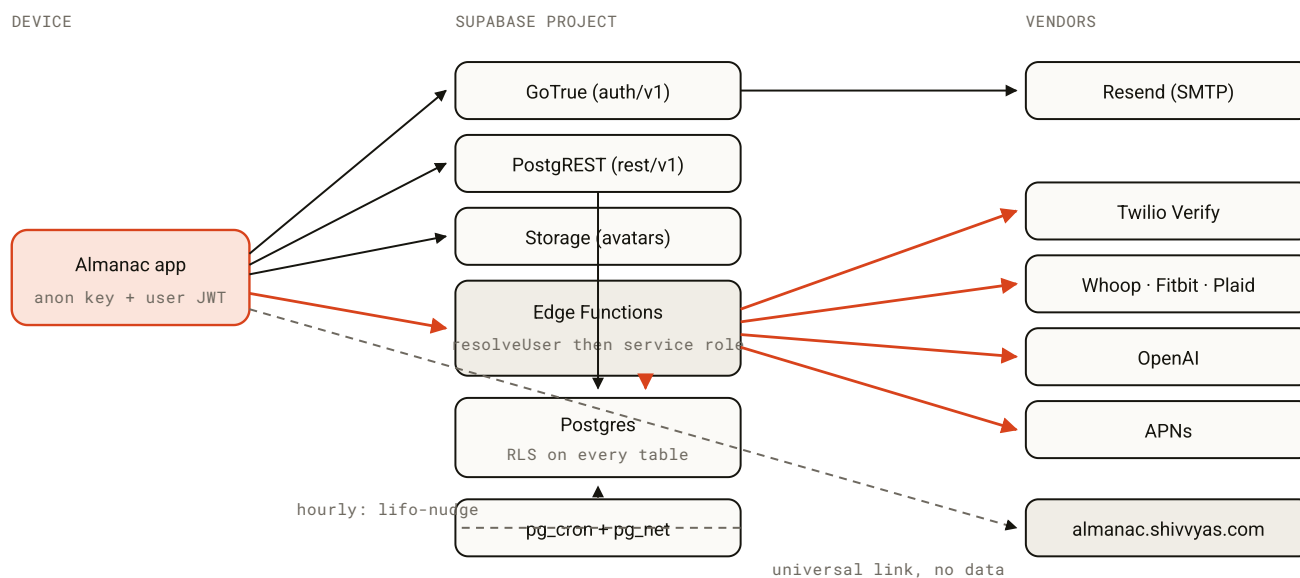
CHECK YOURSELF

- **Why does a Fitbit HRV reading not overwrite a Whoop one, while a Health step count does overwrite yesterday's Health step count?** HRV is `claimedByWearable`, so the secondary strap ranks 2 against the primary's 3. Steps are `healthIsAuthoritative`, so Health ranks 3 against its own 3, and equal rank is allowed to correct itself.
- **What would happen if you deleted `MetricSource.legacy(for:)`?** An unattributed typed weight would rank 0 and the next Health sync would replace it. That is the one path that destroys hand-entered data.
- **Why does Whoop keep a token in the Keychain but Fitbit does not?** Fitbit's refresh token is single use and rotates, so only one writer may hold it; the server row is that writer. Whoop's is held both places, with a server-side lock around refresh.
- **Which HTTP statuses sign a user out, and why not 429?** 400, 401, 403 and 422 are the server refusing the credential. 429 and 5xx are temporary by definition, and opening the app on a plane must not log you out.
- **Why is HealthKit's percent scaled by 100 and what bug did it fix?** `HKUnit.percent()` yields a 0 to 1 fraction; the app stores percentages as percentages, and Health SpO2 was rendering as 1.0.
- **Why does the Whoop bridge send `lifeos://` and not `almanac://`?** Every installed build knows `lifeos`; pointing at `almanac` stranded builds that predate the rename.

20 The backend

One Supabase project: seventeen migrations, thirteen Edge Functions, two email templates, a cron job, and a two-file static site. Everything in this chapter exists for one of three reasons: a secret that cannot ship in an `.ipa`, data that must cross between people, or work that a sleeping phone cannot do.

Revision D described a schema of five tables and two deployed functions. The backend has grown since, but its rules have not: every table has Row Level Security enabled without exception, every policy scopes to `auth.uid()`, every metric column is nullable, and the service-role key exists only in function environments. This chapter walks the configuration, the migrations in order, the functions one by one, the tests, and the operational shape the cron job forces. Every RLS claim below is quoted from the SQL on disk.



The app never holds a secret and never calls a vendor that needs one. Every highlighted arrow passes through a function that first resolves the caller from their own JWT and only then uses the service role, scoped to that id.

20.1 Configuration

`supabase/config.toml` is pushed to the hosted project with `supabase config push`. The parts that decide behaviour:

BLOCK	SETTING	WHY
[auth]	jwt_expiry = 3600 , enable_refresh_token_rotation = true , refresh_token_reuse_interval = 10	An hour-long access token is why SessionKeepAlive exists. Rotation is why concurrent refreshes must be coalesced (19.7).
[auth]	additional_redirect_urls lists almanac://auth-callback and lifeos://auth-callback	Both schemes the app claims.
[auth.rate_limit]	sms_sent = 8 , token_verifications = 8 , email_sent = 30	GoTrue's own limits, in front of the function's abuse guard.
[auth.email]	otp_length = 6 , otp_expiry = 3600 , max_frequency = "1m0s"	The six-box code field in the app.
[auth.email.smtp]	host = "smtp.resend.com" , pass = "env(RESEND_API_KEY)" , admin_email = "env(RESEND_SENDER_EMAIL)" , sender_name = "Almanac"	Custom SMTP is required for a custom template. admin_email is required by the CLI or the push fails.
[auth.email.template.magic_link]	subject = "Your Almanac code" , content_path = ./supabase/templates/magic_link.html	The template puts the token on the page, not a link.
[auth.sms.test_otp]	11234567890 = "123123"	The test number, mirrored as TEST_PHONE in the function.
[auth.external.apple]	enabled = true , client_id = "com.shivvyas.lifeos"	Kept enabled so a config push does not switch Sign in with Apple off.
[functions.whoop-callback] , [functions.whoop-token]	verify_jwt = false	The dashboard resets verify_jwt to true on every deploy. The callback is a browser redirect with no JWT; the token function resolves the user itself.
[edge_runtime]	deno_version = 2	The functions use jsr : imports.

Secrets are set with `supabase secrets set` and never appear in the file: the `env()` references resolve at push time. `supabase/functions/.env` holds local-only values for `supabase functions serve` and is

gitignored along with `.branches`, `.temp` and any `.env.*.local`.

20.2 The migrations, in order

Migrations are the schema's history. Reading them in order is reading the product's history. Each entry below names what the migration creates and quotes the RLS shape it leaves behind.

20260811 · INITIAL_SCHEMA

Five tables mirroring the SwiftData models: `daily_metrics` (unique on `(user_id, date)`, `date` typed as `date not timestamptz` so a device in another zone cannot write to the adjacent day), `workout_records`, `sleep_records` (with a check that `ended_at > started_at`), `whoop_raw` and `sync_state`. A `touch_updated_at()` trigger sets `updated_at = now()` on every update so a client can never advance it past the server's clock and win a conflict it should have lost. Every table gets four policies of the same shape:

```
-- supabase/migrations/20260811164841_initial_schema.sql
create policy "own rows readable" on public.daily_metrics
  for select to authenticated using ((select auth.uid()) = user_id);
create policy "own rows insertable" on public.daily_metrics
  for insert to authenticated with check ((select auth.uid()) = user_id);
create policy "own rows updatable" on public.daily_metrics
  for update to authenticated using ((select auth.uid()) = user_id)
  with check ((select auth.uid()) = user_id);
create policy "own rows deletable" on public.daily_metrics
  for delete to authenticated using ((select auth.uid()) = user_id);
```

`((select auth.uid()))` rather than bare `auth.uid()` is a Postgres planner idiom: the subselect is evaluated once per statement instead of once per row.

20260825 · OTP_ABUSE_GUARD

`otp_events` records every SMS start, failed check, successful check and block, keyed by a peppered SHA-256 of the phone, a "pumping prefix" (country code plus the first digits), and a hash of the IP. RLS is enabled with no policies, so no client can read or write it; only the service role reaches it.

`otp_user_id_for_phone(p_phone)` is a `security definer` function that looks a phone up in `auth.users`, with execute revoked from `anon` and `authenticated` and granted to `service_role` only.

20260826 · PLAID_ITEMS

`(user_id, item_id)` primary key, the Plaid `access_token` in plaintext, and a unique constraint on `(user_id, institution_id)` so the same bank cannot be connected twice. `alter table public.plaid_items enable row level security;` with no policies: the credential is readable only by the service role.

20260827 · LIFO_USAGE

The token ledger: `(user_id, day)` with a `tokens bigint`. RLS on, no policies. `lifo_debit(p_user, p_tokens)` is an atomic upsert-and-add returning the new total, granted to `service_role` only. This is the

database constraint behind the cost ceiling in Chapter 24.

20260827 · FRIENDS_AND_MESSAGES

Three tables. `profiles` is the one table any signed-in user may read (`for select to authenticated using (true)`) because people search for each other by name; inserts and updates are own-row only. `friendships` encodes the state machine in policy:

```
-- supabase/migrations/20260827130000_friends_and_messages.sql
create policy "ask for a friendship"
  on public.friendships for insert to authenticated
  with check (requester = auth.uid() and status = 'pending');
create policy "addressee answers"
  on public.friendships for update to authenticated
  using (addressee = auth.uid())
  with check (addressee = auth.uid() and status = 'accepted');
create policy "either side may end it"
  on public.friendships for delete to authenticated
  using (auth.uid() in (requester, addressee));
```

A request can only be created pending, only the addressee can accept, and only accepted is a valid update target. `messages` lets participants read their conversation and lets a sender insert only when an accepted friendship exists in either direction.

20260827 · FRIENDSHIP_COLUMN_GRANTS

Two lines: revoke `update` on `friendships` from `authenticated`, then `grant update (status)`. The update policy already constrains rows; this constrains columns, so the addressee cannot rewrite who the requester was.

20260828 · NOTES

`note_folders` and `note_documents`, with client-generated UUID primary keys (the device creates pages offline), a `blocks jsonb` column, a `deleted_at` tombstone so deletions sync, and partial indexes for archived pages and journal entries. One policy each, `for all using (auth.uid() = user_id) with check (auth.uid() = user_id)`. The sync that drives this is in Chapter 12.

20260828 · PROFILES_AND_AVATARS, THEN AVATARS_SIGNED_IN_ONLY

The first adds the onboarding fields to `profiles` and creates a public `avatars` bucket. The second, the same day, reverses the public part: the bucket becomes private, accepts only `image/jpeg`, and reads require a signed-in user. The write policies pin the object name to exactly `{uid}/avatar.jpg`:

```
-- supabase/migrations/20260828150000_avatars_signed_in_only.sql
create policy "own avatar insert"
  on storage.objects for insert to authenticated
  with check (
    bucket_id = 'avatars'
    and (storage.foldername(name))[1] = (select auth.uid())::text
    and name = (select auth.uid())::text || '/avatar.jpg'
  );
```

One file per person, in their own folder, no other name possible. The app reads avatars with a signed URL and its own token (Chapter 17).

20260829 · FITBIT_CONNECTIONS, THEN FITBIT_LOCK

A user-keyed row holding both Fitbit tokens, the expiry, the granted scopes and a `needs_reauth` flag. RLS enabled, deliberately no policies, so the row's own owner cannot read it from a client. The second migration adds `refresh_lease_until` and two `security definer` RPCs: `claim_fitbit_refresh`, which sets a lease only if none is live and returns the row, and `read_fitbit_connection`. Both have `execute revoked from public, anon and authenticated`. The migration comment explains why: Fitbit's refresh token is single use, so two devices sharing it would sign each other out, and a single server row with a lock is the only arrangement where exactly one writer rotates it.

20260829 · SHARED_STATS

`profile_stats` holds three figures a person may choose to share (streak, days tracked, workouts) behind a `shares` flag. The select policy is the first one that crosses users:

```
-- supabase/migrations/20260829120000_shared_stats.sql
create policy "own stats and accepted friends' stats"
  on public.profile_stats for select to authenticated
  using (
    user_id = auth.uid()
    or exists (
      select 1 from public.friendships f
      where f.status = 'accepted'
      and ((f.requester = auth.uid() and f.addressee = profile_stats.user_id)
        or (f.addressee = auth.uid() and f.requester = profile_stats.user_id))
    )
  );
```

Every statement is `drop policy if exists then create`, which is what made the migration re-runnable after it first failed halfway.

20260830 · PROACTIVE_NUDGES

`device_tokens` with the APNs token as primary key and the user's time zone, four own-row policies. `nudge_log` keyed `(user_id, day)` with the trigger name and never the sentence, RLS on and no policies. And a change to the ledger: `lifo_usage` gains a `kind` column, its primary key becomes `(user_id, day,`

kind), and `lifo_debit` is dropped and recreated with a third parameter. Chat and nudges do not share a budget, because a heavy chat evening must not eat the next morning's nudge.

THE DEPLOY ORDER THIS MIGRATION FORCES

The old `lifo_debit(uuid, bigint)` is dropped. A deployed `lifo-agent` that still calls it with two arguments fails every debit after the migration runs, and a new `lifo-agent` that calls it with three fails before. So the order is: push the migration, then deploy `lifo-agent` and `lifo-nudge` together, in that window. The plan file for slice 2 records that the migration was applied on 29 August and the functions deployed after it.

20260913 · WHOOP_ACCOUNT_CONNECTIONS

`whoop_connections`, user-keyed, holding the pair and a `refresh_lock uuid` with `refresh_locked_until`. RLS on, then `revoke all ... from anon, authenticated` and `grant all ... to service_role`. `claim_whoop_refresh(p_user_id, p_lock)` takes a thirty-second lock the same way Fitbit's does, keyed by a caller-supplied UUID so the release can be conditioned on still holding it.

20260914 · SOCIAL_GROUPS

The largest migration. Four tables (`social_groups`, `group_members` with an `invited | accepted` status and a `shares_activity` flag, `group_messages` with a `(sender, client_id)` unique key for idempotent sends, `group_activity`) and nine `security definer` functions. The shape is different from the friends tables: clients get `select` only, and every write goes through a function that enforces the rule in code.

```
-- supabase/migrations/20260914090000_social_groups.sql
revoke all on public.social_groups, public.group_members, public.group_messages,
  public.group_activity from anon, authenticated;
grant select on public.social_groups, public.group_members, public.group_messages,
  public.group_activity to authenticated;

create policy "members and invitees see group" on public.social_groups for select to
authenticated
  using (exists(select 1 from public.group_members m where m.group_id=id and
m.user_id=auth.uid()));
create policy "accepted members read chat" on public.group_messages for select to
authenticated
  using(public.is_group_member(group_id));
```

`is_group_member` is itself a definer function so a policy on `group_members` can consult `group_members` without recursion. The functions carry the product rules: only accepted friends can be invited, a group holds at most 50 people, an owner who leaves hands the group to the earliest-joined member or deletes it when alone, and `send_group_message` returns the existing id when a retried `client_id` matches the same body and raises when it matches different text. `group_leaderboard` accepts exactly three metric names and raises on anything else.

20260914 · GROUP_WELLNESS_LEADERBOARDS

A second, separate consent. `shares_wellness` on `group_members`, and `group_wellness_daily` holding three 0 to 100 scores per day, each paired with its source and checked so a score cannot exist without one: `check ((effort is null) = (effort_source is null))`. Own-row select only; `publish_group_wellness` accepts at most two days (today and yesterday), rejects a day outside a narrow range, rejects an `observed_at` more than five minutes in the future or four days in the past, and only overwrites a row with a newer observation. Health scores are never readable as rows by anyone but their owner; other members see only the ranked result of `group_wellness_leaderboard`.

20260915 · WORKOUT_VIDEOS AND ITS SEED

`workout_videos` keyed by `youtube_id` with a `split`, arrays for `goal`, `muscles` and `equipment`, an `intensity` from 1 to 3 and a `verified_at`. Grants are revoked from both roles and `select` is granted back to `authenticated` with a policy of `using (true)`: a catalog, readable by anyone signed in, writable by nobody but the migration. The seed is generated by `scripts/catalog/verify.ts`, carries 122 `insert ... on conflict do update` rows, and ends with a `delete where youtube_id not in (...)` so the seed is a full sync rather than an append.

THE RLS SHAPES, SUMMARISED

SHAPE	TABLES	WHO REACHES THE ROWS
Own rows, four policies	<code>daily_metrics</code> , <code>workout_records</code> , <code>sleep_records</code> , <code>whoop_raw</code> , <code>sync_state</code> , <code>device_tokens</code>	The owner, from the app
Own rows, one <code>for all</code> policy	<code>note_folders</code> , <code>note_documents</code>	The owner, from the app
RLS on, no policies	<code>otp_events</code> , <code>plaid_items</code> , <code>lifo_usage</code> , <code>fitbit_connections</code> , <code>nudge_log</code> , <code>whoop_connections</code>	Service role only, from a function
Readable by any signed-in user	<code>profiles</code> , <code>workout_videos</code>	Search and catalog
Cross-user by relationship	<code>friendships</code> , <code>messages</code> , <code>profile_stats</code>	Both participants, or accepted friends
Select only, writes via definer functions	<code>social_groups</code> , <code>group_members</code> , <code>group_messages</code> , <code>group_activity</code> , <code>group_wellness_daily</code>	Members, with every rule in code

AN RLS MISTAKE DOES NOT THROW

It returns an empty result set, which looks exactly like "no data yet". This is why `SupabaseREST` always sends the user's token and never the anon key alone, and why the two pgTAP files in 20.5 exist.

20.3 The shared door

`_shared/supabase.ts` is forty lines and the most important file in the folder. Every function that touches a user's row calls `resolveUser(req)` first:

```
// supabase/functions/_shared/supabase.ts
/// Resolves the caller from their own token.
///
/// This is a security boundary, not a formality. Every one of these functions
/// reads a bank credential keyed by user, so a service-role client that
/// trusted a user_id from the request body would let any signed-in user pull
/// anyone else's transactions.
export async function resolveUser(req: Request): Promise<string | null> {
  const authorization = req.headers.get("Authorization");
  if (!authorization) return null;
  const client = createClient(SUPABASE_URL, SUPABASE_ANON_KEY,
    { global: { headers: { Authorization: authorization } } });
  const { data, error } = await client.auth.getUser();
  if (error || !data.user) return null;
  return data.user.id;
}
```

The user id comes from GoTrue validating the bearer token, never from the body. Only after that does `serviceClient()` build the privileged client, and every query it runs is filtered by that id. `_shared/plaid.ts` re-exports these so the Plaid functions have one import, and its comment states the rule for the folder: a module named for one integration must not be the front door for another.

20.4 The Edge Functions

Thirteen functions, each a `Deno.serve` that owns the door (method, auth, body parsing, the fetch) and delegates every decision to a pure helper in `_shared/` that has a test beside it. The table gives the contract; the subsections give the parts worth studying.

FUNCTION	AUTH	SECRETS	DOES
otp-start	anon key	TWILIO_*, OTP_PEPPER	Rate-limits, then starts a Twilio Verify SMS
otp-check	anon key	TWILIO_*, service role	Checks the code, mints a GoTrue session
whoop-callback	none (verify_jwt = false)	none	https to lifeos:// redirect; renders the code for the manual flow
whoop-token	user JWT via resolveUser	WHOOP_CLIENT_SECRET	Exchange, refresh under lock, restore, disconnect
plaid-link-token	user JWT	PLAID_CLIENT_ID, PLAID_SECRET	Mints a Link token keyed to the user
plaid-exchange	user JWT	PLAID_*	Public token to Item; refuses duplicates; cleans up on storage failure
plaid-sync	user JWT	PLAID_*	Up to three pages per Item; balances separately
plaid-disconnect	user JWT	PLAID_*	Removes the Item at Plaid before the row
fitbit-token	user JWT	FITBIT_CLIENT_SECRET	Exchange; stores the pair; returns no token
fitbit-sync	user JWT	FITBIT_CLIENT_SECRET	Refresh under lease; nine collections under a quota floor
lifo-agent	user JWT	OPENAI_API_KEY	Budget check, guardrails, chat or one-shot task, streaming, debit
lifo-nudge	service role bearer (cron only)	OPENAI_API_KEY, APNS_*	Hourly trigger evaluation and push

OTP: THE ABUSE GUARD

SMS costs money per message and "SMS pumping" is a known fraud where a bot requests thousands of codes to premium-rate ranges. `allowStart` in `otp_guard.ts` counts `otp_events` rows through PostgREST with `Prefer: count=exact` and `Range: 0-0` (so no row bodies are transferred) across six windows:

KEY	WINDOW	LIMIT	ERROR
phone	45 seconds	0 prior starts	rate_phone
phone	1 hour	5	rate_phone
phone	24 hours	10	rate_phone
IP	1 hour	10	rate_ip
IP	24 hours	25	rate_ip
prefix	1 hour	15	rate_prefix

`allowCheck` blocks after five failed checks in fifteen minutes. Phones are accepted only in two ranges (`+1` with ten digits, `+91` with ten digits starting 6 to 9), and `PhoneOTP` in Swift mirrors the rule so the form refuses a number the server would never send to, without a round trip. The identities are stored hashed with a pepper (`OTP_PEPPER` , falling back to the service key), so the events table holds no phone numbers.

`otp-check` then does something GoTrue has no built-in for: it mints a session for a phone-verified user. `mintSession` finds or creates the user with a synthetic email (`p{digits}@phone.lifeos.internal`), generates a magic link through the admin API, and immediately verifies its `hashed_token` to obtain a session, which is returned verbatim. The lookup uses `otp_user_id_for_phone` with the digits only, because GoTrue strips the leading `+` before storing `auth.users.phone` , and the one Deno test in `otp_session_test.ts` pins exactly that.

`classifyTwilioFailure` maps Twilio's numeric codes to nine kinds, and `SEND_FAILURE_STATUS` maps every kind to an HTTP status; a test checks that no kind can be added to one without the other. The distinction the code comments insist on: 21608 (trial account, unverified number) cannot happen on a paid account and is the operator's problem, while 21610 (opted out) is per number and only the user can undo it. Each gets copy that sends the reader to the right fix.

WHOOOP AND FITBIT TOKENS

Both are described in Chapter 19. The pattern to notice here is shared: read the row, short-circuit if the access token is still good, claim a lock or lease before refreshing, condition the write on still holding it, and release in `finally` . Neither function ever logs a provider's error body on the token path, because OAuth errors can echo credentials.

PLAID: THE EXCHANGE THAT MUST NOT STRAND AN ITEM

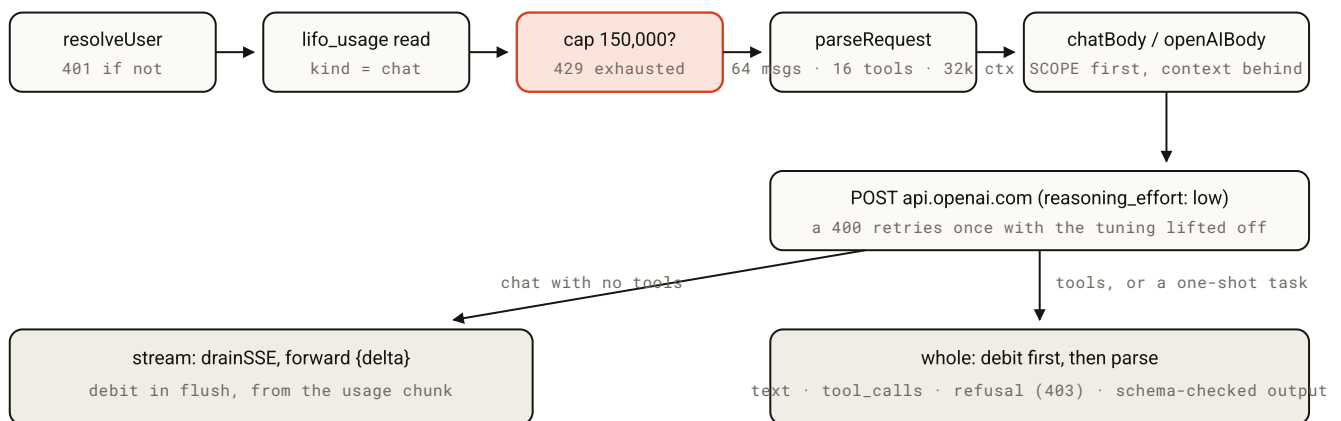
```
// supabase/functions/plaid-exchange/index.ts
// Past this point the Item exists at Plaid and bills monthly. Any failure
// below must not strand it: without a row, plaid-disconnect has nothing to
// look up and the device never learns the item_id, so the Plaid dashboard
// becomes the only remedy. Best-effort remove it before returning.
try {
  const { error } = await db.from("plaid_items").insert({ ... });
  if (error) {
    await removeExchangedItem(exchanged.access_token);
    if (error.code === "23505") {
      return json({ error: "institution_already_connected" }, 409);
    }
    return json({ error: "storage_failed" }, 500);
  }
} catch (storageError) {
  await removeExchangedItem(exchanged.access_token);
  throw storageError;
}
```

Plaid bills per live Item. The function checks for a duplicate institution before spending the exchange, treats a failed lookup as a refusal rather than as "no duplicate", and if the insert fails after the exchange it removes the Item at Plaid before answering. The unique constraint is the last line of defence against the check-then-insert

race, and a 23505 from it is reported the same way as the guard. `plaid-disconnect` is the mirror: it removes the Item at Plaid first and deletes the row only when Plaid confirms it is gone or says `ITEM_NOT_FOUND`.

LIFO-AGENT: THE COACH'S CLOUD TIER

The most consequential function. Its request path is worth a diagram because every step is a guard, and the guards are the cost ceiling.



Nothing reaches the provider without a resolved user, a budget read, a bounded body and the server's own guardrail prompt in first position. Every path that produced a billable call debits the ledger, including refusals and unparseable replies.

The guardrail lives on the server. `SCOPE` in `lifo.ts` tells the model it is LIFO, coaching one person over their own data, declining essays and world knowledge, never diagnosing, never recommending securities, and citing only numbers it was given. It is the first system message on every request, and the client cannot strip what it never carries. A client that sends a message with `role: "system"` is refused outright by `parseRequest`, because a later system message displaces an earlier one in most providers' weighting. The context (the device's rendered digest, or the calendar assistant's rules) arrives in its own field and is placed as a second system message behind `SCOPE`.

Two tasks with schemas, one without. `answer` and `nudge` ask for strict JSON against a named schema, and `parseOutput` validates the reply against that task's own schema rather than a literal field name, throwing loudly on a property type it does not know how to check so a future schema cannot silently skip validation. `chat` declares no schema, because a completion may come back as prose or as tool calls and there is no single shape to check.

The style prompt is copied, and pinned twice. `CONVERSATION` is a verbatim copy of `ResponseStyle.conversation` in Swift: no markdown, no dashes, no quotation marks for emphasis, figures with units, one question at most. The two tiers are meant to write identically, and the copy had drifted at birth, so a Deno test and a Swift test each pin the same prohibitions.

Bounds, not tuning knobs. `MAX_CHAT_MESSAGES = 64`, `MAX_CHAT_TOOLS = 16`, `MAX_CONTEXT_LENGTH = 32000`. Without them the endpoint is a general OpenAI proxy on the project's key for any authenticated user. The device sends at most a 20-turn window, six calendar tools, and a context budgeted at 8,000 characters.

Streaming. A chat turn with no tools may ask for a stream, and the coach does, because the wait for the first word is the whole of how fast it feels. The function re-emits the provider's SSE as its own `{"delta": "..."} frames closed by [DONE]`, so the client never learns OpenAI's chunk format and swapping providers is not an app release. `stream_options.include_usage` is set because usage arrives only in a final chunk, and without it a streamed turn would be free against the cap. A tool-bearing turn is never streamed: a tool call arrives as fragments of a JSON argument string across chunks, and reassembling those on the server would put half the device's round loop up here.

Low reasoning effort. The body sends `reasoning_effort: "low"` and `verbosity: "low"`. The default is medium, and medium is most of the wait; a coaching question over a week of numbers is not a reasoning problem, because the thinking that matters happened on the device when the digest was built. Low also spends fewer tokens, the same lever as the daily cap pulled from the other end. A 400 from the provider retries once with the two parameters lifted off, because a model that stops accepting one of them should cost a slower answer, never no answer.

Debit everything. The non-streaming path debits before it parses. A refusal or a malformed reply still consumed real tokens, and leaving those unmetered would let someone burn unlimited budget by asking off-topic questions in a loop. The debit's own failure is logged and never costs the user their answer. Logs carry ids, task names, token counts and latency; never the prompt, never the key.

LIFO-NUDGE: RULES DECIDE, A MODEL PHRASES

The proactive half. `pg_cron` calls it hourly with the service-role key as a bearer, and the function refuses any other caller. Each run selects the users whose local clock has just reached the send hour (8am, inside a 7am to 9pm quiet floor that widening the send hour can never breach), fetches 94 days of `daily_metrics`, `workout_records` and `nudge_log` for the whole batch in three queries, and evaluates four triggers per user in a fixed order:

1. `strongWeek`: seven-day step mean at least 25 percent above the trailing three weeks, or a personal best in the last three days against 90 days of history. First on purpose: a channel that only carries bad news gets switched off.
2. `shortSleep`: three nights each more than an hour under the person's own median over the prior four weeks, needing at least fourteen nights of baseline. Their own median, not seven hours, so a habitual six-hour sleeper is not told they are failing daily.
3. `stepsCollapsing`: three-day mean under 60 percent of the trailing fortnight, with a baseline of at least 2,000 so someone who does not carry their phone is left alone.
4. `streakAtRisk`: a workout streak of three or more days counted back from yesterday, because at 8am nobody has trained yet.

A trigger that fired inside the seven-day cooldown steps aside. Nothing firing is the common outcome and a valid one; it reaches no model. When one fires, the function first inserts the `nudge_log` row, so two overlapping cron runs both reach here and the unique key on `(user_id, day)` lets exactly one win. Only then does it ask the model to phrase, with a prompt of roughly 300 tokens carrying the trigger's figures and an instruction to use only those. Every fired trigger carries a fallback sentence, so an OpenAI outage delivers the observation plainly rather than swallowing it. A refusal is the one failure that sends nothing, the same distinction `EscalationPolicy` draws on the device between a transient fault and a safety decision. The

phrasing spend debits the `nudge` kind against its own cap of 4,000 tokens, a runaway guard more than a budget.

A forced send exists for proving delivery: `{"test_user_id": ...}` bypasses the triggers, the cooldown and the send hour, writes no log row and spends no tokens, and reports how many tokens were found and how many sends succeeded. Silence being the design, there was otherwise no way to know the APNs half was wired without waiting days for a real trigger.

```
-- supabase/functions/lifo-nudge/schedule.sql (NOT a migration, run once per project)
select cron.schedule('lifo-nudge-hourly', '0 * * * *', $$
  select net.http_post(
    url := (select decrypted_secret from vault.decrypted_secrets where name =
'lifo_project_url')
    || '/functions/v1/lifo-nudge',
    headers := jsonb_build_object('Content-Type', 'application/json',
'Authorization', 'Bearer ' ||
(select decrypted_secret from vault.decrypted_secrets where name =
'lifo_nudge_key')),
    body := '{}':jsonb, timeout_milliseconds := 55000);
  $$);
```

The schedule is deliberately not a migration. It needs the project's own function URL, which differs per environment, and the service-role key, which must never be committed; both are read from Vault. Hourly rather than daily because the send hour is local and a single daily run would serve one time zone.

20.5 Tests

Two kinds. The Deno tests run with `deno test` in `supabase/functions/_shared/` and touch no network: 100 tests across seven files.

FILE	TESTS	PINS
<code>lifo_test.ts</code>	44	Task config, the guardrail's presence on every task, schema validation, refusal handling, the chat body's message order, role filtering, the three bounds, tool-call normalisation, tuning and its removal, streaming gating, SSE reassembly across split reads
<code>nudge_test.ts</code>	25	Day arithmetic across month and year ends, local hour and day in the user's zone, the quiet window, cooldown, each trigger's firing and non-firing cases, trigger order, fallback sentences
<code>fitbit_test.ts</code>	10	Failure kinds, quota header, token form, the scope list holds nothing unread, each collection's path
<code>apns_test.ts</code>	9	JWT header and claims, base64url, PEM tolerance, payload carries no numbers, collapse and expiry headers, dead-token detection from both answers, provider token reuse
<code>otp_twilio_test.ts</code>	7	Twilio code classification and that every kind has a status
<code>plaid_test.ts</code>	4	The three failure kinds and a non-JSON body
<code>otp_session_test.ts</code>	1	A returning user's phone is found in GoTrue's stored format

The pgTAP files in `supabase/tests/` run inside a transaction that is rolled back, so the disposable users and groups they insert never persist. `social_groups.test.sql` switches `request.jwt.claim.sub` between an owner, a friend and an outsider and checks the rules from each seat: an invitee sees the group but not the roster, cannot send chat or read the leaderboard, cannot forge membership with a direct update (error 42501), a retried send does not duplicate, an outsider sees nothing. `group_wellness.test.sql` checks that activity sharing does not grant health-score consent and that no score is stored before the second opt-in. These are the tests that catch an RLS mistake, which otherwise looks like an empty screen.

20.6 Email templates

`magic_link.html` and `confirmation.html` are the same design: a card on the app's canvas colour with an accent band across the head, the dot grid drawn with table cells so it survives image blocking, and the six-digit `{{ .Token }}` in a 42-point monospaced span with 12 pixels of letter spacing. Tables and inline styles throughout, because email clients strip style blocks, ignore flexbox, and Outlook renders through Word. No images anywhere: nothing to host or have go missing in a year. The subject lines are set in `config.toml`, and `supabase config push` is what sends the templates to the hosted project.

20.7 The static site

`web/` is a two-file site deployed on Vercel at `almanac.shivvyas.com`. Its only job is to make Plaid's OAuth redirect land back inside the app.

```
// web/.well-known/apple-app-site-association
{ "applinks": { "details": [ {
  "appIDs": [ "Z42YU5W6WY.com.shivvyas.lifeos" ],
  "components": [ { "/" : "/plaid-oauth*" } ]
} ] } }
```

Banks that use OAuth hand the browser to the bank, and the bank redirects to a URI Plaid was told about in advance. On iOS that must be a universal link, because a custom scheme cannot be registered with Plaid and would not survive the hop through Safari. The association file claims the one path, `vercel.json` forces its content type to `application/json` (without which iOS ignores the file and every OAuth connection silently falls back to Safari), and `plaid-oauth.html` is the fallback page a person sees only when the link does not resolve. That page must not look like an error and must never claim the connection failed, because by then Plaid has usually already succeeded; it offers an `almanac://plaid-oauth` button back into the app. The README lists the four steps the repository cannot do: DNS, the Plaid dashboard's allowed redirect URIs, the `PLAID_REDIRECT_URI` secret, and the Associated Domains capability on the App ID.

20.8 Study notes

READ IN THIS ORDER

1. `supabase/functions/_shared/supabase.ts`, the door every function uses.
2. `supabase/migrations/20260811164841_initial_schema.sql` for the policy shape, then `20260914090000_social_groups.sql` for the definer-function shape.
3. `_shared/lifo.ts` top to bottom, then `lifo-agent/index.ts`.
4. `_shared/nudge.ts`, then `lifo-nudge/index.ts`, then `schedule.sql`.
5. `_shared/otp_guard.ts` and `otp_twilio.ts` together.
6. `plaid-exchange/index.ts` for the cleanup pattern.
7. `supabase/tests/social_groups.test.sql`, to see RLS tested from three seats.

CHECK YOURSELF

- **Why must `resolveUser` run before `serviceClient` in every function?** The service role bypasses RLS. A user id taken from the request body would let any signed-in person read anyone's bank credential.
- **Which tables have RLS enabled with zero policies, and what does that mean?** `otp_events`, `plaid_items`, `lifo_usage`, `fitbit_connections`, `nudge_log`, `whoop_connections`. Nobody but the service role can read or write them, including the row's own owner.
- **Why does the social groups migration grant only `select` to clients?** Every write has a rule (friends only, 50 max, owner succession, idempotent sends) that a row policy cannot express, so writes go through definer functions that raise.
- **Why is the guardrail prompt on the server and not in the app?** A client cannot strip what it never carries, and a client system message is refused so it cannot displace it.
- **What does the streamed path do that makes it not a free path through the cap?** It asks for `include_usage` and debits in `flush`, which runs even when the client abandons the stream.
- **Why is `schedule.sql` not a migration?** It needs the per-environment function URL and the service-role key, both read from Vault at run time.
- **What breaks if `lifo-agent` is deployed before the `proactive_nudges` migration?** Its three-argument `lifo_debit` call fails until the migration creates that signature; deploy order is migration first, then both functions.

21 Data reference

Every type a screen reads through, as it exists in `Life0SKit/Sources/Persistence` today: twenty `SwiftData` models, the value types beside them, and the stores that are the only way anything writes. The package depends on nothing, builds for macOS, and is where the app's most important rule lives.

21.1 Four rules every model obeys

The models below look different but they were written to the same four rules. Learn the rules and most of the reference reads itself.

1. **Every metric is optional.** A missing value and a zero must never be the same thing. Zero steps means you did not move; nil steps means the watch was on the charger. In a health app a false zero is worse than a blank, because it is a wrong number presented with confidence. The optional travels unchanged from the column through the snapshot to the tile, which renders the dash itself.
2. **A field added to an existing model is optional or defaulted.** `SwiftData` will not migrate a store in place when a mandatory attribute has no default, and it does not fail loudly: the container never opens, the shell has no account to show, and the app sits on its spinner. Section 21.13 tells the story of the day this happened.
3. **Enums are stored as raw strings with typed accessors.** `kindRaw`, `statusRaw`, `bucketRaw`, `sectorRaw`. A new case is a code change rather than a migration, and an unknown value read back from an older store falls to a sensible default instead of failing the row.
4. **Views never hold a `@Model`.** Every store hands out a detached value type: `PlanItemSnapshot`, `CalendarEventSnapshot`, `NoteCardSnapshot`, `ChatMessageSnapshot`. A managed object handed to a card is a live handle on the database and a reason every view needs a container to preview. The snapshot costs one map per fetch and buys back both testability and the compile-time guarantee that nothing above the store mutates rows.

WHY RAW JSON BAGS APPEAR SO OFTEN

Five models keep an array or dictionary as encoded `Data` rather than as a relationship:

`DailyMetrics.extrasData`, `WorkoutRecord.setsData`, `NoteDocument.blocksData`, `ChatMessage.eventIDsData` and `SectorScore.evidenceData`. The reasoning is the same each time: the value is only ever read and written as a whole, no query asks for one element of it, and a relationship would cost one migration hazard per field in exchange for nothing. Where a query is needed, the JSON is mirrored out into derived rows instead, which is exactly what `NoteTask` and `NoteLink` are.

21.2 DailyMetrics

One row per calendar day, the join key for the entire application. `#Unique([\ .date])` makes a second row for the same day impossible, and `date` is always `Calendar.startOfDay`, never a timestamp.

PROPERTY	TYPE	WRITTEN BY	NOTES
<code>date</code>	Date	MetricsStore	Unique. Start of day.
<code>weightKg</code>	Double?	Health, Quick log	Renpho writes Health; Health writes here
<code>steps</code>	Int?	Health, Fitbit	De-duplicated by HealthKit across watch and phone
<code>activeEnergyKcal</code>	Double?	Health, Fitbit	
<code>exerciseMinutes</code>	Int?	Health, Fitbit	
<code>sleepMinutes</code>	Int?	Health, Whoop, Fitbit	Attributed to the morning you woke
<code>waterML</code>	Double?	Quick log	No automatic source exists
<code>restingHR</code>	Double?	Health, Whoop, Fitbit	
<code>hrvMs</code>	Double?	Health, Whoop, Fitbit	SDNN from Health, RMSSD from Whoop; the arbiter decides
<code>whoopRecoveryPct</code> , <code>whoopDayStrain</code> , <code>whoopSleepPerformancePct</code>	Double?	Whoop	Exist nowhere else
<code>spo2Percentage</code> , <code>skinTempCelsius</code> , <code>respiratoryRate</code>	Double?	Whoop, Health	The scored daily surface Whoop reports
<code>whoopCalories</code> , <code>whoopAverageHR</code> , <code>whoopMaxHR</code>	Double?	Whoop	From the cycle record
<code>whoopSleepConsistencyPct</code> , <code>whoopSleepEfficiencyPct</code> , <code>whoopSleepDebtMinutes</code>	Double?, Double?, Int?	Whoop	Sleep need components live on <code>SleepRecord</code>
<code>whoopRecoveryIsCalibrating</code>	Bool?	Whoop	Optional so an existing store migrates; a calibrating score is rendered with a marker, never compared
<code>extrasData</code>	Data?	Health	JSON <code>[String: Double]</code> keyed by <code>HealthMetric.rawValue</code> : the long tail of Health metrics with no column of their own
<code>sourcesData</code>	Data?	Health, Whoop, Fitbit, Quick log	JSON <code>[String: String]</code> : who wrote each value, read by the arbiter
<code>updatedAt</code>	Date	MetricsStore	Drives field-level last-write-wins
<code>syncedAt</code>	Date?	reserved	nil means never pushed. Health rows are not pushed today; see Chapter 20

Two design points deserve a closer look. The first is why the named columns stopped growing. Height and max heart rate are absent on purpose because they are constants, and a column restating the same value on every row is noise. Everything else Apple Health can supply lands in the `extras` bag, added by one enum case and one mapping, because most people have no data for most of it and a schema change per metric for fields nothing queries is the wrong trade.

```
// LifeOSKit/Sources/Persistence/DailyMetrics.swift
public func setExtra(_ key: String, _ value: Double?) {
    var bag = extras
    if let value { bag[key] = value } else { bag.removeValue(forKey: key) }
    extras = bag
}
```

Writing `nil` removes the key rather than storing a zero. The bag keeps the distinction the whole model is built on: a key that is missing means no reading, and nothing ever writes a zero to stand in for one. The getter returns an empty dictionary rather than throwing when the bytes cannot be read, because a day whose extras are unreadable still has its named columns, and losing the screen over the tail would be the worse failure.

The second is provenance. `sourcesData` is a parallel bag rather than a field folded into `extras`, because that bag is numbers and a source is a name. It stores a raw string rather than a typed `MetricSource` because `Persistence` sits below `Integrations` and must not import it; the vocabulary lives up there. A store written before provenance existed reads back no attribution, which `MetricArbiter` treats as rank zero for everything except the two typed metrics, weight and water, where an unattributed value is read as typed so that the next Health sync cannot silently replace a figure a person entered by hand.

21.3 UserGoals, goal evaluation and anomalies

`UserGoals` is a singleton by convention: exactly one row, created with defaults on first access by `MetricsStore.goals()`.

PROPERTY	TYPE	DEFAULT	USED BY
<code>stepsGoal</code>	Int	8,000	Dot grid, Dry Run ceiling
<code>sleepMinutesGoal</code>	Int	420	Dot grid, Dry Run ceiling
<code>exerciseMinutesGoal</code>	Int	30	Dot grid, Dry Run ceiling
<code>waterMLGoal</code>	Double	2,500	Dot grid, Dry Run ceiling
<code>requiredCount</code>	Int	3	How many of the four make a good day
<code>trainingGoal</code>	String?	nil	Workout library planner
<code>equipmentRaw</code>	[String]	[]	Workout library filter
<code>sessionMinutes</code>	Int?	nil	Workout library planner

The last three were added on 14 September 2026 for the workout library, and the middle one is the field that briefly shipped without a default. The row converts to `GoalTargets`, a plain `Sendable` struct, before it

reaches any scorer or view model.

EVALUATION

```
// LifeOSKit/Sources/Persistence/GoalEvaluation.swift
public func evaluate(_ reading: DayReading, against targets: GoalTargets) -> DayStatus {
    guard reading.hasAnyData else { return .noData }
    var met = 0
    if let steps = reading.steps, steps >= targets.steps { met += 1 }
    if let sleep = reading.sleepMinutes, sleep >= targets.sleepMinutes { met += 1 }
    if let exercise = reading.exerciseMinutes, exercise >= targets.exerciseMinutes { met += 1 }
}

    if let water = reading.waterML, water >= targets.waterML { met += 1 }
    return met >= targets.requiredCount ? .onTarget : .missed
}
```

A day with no metrics at all is `.noData`, excluded from judgement, because leaving the watch on the charger is not a failure. A day with some data is judged on what it has, so an unlogged metric cannot launder a bad day into a blank one. Exactly meeting a goal counts, since the comparison is greater-or-equal.

`currentStreak` expects statuses most-recent-first, treats `.noData` as transparent, counts `.onTarget`, and stops at the first `.missed`.

ANOMALIES

`AnomalyEvaluation` flags one vital sitting outside its own recent range. The baseline needs at least seven actual readings (`minimumBaselineDays`) or there is no verdict, which is the app-wide rule that absence of data is never a judgement. Thresholds are relative for metrics that scale with the person (heart rate) and absolute for metrics with a physiological unit (a degree of skin temperature means the same at any baseline). The wording rule for anything rendered from an `AnomalyFinding` is descriptive, never diagnostic: "well above your baseline" is a fact, "see a doctor" is a claim the data cannot support.

21.4 Source records and the raw archive

`WorkoutRecord` and `SleepRecord` hold data that is genuinely not daily. They roll up into `DailyMetrics`, which is derived state and always safe to recompute from them.

MODEL	KEY	FIELDS WORTH KNOWING
WorkoutRecord	externalID (provider id or the recorder's UUID)	start , durationMinutes , activityName , energyKcal , strain , averageHR , maxHR , distanceMeters , altitude gain and change , percentRecorded , sportID , six optional zone-minute columns, setsData (reps per set as JSON), split and videoID for sessions that followed a catalog video
SleepRecord	externalID	start , end , attributedDate (the morning you woke), performance, consistency and efficiency percentages, light, REM, SWS, awake and no-data minutes, cycle count, respiratory rate, sleep need, debt, need from strain, need from nap (negative or zero), disturbance count, isNap: Bool?
WhoopRawRecord	#Unique([\ .kind, \.externalID])	kind is a String, not an enum, so a collection never seen lands as data; payload , receivedAt

Three small decisions on `WorkoutRecord` show the rules at work. Six zone columns rather than one array, because `SwiftData` stores scalars cleanly and an array attribute would be a migration this does not need. `percentRecorded` kept, because a workout at 40 percent recorded is not a workout with a low strain, and collapsing the two would be the false zero the app refuses. `setsData` as JSON with empty sets stored as nil, so an existing store migrates without a plan and a session with no sets is not a session with an empty list.

`WhoopArchive` is the reader and writer of the raw archive. It upserts on `(kind, externalID)` because a re-sync returns records already seen, and it has a batched `store(_:)` that saves once per page, because a save per record turns a fourteen-day sync into hundreds of disk transactions on the main actor. `FitbitSync` reuses the same archive under `fitbit-` kinds, since it was already a generic raw-record store and only its name is Whoop-specific. The archive is why a derivation bug six months out is fixable without asking a rate-limited API for the data again.

21.5 Money

Four models and three value types. The one sentence to remember is written into the type: **positive is money in, negative is money out**. Plaid is the opposite, its `amount` is positive for outflows, and the ingestion layer negates on the way in. A silent sign flip turns income into expenses and every rollup lies, which is why the convention is a doc comment on the class rather than a note in a spec.

MODEL	FIELDS	INVARIANT
MoneyEntry	id, externalID (Plaid transaction_id), sourceRaw, date, amount, currencyCode, merchant, category, categoryCode, merchantID, logoURL, accountID, accountName, pending, timestamps	date is start of day; externalID is the upsert key; categoryCode keys arithmetic and category is display text only
MoneyAccount	externalID, name, type, subtype, mask, availableBalance, creditLimit, currentBalance, currencyCode	credit and loan types contribute - abs(balance) to net worth; creditLimit is never defaulted to zero because utilisation against zero computes to 100 percent and means nothing
SpendBucket	name, monthlyLimit, claimedRaw, sortOrder	limit is positive; a claim key is held by at most one bucket, enforced on write

The spending predicate is one property so that the category list always sums to the figure above it:

```
// LifeOSKit/Sources/Persistence/MoneyEntry.swift
public var isSpending: Bool {
    amount < 0 && !pending && !MoneyCategoryRule.isTransferLike(categoryCode)
}
```

MoneyCategoryRule.isTransferLike keeps transfers between your own accounts and credit card payments out of both income and expenses: they appear as a real debit and a real credit, so counting them leaves net correct while overstating both sides, and the savings rate is computed from those two. It matches the underscore-delimited prefix (TRANSFER_IN_, TRANSFER_OUT_) rather than a bare prefix, because without the underscore a code that merely starts with the same letters would be excluded as if it were a transfer.

MoneySummary.savingsRate is nil when income is zero. Undefined is not zero, and this is the sharpest case of the app's central rule. MoneyParts splits an amount for display and rounds to the cent before splitting, because splitting a Double first lets a fraction of .995 round up to 100 cents and render as \$19.100. It lives in the kit, where it is tested, rather than in a view where it is eyeballed. MoneyEntry.claimKey is categoryCode ?? category : Plaid's raw detailed code when there is one, else a manual entry's free-form category.

21.6 Plan entries and habit ticks

PlanEntry is one model carrying five kinds: goal, habit, note, content and journal. A block renderer can lay out any entry without knowing which feature produced it, and a new kind is an enum case rather than a table. PlanStatus is todo, inProgress, done, blocked or scheduled, and both enums are stored raw so an unknown value reads as .note and .todo.

HabitTick is one completion of a habit on one day, separate from the entry because a habit's history is a set of days rather than a field, and because it lets the existing dot grid render a habit exactly like the Today grid. PlanItemSnapshot is the detached form, with fraction derived from progressValue and progressTarget and clamped to 0...1.

The plan tab's goals and journal entries were migrated onto note pages on 27 and 28 August 2026.

`PlanNoteMigration` runs once, idempotent through `NoteDocument.originPlanEntryID` rather than a flag, so a second device does not produce a second copy of everything the first one migrated. The rows it deliberately leaves behind still score months that were already closed, which is why `NoteEvidence` returns both.

21.7 Calendar events and chat messages

`CalendarEvent` is a cache of provider events, never the source of truth for them. The natural key is `(source, sourceID)`, and `sourceID` is EventKit's identifier qualified with the occurrence start for recurring rows, because EventKit reuses one identifier across a series. `CalendarEventSnapshot` is the value form, and being `Sendable` is the point: a tool result crosses into an engine, and a `@Model` cannot make that trip. `CalendarEventDraft` is what a caller supplies to create or edit, with no id and no source, because routing decides those. `CalendarDayIndex.group` indexes overlapping events onto every visible day with an exclusive end date.

`ChatMessage` persists a conversation locally only. It carries `toolSummaries`, rendered as activity chips, and `eventIDsData`, the ids of the events a reply was about. Ids rather than the events, on purpose: an event can be moved or deleted after the sentence about it was written, and a card showing what was true last Tuesday is worse than no card. They are re-resolved against the calendar when the conversation reopens through `CalendarStore.events(withIDs:)`, so a deleted event simply stops appearing. `ChatStore.append` takes an `at` date for tests, because appends in a tight loop share a wall-clock timestamp and the recency ordering must not be flaky over ties.

21.8 Notes

Notes are the largest model family and the only one synced to Supabase. Four persisted models, two derived, and a set of value types.

MODEL	ROLE	FIELDS WORTH KNOWING
<code>NoteFolder</code>	A folder on one of the PARA shelves	<code>name</code> , <code>icon</code> , <code>bucketRaw</code> , <code>accentRaw</code> , <code>parentID</code> (nests arbitrarily), <code>sortOrder</code> , <code>syncedAt</code> , <code>deletedAt</code> (a tombstone, so a folder removed on one device disappears on the other)
<code>NoteDocument</code>	One page: note, journal entry or task list	<code>title</code> , <code>icon</code> , <code>kindRaw</code> , <code>bucketRaw</code> , <code>accentRaw</code> , <code>folderID</code> , <code>blocksData</code> , <code>drawingData</code> (a <code>PKDrawing</code>), <code>entryDate</code> , <code>dueDate</code> , <code>statusRaw</code> (reuses <code>PlanStatus</code>), <code>isFavorite</code> , <code>openedAt</code> , <code>archivedAt</code> , <code>filedAt</code> , <code>syncedAt</code> , <code>deletedAt</code> , <code>originPlanEntryID</code>
<code>NoteTask</code>	Derived: one to-do block hoisted out of its page	<code>id</code> is the block's own id; scoped by <code>documentID</code> ; <code>text</code> , <code>isChecked</code> , <code>indent</code> , <code>dueDate</code> , <code>goalID</code> , <code>sortOrder</code> , <code>documentUpdatedAt</code> (denormalised so a cross-page list can order pages)
<code>NoteLink</code>	Derived: one <code>[[wiki link]]</code> edge	<code>sourceID</code> , <code>targetTitleFolded</code> (lowercased), <code>targetID</code> (nil when the page does not exist yet; non-nil is not proof it still does)

Two fields on `NoteDocument` carry whole features. The Inbox is `filedAt` and nothing else: every page has a bucket from the moment it is created, so a bucket cannot answer "has anyone decided where this goes", and

the stamp is written by filing, never by editing, so a page you keep writing in stays in the Inbox until you decide otherwise. The Archive shelf is `archivedAt`, a flag rather than a bucket, so a restored page returns to the shelf it came from instead of needing to be refiled. Every field carries a default because this model will keep growing and rule two applies.

`NoteBlock` is a value type with a `kind` (paragraph, three headings, bulleted, numbered, todo, quote, callout, code, divider, sketch), `text`, `isChecked`, `indent` 0 through 4, optional `drawing` and `sketchHeight` for sketch blocks, and optional `dueDate` and `goalID` for to-dos. The whole array is stored as JSON. A block editor's trick is that the document is a list of small typed pieces rather than one string with formatting hung off ranges, so splitting, merging and turning a paragraph into a to-do all become list operations, and none of them can corrupt the others. Nesting is purely visual: a nested item is still a sibling in the array.

`NoteBlockParser.shortcut` resolves Notion-style typed prefixes, and the order of its candidates is the whole correctness argument: `- [ ]` starts with `-`, so checking kinds in declaration order would turn every to-do typed by hand into a bullet reading `[ ] whatever`. Longest match first is the only rule that cannot be broken by adding a kind later. `NoteLinkScanner` finds `[[targets]]`, preserving case in the text and folding it at lookup. `NoteBucket` is the four PARA shelves as an enum rather than four folders a person could rename into meaninglessness, with `research` standing in for PARA's vaguer "resources". `NoteAccent` is eight named pastels, colour living in the design system so that `Persistence` never imports SwiftUI and `swift test` keeps running on macOS.

The value types: `NoteCardSnapshot` (a page flattened for a card, with `doneCount`, `taskCount`, `hasInk`, `linkCount`), `NoteFolderSnapshot` (a sidebar row with its children already attached, built once in the store rather than by the view walking a flat list on every render), `NotesSnapshot` (everything the sidebar needs in one read), `NoteBacklink`, `NoteStreamChip` and `NoteStream` (a chip resolves to either cards or tasks, because flattening to-dos into cards would lose which page each came from). `NoteEvidence` is the bridge the monthly close reads: journal dates for the journalling streak, and project statuses where a page with to-do blocks reports what those blocks say rather than its own menu property.

21.9 Sectors

`LifeSector` names the nine sectors: family, romance, soul, friends, growth, money, mission, body, mind. Raw values are written to the store, so a case may be added but never renamed, since a rename orphans every score recorded against it. `boardOrder` is kept separate from `allCases` so adding a sector cannot silently reshuffle the grid.


```
// LifeOSKit/Sources/Persistence/SectorScore.swift
@Model
public final class SectorScore {
    public var sectorRaw: String
    /// Always the first day of the month being scored.
    public private(set) var month: Date
    /// What the rule said, or nil when there was no evidence at all.
    public var proposedScore: Int?
    /// What the person said. Nil until they pass through this sector.
    public var userScore: Int?
    public var closedAt: Date?
    /// The rows behind the proposal, frozen at close time.
    public var evidenceData: Data?
}
```

Both scores are kept permanently, because the distance between what the rule computed and what the person felt is the most interesting number here, and averaging them would erase it. The evidence is archived rather than recomputed: change a goal in March and February's reasoning must still read the way it did in February. `month` is `private(set)` and normalised through `Date.startOfMonth` on the way in, so closing August on 3 September files the score under 1 August.

`CheckInAnswer` stores one answer to one question for one sector-month, as text whatever the question's shape, because its value is comparison against last month rather than arithmetic. `Evidence` is the one shared scoring rule: a weighted mean of `EvidenceRow`s on a 0...10 scale, clamped on the way in rather than trusted, with `proposedValue` nil when there is nothing to go on. Deliberately not zero: a sector with no evidence has not been judged badly, it has not been judged. `proposedScore` is expressed in terms of `proposedValue` so a projection sorting on the unrounded value can never rank against a different number than the one shown.

21.10 The workout catalog and bookmarks

`CatalogVideoRow` is the wire row, snake_case on the server and decoded by the client; `CatalogVideo` is the cached copy with `#Unique([\.youtubeID])`, the same ten fields plus `cachedAt`, a computed `thumbnailURL` on `i.ytimg.com` and `durationMinutes`. The server is the source of truth: `CatalogStore.upsert` replaces the whole set, updating matches, inserting new rows and deleting absent ones, so a video pulled from the catalog disappears here. It builds its lookup with `uniquingKeysWith` rather than `uniqueKeysWithValues`, because a trap on a malformed response is a crash where keeping the last row is not.

`WorkoutBookmark` is a person's mark on a video: `saved`, `scheduledFor` (a calendar day, never a timestamp), or both. One row per video and absent means neither, so `BookmarkStore.toggleSaved` and `schedule` both delete a row that ends up neither saved nor scheduled. The store reads `row.saved` into a local before the delete, because a deleted model is not a thing to ask questions of, even one it answered correctly a line earlier. Bookmarks are local only.

21.11 Accounts, scopes and the container

Until 27 August 2026 the app had one store file, no owning column on any model, and one keychain slot. A second person signing in on the same device inherited the first person's notes, journal, chat, money and health data. The server was never the problem, since Row Level Security scopes every row to `auth.uid()`; the device was.

`UserScope` is the answer: a struct holding the Supabase user id, with `storeURL(base:)` under Application Support in a directory named for the account, and `defaultsSuiteName` for a `UserDefaults` suite of the account's own. Isolation is by file rather than by an owner column on forty models, because a column means a predicate on every read, and a predicate forgotten once is a leak that looks like working software. A separate file cannot leak: the rows are not there to be read. The defaults suite matters as much as the store: sync cursors, connection tokens and "have they seen the tour" were all in the shared suite, and a second account would have resumed a notes pull from the first account's cursor and silently missed every row written in between. Device preferences (appearance, the assistant's voice, which engine answers) stay in `standard` on purpose, because they belong to the phone rather than to whoever is signed in on it.

```
// LifeOSKit/Sources/Persistence/LifeOSContainer.swift
public static let schema = Schema([
    DailyMetrics.self, UserGoals.self, WorkoutRecord.self, SleepRecord.self,
    WhoopRawRecord.self, PlanEntry.self, HabitTick.self,
    MoneyEntry.self, MoneyAccount.self, SpendBucket.self,
    SectorScore.self, CheckInAnswer.self, CalendarEvent.self, ChatMessage.self,
    NoteDocument.self, NoteFolder.self, NoteTask.self, NoteLink.self,
    CatalogVideo.self, WorkoutBookmark.self,
])
```

Twenty models. `LifeOSContainer.make(for:)` opens one account's store; `make(inMemory:)` serves tests and previews. Two adoption paths move a store rather than copy it, which is atomic and costs the same for one note or a thousand: `adoptLegacyStore(into:)` moves SwiftData's `default.store` into the first account that signs in after the upgrade, so scoping the store never looks like the app deleting a year of notes, and `adopt(_:into:)` hands one scope's store to another. Both refuse when the destination already has a store, because choosing which of two rows wins for every model in the schema has no obviously right answer, and both move the `-wal` and `-shm` files alongside the database, since moving the database alone leaves the most recent writes behind in a log the new file cannot find.

`AccountDeviceAccess` records that device permissions are app-wide in iOS but permission to import that device's calendar into an account must also be granted by that account. `SampleMetricsPurge` deletes every `DailyMetrics` row once, removing the sixty days of invented history a fresh install used to seed. The seeded values did more than look wrong: most Health fields were fill-gaps-only, so every seeded day refused the real reading for as long as it existed. Only the derived table is removed; workouts, sleep and Whoop records are left and a resync rebuilds the days.

21.12 The stores

Every store is a `@MainActor` value type over a `ModelContext`, with the calendar injected so tests are not at the mercy of the machine's locale. The pattern is `MetricsStore`'s, and every later store copied it.

STORE	READS	WRITES	TH
MetricsStore	metrics(from:to:) most-recent-first, workouts(on:), workouts(from:to:), sleepRecords(from:to:includingNaps:), goals()	upsert(date:apply:), upsertBatch(dates:apply:), upsertSleepRecord, upsertWorkoutRecord	On me so car sal ba
MoneyStore	entries(from:to:) always bounded, monthEntries(containing:), accounts(), buckets()	add, delete, ingest, remove(externalIDs:), upsertAccounts, bucket CRUD, claim, unclaim	Up id; ins be is u init tho the key
PlanStore	entries(kind:), recentTicks, tickedHabitIDs(on:), streak(for:)	add, delete, setStatus, toggleTick	At ent del
NotesStore	documents, folders, cards, recent, favorites, snapshot, search, backlinks, indexedTasks, indexedLinks, inbox, stream(for:), pendingDocuments, pendingFolders	createFolder, createDocument, capture, journalEntry, update(_:blocks:), rename, icon, accent, status, due date, drawing, move, favourite, markOpened, archive, delete, markSynced	Ev the del ma tou so flig
CalendarStore	events(from:to:) half-open, events(withIDs:), snapshot(id:), freeSlots	apply(_:window:authoritative:)	Re fet (s so loc ab: on fet
SectorStore	scores(forMonth:), score(_:month:), history, scoredCounts(before:), answers	record, commit(userScore:to:), saveAnswer	A c ne re uni co se
ChatStore	recent(conversationID:limit:) oldest first, latestConversationID	append	Lo ne
CatalogStore, BookmarkStore	all, bookmark(for:), scheduled(on:)	upsert, toggleSaved, schedule	Th wh bo del

`MoneyIngesting` is the write side of `MoneyStore` as a protocol, so `PlaidSync` can be tested against a store that fails on demand. Failure is the interesting case: the cursor must not move when the ingest did not happen. `CalendarStore.freeSlots` has a `nonisolated static` pure core that merges busy intervals, subtracts them from the window and drops gaps shorter than the requested duration; all-day events do not block time, because "Anniversary" is context, not a meeting.

Three one-time migrations sit beside the stores. `PlanNoteMigration` is described above.

`NoteIndexMigration` stamps every page that predates the Inbox as filed, then builds the index, idempotent through a marker in the account's own defaults suite; the marker is only spent when there was history to interpret, because on a fresh install the first launch runs before the first pull and consuming the one-shot then would strand every page that arrives afterwards in the Inbox. `NoteIndexer` is the only writer of `NoteTask` and `NoteLink`. It deletes and reinserts rather than diffing, never saves on its own because it runs inside the transaction already changing the document, and fetches rows to delete rather than using the batch `delete(model:where:)`, because the batch form runs against the persistent store and cannot see rows inserted earlier in the same unsaved transaction. `rebuildAll` is the opposite: the batch form is correct there because everything is cleared before anything is inserted. It builds the folded title index once for the whole pass, since building it per document made a rebuild quadratic in the size of the library.

21.13 Adding a field without breaking installed stores

On 15 September 2026 the goals row gained `equipmentRaw: [String]`, mandatory and with no default. `SwiftData` refused to migrate any store written before it. The container never opened, the shell had no account to show, and the app sat on the signed-in spinner. The error, when it surfaces at all, reads "Validation error missing attribute values on mandatory destination attribute". The fix was one character group, `= []`, and a test that holds the added columns to the rule:

```
// LifeOSKit/Tests/PersistenceTests/SchemaMigrationTests.swift
@Test func fieldsAddedToTheGoalsRowHaveDefaults() throws {
    let goals = try #require(LifeOSContainer.schema.entities.first { $0.name == "UserGoals" })
    let added = ["trainingGoal", "equipmentRaw", "sessionMinutes"]
    for name in added {
        let attribute = try #require(goals.attributesByName[name], "missing \(name)")
        #expect(attribute.isOptional || attribute.defaultValue != nil,
            "\(name) has no default and is not optional")
    }
}
```

The rule, and how the models already follow it: `whoopRecoveryIsCalibrating: Bool?` and `SleepRecord.isNap: Bool?` are optional rather than defaulted Booleans; `ChatMessage.eventIDsData: Data?` and `WorkoutRecord.setsData: Data?` are optional JSON; every field on `NoteDocument` and `NoteFolder` carries an initial value. When you add a field to a model that has rows on somebody's phone, make it optional or give it a default, and add its name to the list in that test.

WHY THE APP HANGS INSTEAD OF CRASHING

`AccountSession` opens the container for the current account after the session is restored. A container that throws leaves the session without a store, and the shell renders its waiting state until one arrives. Nothing is wrong with the shell; the failure is a step earlier, which is why the symptom looks like a network problem and is not.

21.14 Study notes

READ IN THIS ORDER

1. `LifeOSKit/Sources/Persistence/DailyMetrics.swift`, for the rule and the two bags.
2. `MetricsStore.swift`, for the upsert-as-merge pattern every store copies.
3. `GoalEvaluation.swift` and `Streak.swift`, the shortest pure logic in the app.
4. `MoneyEntry.swift`, for a type that carries its own sign convention.
5. `NoteDocument.swift` then `NoteBlock.swift`, for JSON-in-a-column and the parser order argument.
6. `NoteIndexer.swift`, for the fetch-versus-batch-delete distinction.
7. `UserScope.swift` and `LifeOSContainer.swift`, for per-account stores and the two adoption moves.
8. `Tests/PersistenceTests/SchemaMigrationTests.swift`, then add a field to a model in a scratch branch and watch it fail.

CHECK YOURSELF

- **Why is `savingsRate` nil rather than zero when there is no income?** Because undefined is not zero; rendering 0 percent would be a confident wrong number.
- **What breaks if `NoteIndexer.reindex` used `context.delete(model:where:)`?** A second reindex in the same unsaved transaction would duplicate rows, because the batch delete cannot see rows inserted earlier in that transaction.
- **Why is `LifeSector` a case you may add but never rename?** Raw values are stored; a rename orphans every score already recorded against it.
- **Why does the store move `-wal` and `-shm` with the database?** SQLite keeps the most recent writes in the write-ahead log; moving the database alone leaves them in a file the new location cannot find.
- **How does the Inbox know a page is unfiled?** `filedAt` is nil. Buckets cannot answer, because every page has one from creation.
- **Why does `MoneyStore.ingest` fetch all ids up front?** `externalID` is unindexed and this runs on the main actor; one scan per page instead of one per row.

22 Production edge cases

The register below is the difference between a demo and something you can rely on. Each row is a condition that will occur in normal use, not a theoretical one, and each names where in the code or the test suite the behaviour is pinned. The feature chapters cross-reference the rows that apply to them.

22.1 How to read this register

Three columns. **Condition** is what happens in the world. **Behaviour** is what the app does about it, in the present tense, because every row here describes code on disk today. **Pinned by** names the test, native check or pure function that holds the behaviour in place, so that a future edit which changes it fails somewhere visible. A row marked *by design* has no test because the behaviour is structural: a type that cannot cross a boundary, a table with no policies. A row marked *hardware* is unverified in the simulator and waits on a physical device; those are listed again in Appendix A.

Two themes recur. The first is the rule from 21.1: absence is never zero, and a screen with no data shows its designed empty state rather than a number nobody earned. The second is that destructive outcomes, signing someone out, moving a sync cursor, replacing a cache, happen only on a definitive answer from the server and never on a transport failure, a rate limit or an empty response that is more likely a bug than a fact.

22.2 Time and calendars

CONDITION	BEHAVIOUR	PINNED BY
Sleep spans midnight	Attributed to the morning you woke, through <code>SleepRecord.attributedDate</code> . A nap is stored with <code>isNap</code> and excluded from night-by-night views	<code>WhoopTests</code> , <code>WhoopSleepMath</code>
A Whoop cycle boundary is not a calendar day	Cycles are attributed to the day the cycle starts, sleep to the day it ends. Documented so the mismatch is not read as a bug	<code>WhoopDerivation</code>
Travel across time zones	"Today" follows the device's current zone; a day can be shortened or lengthened. The nudge job counts windows back from the user's local day, not UTC midnight, and a device token without a usable zone never sends	<code>nudge_test.ts</code> ; <code>device_tokens.timezone</code> not null
DST transition	<code>Calendar.startOfDay</code> handles the 23 or 25 hour day; nothing computes days by adding 86,400 seconds	by design
Month boundaries on the grid	Leading blanks computed from <code>firstWeekday</code> ; padded to whole rows	<code>MonthGridLayoutTests</code>
A month is scored on the third of the next month	Filed under the first of the month scored, through <code>Date.startOfMonth</code> , which is enforced on <code>SectorScore.month</code> as <code>private(set)</code>	<code>SectorScoreTests</code> , <code>SectorStoreTests</code>
The planner names a weekday in its reason	Weekday symbols are read through an <code>en_US_POSIX</code> copy of the calendar, because a host calendar with an empty locale returns abbreviations	<code>WorkoutPlannerTests</code>
The clock moves backwards	A future last-sync is treated as stale and the sync runs; a session keep-alive delay never drops below five seconds, so a jumped clock cannot spin against the auth endpoint	<code>SyncStalenessPolicyTests</code> , <code>SessionKeepAliveTests</code>
First open of the morning, last sync at 23:50	The day boundary triggers a sync regardless of staleness, because the night's sleep is new information a minutes-old sync knows nothing about	<code>SyncStalenessPolicyTests</code>
Device clock skew against the server	Can let a stale local write win a field-level merge. Accepted and documented; the server's touch trigger stops a client advancing <code>updated_at</code> past the server's clock on synced tables	known limitation; <code>touch_updated_at()</code>

22.3 Apple Health

CONDITION	BEHAVIOUR	PINNED BY
Read access denied	Undetectable by design: HealthKit reports only whether it asked, since revealing a refusal would leak that the user has no data of that type. The reader queries anyway, an empty result renders the empty state, and the settings row links out to Health rather than claiming a connection state	<code>HealthKitReader doc;</code> <code>HealthKitError.authorisationRefused</code> is only for the write request
iPhone and Watch both count steps	De-duplicated by <code>HKStatisticsCollectionQuery</code> . Raw samples are never summed	by design
Health corrects a sample after it was read	Observer queries fire on change rather than a timer polling; the sync window always re-reads today and the day of the last sync, because a night's sleep lands on the morning it ends	<code>HealthSyncWindowTests</code>
Thirty-one days of history missing	The window never goes further back than thirty days, the month the dot grid shows. Health keeps the data, so a longer backfill can be added when something needs it	<code>HealthSyncWindow.maximumDays</code>
Health and Whoop write the same day	<code>MetricsStore.upsert</code> merges; each source touches its own columns and <code>MetricArbiter</code> decides contested ones by rank	<code>MetricArbiterTests</code> , <code>HealthApplyTests</code>
Health sends a value for a metric a person typed	Manual rank 4 beats every automatic source; a value with no recorded source for weight or water is read as typed, so the upgrade to provenance cannot let a sync replace a hand-entered figure	<code>MetricArbiterTests</code> , <code>MetricSource.legacy(for:)</code>
Same source syncs again with a corrected value	Equal rank wins, because greater-or-equal is the comparison; a source may correct itself	<code>MetricArbiterTests</code>
Weight recorded in pounds	Read through <code>HKUnit</code> conversion, stored in kilograms	by design
Cycle tracking	Asked as a second, separate prompt, defaulted from Health's own sex characteristic, offered either way	<code>HealthKitReader.requestCycleAuthorization</code>
Simulator has no health data	Expected. The design preview fixtures and the checks harness drive every screen without Health	<code>HealthActivityDesignPreview</code>

CONDITION	BEHAVIOUR	PINNED BY
Seeded sample history on an old install	<code>SampleMetricsPurge</code> deletes every daily row once; workouts, sleep and Whoop records rebuild them on resync. Typed water and weight from before the purge are lost, and this is stated in the type's doc comment rather than hidden	<code>SampleMetricsPurge</code>

22.4 Whoop

CONDITION	BEHAVIOUR	PINNED BY
Access token expired	Refreshed before each sync through the <code>whoop-token</code> function; the account-owned credential row is claimed under a 30 second lock so two devices cannot rotate the same refresh token	<code>WhoopTokenTests</code> ; <code>claim_whoop_refresh</code>
Refresh refused (400 <code>invalid_grant</code> or 401)	Tokens cleared, "Whoop sign-in expired" in Settings. Any other refresh error keeps the tokens	<code>WhoopSyncError.reauthenticationRequired</code>
A collection endpoint answers 401 with a live token	Tokens are kept and the user is told to disconnect and reconnect to grant newer scopes. Clearing them here once made connect-then-sync an unbreakable loop	<code>WhoopSyncError.accessDenied</code> , <code>WhoopTests</code>
Rate limited (429)	Surfaced immediately; no retry loop	<code>WhoopTests</code>
Record not yet scored	<code>PENDING_SCORE</code> and <code>UNSCORABLE</code> records are archived but not derived; a later sync's re-derive picks them up	<code>WhoopTests</code>
Strap not worn	No recovery record; fields stay nil; the screen shows its empty state and charts show a gap	by design
Recovery still calibrating	<code>whoopRecoveryIsCalibrating</code> is stored and rendered with a marker in the digest; it is never compared unqualified	<code>WhoopArchiveTests</code> , <code>MetricsDigestTests</code>
Empty <code>zone_durations</code> object	Treated as no reading, not six zeros	<code>WhoopWireFormatTests</code>
Payload shape differs from the docs	Raw payloads are retained; <code>WhoopDerivation.rederive()</code> rebuilds everything with zero API calls. The custom date decoder exists because fractional seconds once broke stock ISO 8601 parsing on every record	<code>WhoopArchiveTests</code> , <code>WhoopWireFormatTests</code>
Both straps report the same metric	The primary wearable ranks 3, the other 2, for metrics a wearable claims; Health ranks 3 only for metrics it is authoritative on	<code>WhoopArbitrationTests</code> , <code>MetricSourceTests</code>
Several WHOOP straps broadcasting in a gym	Auto-pair only when exactly one WHOOP is discovered. Another person's strap is not a reading this person owns	<code>WhoopAutoPair</code>
Auto-reconnect at the start of a new session	Expected to read "Reconnecting to WHOOP" then "Live · WHOOP" with no tap	hardware, pending

22.5 Fitbit

CONDITION	BEHAVIOUR	PINNED BY
Two devices refresh at once	Fitbit rotates the refresh token on every use and the old one dies, so the credential lives in one server row and the refresh runs under a lease held in a column, not a transaction lock, because the exclusion must outlive an HTTP round trip. The loser sees <code>busy</code> and retries later	<code>fitbit_lock.sql</code> ; <code>fitbit_test.ts</code>
Refresh refused outright	<code>needs_reauth</code> ; the app shows the reconnect row	<code>FitbitSyncError.reauthenticationRequired</code>
Quota running low (150 requests an hour)	The function reads <code>Fitbit-Rate-Limit-Remaining</code> on every response and stops while it still holds headroom, returning a resume cursor. A 30 day backfill costs about 40 requests	<code>fitbit-sync/index.ts</code> ; <code>FitbitSyncTests</code>
Some collections land and some do not	The days that arrived are written; <code>partial(failed:)</code> names what is missing and the last-synced stamp is not written clean	<code>FitbitSyncTests</code>
A range request exceeds Fitbit's per-collection day cap	The function splits by each collection's own cap, because exceeding it is a 400 that names nothing useful	<code>fitbit-sync/index.ts</code>
Raw JSON needs re-deriving	Stored in <code>WhoopArchive</code> under <code>fitbit-</code> kinds keyed by day window, so a re-sync replaces rather than duplicates	<code>FitbitDerivationTests</code>

22.6 Banks and Plaid

CONDITION	BEHAVIOUR	PINNED BY
Ingest throws mid-page	The cursor does not move. Store the data, then move the cursor: a cursor advanced before a successful ingest is a page Plaid will never send again	<code>PlaidSyncTests</code> , assertions written so they cannot pass on an empty store
App reinstalled	The cursor lives in <code>UserDefaults</code> rather than the Keychain on purpose: the Keychain survives a reinstall and the database does not, and a cursor that outlived its database would replay nothing into an empty app	<code>PlaidItemStore</code> doc
Bank invalidates the stored login	<code>item_login_required</code> puts the institution in <code>needsReconnect</code> ; nothing about it is retried	<code>PlaidSyncTests</code> , <code>PlaidClientError.itemLoginRequired</code>
A bank is still sending its history	The item is remembered before anything else, so a connected bank still fetching is not mistaken for one never connected, and the screen says the history is on its way	<code>PlaidSync.run</code>
Same institution connected twice, concurrently	The function's check-then-insert is backstopped by a unique constraint on <code>(user_id, institution_id)</code>	<code>plaid_items.sql</code>
A link removed on another device	Items absent from the sync response are removed from the local cache	<code>PlaidSyncTests</code>
Plaid amount sign	Negated on the way in; positive is money in everywhere inside the app	<code>PlaidMappingTests</code>
Plaid date string	Parsed in the same calendar <code>MoneyStore</code> normalises with, so a transaction cannot land on the adjacent day	<code>PlaidMappingTests</code>
Transfers and card payments	Excluded from income and spending by the underscore-delimited prefix rule	<code>MoneyTests</code> , <code>PlaidCategoryTests</code>
Old rows have no merchant logo	Every item's cursor is cleared once, so the next sync replays history through the same keyed upsert. Not billed, because Plaid charges per connected item, not per sync call	<code>PlaidItemStore.resetCursors</code>
A bank's OAuth login redirects to a browser	The universal link <code>almanac.shivvyas.com/plaid-oauth</code> lands back in the app; the static site exists only to serve the association file with the right content type	<code>web/README.md</code> ; device only
Storage fails after the exchange	The live credential is never stranded: the function fails the exchange rather than leaving a token nobody can find	<code>plaid-exchange</code> , <code>commit 1dcafaf</code>

CONDITION	BEHAVIOUR	PINNED BY
An amount like 19.995	Rounded to the cent before splitting into whole and cents, never after	<code>MoneyPartsTests</code>

22.7 Calendar and EventKit

CONDITION	BEHAVIOUR	PINNED BY
A recurring series	Each occurrence is keyed by identifier plus occurrence start, because EventKit reuses one identifier across the series. Gated writes refuse recurring rows	<code>EventKitSourceIDTests</code> , <code>CalendarToolsTests</code>
A source throws during a sync pass	Only sources actually fetched are authoritative; rows for a source that threw are left untouched rather than treated as zero events	<code>CalendarStoreTests</code> , <code>CalendarSyncTests</code>
An event deleted in another app	Absent from the fetch inside the window, so deleted from the cache; rows outside the window are never touched	<code>CalendarStoreTests</code>
An event moves into the window from outside it	Matched by key across the whole cache, not only the window, so it updates in place and keeps its local id	<code>CalendarStoreTests</code>
An all-day event	Does not block free time and is never clamped to a minimum length	<code>CalendarFreeSlotTests</code>
Calendar permission granted on the device but not for this account	<code>AccountDeviceAccess.allowsCalendar</code> requires both	<code>AccountDeviceAccessTests</code>
A write lands	The provider is re-synced afterwards, because providers normalise all-day handling and timezones and trusting the local draft would drift	<code>CalendarSyncTests</code>
No writable source	<code>CalendarSyncError.noWritableSource</code> , surfaced, not retried	<code>CalendarSyncTests</code>

22.8 Sessions and accounts

CONDITION	BEHAVIOUR	PINNED BY
Access token expired at launch	<code>SessionRefresher</code> refreshes. A refusal (400, 401, 403, 422) clears the store and returns to sign-in; a transport error, 429 or 5xx keeps the session and the app keeps working	<code>SessionRefreshTests</code>
App kept open past the hour	<code>SessionKeepAlive</code> schedules a renewal 45 seconds before expiry, inside the refresher's own minute of slack, so friends, the cloud coach and notes sync do not all start failing with <code>JWT expired</code> at once	<code>SessionKeepAliveTests</code>
Refresh response is sparse	The new session carries forward the previous refresh token, user id, phone, email and <code>hasProfile</code>	<code>AuthSessionProfileTests</code>
Refresh returns a different user id	Cleared and rejected	<code>SessionRefreshTests</code>
Sign out	Stays signed out: the keychain slot is cleared, the device token row is deleted, and a stale refresh in flight cannot resurrect the session because the refresher checks the store still holds the token it started with	<code>SessionRefreshTests</code> ; commit 1ebc667
Two people on one device	Each account has its own store file and defaults suite; switching closes one and opens the other. A push token registered under B replaces A's row because the token is the primary key	<code>UserScopeTests</code> , <code>AccountStoreTests</code> ; <code>device_tokens</code>
Upgrade from the single-store build	The session in the old keychain slot is adopted into the roster and the legacy store is moved, once, into the first account that signs in	<code>AccountStoreTests</code> , <code>LifeOSContainer.adoptLegacyStore</code>
SMS provider fails	Twilio codes 21608, 21610, 30034 and 60410 map to sentences a person can act on; email is the default channel and any phone failure offers "Use email instead"	<code>otp_twilio_test.ts</code> , <code>AuthErrorMappingTests</code>
Wrong door taken (sign in versus sign up)	Harmless: the branch after verify routes on <code>AuthSession.hasProfile</code> , not on the door	<code>AuthSessionProfileTests</code>
OTP abuse	Hashed phone, prefix and IP events in <code>otp_events</code> ; a blocked attempt is logged and refused before Twilio is called	<code>otp_session_test.ts</code>
A schema field with no default	The container never opens and the shell waits on a store that never arrives. See 21.13	<code>SchemaMigrationTests</code>

22.9 Notes sync

CONDITION	BEHAVIOUR	PINNED BY
An edit made while a push is in flight	Push before pull, always; the synced mark never touches <code>updatedAt</code> , so the edit stays dirty and goes out on the next pass	<code>NotesStoreTests</code> , <code>NoteSyncIndexTests</code>
Same row changed on two devices	<code>updatedAt</code> decides and a tie goes to the local row, because ties come from the same device writing both	<code>NoteSync</code> doc, <code>NoteWireFormatTests</code>
A page deleted on one device	A tombstone, not a delete, so the other device can pull the fact rather than fail to observe an absence	<code>notes.sql</code> , <code>NotesStoreTests</code>
A child folder reaches the server before its parent	Allowed: <code>parent_id</code> is deliberately not a foreign key, because rows arrive in whatever order the client batches them	<code>notes.sql</code>
Several triggers fire on a cold start	Coalesced into one pass; a second caller joins the pass already running	<code>NoteSync</code>
Signed out	Notes work fully offline; a guest simply never pushes. Nil token is not an error	by design
Fresh install pulls a library	Pulled pages are indexed on arrival and stay out of the Inbox; the index migration marker is only spent when there was history to interpret	<code>NoteIndexMigrationTests</code> , <code>NoteSyncIndexTests</code>
A page's JSON cannot be decoded	The editor gets a blank document and the original bytes stay on disk until saved over	<code>NoteDocument.blocks</code>
A link points at a renamed or deleted page	<code>targetID</code> may dangle until the next launch's <code>rebuildAll</code> ; consumers treat a miss as survivable	<code>NoteIndexerTests</code>
Search typed quickly	One shelf per query, not one per keystroke	commit 2218d36
Access token captured at attach	Never: the token is read from the keychain per pass, because the shell refreshes it on launch and foreground and a captured one is stale within the hour	<code>RootView.attachAll</code>

22.10 Live sessions and sensors

These are pinned by the native checks harness rather than by `swift test`, because the recorder needs `HKWorkoutSession`, Bluetooth and ActivityKit. Launch with `--design-preview --page=checks` on a simulator; 34 of 34 pass as of 15 September 2026. The pure parts, zones, effort, capacity and the readout, are in `LiveEffortTests` and `LiveSessionReadoutTests`.

CONDITION	BEHAVIOUR	PINNED BY
Health access refused at start	The timer starts anyway; the session saves locally without claiming a Health save	checks: "Timer starts without Health access", "Finish saves locally without claiming a Health save"
Paused	Readings accrue nothing and the elapsed time holds; the first reading after resume credits nothing and the second accrues	checks: "Paused readings accrue nothing", "First reading after resume credits nothing"
App relaunched mid-session	The draft restores the timer, capacity, effort and the video being followed; a timer-only draft restores without Health recovery	checks: "Draft restores capacity and effort", "A restored draft keeps its video"
Save tapped twice	One workout record	check: "Repeated save creates one workout"
Account switched mid-session	Recording stops and late starts are rejected; another account cannot see the draft or the remembered sensor	checks: "Account transition stops recording", "Other account cannot see the draft"
No recovery reading today	Capacity falls back to Apple Health HRV against a week of baseline rows, then to a neutral default; the tile says which	<code>LiveEffortTests</code> , <code>CapacityInputs</code>
Readings arrive every second	The readout is recomputed every second, but the draft is written at most every ten seconds and the Live Activity publish is throttled to a three second minimum interval	<code>ActivityRecorder</code> doc; <code>LiveSessionReadoutTests</code>
A stale Live Activity card from a session nobody adopted	Ended; only one activity belongs on screen	<code>ActivityRecorder</code> , <code>WatchSessionBridge.endPlaceholder</code>
A freeform walk after a cancelled video start	Not stamped with the video: <code>following</code> belongs to the start that sets it, and every other start clears it	check: "A failed start does not stamp the next session"
Rep removed below zero	Floors at zero; an unknown count stays unknown rather than becoming 0	check: "Remove rep never goes below zero"
Lock-screen BPM updates while the phone stays locked for two minutes	Expected to keep updating; if it stops, add <code>workout-processing</code> to <code>UIBackgroundModes</code>	hardware, pending
Dynamic Island expanded presentation	Unverified; only the compact state was captured	pending capture

22.11 The watch hand-off

CONDITION	BEHAVIOUR	PINNED BY
Watch never answers	Ten seconds, shown as "Asking your Apple Watch...", then the phone runs the session itself	check: "A watch that does not answer falls back to the phone"
Health saving turned off	The watch is never offered the session, because the watch writes to Health	check: "Health saving off keeps the session on the phone"
Health refused on the phone	Falls through to the phone path; the first permission prompt is on the phone, never the wrist. A refused session sends <code>discard</code> so nothing reaches Health	watch report, C1
A packet with an unknown version	Dropped without changing state	check: "A packet with an unknown version changes nothing"; <code>WatchWireTests</code>
A packet arrives for a phone-owned session	Never reaches it	check: "A watch packet never reaches a phone session"
Person starts on both devices	The recorder keeps the phone session and discards the mirrored one with a notice; it never merges and never ends the wrist session, since ending means saving there	spec 9; <code>ActivityRecorder+Watch</code>
Automatic rep arrives after a manual +1	Manual and automatic reps are tracked separately and summed, so the tap is never overwritten	watch report, task 5
Mirrored session arrives with no recorder listening	The bridge requests the Live Activity itself and the recorder's timer adopts that id instead of asking for a second card	watch report, I3
Last command sent	The session is dropped in the send's completion handler, not the same run-loop turn, or the command is cancelled before the watch hears it	<code>WatchSessionBridge.end(after:)</code>
Watch app not in the foreground	<code>watchAvailable</code> is paired, activated and installed, never <code>isReachable</code> , which is true only while the watch app is already open	watch report, I1
Phone relaunched during a mirrored session	Packet-driven re-attach; whether <code>recoverActiveWorkoutSession</code> also returns it is unverified	hardware, pending
Rep accuracy on real movement	Unmeasured; constants are pinned by six synthetic tests and tuned from a hardware session	<code>RepCounterTests</code> ; hardware, pending

22.12 Video and the catalog

CONDITION	BEHAVIOUR	PINNED BY
Embed loaded straight into the web view	Error 153 on every video. The iframe is wrapped in a page of the app's own with an <code>origin</code> matching its <code>baseURL</code> , both on <code>youtube-nocookie.com</code> ; six of six seed ids then render	library report, embed spot-check
Owner has disabled embedding	YouTube's own card carries a "Watch on YouTube" button; the player also shows "Open in YouTube" when the page fails to load	<code>YouTubePlayerView</code>
A row's id is not a YouTube id	Validated against <code>^[A-Za-z0-9_-]{11}\$</code> before anything is interpolated; the row says it will not play here	<code>YouTubePlayerView</code> , <code>verify_lib_test.ts</code>
Phone rotates or full screen is toggled	The web view is owned by the screen, not the layout branch, so the video never restarts	<code>YouTubePlayerView</code> doc
Catalog refresh comes back empty	The cache is kept and the screen says the last downloaded one is showing. An empty answer is far more often an unseeded table or a policy that stopped matching than a library that emptied	<code>CatalogRefreshTests</code>
First open with no connection	Empty state with <code>wifi.slash</code> : the library needs a connection the first time	library report
A video deleted or privated upstream	Stays in the cache until the next refresh removes rows absent from the server	<code>CatalogStore.upsert</code>
Duplicate id in a response	Last row wins; no trap	<code>CatalogStore.upsert</code>
Catalog edited by hand	The verify script refuses an empty catalog, checks every id against <code>oEmbed</code> for title, channel and duration, and every seed ends with a delete naming every verified id, so a seed run is a full sync	<code>verify_lib_test.ts</code> (7 tests)
Explicit split chip chosen	Never widened away by the planner's fallback	<code>WorkoutLibraryFilterTests</code>
Saved chip	Shows saved rows regardless of today's plan, filtered only by the query	<code>WorkoutLibraryFilterTests.savedListIgnores</code>
Landscape overlay on iPhone	Unreachable today: the iPhone app is portrait-only; the overlay sits behind <code>VideoWorkoutLayout.landscapeOverlay</code>	spec 5.3; pending

22.13 LIFO and the cloud tier

CONDITION	BEHAVIOUR	PINNED BY
Daily token cap reached (150,000 tokens)	The function answers 429 <code>exhausted</code> before any call; the router falls back to the on-device model and the reply is marked <code>degraded</code> , never mentioning the allowance	<code>lifo_test.ts</code> , <code>RemoteEngineTests</code> , <code>CoachRouterTests</code>
Model refuses	403 <code>refused</code> with the model's words; surfaced plainly, never retried with a reworded prompt. A refusal still consumed tokens and is still debited	<code>lifo_test.ts</code> , <code>RemoteEngineTests</code>
Reply does not parse against the task's schema	502 <code>upstream_failure</code> , debited, surfaced as unavailable with retry	<code>lifo_test.ts</code> (<code>parseOutput</code>)
Provider rejects the tuned body (400)	Retried once without <code>reasoning_effort</code> and <code>verbosity</code> ; a 400 is the only status a smaller body can fix	<code>lifo_test.ts</code> (<code>withoutTuning</code>)
DNS, timeout, connection reset	No billable call happened, nothing debited, 502; the client shows retry	<code>lifo-agent/index.ts</code>
Stream abandoned half way	<code>flush</code> still runs when the body is cancelled, so a walked-away-from answer is still paid for	<code>streamToClient</code>
Provider omits usage	Zero tokens are never debited, but the path is logged; an unmetered path is a free one	<code>lifo-agent/index.ts</code>
Prompt too large for the on-device window	The digest drops whole days, oldest first, until the render fits an 8,000 character budget; if it still does not fit, <code>tooLarge</code> is surfaced as our fault, not the device's	<code>ContextBundleTests</code> , <code>MetricsDigestTests</code> , <code>CoachRouterTests</code>
On-device model unavailable	Three reasons distinguished: not eligible (resolved once), Apple Intelligence off (re-checked on foreground), model not ready (re-checked per request)	<code>ModelAvailabilityTests</code>
On-device error	<code>EscalationPolicy</code> maps each case: context window exceeded escalates, assets unavailable and rate limited retry locally, decoding failure retries then escalates, refusal surfaces, unsupported guide is a programmer error	<code>EscalationPolicyTests</code>
Guest (no session)	<code>notSignedIn</code> ; the cloud tier is never called and the screen offers a sign-in door	<code>RemoteEngineTests</code>
A thread that writes its own system message	Refused by the server before the model sees it	<code>lifo_test.ts</code> (<code>parseRequest</code>)

CONDITION	BEHAVIOUR	PINNED BY
A tool the model asks to run twice	Tool execution is capped per turn and gated writes suspend until the card is answered; nothing the model emits can reach the call itself	<code>ToolInvokerTests</code> , <code>ToolGate</code>
Cloud write confirmed, then the device retried	Never: once a write has been confirmed the turn does not fall back to the device	<code>commit 759fd65</code> , <code>RemoteChatEngineTests</code>
Brief regenerating on every open	One brief per calendar day through <code>BriefCache</code> , with an in-flight map so two callers for the same day share one generation	<code>BriefCacheTests</code>
The two conversational prompts drift apart	A Deno test and a Swift test each pin the same prohibitions and permissions in <code>ResponseStyle.conversation</code> and the server's <code>CONVERSATION</code>	<code>ResponseStyleTests</code> , <code>lifo_test.ts</code>
Microphone opened in a loud room	The endpointer learns the noise floor from non-voice frames, climbs slowly and drops at once; a backstop ends the turn when every frame reads as voice, and an opened mic with nothing said closes itself	<code>SpeechEndpointerTests</code>
Speech reply over 1,200 characters	Clipped before it is sent, because speech is billed per character	<code>ElevenLabsVoiceClient.maxCharacters</code> , <code>SpokenReplyTests</code>

22.14 Nudges and push

CONDITION	BEHAVIOUR	PINNED BY
Two cron runs overlap	The job inserts into <code>nudge_log</code> before it sends; the primary key <code>(user_id, day)</code> makes the second run conflict and stop	<code>proactive_nudges.sql</code>
Same trigger fires again on Thursday	Seven day per-trigger cooldown	<code>nudge_test.ts</code>
8am in Auckland, midnight in Los Angeles	Hourly schedule; each run selects only users whose local clock has just reached the send hour, and nothing sends outside 7 to 21 local	<code>nudge_test.ts</code> , <code>schedule.sql</code>
OpenAI unreachable	The fired trigger's own fallback sentence is delivered; an outage never swallows an observation	<code>nudge_test.ts</code>
Nudge budget spent (4,000 tokens)	Fallback sentence; a heavy chat evening cannot eat the next morning's nudge because chat and nudges are separate ledger rows	<code>lifo_debit(uuid, bigint, text)</code>
Phone off for three days	One collapse id for the channel and a 12 hour expiry, so it gets today's nudge or none, never three	<code>apns_test.ts</code>
APNs provider token minted too often	One ES256 token reused for fifty minutes; Apple throttles a sender that mints more than once every twenty minutes	<code>apns_test.ts</code>
Wrong APNs host for the build	400 <code>BadDeviceToken</code> on every send, which is why <code>APNS_SANDBOX</code> is configuration rather than a guess	<code>apns.ts</code>
APNs not configured	The job returns <code>apns_not_configured</code> and sends nothing	<code>lifo-nudge/index.ts</code>
Permission prompt timing	Never at launch; asked after the app has shown something worth being interrupted about	<code>PushService</code> doc
Token arrives after the sign-in that asked for it	Kept in defaults as well as memory so sign-out can delete the row after a relaunch	<code>PushTokenTests</code>

22.15 Social

CONDITION	BEHAVIOUR	PINNED BY
A slow search answers after a newer one	The newer search can never lose to it	<code>FriendsViewModel</code> , commit 8e2d616
A send fails	The row is never lost; <code>client_id</code> is unique per sender so a retry cannot duplicate it	<code>social_groups.sql</code> , <code>SocialAPITests</code>
PostgREST refuses a request	The server's sentence is carried through, because an expired token, a missing grant and a malformed filter are otherwise the same refusal	<code>SocialAPITests</code> , <code>SharedStatsClientTests</code>
Stats sharing off	Nulls, not zeros, in <code>profile_stats</code> ; the screen omits what it does not know	<code>shared_stats.sql</code>
A group invitation	Grants nothing until accepted; chat and leaderboards need <code>accepted</code>	<code>social_groups.test.sql</code>
Wellness scores	Need their own consent flag; streak sharing does not silently expand to HRV-derived scores, and a score without a source is unrepresentable	<code>group_wellness.test.sql</code>
Avatar replaced on another device	Three-case lookup of the object's timestamp; "no object" and "no timestamp" want opposite answers, so they are not collapsed into one nil	<code>ProfileClientTests</code>
Profile never reached the server	Not lost; retried on the next foreground	commit b2bc7cc

22.16 UI and data display

CONDITION	BEHAVIOUR	PINNED BY
Metric missing	A dash, never 0, rendered by the tile from a <code>String?</code>	<code>MetricTile</code>
Goal changed retroactively	All history re-evaluates on the next load; the grid measures against your current standard	<code>GoalEvaluationTests</code>
Streak across a no-data gap	The gap is transparent	<code>StreakTests</code>
Weight delta with fewer than seven days	Suppressed rather than computed from a short window	<code>TrendSeriesTests</code>
A font declared outside the scale	<code>scripts/check-typography.sh</code> fails the run; the app once carried 376 hand-written font calls across 28 sizes	script
Dark mode	Every hue has a dark variant; no colour is defined only inside a scheme branch	<code>TokensTests</code>
A metric with no readings	Averages are nil rather than zero when nothing contributed	<code>MetricsDigestTests</code>
Cents in a money row	Whole and cents split in the kit, two digits always	<code>MoneyPartsTests</code>
Donut categories	Top five plus Other, one ink at stepped opacity; identity is never colour alone	<code>SpendSeriesTests</code>
iPad rail overlapping the pane	The rail's real width is reserved; it withdraws while the pane is worked with and returns when that stops	<code>LayoutMetricsTests</code>
Sector with no evidence	No band and no score; the board's "decided" mean is over sectors with a band only, so silence never reads as certainty	<code>BoardSummaryTests</code> , <code>SectorBandTests</code>
Coast projection for Money	Continuing at the current pace, not an empty tail, because spending nothing is the best case and an empty tail would put the floor above the ceiling	<code>ProjectedInputsTests</code>

22.17 Study notes

READ IN THIS ORDER

1. `LifeOSKit/Sources/Integrations/SessionRefresh.swift` , the clearest statement of "destroy only on a definitive refusal".
2. `SyncStalenessPolicy.swift` and `SessionKeepAlive.swift` , two pure clocks.
3. `MetricArbiter.swift` and `MetricSource.swift` , then the tests beside them.
4. `PlaidSync.swift` , for store-then-move-cursor.
5. `supabase/functions/lifo-agent/index.ts` , for every failure path a cloud call has.
6. `supabase/functions/_shared/nudge.ts` and `apns.ts` , for the fallback and collapse rules.

7. `LIfе0S/Features/Activity/ViewModel/ActivityRecorderChecks.swift` , the native checks that stand in for tests.
8. `docs/design/watch-companion/report.md` , for what is still unverified.

CHECK YOURSELF

- **Which HTTP statuses sign a person out, and why not 429?** 400, 401, 403 and 422 are the server refusing the credential; 429 and 5xx are the server failing to answer.
- **Why does the Plaid cursor live in UserDefaults and not the Keychain?** The Keychain survives a reinstall and the database does not; a surviving cursor would replay nothing into an empty app.
- **What happens to an empty catalog response?** The cache is kept; an empty answer is believed only when there was nothing to lose.
- **Why is a refused model reply still debited?** It consumed real tokens; an unmetered path would let off-topic questions burn budget for free.
- **How can two cron runs not double-send?** The log insert precedes the send and the primary key makes the second insert conflict.
- **What does a watch hand-off do when Health saving is off?** It is never offered; the watch writes to Health, so the phone keeps the session.

23 Operations

How to build the app from a clean machine, what has to exist on the server side before it signs anyone in, how a build reaches TestFlight, and how the code is kept honest: 1,298 package tests, 100 Deno tests, 7 catalog tests, a native checks harness, a typography guard and a way of working that survives six sessions editing one checkout.

23.1 Setup runbook

1 · LOCAL BUILD

```
git clone https://github.com/shivvyas2/LifeOS.git
cd LifeOS
cp Config/Secrets.example.xcconfig Config/Secrets.xcconfig
swift test --package-path LifeOSKit      # no simulator needed
open LIfеOS.xcodeproj
```

The package declares an iOS 26 platform and also builds for macOS purely so that `swift test` runs from the terminal. The Xcode project holds four targets: `LIfеOS` (the phone and iPad app, still spelled with the capital I because renaming a target churns the project file for no user-visible gain), `AlmanacWatch`, `AlmanacWatchWidgets` and `AlmanacWidgets`. The display name under the icon is Almanac; the bundle id is `com.shivvyas.lifeos`; the team is `Z42YU5W6WY`. The app's folders under `LIfеOS/` are picked up automatically, as described in Chapter 2.

A MISSING SECRETS.XCCONFIG DOES NOT FAIL THE BUILD

xcconfig substitution of a missing variable yields an empty string. A clean checkout builds, archives and launches, then shows a sign-in screen that can never sign anyone in. Confirm the file exists and is populated before archiving on any machine that is not the usual one.

2 · SUPABASE

```
supabase link --project-ref <ref>
supabase db push                                # 18 migrations: schema, RLS, functions, seed
supabase config push                            # auth: email OTP via Resend, SMS, Apple, templates
supabase secrets set \
  OPENAI_API_KEY=... \
  WHOOP_CLIENT_ID=... WHOOP_CLIENT_SECRET=... \
  FITBIT_CLIENT_ID=... FITBIT_CLIENT_SECRET=... \
  PLAID_CLIENT_ID=... PLAID_SECRET=... PLAID_ENV=sandbox \
  PLAID_REDIRECT_URI=https://almanac.shivvyas.com/plaid-oauth \
  TWILIO_ACCOUNT_SID=... TWILIO_AUTH_TOKEN=... TWILIO_VERIFY_SERVICE_SID=... \
  OTP_PEPPER=... \
  APNS_KEY_ID=... APNS_TEAM_ID=Z42YU5W6WY APNS_BUNDLE_ID=com.shivvyas.lifeos \
  APNS_KEY="$(cat AuthKey.p8)" APNS_SANDBOX=true
supabase functions deploy
```

Then put the project URL and anon key into `Config/Secrets.xcconfig`. Thirteen Edge Functions deploy from `supabase/functions/`: `otp-start`, `otp-check`, `whoop-callback`, `whoop-token`, `fitbit-token`, `fitbit-sync`, `plaid-link-token`, `plaid-exchange`, `plaid-sync`, `plaid-disconnect`, `lifo-agent`, `lifo-nudge`, and the shared modules under `_shared/`. Two of them declare `verify_jwt = false` in `config.toml`: `whoop-callback`, because a browser following Whoop's 302 carries no Authorization header and cannot be made to, and `whoop-token`, which validates the bearer itself with `resolveUser` before touching account data. Stating it in config rather than passing `--no-verify-jwt` means a deploy from anywhere gets it right.

Two server-side steps cannot live in a migration. The hourly nudge schedule needs the project's own function URL and the service role key, so `supabase/functions/lifo-nudge/schedule.sql` is run once from the SQL editor after storing both in Vault. And the auth email templates must be pushed, because Supabase's stock templates send a magic link and the app's sign-in is a six-digit code that has to be typable.

3 · PROVIDER DEVELOPER ACCOUNTS

PROVIDER	WHAT TO REGISTER	WHERE THE PIECES GO
Whoop	An app at <code>developer.whoop.com</code> (needs an active membership). Redirect URI is the <code>https://whoop-callback</code> function, which bridges to the app's own scheme; the app answers to both <code>lifeos://</code> and <code>com.shivvyas.almanac.whoop</code> . Request every scope listed in <code>WhoopCollection</code> including <code>offline</code>	Client id in the <code>xcconfig</code> ; secret in function env
Fitbit	An app with a custom-scheme redirect, <code>lifeos://fitbit-callback</code> . Fitbit permits a custom scheme for a native app, so no bridge function	Client id in the <code>xcconfig</code> ; secret in function env; tokens live server-side under a lease
Plaid	Products Transactions and Accounts. Allowed redirect URI <code>https://almanac.shivvyas.com/plaid-oauth</code> . Plaid refuses to mint a link token carrying a redirect it has not been shown, so register this before setting <code>PLAID_REDIRECT_URI</code>	Client id and secret in function env only; no Plaid call ever originates on the device
The universal link site	The <code>web/</code> folder deploys to Vercel as project <code>almanac-links</code> ; point <code>almanac.shivvyas.com</code> at it and add the domain. <code>vercel.json</code> forces <code>application/json</code> on the association file, without which iOS ignores it and every OAuth bank silently falls back to Safari	Check with <code>curl -sI</code> : wants 200 and that content type
OpenAI	A key with access to <code>gpt-5-mini</code> . The model name is in <code>_shared/lifo.ts</code> , so a model change is a function deploy	Function env only
ElevenLabs	A key for Scribe (speech to text) and Flash v2.5 (text to speech)	Today in the <code>xcconfig</code> and therefore in the bundle; moving it behind an Edge Function and rotating it is item 2 of the privacy plan in Appendix A
Twilio Verify	A Verify service. The committed config enables SMS with placeholder SIDs, and every number in <code>[auth.sms.test_otp]</code> short-circuits before Twilio is called, so the flow is fully exercisable with no account	Function env; real numbers start working when real credentials replace the zeros
Resend	SMTP for the email code; <code>admin_email</code> is required in config whenever SMTP is enabled	<code>config.toml</code> with <code>RESEND_API_KEY</code> in the environment when pushing
APNs	A <code>.p8</code> key with the Push Notifications capability. Sandbox host for development builds, production for TestFlight; the wrong one answers 400 <code>BadDeviceToken</code> on every send	Function env; the sandbox flag is configuration, not a guess

4 · THE WORKOUT CATALOG

```
deno run --allow-net --allow-read --allow-write scripts/catalog/verify.ts \
  --out supabase/migrations/<stamp>_workout_videos_seed.sql
supabase db push
```

Curation is an edit to `scripts/catalog/catalog.json` plus a verify run. The script checks every id against YouTube's keyless oEmbed endpoint for title, channel and plausible duration, refuses an empty catalog, and

writes a seed whose last statement deletes every row not in the verified list, so a seed run is a full sync. The first seed is a migration Supabase has already recorded and will never re-run, which is why later passes write a new timestamped file with `--out`. The catalog ships at 122 verified videos.

5 · DEVICE

HealthKit background delivery, the Whoop and Fitbit OAuth round trips, the Plaid universal link, a real `HKWorkoutSession`, the watch hand-off and push delivery cannot be verified in the simulator. All need a physical device with a signing team, and the watch checks need a paired watch. The list of what is still pending on hardware is in Appendix A.

23.2 Secrets

There is no runtime `.env` on iOS. The app is a signed bundle; anything shipped inside it is extractable. The governing rule is therefore: **what must never leak lives server-side.**

TIER	LOCATION	CONTENTS	IN GIT?
1 · Build config	<code>Config/Secrets.xcconfig</code>	Supabase URL and anon key, Whoop client id and redirect, Plaid redirect, Fitbit client id and redirect, ElevenLabs key	Gitignored. Template committed as <code>Secrets.example.xcconfig</code>
2 · Runtime	Keychain, per account slot	Supabase session (access and refresh token), Whoop tokens	Never
2b · Runtime	<code>UserDefaults</code> , per account suite	Plaid sync cursors, connection state, tour and migration markers, the APNs token	Never; dies with the app container on purpose
3 · Server	Edge Function environment	<code>SUPABASE_SERVICE_ROLE_KEY</code> , <code>OPENAI_API_KEY</code> , <code>WHOOO_CLIENT_SECRET</code> , <code>FITBIT_CLIENT_SECRET</code> , <code>PLAID_SECRET</code> , Twilio credentials, <code>OTP_PEPPER</code> , <code>APNS_KEY</code>	Never. Never on the device either
3b · Database	Tables with no client policies	<code>plaid_items.access_token</code> , <code>fitbit_connections</code> tokens, <code>whoop_connections</code> tokens, <code>lifo_usage</code> , <code>otp_events</code>	Readable only by the service role from inside a function

Nothing in tier 1 is truly secret except the ElevenLabs key. The anon key is designed to ship in clients and is safe precisely because RLS gates every row; the OAuth client ids are public by design. The xcconfig has one trap: `//` starts a comment, so every URL in it breaks the scheme separator with `$()` or the value truncates to `lifeos:`.

SERVICE_ROLE IS THE ONE THAT MATTERS

It bypasses RLS entirely. It belongs only in an Edge Function environment and in Vault for the cron schedule: never in an xcconfig, never in the app target, never in git. Every other secret in this system is recoverable by rotation; this one grants total read and write access to everything, including three tables of plaintext bank and wearable credentials that are an accepted single-user trade and must be encrypted before a second person's data lands in them.

23.3 Shipping to TestFlight

Everything the repo can settle is settled: display name, bundle id, team, iOS 26 floor, export compliance declared exempt. The parts that need an Apple ID are in `docs/handbook/testflight-release.md`; the two that bite are here.

Bump the build number first, and commit it. App Store Connect rejects a `(version, build)` pair it has already seen, and it rejects it after the upload finishes. `CURRENT_PROJECT_VERSION` is 17 today, in both Debug and Release; raise it for every upload, even a re-upload, and a rejected upload still burns the number. Through build 15 the number in git never moved off 1 because each archive was made from an uncommitted edit, so nothing in the repository could answer what was uploaded last. Committing it is what made the guess go away.

Then: destination Any iOS Device, Product → Archive, Distribute App → TestFlight, accept automatic signing (the Apple Distribution identity, not the Apple Development one a local archive uses). Internal testers get the build with no review. Two warnings are not failures: the missing dSYM for `LinkKit.framework` is Plaid shipping no debug symbols in any slice, and must not be silenced by stripping the app's own symbols; and the first Live Activity request on a fresh install raises the system's own consent alert.

The app icon is rendered from `docs/brand/lifeos-icon.svg` with headless Chrome at 1024 pixels. The original artwork clipped to a squircle and left the corners transparent; iOS applies its own mask and rejects an icon with an alpha channel at validation, so the shipping SVG lets the gradient run to the edges. Check `hasAlpha` reads `no` with `sips` before committing.

23.4 Testing strategy

Logic that can be tested without a simulator lives in the package and is developed test-first. Feature view models are exercised through the native checks harness against the real store, because the Xcode project still has no test target. SwiftUI view bodies are verified by design-preview captures and a build, because asserting on view structure is ceremony that breaks on every layout change without catching real defects.

The counts below are from runs on 15 September 2026, on this machine, against the tip of `main`:


```
$ swift test --package-path LifeOSKit 2>&1 | tail -2
Test run with 1298 tests in 173 suites passed after 4.589 seconds.

$ cd supabase/functions && deno test --allow-all 2>&1 | tail -2
ok | 100 passed | 0 failed (1s)

$ cd scripts/catalog && deno test --allow-all 2>&1 | tail -2
ok | 7 passed | 0 failed (42ms)
```

TARGET	WHAT IS TESTED	HOW
PersistenceTests	Goal evaluation, streaks, anomaly baselines, upsert invariants, money rollup and cents, buckets, plan ticks, calendar windows and free slots, chat recency, the whole notes store, block editor and parser, indexer and index migration, sector store, catalog and bookmarks, user scopes, the schema default rule	Swift Testing over an in-memory ModelContainer ; a fixed calendar injected
IntegrationsTests	Whoop wire format, pagination, arbitration and tokens; Fitbit wire format, derivation, OAuth and sync; Plaid client, mapping, item store and sync; HealthKit apply and window; the metric arbiter; calendar merge and sync; note wire format and sync indexing; auth error mapping, session refresh, keep-alive, profiles, social, groups, shared stats, push tokens; live effort; staleness	Recorded payload fixtures; injected URLProtocol stubs and fake APIs; no network
InsightsTests	Routing and escalation, the digest and context bundle under budget, brief cache, coach tasks and tools, tool schema and invoker, chat wire and the remote chat engine round loop, response style, spoken reply, speech endpointer and capture plan, spend series, recurring spend, model availability	No model weights, no network; scripted streams
SectorsTests	Every scorer, evidence factory, check-in, close schedule and progress, month windows and inputs, projections, levers and leverage, bands, history and observations, budget periods, the workout planner and filter, catalog refresh	Pure functions; deterministic fixtures
AssistantTests	The six calendar tools against a fake calendar and the assistant context	Fake CalendarWriting
DesignSystemTests	Tokens and dark variants, layout metrics, month grid maths, trend series, bar chart geometry, sleep composition, sign-in chrome, the scanline slicer	Swift Testing, fixed UTC calendar
AppSurfacesTests , MotionTests	The readout the widget decodes, HUD placement, ring maths, watch wire packets; the rep counter's six synthetic signals	Foundation only, so both link into the watch
Deno, supabase/functions	OTP session and Twilio classification, Plaid helpers, Fitbit quota and lease arithmetic, LIFO task config, budget cutoff, SSE framing, stream chunks, refusals, request parsing, the nudge triggers and fallbacks, APNs token and payload rules	Pure functions behind injected fetch fakes
Deno, scripts/catalog	Catalog parsing, title and channel matching, duration plausibility, seed SQL, the empty-catalog refusal	Pure
SQL, supabase/tests	Group membership and wellness leaderboard policies	pgTAP-style assertions against the local database
Native checks	The recorder, sensors, drafts, watch hand-off and video stamping	--design-preview --page=checks on a simulator; 34 of 34

TARGET	WHAT IS TESTED	HOW
Views	Both themes, every hue, empty and live states	Design-preview captures in <code>docs/design/</code> , reviewed in each slice's QA report
Integrations on hardware	Background delivery, OAuth round trips, real sensors, watch, push	Manual, physical device

THE ONE TEST THAT MATTERS MOST

For every screen, an empty store must produce a snapshot whose readings are all nil. It is a four-line test per feature and it is the regression guard against the single worst bug this app can have: a confident zero where there is no data.

Two habits keep the suite useful. Assertions are written so they cannot pass for the wrong reason: the Plaid cursor tests were rewritten so the "cursor did not move" assertion fails on an empty store, and the money tests dropped a stale comment that described a case they did not cover. And when a check fails during a slice, the rule is fix the cause, never the check: the strength-session check that regressed "Repeated save creates one workout" was fixed by deleting the row it left in the shared context, not by relaxing the assertion.

23.5 The design preview and native checks harness

Since 14 September 2026 the app has a DEBUG-only launch argument, `--design-preview` , that mounts a fixture instead of the signed-in shell. It exists because the recorder, the sensors and the Live Activity cannot be reached by `swift test` , and because every design QA capture needs the same screen in the same state on every run, with no account loaded.

ARGUMENT	EFFECT
<code>--page=gallery</code>	The default: a gallery of surfaces
<code>--page=activity</code>	Begin Activity over <code>HealthActivityFixture</code> , a fixed WHOOP recovery of 82 percent and a live reading of 152 bpm
<code>--page=player --video=<id></code>	The video player on one catalog id, used for the embed spot-check
<code>--page=checks</code>	Runs <code>ActivityRecorderChecks</code> and prints one PASS or FAIL line per check to the console
<code>--page=inbox</code> , <code>--page=settings</code>	The notification inbox and the settings screen
<code>--live</code> , <code>--strength</code> , <code>--dark</code> , <code>--live-activity</code> , <code>--hud-collapsed</code> , <code>--hud-expanded</code>	Drive the in-session fixture: a running session, a strength activity with reps, dark mode, a real Live Activity request, the rings folded to a bar

The checks are the recorder's test suite. Each is a named closure that builds a recorder over an in-memory store with injected answers for Health authorization, the watch bridge and the age, drives it, and asserts. They cover the paths a unit test cannot: a timer starting without Health access, paused readings accruing nothing, a draft restoring after relaunch, a second account being unable to see the first's draft, a watch that never

answers, a packet with an unknown version. The rule that applies to them is the same as for tests: fix the cause, never the check.

23.6 Driving the simulator from the terminal

`idb` is not installed and `simctl` cannot tap, so UI automation goes through clicks at absolute screen coordinates with `cliclick`, and only while the Simulator is frontmost. The recipe that works, run before every click:

```
osascript -e 'tell application "System Events" to set frontmost of process "Simulator" to true'
sleep 1
cliclick -w 150 c:X,Y
```

Coordinates come from the Simulator's own geometry: the device screen is drawn into the first `AXGroup` of the window, so read its position and size through System Events and scale a pixel measured on the full-resolution screenshot (1206 by 2622 for an iPhone 17) into it. Re-read the group after every boot or window move, because the origin changes. Screenshot after every tap rather than firing three and checking at the end, since a tap during a transition is swallowed. The simulator keychain survives `simctl uninstall`, so a reinstall skips onboarding; `xcrun simctl keychain <sim> reset` clears it while the device is booted. Never tap within a hundred pixels of a Plaid entry point without a screenshot confirming what is under the point: on 1 September 2026 a mapped tap landed seventy pixels high, opened Plaid Link and sent a real login code. Build destinations must be iOS 26 runtimes; check `xcodebuild -showdestinations`, not `simctl list devices`.

23.7 Working in parallel

Five or six sessions routinely run against the one checkout at `/Users/shivvyas/LifeOS`, switching branches and pushing under each other, so `HEAD` in the shared checkout can move mid-task. The conventions that make this survivable:

- **Never commit to main.** A branch per logical change: `feat/`, `fix/`, `docs/`, `chore/`. Branches are reviewed and merged by hand; unrelated changes mixed into one commit or dumped on main make that impossible.
- **Conventional commits.** `type(scope): imperative summary` with a short body explaining what and why, matching the history. No em dashes anywhere in a message, and no attribution trailers.
- **A private worktree for any multi-commit task.** `git worktree add .claude/worktrees/<name>`, then copy the untracked `Config/Secrets.xcconfig` in, because the app will not sign in without it. Never under a temporary directory that a tool may clear mid-task. Re-read `origin/main` immediately before every push.
- **Each slice leaves a paper trail.** A spec under `docs/superpowers/specs/`, a plan under `docs/superpowers/plans/`, and a QA report under `docs/design/<slice>/report.md` with the captures beside it. The report records every deviation from the plan and every check still pending on hardware.

23.8 Guards that run without a test

`scripts/check-typography.sh` greps the app and package sources for any font declared outside `LifeOSType`: a literal point size, a UIKit `systemFont(ofSize:)`, a rounded or serif design, or one of Apple's semantic text styles. The app once carried 376 hand-written font calls across 28 sizes because nothing stopped a screen inventing its own. Adding a step is a deliberate edit to `Typography.swift`, not something that happens by accident on a Tuesday. It exits non-zero on any hit.

The schema test in 21.13 is the other guard worth knowing by name. When you add a field to an existing model, add its name to the list there; the test reads the schema itself rather than the source, so it catches a default removed by a refactor as well as one never written.

23.9 Study notes

READ IN THIS ORDER

1. `Config/Secrets.example.xcconfig`, every key annotated with where its real secret lives.
2. `supabase/config.toml`, the auth and function blocks.
3. `docs/handbook/testflight-release.md`.
4. `LIfiOS/App/LIfiOSApp.swift` and `App/Surfaces/SurfaceDesignPreview.swift`, for how `--design-preview` takes over the scene.
5. `LIfiOS/Features/Activity/ViewModel/ActivityRecorderChecks.swift`.
6. `scripts/check-typography.sh` and `scripts/catalog/verify.ts`.
7. `LIfiOSKit/Package.swift`, for the eight targets and their dependency edges.

CHECK YOURSELF

- **Why does a clean checkout build without secrets?** `xcconfig` substitution of a missing variable is an empty string, not an error.
- **Why does `whoop-callback` skip JWT verification?** A browser following Whoop's redirect carries no bearer; the function holds no secret and only redirects into an app that checks PKCE and state itself.
- **What must happen before every TestFlight upload?** Raise `CURRENT_PROJECT_VERSION` in both configurations and commit it.
- **Where do the recorder's tests run?** On a simulator, through `--design-preview --page=checks`, because they need HealthKit, Bluetooth and ActivityKit.
- **Why a worktree under `.claude/worktrees` and not a temporary directory?** A temporary directory can be cleared mid-task; the worktrees directory persists and is where every other session puts its own.

24 Cost and the allowance

Almanac ships free, so every user costs money and none of them pay it. That single fact caps the architecture. This chapter says where the money goes, which lines are metered by a constraint in the database rather than by good intentions, and what the app costs to run today.

24.1 The ceiling

Total running cost must stay under roughly **two dollars per user per calendar month**, covering cloud inference, speech, bank data, the database, and notifications. There is no revenue per user, so this comes out of pocket and unit economics decide what can be built. The number was fixed on 25 August 2026 and nothing since has moved it.

The requirement is that spending *cannot* exceed the allowance, not that it usually does not. Those are different engineering problems, and only the first survives concurrency and a looping client. So the lines that scale with use (cloud tokens) are capped in Postgres, the lines that scale with connections (Plaid) are bounded by a count the person controls, and the lines that scale with nothing (Supabase, APNs, Vercel, YouTube) are on free tiers whose limits are named below.

THE ALLOWANCE IS INTERNAL

Users are never shown a balance, a meter, a percentage, or a dollar figure. It exists to bound what the app costs to run, not to be a feature. The only moment it becomes visible is when it runs out, and then it is one sentence: `exhausted` comes back as a 429 from `lifo-agent`, and Chapter 14 shows how the coach screen turns that into words.

24.2 Where the money goes

Every line below is either metered, bounded, or free. Nothing in the app calls a paid service without one of the three applying.

LINE	PROVIDER	SCALES WITH	HOW IT IS BOUNDED	WHAT LEAVES THE DEVICE
Coach chat, answers, sector notes	OpenAI <code>gpt-5-mini</code> through <code>lifo-agent</code>	Turns per user	150,000 tokens per user per day, kind <code>chat</code>	The off-device render of <code>ContextBundle</code> : derived numbers, never raw series
Morning nudge	OpenAI <code>gpt-5-mini</code> through <code>lifo-nudge</code>	One send per user per day	4,000 tokens per user per day, kind <code>nudge</code> ; a fallback sentence when spent	The trigger's few figures; about 300 tokens in, 60 out
Speech in (Scribe) and out (Flash v2.5)	ElevenLabs, called from the app	Spoken turns, characters spoken	Voice off by default; replies capped at 1,200 characters; key rotation is the only kill switch today	The recorded WAV of a question; the text of a reply
Bank sync	Plaid, four Edge Functions	Connected Items per user per month	One Item per bank; a failed exchange removes the Item so nothing bills for a row that was never stored	Nothing; transactions come back to the phone and stay there
Sign-in codes	Resend (email), Twilio Verify (SMS)	Sign-ins	Email is the default and free; SMS only for US and India numbers, 45 s guard, pumping-prefix limits	An address or number, hashed with a pepper in <code>otp_events</code>
Database, auth, functions, storage	Supabase	Rows, function invocations, avatar bytes	Free tier; see 24.4 for the limits that would push it over	Notes, profile, avatar, friends, messages, shared stats, tokens
Push	APNs	Sends	Free; one send per user per day by design	A sentence, a trigger name, a day
Universal link site	Vercel	Requests	Free hobby tier; two static files	Nothing
Workout videos	YouTube embeds	Plays	Free; the catalog is 122 curated ids in a table, no API key	The video id and the app's embed origin
Wearables and Health	Whoop and Fitbit APIs, HealthKit	Syncs	Free at the developer tier; rate-limited, not billed	An OAuth token; readings come back to the phone

Three of these lines are the whole cost story: cloud inference, speech, and Plaid. The rest round to zero at the scale of a personal app with a handful of friends on it.

24.3 The token ledger

The cloud tier is metered by one table and one function. `lifo_usage` holds one row per user, per day, per kind; `lifo_debit` adds to it in a single statement. Both live in `supabase/migrations/20260827090000_lifo_usage.sql`, with the `kind` column added by `20260830090000_proactive_nudges.sql` when the nudge got its own allowance.


```

-- supabase/migrations/20260830090000_proactive_nudges.sql
alter table public.lifo_usage
  add column kind text not null default 'chat';
alter table public.lifo_usage drop constraint lifo_usage_pkey;
alter table public.lifo_usage add primary key (user_id, day, kind);

create function public.lifo_debit(p_user uuid, p_tokens bigint, p_kind text default 'chat')
returns bigint language sql security definer set search_path = public as $$
  insert into lifo_usage (user_id, day, kind, tokens)
  values (p_user, current_date, p_kind, p_tokens)
  on conflict (user_id, day, kind)
  do update set tokens = lifo_usage.tokens + excluded.tokens
  returning tokens;
$$;

revoke execute on function public.lifo_debit from public, anon, authenticated;
grant execute on function public.lifo_debit(uuid, bigint, text) to service_role;

```

Three things in that SQL carry the cost model. The upsert-and-add is one statement, so two concurrent turns cannot both read the same balance and each write their own. The table has RLS enabled with zero policies, so no client can read or forge its own usage; the service role inside an Edge Function is the only writer and reader. And the function's execute grant is revoked from every client role, so even a definer function cannot be called from the app.

The caps themselves are constants in the function, not in the schema, so changing one is a deploy rather than a migration:

```

// supabase/functions/_shared/lifo.ts
export const DAILY_TOKEN_CAP = 150_000;
/// The nudge's own daily allowance, separate from DAILY_TOKEN_CAP.
export const NUDGE_TOKEN_CAP = 4_000;

export const MAX_CHAT_MESSAGES = 64;
export const MAX_CHAT_TOOLS = 16;
export const MAX_CONTEXT_LENGTH = 32_000;

```

The door in `lifo-agent/index.ts` reads the day's `chat` row before calling the provider and refuses with `exhausted` at or above the cap. After the reply it debits whatever the provider reported in `usage.total_tokens`, on every path: a normal answer, a refusal, a tool call, and a streamed turn, where the debit runs in the stream's `flush` because usage only arrives with the last chunk. The debit is deliberately non-fatal: a failed write to the ledger costs a log line, never the person's answer.

THIS IS A CAP, NOT A RESERVATION

Revision D described a proxy that reserved the worst-case cost before calling and settled afterwards under a check constraint. That is not what shipped. What shipped reads the balance, calls, then adds what was billed. A turn that starts at 149,999 tokens will complete and push the row past the cap; the next turn is refused. The overshoot is bounded by one turn, and one turn is bounded by `MAX_CONTEXT_LENGTH` plus the model's own output limit, which is why the three bounds above matter as much as the cap. The reservation design is worth revisiting before a second person's turns and nudges share the same provider key.

WHAT A TURN COSTS IN TOKENS

Prices are not in the repository, on purpose: the model name is a string in `_shared/lifo.ts` and the key is in the function environment, so a price change is never a code change. The arithmetic that *is* in the code is the shape of a turn.

TURN	INPUT, ROUGHLY	OUTPUT, ROUGHLY	WHERE THE NUMBER COMES FROM
Coach chat, one question	SCOPE plus up to 32,000 characters of context plus the thread; about 8,000 to 10,000 tokens at the cap	A few hundred tokens; <code>verbosity: "low"</code> and <code>reasoning_effort: "low"</code>	<code>MAX_CONTEXT_LENGTH</code> , <code>TUNING</code>
Daily brief, answer, sector note	The task prompt plus the digest render; smaller than chat because there is no thread	One paragraph or one sentence, schema-constrained	<code>TASKS</code> table
Morning nudge	About 300 tokens	About 60 tokens	Comment in <code>_shared/nudge.ts</code>

At 150,000 chat tokens a day the cap allows something like fifteen to twenty full-context turns, or far more short ones, per person per day. At the list price OpenAI published for `gpt-5-mini` when this was written (a quarter of a dollar per million input tokens and two dollars per million output, a figure to re-check rather than trust), a person who hit the cap every single day of a month would cost about a dollar and a quarter in input and well under a dollar in output. Nobody hits the cap every day. The nudge, at 4,000 tokens a day, cannot reach a cent a month. The ceiling holds with room in it, and the cap is what makes that a statement about the worst day rather than the average one.

WHY LOW REASONING EFFORT

Medium effort is most of the wait and most of the output tokens. The thinking already happened on the device when `MetricsDigest` and `ContextBundle` were built: the model is handed sums, averages and deltas, not a year of rows to reason over. A question over a week of a person's own numbers is not a reasoning problem. If the provider ever rejects the tuning parameters, the door retries once without them, so a rejected parameter costs a slower answer, never no answer.

24.4 The lines that are not metered

SPEECH

ElevenLabs is the one paid service the app calls directly, and the one line the ledger does not see. Speech to text runs on `scribe_v2` per recorded question; text to speech runs on `eleven_flash_v2_5` and is billed per character. Three things bound it. Spoken replies are off until a person turns them on (`AssistantVoice.enabledKey` defaults to nothing). The text sent to be spoken is cleaned of markdown and cut at `ElevenLabsVoiceClient.maxCharacters`, 1,200 characters, on a sentence boundary, which is about ninety seconds of speech. And every spoken reply is also a chat turn, so it already sits under the token cap.

THE KEY SHIPS IN THE BUNDLE

`ELEVENLABS_API_KEY` is read from `Config/Secrets.xcconfig` into `Config/App-Info.plist` as `ElevenLabsAPIKey`, and `AppConfig.elevenLabsAPIKey` hands it to both clients. A signed bundle is extractable, so a stranger with the .ipa has the key, and there is no per-user cap on it because there is no server between the app and ElevenLabs. Today the only kill switch is rotating the key in the ElevenLabs dashboard and shipping a build. Moving speech behind an Edge Function with its own kind in `lifo_usage` is item 2 of the privacy plan in Appendix A, and it is a cost item as much as a privacy one.

PLAID

Plaid bills per connected Item per month, and the app can never call Plaid except through the four Edge Functions in Chapter 20. Two rules keep the count honest. `plaid-exchange` removes the Item at Plaid if inserting its row fails, so nothing bills for a bank the app cannot find again. And `plaid-disconnect` removes the Item at Plaid before deleting the row, so a disconnected bank stops billing the month it is disconnected. The environment is set by `PLAID_ENV`, which defaults to `sandbox`; sandbox Items are free, which is how the whole bank path was built and tested before a real institution was touched. The price per Item is Plaid's and is not in the repository.

SUPABASE

Everything server-side runs on one Supabase project on the free tier. The rows it holds per user are small: a profile, a few hundred note pages at most, friendships, messages, shared stats, and one token row per connected provider. Health rows, transactions and sector scores never reach it. The lines that would push a project off the free tier, in the order they would go, are Edge Function invocations (every coach turn, every sync, every OTP), then storage bytes (avatars are pinned to one JPEG per user at 512 points, so this is bounded by user count), then database size (notes are the only unbounded table, and ink drawings above 1.5 MB stay on the device). At a personal scale none of these is close.

SIGN-IN

Email codes go through Resend's SMTP, configured in `config.toml` with a sixty second `max_frequency`, and are free at this volume. SMS is the expensive channel and is treated that way: phone sign-in is offered only for US and India numbers, the `otp-start` guard refuses a resend inside 45 seconds and rate-limits by pumping prefix, and the committed configuration short-circuits every test number before Twilio is called. The Twilio price per verification is Twilio's and is not in the repository.

FREE BY CONSTRUCTION

APNs costs nothing and the nudge sends at most once per user per local day, claimed by the primary key on `nudge_log` before the model is asked. The universal-link site is two static files on Vercel's hobby tier. YouTube embeds are free and keyless; the catalog is a read-only table of 122 verified ids seeded by a script that checks each one against the oEmbed endpoint, which is also keyless. Apple's Foundation Models cost nothing per call, which is what makes the on-device tier a real fallback rather than a cheaper cloud.

24.5 Why the ceiling holds

The obvious approach does not work:

```
read spend -> if under cap -> call -> add cost
```

Two concurrent requests both read a balance under the cap and both proceed. The app's answer is not to pretend this cannot happen but to make the damage one turn wide and every turn small.

PROPERTY	HOW
The add is atomic	<code>lifo_debit</code> is one <code>insert ... on conflict do update</code> ; two debits never lose each other
The overshoot is bounded	One turn at most past the cap, and a turn is bounded by 64 messages, 16 tools and 32,000 context characters before the model is called
Every path is metered	Refusals, tool calls and streamed turns all debit; an unmetered path would be a free one, and a looping client would find it
Budgets do not bleed	Chat and nudge are separate kinds with separate caps; a heavy evening of questions cannot cost the next morning's nudge
The client cannot read or write the ledger	RLS with no policies, execute revoked from every client role
The prompt cannot be widened from the client	SCOPE is prepended on the server and a thread carrying its own system message is refused
A missing key fails closed	No <code>OPENAI_API_KEY</code> means the nudge sends its fallback sentence and the coach falls back to the device; nothing is attempted at a guessed cost

The nudge adds one more guard the chat does not need: the rule decides whether to send and the model only phrases. Letting a model decide would bill every user every night and calibrate toward yes. Rules are free and instant, and the fallback sentence in `_shared/nudge.ts` is what gets delivered when the model is down or the 4,000-token allowance is spent, so an outage costs nothing and still says something.

24.6 What it costs today

Measured usage does not exist yet: the app has one real user and a few TestFlight testers, and the ledger is the only meter. What can be said honestly is the shape.

LINE	TODAY	AT THE CEILING
Cloud inference	Cents per month; most days do not reach a tenth of the cap	About a dollar and a quarter if the 150,000 cap were hit every day, plus the nudge at under a cent
Speech	Zero unless a person turns voice on	Unbounded per person today; the 1,200-character cut and the chat cap are the only limits
Plaid	Sandbox, free	One Item per connected bank, Plaid's monthly rate each
Everything else	Free tier	Free tier
Allowance	\$2.00 per user per month	

The sensitive input is not the per-token price, which the ledger bounds, but the two lines the ledger does not see: how many people turn voice on, and how many banks each connects. The first is fixed by moving speech server-side under its own kind. The second is a product decision, and the honest reading of the ceiling is that a person with three banks and voice on is over it, and a person with one bank and voice off is nowhere near it.

WHY NOT THE CHEAPEST MODEL

Providers list models an order of magnitude cheaper than `gpt-5-mini`. They are not better than Apple's on-device model at the questions this app gets, so escalating to one means paying for a worse answer than the free tier already produced. The selection rule is not cheapest available but cheapest model meaningfully more capable than the device, and the cap is what makes that affordable. Changing the model is an edit to one string in `_shared/lifo.ts` and a function deploy, not an App Store release.

24.7 Study notes

READ IN THIS ORDER

1. `supabase/migrations/20260827090000_lifo_usage.sql` : the ledger and why it has no policies.
2. `supabase/migrations/20260830090000_proactive_nudges.sql` : the `kind` column and the rewritten debit.
3. `supabase/functions/_shared/lifo.ts` : the caps, the bounds, the task table, the tuning and its retry.
4. `supabase/functions/lifo-agent/index.ts` : the balance read, the call, the debit on every path.
5. `supabase/functions/lifo-nudge/index.ts` and `_shared/nudge.ts` : rules decide, the model phrases, the fallback sentence.
6. `LIfiOS/Features/Coach/Model/ElevenLabsVoiceClient.swift` : the character cap and the cleaning before send.
7. `supabase/functions/plaid-exchange/index.ts` and `plaid-disconnect/index.ts` : the two rules that keep the Item count honest.

Why are the caps constants in the function rather than a check constraint in the schema?

Changing a cap is then a deploy, not a migration, and the ledger stays a plain count that any future cap can read.

What stops a client from asking for the same answer in a loop for free?

Every path debits, including refusals and streamed turns, and the ledger is unreadable and unwritable from the client.

By how much can a person exceed the daily chat cap?

By one turn, because the check happens before the call and the debit after; the turn itself is bounded by the three request limits.

Which paid line is not metered at all today, and what bounds it?

ElevenLabs speech; voice off by default, a 1,200-character cut, and the fact that every spoken reply is also a capped chat turn.

Why does the nudge have its own kind and cap?

So a heavy chat evening cannot spend the next morning's nudge, and so the nudge, at 4,000 tokens, can never cost more than a fraction of a cent.

APPENDICES

Reference

Why the product exists, every decision that shaped it, and a glossary with a reading order for studying the code.

- A. Product thesis and roadmap
- B. Decision log
- C. Glossary and reading order

A Product thesis and roadmap

Why the app exists, in the words that were used before any of it was built, and where it stands on 15 September 2026: what shipped in the three weeks since Revision D, and what is still open.

A.1 The thesis

Every health app you already have is a silo. Whoop knows your recovery but not that you skipped breakfast. Apple Health knows your steps but not that you blew the grocery budget and slept badly because of it. Each one gives advice that is correct in isolation and useless in context.

Almanac is built on a single bet: **the value is in the join, not the collection**. A coach that can see you missed meditation, logged five hours of sleep, wrote "low focus" in a note, and went over budget can say something no single-purpose app can. That is the entire reason to build one app instead of using five.

Everything in the architecture follows from that bet. There is one table keyed by date that every source writes into and every reader reads from. The nine sectors of the Life board exist to give the join a shape a person can act on: not "here is a chart" but "here is the one lever that still moves this month." And LIFO, the coach, is handed the join already summed rather than being asked to reason over rows, which is what makes it affordable, private, and fast.

A.2 Product PR FAQ

WORKING BACKWARDS

Almanac is a personal almanac of your own life. It pulls together everything your devices already know about you: weight from your scale, recovery from your strap, movement from your watch, spending from your bank, the day from your calendar, and what you wrote down. It answers one question every morning, am I on track, and one question every month, where did the month go. Open it and a grid of dots shows the month at a glance. Ask LIFO anything and it answers from your numbers, out loud if you like. Nine cards score the parts of your life that matter, and the ones no device can measure get asked, not guessed.

Who is this for?

It began as one person's app, sideloaded on the author's own devices, and that constraint shaped the architecture. It is now on TestFlight with accounts, friends and groups, and the load-bearing assumption has shifted from "one user" to "every user's data stays on their own phone." The server holds what has to be shared or what has to survive a reinstall, and nothing else.

Why not just use Apple Health?

Apple Health is an excellent store and a poor interpreter. It will show you a chart of anything and tell you the meaning of nothing. Almanac reads from it, then adds the layer Health deliberately does not: a definition of a good day, a streak that survives days you forgot to log, a monthly score with the evidence behind it, and a coach that reads across domains.

Does my data go to a server?

Your health rows, your transactions and your sector scores never do. Your notes, your profile, and the three figures you choose to share with friends do, under row-level security that names friendship in the policy. When LIFO answers from the cloud it is handed derived numbers, never a heart-rate series, and every answer carries a disclosure you can open that shows exactly what was sent. Chapter 1 draws the boundary; Chapter 14 enforces it.

What is explicitly not in this product?

Screen time analysis. It was requested and it cannot be built, because Apple's API forbids reading the values into the app; see Chapter 19. Food logging by photo, which Revision D promised, has not been built and is not scheduled. Guest mode was removed: a store belongs to an account.

What is the hardest part?

Not the integrations. It is friction: any app that requires daily manual logging dies in about eleven days. Every product decision here is subordinate to keeping the daily cost near zero, which is why a note is one line into an Inbox, a workout is one tap on a video, and the sectors that need a human answer ask two questions a month, not twenty.

What does it cost to run?

Under two dollars per user per month, by design and by a ledger in the database. Chapter 24 has the arithmetic and the two lines that are not yet metered.

A.3 What shipped since Revision D

Revision D was written on 25 August 2026. Between then and 15 September, 436 commits landed on `main`. Each project got its own spec and plan under `docs/superpowers/`, and each rewrote or added one chapter here when it landed. Grouped by area, with the dates the work was merged:

AREA	DATES	WHAT LANDED	CHAPTER
Sign-in and accounts	25 to 27 Aug	Email six-digit code as the default channel; the returning-user door; Twilio failures mapped to causes a person can act on; a six-box code field; every account gets its own store and several can share a device; the session renews before the hour is up; sign out stays signed out	4
Life board and sectors	25 to 27 Aug	The nine sectors named; evidence rows and one shared scoring rule; scorers for Body, Money, Mission, Growth, Mind and Soul; check-in questions for the relational sectors; the on-device sector note; the Life board as the fifth tab, the wallet deck, the monthly close; sector history in four bands; the in-flight month scored as a reachable band, coasting and perfect-finish projections, levers ranked by what still moves the score	13
Money	26 Aug to 1 Sep	Real bank sync through Plaid: four Edge Functions, credentials behind service-role-only RLS, a per-Item cursor on the device, transfers kept out of income and spending; budgets as buckets with a single holder per claim key and an unclaimed-spend band; the rebuilt finance tab as banded sections behind a rail; merchant entity, mask, subtype and both balances decoded; cents exact through MoneyParts ; merchant logos; drill-down pages by category and merchant; the three charts; collapsible cards; the sample month removed	11
Calendar and Today	26 Aug to 1 Sep	The calendar core: EventKit behind a source protocol, a windowed SwiftData cache, dedup merge, free slots; agenda, upcoming and the day schedule on Today; add and edit events from the agenda; a month you can walk through; a metric opens on a page of its own; today's numbers keep moving while the app is open	6
The calendar assistant	26 to 27 Aug	The Assistant target with six calendar tools; on-device session turns binding tools; the confirmation broker gating and capping writes; chat conversations persisted locally; the assistant sheet from Today; the calendar drawn as a day you can move around in	15
LIFO	26 Aug to 15 Sep	The <code>lifo-agent</code> Edge Function with a JWT gate, server-side task config, guardrails, reply parsing and a daily token ledger; the device-assembled context bundle with audience renders; the widened Whoop digest; cloud first and the device after; the conversational core with tools run against the cloud from the device; tier preference; LIFO remembers the conversation; the aura, then the orb, then text and voice split into two screens; three voices; streaming answers; the mic closing when the sentence ends; one spoken line first with the cards following the voice; proactive nudges, LIFO speaking first	14
Health and wearables	26 to 27 Aug	Apple Health read in full, showing what exists and filling the gaps Whoop leaves; Health connected from onboarding and Settings; cycle data asked separately and only where it applies; Fitbit as a second wearable with server-held tokens under a refresh lease; the arbiter merging contested columns by rank instead of a boolean; the fortnight trends as minimal lines; the sleep dashboard	7, 19
Notes	26 Aug to 13 Sep	The notes tab rebuilt as a PARA library with Supabase sync; iPad three columns, hardware keyboard, the Pencil writing and drawing in place; the index layer: to-do rows, a wiki-link edge set, a title index rebuilt once; a capture-first Inbox; the phone drawer, later replaced by a sheet; the plan migrated into pages, habits kept as plan entries	12
Profile and social	26 to 28 Aug, 14 Sep	A profile screen behind the avatar with settings inside it; the profile and avatar kept on the server, the bucket closed to the anonymous internet; profiles, friendships and messages with RLS doing the guarding; find people, message a friend, share	16, 17

AREA	DATES	WHAT LANDED	CHAPTER
		three figures; requests and conversations given a place to wait; groups and wellness leaderboards	
Design and shell	26 to 28 Aug, 14 Sep	The app renamed Almanac with a new icon; every font on one type scale; the iPad rail that steps aside; landscape layouts; the three actions collapsed into a fan, later flattened into every tab's bar; an animated first-run tour and the dotted cat; push-state tints and the live gradient	3, 5
Live sessions	14 Sep	Heart-rate zones from age, estimated effort from seconds in zone, capacity, ceiling, battery and push state; a live readout the widget can decode; the glass HUD and the gradient hero; a Live Activity with live readings; sensors that reconnect on their own; the glass rings over the hero and the video	8, 18
Watch companion	14 to 15 Sep	A pure rep counter on wrist acceleration; watch wire types; the session handed to a nearby Apple Watch and run and mirrored from the wrist; reps and sets on every surface; correct a rep and recover an active workout	9
Workout library	14 to 15 Sep	A verified table of curated videos and a read-only client; catalog cache, training preferences, workout split and video on the record; a planner for today's workout; the library with today's plan; a video played behind the live HUD with the nocookie origin fix; bookmarks, scheduling and search; full screen and landscape video with a HUD you can move; the equipment column given a default so installed stores migrate	10

THE V1 TO V7 SLICE SCHEME IS RETIRED

Revision C numbered work V1 to V7 and Revision D carried a six-row project table. Neither maps onto what happened: the calendar shipped before nutrition, the Life board was not in either list, and the API backbone on Cloud Run was never built because Supabase Edge Functions turned out to be enough. The table above is the record; the table below is the live list.

A.4 What is still open

Everything here is known, written down somewhere in the repository, and not done. The order is roughly the order it should be picked up.

ITEM	STATE	WHERE IT IS RECORDED
Privacy plan item 1: the off-device prompt sends spending-by-category totals, never transaction rows	Done on the branch <code>feat/privacy-aggregate-prompts</code> , not merged	Worktree <code>.claude/worktrees/privacy-aggregates</code>
Privacy plan item 2: move ElevenLabs behind an Edge Function (<code>lifo-speak</code>) and rotate the key that ships in the bundle	Open; the function does not exist	24.4, Chapter 14
Privacy plan item 3: the <code>profiles</code> select policy is <code>using (true)</code> and the row now carries birth date, height and gender; split or tighten it	Open	Chapter 20
Privacy plan item 4: encrypt <code>plaid_items.access_token</code> , and the Whoop and Fitbit tokens, at rest with the key in the function environment only	Open; accepted as a single-user trade today	Chapter 23, secrets tier 3b
Privacy plan item 5: delete the unused server-side health tables from the initial schema; <code>lifo-nudge</code> still reads <code>daily_metrics</code> , so that read must move first	Open	Chapter 20
Privacy plan item 6: file protection on the SwiftData store	Open	Memory note, 2 September 2026
Privacy plan item 7: an account deletion path; sign out leaves the local store on disk and Apple requires deletion	Open	Chapter 4
Privacy plan item 8: a <code>PrivacyInfo.xcprivacy</code> manifest, then a Privacy screen in Settings stating the three guarantees	Open	Memory note, 2 September 2026
The assistant's one-time privacy disclosure before first use, promised in the calendar spec	Not implemented; every LIFO answer carries a per-answer disclosure instead	Chapter 15
Five watch hardware checks: watch launch and heart-rate hand-off within ten seconds; a watch-started session raising the phone Live Activity; rep accuracy over ten slow curls; plus the two checks after them and one recovery-behaviour check	Pending on a physical iPhone and Watch; only the no-watch fallback is exercised by the simulator checks	<code>docs/design/watch-companion/report.md</code> , Chapter 9
The landscape capture of the video player	Pending; the layout is reachable now that <code>OrientationLock</code> lets the player rotate, but no capture has been committed	<code>docs/design/workout-library/report.md</code> , Chapter 10

ITEM	STATE	WHERE IT IS RECORDED
Proactive LIFO slice 3: device-side triggers, sector snapshots on the server, open commitments	No code; slices 1 and 2 shipped	docs/superpowers/specs/2026-08-28-proactive-lifo-design.md
The notes Inbox screen and its view model compile but nothing routes to them since the phone shell changed on 13 September	Dead code to delete or re-route	Chapter 12
CoachViewModel still builds a CoachRouter it never calls	Safe to delete	Chapter 14
The savings-goal band reads two defaults keys nothing in the app writes	Open; "Set a goal" opens the bucket editor instead	Chapter 11
scripts/check-typography.sh fails on three font calls in the intro and signup screens	Open	Chapter 3
The cloud budget is a cap with a one-turn overshoot, not a reservation	Accepted for now; revisit before a shared provider key serves many people	24.3
Food logging by photo	Not started; the vision role in the router has no task behind it	Revision D, Appendix A
The feature layer has no unit test target; view models are covered by native checks and the design preview	Open since Revision B	Chapter 23

WHY THE CHEAP MODULES WERE NOT AN AFTERTHOUGHT

The Life board, the workout library and the nudges are each one or two SwiftData models, one hue, one feature folder and a handful of pure functions in the package, assembled from components that already existed. They were cheap because the foundations were built properly: the design system, the daily spine, the MVVM template, the preview harness. They also carry disproportionate value, because every domain added to the join makes the coach materially better at its job.

A.5 Study notes

READ IN THIS ORDER

1. `docs/superpowers/specs/2026-08-10-life-os-design.md` : the original design, to see what survived.
2. `docs/superpowers/specs/2026-08-25-life-sectors-spine-design.md` : the nine sectors, the largest addition since.
3. `docs/superpowers/specs/2026-08-26-lifo-agent-design.md` and `2026-08-28-proactive-lifo-design.md` : the coach's cloud tier and its slices.
4. `docs/design/watch-companion/report.md` and `docs/design/workout-library/report.md` : what a QA report looks like and what "pending" means in one.
5. The memory note on the privacy plan, dated 2 September 2026.

What is the one bet every architectural decision follows from?

The value is in the join, not the collection: one table keyed by day, every source writes in, every reader reads out.

Which Revision D promise was never built, and why does it not matter to the architecture?

The Cloud Run API backbone; Edge Functions carried every server job that arrived, including long Whoop syncs under a lease.

Which open item is a cost problem as much as a privacy one?

Moving ElevenLabs behind an Edge Function: the key in the bundle has no per-user cap.

What changed about the load-bearing assumption between Revision D and E?

From "one user, no accounts" to "many accounts, each person's data on their own phone"; the server holds only what must be shared or must survive a reinstall.

B Decision log

Every decision that shaped the app, with the choice made and the reason it was made. Rows from Revision D are kept where they still hold and corrected where they no longer do; the rest were collected from the specs, the commit history and the chapters of this edition.

The log is grouped by area. A row belongs here when a reasonable engineer could have chosen otherwise, so that the next person, usually the author, does not rediscover the reason as a mystery bug. Where a row reverses an earlier one, the earlier one is gone rather than struck through; git has the history.

B.1 Product and scope

DECISION	CHOICE	REASON
Audience	One person at first; accounts and friends since 27 August	The single-user constraint shaped the architecture; what survives it is that every person's data stays on their own phone
App name	Almanac, from 26 August; package LifeOSKit and bundle id unchanged	Renaming targets and identifiers churns the project and the keychain for no user-visible gain
Tab bar	Five tabs: Today, Health, Money, Notes, Life	Health absorbs activity, weight, recovery and wellness; the bar is the scarcest surface in the app
Settings placement	Behind the avatar on Today, as a full profile page with settings inside it	Spends no tab slot on a screen opened once a month; the profile is what a person taps first
Quick actions	Three actions (assistant, LIFO, begin activity) drawn flat in every tab's bar	The bottom-corner fan hid the most-used control on four of five tabs
Guest mode	Removed	A store belongs to an account; signed-out data would become the next sign-in's
Screen time	Cut	Apple's API forbids reading the values into the app
Food logging	Photo to AI estimate, decided; not built	Lowest friction if it is ever built; the vision role in the router has no task behind it
Every metric optional	Yes	A false zero is worse than a blank in a health app
Missing-value rendering	Tiles take a nil String and render the dash	A rule implemented in nine places is broken in one of them
Dot grid rule	3 of 4, configurable	A vague rule makes the whole grid meaningless
No-data days	Excluded from streaks	Punishing a charging night teaches distrust of the streak
Cost ceiling	About two dollars per user per month	The app ships free, so unit economics cap architecture and model choice

B.2 Architecture and data

DECISION	CHOICE	REASON
UI architecture	MVVM, one folder per screen	Derivation becomes testable without a simulator; a file's path names its layer
Views and the database	No @Query in a view	Keeps persistence types out of layout code and derivation out of body
Store surface	Protocols returning value types	Stubbable in tests, and nothing above the store can mutate the database
Shared UI	Components/, split Model, ViewModel, View	A shared piece that owns state has an obvious home
View model injection	attach(context), not protocol init	Pragmatic; it is also why the feature layer has no unit tests and leans on native checks
Reload trigger	ModelContext.didSave plus foregrounding	A save is the only thing that can change what a screen shows
Data topology	Local first; Supabase holds what must be shared or must survive a reinstall	A health app that waits on the network is one you stop opening
Conflict resolution	Field-level last write wins	Row-level would let a nightly job discard a manual entry
Merge rule across sources	Rank per metric via MetricArbiter; same rank may overwrite	A boolean fillGapsOnly could not tell a Whoop value from a typed one once a second strap existed; equal rank lets a source correct itself
Unattributed typed metrics	Read as manual, rank 4	Otherwise the first Health sync after upgrade replaces a weight the person typed
Provenance storage	String bag on DailyMetrics, not a typed enum	Persistence sits below Integrations in the package graph and cannot import MetricSource
Long-tail Health metrics	JSON extras bag keyed by HealthMetric.rawValue	Forty-seven metrics would mean a schema change each; a missing key stays distinct from zero
Enum storage	Raw strings with typed accessors	A new case is a code change rather than a migration
New @Model fields	Optional or defaulted, guarded by SchemaMigrationTests	A mandatory array with no default stopped installed stores opening and hung the app on the spinner
Account isolation	One SQLite file per account under Accounts/uid, plus a defaults suite per account	A userID column needs a predicate on every read of twenty models; a file cannot leak rows it does not hold
Legacy store on upgrade	Moved into the first account once, with -wal and -shm	Scoping without moving would look like the app deleting a year of notes
Re-root per account	.id(scope.id) on AppShell	A stale snapshot on screen after a switch is the same class of leak as a stale row
Two locks before RootView	A session and an open store	Mounting the tab tree with no container reaches the first fetch and CoreData throws an uncatchable exception

DECISION	CHOICE	REASON
Width class	Package enum LayoutWidth, mapped once in RootView	UserInterfaceSizeClass is UIKit-only and the package builds for macOS so tests run without a simulator
Privacy boundary	Engines take a digest, never a row; audience decides the render	DailyMetrics is not Sendable, so the compiler stops a raw row crossing; the off-device render never carries raw physiology

B.3 Accounts and sign-in

DECISION	CHOICE	REASON
Sign-in	One-time code, no password	No secret at rest and no reset flow to build
Email as default channel	Email first; phone via a Twilio Verify Edge Function, US and India only	Free through Resend with none of the SMS failure modes; bounded regions stop SMS pumping
Email code	Six digits, same as phone, custom SMTP pushed to the project	The stock magic link cannot be typed into the app
Returning-user routing	AuthSession.hasProfile from user_metadata, decoded by hand with a false default	With OTP sign-in and signup are the same steps; only the account, checked after verify, tells them apart; a synthesised decoder would throw on old keychain sessions
Resend countdown	60 seconds	GoTrue's email max_frequency is 60 s; the phone guard is 45 s; the countdown must clear the stricter
Auth client	Hand-written REST, no SDK	Three calls; a dependency pulls in far more surface than that
Session refusal	Only 400, 401, 403 and 422 sign a person out	429, 5xx and transport errors are failures to answer, not refusals; signing out is destructive
Session keep-alive	Renew 45 s before expiry with a 5 s floor	Launch and foreground refreshes miss a person who keeps the app open past the hour
Concurrent refresh	Coalesced onto one task	Supabase rotates the refresh token on use; a second concurrent refresh presents a retired token
Refresh URL	Query string kept via URLComponents	appendingPathComponent encoded the question mark and every session died after an hour
Account roster	Ids and labels in defaults, one keychain item per user	The roster is not a secret; refresh tokens are; enumerating accounts must not need a keychain read each
Profile of record	A profiles row plus a private avatars bucket, cached per account on the device	user_metadata rides in the JWT; the old device cache was not keyed by account
Avatar freshness	Bucket listing updated_at stamp, three-case AvatarStamp	The path never changes, so a copy on disk cannot tell a replaced photo from an unchanged one
Avatar storage	Private bucket, JPEG only, name pinned to uid/avatar.jpg, 512 pt at renderer scale 1	A public bucket was reversed the same day; a tighter client limit would reject older installs silently
Height storage	Always centimetres; imperial fields translate only	Storing the typed unit goes wrong after a locale change
First-run tour	Overlay, not fullScreenCover, keyed on a per-account flag	iOS can restore a presented cover at launch and two covers contending for one window drops one
Push permission	Never at launch; asked from the notification inbox	A prompt before the app has shown anything worth interrupting for is the fastest way to a permanent no

DECISION	CHOICE	REASON
Push registration	The token is the primary key, keyed to the active account; deregister before the keychain clears	A token left under a signed-out account pushes one person's data onto another's phone

B.4 Design system and layout

DECISION	CHOICE	REASON
Type scale	Nine text steps, three numeral steps, the masthead as its own step; enforced by check-typography.sh	376 hand-written font calls across 28 sizes; a scale that cannot express the biggest thing in the app is missing a step
Numerals	Rounded face for digits only, never words	Rounded digits read faster at a glance; a rounded label beside a sans label is drift
Colours	AdaptiveColor light and dark pairs resolved by the view	UIKit dynamic colours are iOS-only; explicit pairs let terminal tests compare both halves
Liquid Glass	Real .glassEffect in GlassPanel, one place	The system renders true refraction; every panel gets it from one primitive
iPad rail	An overlay that withdraws on drag and keyboard and returns after 1400 ms or on tab change	Overlays claim no layout room; the delay stops flicker between flicks of a fast scroll
railInset	112 points applied by screens, not a safe area	Safe areas do not survive NavigationStack into a background-ignoring ScrollView
Orientation	Portrait on the phone by default; only the video player asks for landscape and releases it	Nothing else is laid out for landscape; release also requests portrait so the next screen is not stuck sideways
Code field	One hidden text field behind six boxes; digit logic as a free function	Keyboard, autofill and paste keep working; a test never touches a View type
Intro mascot	A SceneKit character with a shader for ink lines, 30 fps, paused when inactive or under Reduce Motion	Lit 3D beats an image sequence and costs little
Deck palette	Nine feature-local tones with one shared ink	ModuleHue has six cases; overlapping neighbours need distinct colours
Money charts	One ink at stepped opacity, no y-axis	The six pastels fail as a categorical palette; the figure is the number, the marks are the shape
Close banner	Dashed outline, not AlertBanner	The alarm styling is the anomaly surface; admin must not spend it
Harness	--design-preview --page= launch argument in DEBUG builds	Repeatable state without an account; a flag cannot be tripped by a stray argument
Native checks	In-process PASS/FAIL checks over real view models and defaults suites	The feature layer has no unit tests; checks are the substitute and QA reports paste their output

B.5 Health and wearables

DECISION	CHOICE	REASON
Weight source	HealthKit via Renpho's own sync	No Renpho integration needed at all
Health tab shape	Two halves, Health and Fitness, over one week strip	Both halves read the same day, so a comparison across dates can never be drawn by accident
Health freshness	HKObserverQuery per metric, 15 s throttle, foreground only; no background delivery	Timers are either wasteful or not live; background delivery needs an entitlement to write rows nobody is looking at
Health sync window	Today plus the day of the last sync and since; 30-day backfill cap	Sleep lands on the morning it ends; the grid shows a month
Health denial	Never claimed; the no-data copy points at Health, Sharing	HealthKit hides read-grant status by design
Cycle data	A second permission prompt, only when cycle tracking is on; the default from Health's own characteristic	A sheet asking everyone for menstrual data misreads most people; the signup gender is a form of address, not a biological field
Whoop	Direct API	Recovery and strain exist nowhere else
Whoop client secret	Edge Function only	Would otherwise be extractable from the binary
Whoop redirect	An https Edge Function bridging to lifeos://, not almanac://	Whoop requires https; every installed build knows lifeos; pointing at almanac stranded builds predating the rename
Whoop sign-in browser	Safari via UIApplication.open	Cloudflare's bot challenge never clears inside ASWebAuthenticationSession
Whoop ingestion order	Archive raw, then derive	A derivation bug six months out must be fixable without re-fetching against a rate limit
Whoop scopes	Colocated with endpoint paths in WhoopCollection	Listing them separately drifted once and 401'd freshly issued tokens
Whoop collection 401	Keep tokens, tell the person to reconnect	Clearing them once made connect then sync an unbreakable loop
Whoop sync window	Rolling 14 days, full re-fetch, no cursor	Upserts make re-syncing idempotent, so which records are new never needs answering
Whoop connection	Held server-side too; restored on a fresh install; refreshed under a UUID lock	A reinstall should not need a new authorization; concurrent refreshes would invalidate each other
Sleep attribution	The day the sleep ends	A night belongs to the morning you woke up
Whoop profile scope	Not requested	Name and email are identity, not signal
Fitbit tokens	Server row only, never on the device; one writer under a 30-second lease	The refresh token is single use

DECISION	CHOICE	REASON
Fitbit scopes	One space-separated parameter, pinned on both sides	Repeating the parameter narrows the grant silently and surfaces as a 403 much later
Sync staleness	One shared SyncStalenessPolicy	Every connection row answers 'stale since when' the same way

B.6 Live sessions and the watch

DECISION	CHOICE	REASON
Session timer	Wall-clock <code>ActivitySessionState</code> , no tick	Backgrounding never stops it; <code>Text(timerInterval:)</code> lets the lock screen count on its own
Effort model	Tanaka max heart rate, Whoop zone bands, weighted seconds through a saturating curve pinned by calibration tests	Agrees with the app the person already trusts; retuning is a diff, not a drift
Effort credit cap	5 seconds per reading	A gap in the stream is not effort that was measured
No birth date	No zones, effort or battery; a hint line instead	A zone on an assumed age is a reading the person never gave
Capacity resolution	Whoop first, then Health HRV against a 7-day baseline, then nil with the yellow ceiling	Cautious guidance without claiming a capacity that was not measured
Readout placement	<code>LiveSessionReadout</code> and <code>PushState</code> in <code>AppSurfaces</code> ; the effort model in <code>Integrations</code>	The widget must decode the readout without <code>HealthKit</code> or <code>SwiftData</code>
Live Activity publishing	A pure <code>LiveActivityThrottle</code> : 3 bpm step, 3 s spacing, 10 s floor, immediate on pause, push and reps	Stay inside <code>ActivityKit</code> 's budget while a strap streams every second, and test the rule
Draft writes	At most every 10 s on readings, always on state changes	One <code>UserDefaults</code> write per second is waste
Save identity	<code>WorkoutRecord.externalID</code> is 'almanac:' plus the timer's UUID, looked up before insert	Finish twice, or a half-completed save, yields one workout
Sensor auto-pair	Remember the last peripheral; auto-pair only when exactly one WHOOP is nearby	Another person's strap in a gym is not a reading this person owns
Session ownership	The watch owns the <code>HKWorkoutSession</code> when paired and Health saving is on; the phone never saves a Health workout for it	Exactly one workout in Health
Watch availability	activated and <code>isPaired</code> and <code>isWatchAppInstalled</code> , not <code>isReachable</code>	<code>isReachable</code> is true only while the watch app is already in the foreground
Hand-off authorisation	The phone asks Health before <code>startWatchApp</code>	Authorisation is shared; the first prompt should not land on the wrist during the phone's countdown
Wire format	Versioned, with a kind field; unknown versions ignored	Packet and command otherwise share every required key
discard versus end	Separate commands	On the wrist end means save; a refused session must not write a stub
Bridge teardown	Drop the session in the send's completion handler	Releasing in the same run-loop turn can cancel the send
Rep counting	A pure <code>RepCounter</code> in its own <code>Motion</code> target; manual and automatic tallies kept separate; labelled auto only on the watch	Testable on macOS; an automatic rep must add to taps, not overwrite them

DECISION	CHOICE	REASON
Rep axis lock	Only on a swing that is still rising	Settling can end mid-fall and lock the axis backwards
Widget data	One SurfaceSnapshot in the App Group, expiring at the earlier of midnight and six hours, tombstoned on sign-out	A reader can never join two accounts' fields; stale steps are worse than a blank

B.7 Workout library

DECISION	CHOICE	REASON
Catalog	A read-only Supabase table; revoke all then grant select; seed generated by an oEmbed-verified script with a trailing delete	Zero per-user API cost; default grants are revoked; a seed run is a full sync
Empty catalog response	Never replaces a non-empty cache	An unseeded table or a broken policy looks like an empty list
Planner	A pure function with four rules; push, pull, legs rotation from the last seven days of WorkoutRecord.split	Free, deterministic, every rule a test, and the reason string is the rule in words
Filter widening	Length, then split if no chip was tapped; intensity cap, equipment and query never widen	A tapped chip is an instruction; a battery ceiling and a kit list are facts
Saved list	The saved rows themselves, newest first, narrowed only by search	A heart is a decision already made; intersecting with the plan hid saves
YouTube embed	An iframe in the app's own page, based at youtube-nocookie.com with a matching origin; fs=0; ids validated	Errors 153 and 152 otherwise; YouTube's own full screen leaves no place for the HUD
Player web view	One WKWebView held in screen state across portrait, full screen and landscape	A branch change gives a new view identity and restarts the video
HUD position	An anchor as a fraction of free travel per placement, settled off the control strip on release	Survives size changes; keeps the terms' clearance wherever the rings end up

B.8 Money

DECISION	CHOICE	REASON
Money sign	Positive is money in, opposite to Plaid	Written into the type, because a silent flip makes every rollup lie
Savings rate with no income	nil, not zero	Undefined is not zero; the sharpest case of the app's central rule
Plaid calls	Edge Functions only, never the app	Every endpoint needs the client id and secret; an .ipa is a zip file
Where transactions rest	The device only; Postgres holds one row per bank with the token	A database compromise exposes a credential, not a ledger
Plaid cursor owner	The device, per Item, in the account's defaults	A server-advanced cursor loses a page to an app that crashed mid-ingest
Sync ordering	Ingest, remove, upsert accounts, then set the cursor	A cursor advanced before a successful ingest is a page Plaid never resends
Plaid exchange cleanup	Remove the Item at Plaid if the row insert fails	A stranded Item bills monthly with nothing left to find it by
Transfers and card payments	Excluded from income and spending by underscore-prefixed category code	They inflate both sides; the savings rate is computed from those two
Budgets	Buckets that claim raw category codes, one holder per key	Raw strings and merchant rules both fail silently when Plaid renames
Money cents	MoneyParts rounds to total cents first, then splits	19.995 must never render as 19.100
Sample money data	Removed entirely	Real rows only; an empty month shows the empty state
Merchant logos	Plaid's logo_url stored, shown in a paper-white circle, with a one-time cursor replay	Already in paid responses; reverses the inference-layer spec's no-logo decision
Money detail	Its own view model	Six months of rows must not reload on every save in the app
Money tab layout	Six sections behind a rail on the phone and chips on the iPad, bands on pure white	One long scroll made every question a scroll past the other five; two vertical rails on an iPad compete

B.9 Notes and the plan

DECISION	CHOICE	REASON
Notes hierarchy	PARA as an enum, not user folders	Sorting by actionability is the method; four renameable folders lose it
Note body	Blocks as JSON in the page, not SwiftData rows	Only ever read as a document; a relationship adds a migration hazard per field for no query
Cross-note queries	Derived NoteTask and NoteLink rows rewritten on every save, never pushed	Rebuildable from documents, so an indexer bug can only show a stale list
Inbox	One nullable filedAt, stamped by filing, never by editing	A bucket cannot say whether anyone decided
Index rebuild	The title index built once per rebuildAll and passed down	Per-page building made a rebuild quadratic on the main actor
Note sync order	Folders first, documents in batches of 25, push before pull	A page before its folder lands at the root; a pull first could flatten an unsent edit
Oversized ink	Kept local above 1.5 MB; the page still syncs its text	One runaway drawing must not stall every other page
Plan entries	One model, five kinds	A block renderer can lay out any entry; a new kind is an enum case, not a table
Habits	Stay as PlanEntry and HabitTick, not pages	Today ticks them live; a document version would break Today or duplicate the habit
Plan migration	Idempotent through originPlanEntryID; old rows left in place	A defaults flag is the wrong question once a second device syncs the pages; closed months keep their evidence

B.10 The Life sectors

DECISION	CHOICE	REASON
Sector scoring	The app proposes, the person decides; both numbers kept	Romance and Soul have no honest data source; the gap between the two columns is the signal
Sector rule	One weighted mean, weights in code, nil with no evidence	One path to test and explain; editable weights are a dial; zero in month one is demoralising
Sector prose	Rules score; the on-device model explains one sentence from the rows only	A model-judged score differs run to run and cannot be explained
Sector evidence	Archived at close, never recomputed	A goal changed in March must not rewrite February's reasoning
Sector screens	All nine open the same history screen; Body, Money and Mission add an Open row	The tabs do not show score history
Observations	Deterministic rules needing three calendar-adjacent months	A single unusual month must never fire one; free and instant
In-flight scoring	Projected MonthInputs through the same SectorEvidenceFactory, no new maths	A band that disagrees with the close is worse than no band
Coasting	Continue at month-to-date pace, not an empty tail	An empty tail puts Money's floor above its ceiling
Ceiling	Remaining days at GoalTargets, max against the running average	Reachable by definition; never asks for less than is already done
Money in flight	No ceiling without buckets	Zero spend is fantasy; any other figure is invented
Close scheduling	The oldest gap within twelve months, else the previous month	Rows only exist after a close, so a pure row query can never start the first one
Free-text answers	Debounced 600 ms, flushed on Skip, Next and dismiss; choices saved immediately	Every save reloads nine view models

B.11 LIFO and the assistant

DECISION	CHOICE	REASON
Coach tiers	Cloud first, the device as fallback; the person can pin either	The on-device model answered worse on the questions this app gets; the daily cap makes the cloud affordable; someone who chose the phone is owed that literally
Model selection	The app names a task; the function maps task to model	Changing a model is a function deploy, not an App Store release
Provider abstraction	Use Apple's session and tool types on the device; a thin wire format to the cloud	No provider SDK on the device; the server re-emits deltas in its own shape
Refusals across tiers	Never re-asked of the other model	A second model is not a second opinion on a refusal, it is a way around one
Prompt render per audience	onDevice may carry raw Whoop physiology; offDevice never does	HRV, SpO2, skin temperature and respiratory rate stay on the phone
Context on the wire	A context field, never a client system message	The server places it behind SCOPE; a client system message would land after the guardrail and outweigh it
Guardrail location	SCOPE lives in the Edge Function	A client cannot strip what it never carries
Two conversation prompts	Verbatim copies pinned by a Swift test and a Deno test	Two prompts meant to agree do not stay agreeing unless something checks
Streaming	Only tool-less turns stream, with usage in the last chunk	Tool calls arrive as fragmented JSON; an unmetered stream is a free path through the cap
Metering	Debit what the provider billed on every path, including refusals	An unmetered path is a free one
Budget kinds	lifo_usage keyed on user, day and kind; the nudge has its own 4,000-token cap	A heavy chat evening must not eat the next morning's nudge
Reasoning effort	Low, with one retry without tuning on a 400	The thinking already happened on the device when the digest was built
Spoken line	One SAY: line asked for first, split on the phone	One call, one debit; the voice starts before the last table row exists
Held cards	The answer held until playback starts, two-second ceiling	What is said comes first; silence is acceptable, a blank screen is not
Endpointing	A room-learned noise floor, two thresholds, an audio-time clock	A fixed level fails in either a bedroom or a cafe; a busy thread must not end a turn on top of the speaker
Two transcribers	Scribe owns the sentence; Apple only draws it	Letting both emit a final sends the answer to whichever finished first
Voice default	Off; three stock voices	A coach that starts talking in an unknown room is worse than silence; forty voices is a settings screen nobody finishes
Spoken reply cap	1,200 characters, cut on a sentence boundary	Billed per character; a voice stopping mid-number sounds like a fault

DECISION	CHOICE	REASON
Nudges	Rules decide, the model phrases; a fallback sentence per trigger	Letting a model decide bills every user every night and calibrates toward yes
One nudge per day	Primary key on nudge_log, inserted before phrasing	Two overlapping cron runs conflict and stop without any locking logic
Nudge payload	Sentence, trigger, day; never the figures	A phone off for days would open on stale numbers; the tap re-evaluates locally
Confirmation gate	A continuation inside ToolInvoker	Nothing the model emits can resume a gated call; structural, not a prompt request
Confirmed write on cloud failure	No device retry	The write already happened; a fresh invoker would raise a second card for the same create
Cards from tools, not prose	CalendarEventCollector carries values beside the text	Parsing a reply back into events trusts the model to restate a date correctly
Reply memory	Event ids on ChatMessage, re-resolved on open	Events move and die after the sentence; a stale card is worse than none
Free time	All-day events do not block	An anniversary is context, not a meeting
Recurring events	Read-only in the app; ids qualified with the occurrence start	Occurrences share one EKEvent id; a qualified id fails writes loudly instead of editing the wrong occurrence
Calendar cache	A SwiftData window of 30 days back and 90 forward; writes go to the provider and re-sync	Providers normalise all-day and time zones; trusting the local draft drifts
Calendar permission	The system grant and a per-account flag	A grant for one Almanac account must not authorise another

B.12 Social

DECISION	CHOICE	REASON
Social authorisation	RLS and security definer RPCs; client filters for display only	The client cannot be trusted to filter for privacy
Chat transport	PostgREST over URLSession, polling while open	No SDK, no realtime channel, nothing per user to pay for
Friendship update	A column grant restricts update to status	Postgres enforces column grants before policies; accept can never smuggle another field
Shared figures	A separate profile_stats table, not columns on profiles	profiles is world-readable for search; its own table can name friendship in the policy
Sharing off	Push explicit nulls	An upsert leaves omitted columns standing
Optimistic sends	Provisional rows kept in a separate pending array	A poll may only ever assign messages, so it cannot erase a send in flight
Group writes	Every write is an RPC with row locks	Invitee-must-be-friend, the 50 cap and owner succession need atomicity a bare insert cannot give
Group message retry	A client-generated idempotency key, unique per sender	A timed-out send that landed cannot post twice
Leaderboard scores	Effort, Recharge and Rest computed on the device; only scores, sources and day published	Raw HRV, heart rate and sleep never reach the endpoint
Recharge baseline	The same provider, seven distinct earlier days within 28, today excluded	Mixed providers and duplicate days would fake a baseline
Wellness leaderboards	A separate consent flag; score and source checked together; at most two days	Activity sharing must not imply health-score sharing

B.13 Backend and operations

DECISION	CHOICE	REASON
Server logic	Supabase Edge Functions, not a Cloud Run API	Every job that arrived fit in a function; the backbone in Revision D's roadmap was never needed
The function door	resolveUser: the caller id from their own JWT, never from the body	A function that trusts a user id in the body is a proxy for anyone
lifo-agent bounds	64 messages, 16 tools, 32,000 context characters	Without them the function is a general proxy on the project key
Cron schedule	schedule.sql run once, keys in Vault, not a migration	It needs the per-environment URL and the service-role key
OTP privacy	Numbers hashed with a pepper in otp_events; pumping-prefix limits	The abuse guard must not become a table of phone numbers
Secrets	What must never leak lives server-side; nothing in the xcconfig is a secret except the ElevenLabs key	A signed bundle is extractable
Build number	Tracked in git, bumped and committed before every upload	App Store Connect rejects a pair it has seen, after the upload finishes
Tests	Logic in the package, test-first, run from the terminal on macOS; view models covered by native checks	Anything that needs a simulator is slow to run and slower to trust
Parallel work	Each session in its own worktree under .claude/worktrees	Peers move HEAD on the shared checkout
Commits	A branch per change, conventional commits, never on main	History that reads as a list of decisions

B.14 Study notes

HOW TO USE THIS APPENDIX

1. Before changing a rule, search this table for the noun; if a row exists, the reason column is the thing to argue with, not the code.
2. When a row turns out wrong, replace it rather than adding a second; the log records the current decision, git records the old one.
3. Every spec under `docs/superpowers/specs/` has a decisions or alternatives section; a decision that lands in code lands here in the same change.

Which single decision do the most other rows depend on?

One table keyed by day that every source writes into; the arbiter, the digest, the sectors and the widgets all read from it.

Which Revision D rows were reversed rather than kept?

Four tabs, settings behind a gear, on-device first for the coach, and the personal-only audience.

What do the Money, Social and LIFO areas have in common in their reasons?

Each keeps a rule in a place the client cannot reach: a definer function, a server-side prompt, a device-only ledger.

C Glossary and reading order

The words this book uses in a particular sense, each in one line, and a route through the code for someone studying it one evening at a time.

C.1 Glossary

Terms are alphabetical. A term in `code` is a real symbol in the repository; the rest are the book's names for things the code has no single name for.

TERM	MEANING
AccountSession	The app-level object that owns which account scope and ModelContainer are open, and tears surfaces down when the account changes.
AccountStore	The roster of signed-in accounts in defaults plus one keychain session item per user id.
Agenda card	The week strip and day rows drawn under a calendar assistant reply, read live from the calendar store rather than parsed from the prose.
Arbiter	MetricArbiter.resolve, the pure rank-based rule that decides which source's value a DailyMetrics column keeps when two sources write the same day.
AssistantTurn	The chat router in Insights: cloud first, the device after; refusals and confirmed writes are never retried on the other tier.
Audience	Who a prompt render is for. An onDevice render may carry raw physiology; an offDevice render never does.
AvatarStamp	missing, unknown, or at(updated_at): the three answers to whether a person's avatar must be downloaded again.
Band (sector)	The floor-to-ceiling range an in-flight sector month can still close in, drawn on the Life board before the month ends.
Block	One typed line of a note: text, heading, to-do, sketch. A note document is an array of blocks stored as JSON.
Bucket (money)	A budget that claims one or more Plaid category codes and counts the month's spend against a limit; the unclaimed band shows what no bucket claims.
Bucket (notes)	One of the PARA folders a page can be filed into: Projects, Areas, Research, Archive.
Capacity	Today's battery percentage and its source, Whoop recovery or Health HRV, computed once at the start of a live session.
ChatEngine	The protocol both chat tiers implement: a thread and tools in, a reply and partial text out.
Check-in	Two or three questions asked at the monthly close for a sector with no tracked data, scored by CheckInScorer.
Claim key	The raw category code, or a manual entry's category, that a budget bucket counts. Held by at most one bucket.
Close (monthly)	The nine-step walk through the sectors of a month that has ended, one screen per sector, producing a SectorScore row each.
Coasting	A projection where the rest of the month continues at the month-to-date pace; the floor of the in-flight band.
Collapse id	apns-collapse-id set to lifo-nudge so a phone that was off receives one nudge, not several.
Composition root	RootView: owns every view model, attaches the model context, reloads on ModelContext.didSave and presents every sheet.
ConfirmationBroker	The actor whose continuation suspends a gated calendar write until the person answers the confirmation card.
ContextBundle	The digest plus money and sector figures, rendered under one character budget for a given audience.

TERM	MEANING
Cursor (Plaid)	Plaid's transactions sync bookmark, held on the device per Item and advanced only after a successful ingest.
DailyMetrics	The one SwiftData row per calendar day that every source writes into and every screen reads from. Every field is optional.
Dash rule	A missing value renders as a dash, drawn by the component from a nil String, never as a zero.
DayStatus	The evaluated result of a day against GoalTargets: onTarget, missed or noData. noData never breaks a streak.
Decided	The share of an in-flight sector's score that can no longer move: one minus the band's width over ten.
Definer function	A security definer RPC in Postgres that runs with its owner's rights and takes the caller only from auth.uid(); used where a row policy cannot express the rule.
Derived row	A NoteTask or NoteLink mirrored out of a page's blocks by the indexer; never synced, always rebuildable.
Design preview	The --design-preview --page= launch harness that renders any screen from an in-memory fixture without an account.
Digest	MetricsDigest, the value type of derived daily numbers that is the only health data an engine ever sees.
Effort	Estimated exertion from seconds spent in each heart-rate zone, weighted and passed through a saturating curve; a daily credit against capacity.
Effort, Recharge, Rest	The app's own 0 to 100 daily comparison scores for group leaderboards; not a vendor's strain, recovery or sleep score.
Endpointer	SpeechEndpointer, the pure state machine that decides from microphone level alone when a spoken question is over.
Evidence	The rows behind a proposed sector score, each normalised to 0 to 1 and weighted; their weighted mean scaled to 0 to 10.
Extras bag	DailyMetrics.extrasData, a JSON dictionary of the long-tail Health metrics that have no named column; an absent key means no reading.
Floor and ceiling	Where an in-flight sector closes if the month coasts, and if the remaining days hit the person's targets.
GoalTargets	The person's targets for steps, sleep, water and weight, read from UserGoals and evaluated per day.
Hand-off	The phone starting a workout on a paired Apple Watch and mirroring it, so the watch owns the HKWorkoutSession and the phone shows it.
hasProfile	The AuthSession flag, from user metadata, that tells a returning person from a new one after the code is verified.
HUD	SessionRings, the glass heads-up display of three rings and a rep count over a live session or a workout video; collapses to a bar and can be dragged.
Inbox (notes)	Pages whose filedAt is nil: captured but not yet given a home in a PARA bucket.

TERM	MEANING
Ink layer	The page-wide PencilKit annotation canvas on a note, distinct from an inline sketch block.
Item (Plaid)	Plaid's name for one connected bank login; billed monthly; one sync cursor each.
Lease	A short server-side lock, thirty seconds for Fitbit, so exactly one caller rotates a single-use refresh token.
Lever	One of seven dimensions (sleep, exercise, steps, water, journal, habits, spend) a projection can perfect on its own to show what still moves a score.
LifeSector	The enum of nine sectors: Family, Romance, Soul, Friends, Growth, Money, Mission, Body, Mind.
lifo_usage	The per-user, per-day, per-kind token ledger the daily caps read and lifo_debit adds to atomically.
LiveActivityThrottle	The pure decision on whether a new readout deserves an ActivityKit update, so a strap streaming every second stays inside the budget.
LiveSessionReadout	The one value the HUD, the lock screen and the Dynamic Island all render; every reading optional.
Module hue	ModuleHue, the one colour a domain screen owns, drawn as a pastel wash on its canvas.
MoneyParts	The whole and cents of an amount, rounded to total cents first and then split, so a figure never renders as 19.100.
Native checks	ActivityRecorderChecks and SurfaceNativeChecks: in-process PASS/FAIL runs over real view models that the QA reports paste verbatim.
NoteIndex	The derived task rows, link edges and title index rebuilt from note documents and kept current on every store write.
Nudge	The once-a-day push chosen by a rule and phrased by a model; LIFO speaking first. The fallback sentence covers an outage.
nudge_log	One row per user per local day recording that a nudge went and which trigger fired; never the text.
Observation	A deterministic note on a sector that needs three calendar-adjacent months, such as a repeated answer or a low score.
Origin id	The stable id a note page or migrated plan entry carries so a second device does not produce a second copy.
OTP	The one-time six-digit code sent by email or SMS; the only credential the app has.
PARA	Projects, Areas, Resources, Archive: the filing method the notes library sorts by actionability.
Pepper	OTP_PEPPER, mixed into the hash so otp_events holds no phone numbers.
Pill bar	PillNavBar, the floating capsule of tab buttons; horizontal on a phone, a vertical rail on an iPad.
Placeholder session id	The Live Activity the watch bridge requests during a background launch, adopted by the timer once the session is real.

TERM	MEANING
Primary wearable	The strap the person named as authority for contested metrics; defaults to Whoop.
Provenance bag	DailyMetrics.sourcesData, the parallel dictionary recording which MetricSource wrote each metric, read back by the arbiter.
Provisional row	A chat message shown before the server confirms it, with a negative id, kept in a separate pending array.
Pumping prefix	A country code plus the first national digits, used to rate-limit SMS fraud across a number range.
Push state	easy, onTrack, nearLimit or overLimit: how hard a live session presses against the day's capacity, with its headline and tint.
Quick actions	The three top-bar actions on every tab: the calendar assistant, LIFO, and begin activity.
<code>railInset</code>	The 112-point leading room regular-width screens leave for the iPad rail.
Readout	See LiveSessionReadout.
Refresh lease	See Lease.
<code>RepCounter</code>	The pure rep detector on wrist acceleration: two moving averages, an axis lock, signed hysteresis, a timing window and a refractory period.
<code>resolveUser</code>	The Edge Function door: the caller's id from their own JWT, never from the request body.
SAY: line	The one spoken sentence the model writes first when a reply will be read aloud; SpokenReply splits it from the shown answer.
<code>SCOPE</code>	The server-held guardrail system prompt every lifo-agent request starts with; a client cannot strip what it never carries.
Sector	One of nine areas of life scored monthly on the Life board.
<code>SectorEvidenceFactory</code>	The one place that turns a month's inputs into evidence rows, used by the close and the in-flight band alike.
Security definer RPC	See Definer function.
<code>SessionKeepAlive</code>	The pure schedule that renews an unexpired session 45 seconds before it lapses.
<code>SleepRecord</code>	One night of sleep as a source record; attributed to the day it ends.
Source ranking	manual 4, authority 3, secondary 2, fallback 1, unattributed 0; the primary wearable is the authority.
Spine	The DailyMetrics table seen as the thing every source writes into and every reader reads from.
<code>SurfaceRoute</code>	The almanac:// deep link (today, health, activity, notifications) the widgets and pushes open, strictly parsed.
<code>SurfaceSnapshot</code>	The credential-free App Group value the widgets and the watch read; a tombstone when ownerID is nil.
Tier	Which model a task needs at minimum: the device, or a named cloud task. A floor, not a model.

TERM	MEANING
Together	The social hub reached from the profile: People, Groups, Rankings.
Tombstone	A row with deletedAt set, kept until the server confirms the delete; also the SurfaceSnapshot written on sign-out.
ToolInvoker	The per-turn executor of tool calls: a six-call cap, the confirmation gate, error shaping, activity chips.
TrendSeries	A fixed window of readings with gaps kept as nil; the average ignores gaps and the latest is the last non-nil.
Two locks	The session and the open store, both required before RootView mounts.
Universal link	https://almanac.shivvyas.com/plaid-oauth , claimed by the app through apple-app-site-association so an OAuth bank lands back in the app.
UserGoals	The stored targets and the dot-grid rule (three of four by default).
UserScope	An account's id and the path to its own store file and defaults suite.
WatchWire	The versioned, kindred packet and command shapes exchanged between the watch and the phone.
Widening	The library filter relaxing length, then split, when today's plan finds nothing; a tapped chip never widens.
Wiki link	[[title]] inside a note block; resolved by folded title at index time into a NoteLink edge, which may dangle.
WorkoutBookmark	A saved and or scheduled mark on a catalog video; deleted when it is neither.
WorkoutRecord	One workout as a source record, with split, reps, sets, video id and an externalID that makes a repeated save one workout.

C.2 Reading order

Ten sittings, each an evening. The order builds: the package before the app, the shell before the screens, one feature at a time, the backend where a feature first needs it. Read each file top to bottom, then its test file if one is named, then grep the type across the app to see where it is used. Every chapter's own study notes go deeper into its files; this is the route between them.

SITTING ONE: THE PACKAGE AND THE SPINE

1. `LifeOSKit/Package.swift` : the eight targets and which may import which.
2. `LifeOSKit/Sources/Persistence/DailyMetrics.swift` : every field optional; the extras and sources bags.
3. `LifeOSKit/Sources/Persistence/MetricsStore.swift` : the protocol every screen reads through.
4. `LifeOSKit/Sources/Integrations/MetricSource.swift` : the ranks.
5. `LifeOSKit/Sources/Integrations/MetricArbiter.swift` : the merge rule, then its tests in `IntegrationsTests`.
6. `LifeOSKit/Sources/Persistence/LifeOSContainer.swift` : the schema list and the migration rule.
7. `LifeOSKit/Sources/Persistence/UserScope.swift` : one store per account.

SITTING TWO: THE SHELL

1. `LIfе0S/App/LIfе0SApp.swift` : @main to the first tab.
2. `LIfе0S/App/AppShell.swift` : the two locks and the re-root.
3. `LIfе0S/App/AccountSession.swift` : switching accounts.
4. `LIfе0S/App/RootView.swift` : the composition root; read `attachAll` and `reloadAll` first.
5. `LifeOSKit/Sources/Integrations/SupabaseAuth.swift` : the three auth calls.
6. `LifeOSKit/Sources/Integrations/SessionRefresh.swift` and `SessionKeepAlive.swift` : when a session is refused and when it is renewed.
7. `LIfе0S/Features/Onboarding/View/SignupScreens.swift` : the code field and the returning-user door.

SITTING THREE: THE DESIGN SYSTEM

1. `LifeOSKit/Sources/DesignSystem/Tokens.swift` : colours and hues.
2. `LifeOSKit/Sources/DesignSystem/Typography.swift` : the one type scale.
3. `LifeOSKit/Sources/DesignSystem/LayoutMetrics.swift` : phone against iPad.
4. `LifeOSKit/Sources/DesignSystem/PillNavBar.swift` and `GradientCanvas.swift` : the shell's chrome.
5. `LifeOSKit/Sources/DesignSystem/SoftCard.swift` and `MetricTile.swift` : the card vocabulary and the dash rule.
6. `LifeOSKit/Sources/DesignSystem/MonthGridLayout.swift` : calendar maths under test.
7. `scripts/check-typography.sh` : the guard.

SITTING FOUR: TODAY AND HEALTH

1. `LIfeOS/Features/Today/ViewModel/TodayViewModel.swift` : one snapshot per screen.
2. `LIfeOS/Features/Today/Model/TodaySnapshot.swift` and `TodayMetric.swift` : what a screen is handed.
3. `LifeOSKit/Sources/Persistence/CalendarStore.swift` and `Integrations/CalendarSync.swift` : the windowed cache and the merge.
4. `LIfeOS/Features/Health/View/HealthHubScreen.swift` : two halves over one week strip.
5. `LifeOSKit/Sources/Integrations/HealthKitReader.swift` and `HealthApply.swift` : reading everything Health holds and applying it through the arbiter.
6. `LifeOSKit/Sources/Integrations/HealthSyncWindow.swift` : which days a sync re-reads.

SITTING FIVE: WEARABLES AND THE BACKEND DOOR

1. `LifeOSKit/Sources/Integrations/WhoopClient.swift`, `WhoopSync.swift`, `WhoopDerivation.swift` : archive raw, then derive.
2. `LifeOSKit/Sources/Integrations/FitbitSync.swift` and `FitbitDerivation.swift` : the second wearable.
3. `supabase/functions/_shared/supabase.ts` : `resolveUser`, the door.
4. `supabase/functions/whoop-token/index.ts` and `fitbit-token/index.ts` : tokens held server-side under a lease.
5. `supabase/migrations/20260811164841_initial_schema.sql` : the first schema and what is now unused.
6. `supabase/config.toml` : auth channels, `verify_jwt` overrides.

SITTING SIX: LIVE SESSIONS AND THE WATCH

1. `LifeOSKit/Sources/Integrations/LiveEffort.swift` : zones, effort, capacity, ceiling; then `LiveEffortTests`.
2. `LifeOSKit/Sources/AppSurfaces/LiveSessionReadout.swift` : the one value every surface renders.
3. `LIfeOS/Features/Activity/ViewModel/ActivityRecorder.swift` : the session lifecycle.
4. `LIfeOS/Features/Activity/ViewModel/LiveHeartRateSensor.swift` : reconnecting on its own.
5. `AlmanacWidgets/WorkoutLiveActivity.swift` : the lock screen and the island.
6. `LifeOSKit/Sources/Motion/RepCounter.swift` and its tests : reps from acceleration.
7. `LIfeOS/App/Surfaces/WatchSessionBridge.swift` and `AlmanacWatch/WatchWorkoutController.swift` : the hand-off from both ends.

SITTING SEVEN: THE WORKOUT LIBRARY

1. `LifeOSKit/Sources/Sectors/WorkoutPlanner.swift` : four rules and a reason.
2. `LifeOSKit/Sources/Sectors/WorkoutLibraryFilter.swift` : widening that names its step.
3. `LifeOSKit/Sources/Persistence/CatalogVideo.swift` and `WorkoutBookmark.swift` : the cache and the marks.
4. `LIfeOS/Features/Workouts/ViewModel/WorkoutLibraryViewModel.swift` : the plan, the list, the refresh.
5. `LIfeOS/Features/Workouts/View/YouTubePlayerView.swift` : the nocookie origin fix.
6. `LIfeOS/Features/Workouts/View/VideoWorkoutScreen.swift` and `DraggableHUD.swift` : one web view across three layouts.
7. `scripts/catalog/verify.ts` : how a video earns its row.

SITTING EIGHT: MONEY AND THE LIFE BOARD

1. `LifeOSKit/Sources/Persistence/MoneyEntry.swift` and `MoneyStore.swift` : the sign convention and the rollup.
2. `LifeOSKit/Sources/Integrations/PlaidSync.swift` and `PlaidMapping.swift` : ingest, remove, upsert, then the cursor.
3. `LifeOSKit/Sources/Persistence/SpendBucket.swift` and `Sectors/BudgetPeriod.swift` : buckets and the assessment.
4. `LifeOSKit/Sources/Sectors/SectorEvidenceFactory.swift` : every score goes through here.
5. `LifeOSKit/Sources/Sectors/ProjectedInputs.swift` and `Leverage.swift` : coasting, perfect finish, levers.
6. `LIfeOS/Features/Life/ViewModel/MonthlyCloseViewModel.swift` : the walk and its pure policies.

SITTING NINE: LIFO

1. `LifeOSKit/Sources/Insights/Context/MetricsDigest.swift` : the privacy boundary in one parameter.
2. `LifeOSKit/Sources/Insights/Context/ContextBundle.swift` : the character budget.
3. `LifeOSKit/Sources/Insights/AssistantTurn.swift` : cloud first, device after.
4. `LifeOSKit/Sources/Insights/Chat/RemoteChatEngine.swift` and `ChatWire.swift` : the wire to lifo-agent.
5. `LifeOSKit/Sources/Insights/ToolInvoker.swift` and `ToolGate.swift` : the confirmation gate.
6. `supabase/functions/_shared/lifo.ts` and `lifo-agent/index.ts` : SCOPE, the caps, the debit.
7. `LIfeOS/Features/Coach/ViewModel/CoachViewModel.swift` and `Model/SpeechListener.swift` : speech in and out.
8. `supabase/functions/_shared/nudge.ts` : rules decide, the model phrases.

SITTING TEN: NOTES AND SOCIAL

1. `LifeOSKit/Sources/Persistence/NoteDocument.swift` and `NoteBlock.swift` : blocks as the unit.
2. `LifeOSKit/Sources/Persistence/NoteIndexer.swift` : the derived rows and the title index.
3. `LifeOSKit/Sources/Integrations/NoteSync.swift` : folders first, batches of 25, push before pull.
4. `LifeOS/Features/Notes/ViewModel/NotesViewModel.swift` : the shelf, the sheet, the commands.
5. `LifeOSKit/Sources/Integrations/SocialAPI.swift` and `GroupAPI.swift` : the thin PostgREST client and the RPCs.
6. `supabase/migrations/20260827130000_friends_and_messages.sql` and `20260914090000_social_groups.sql` : RLS doing the guarding.
7. `LifeOSKit/Sources/Integrations/SocialWellness.swift` : Effort, Recharge, Rest.

AFTER THE TENTH SITTING

Run `swift test --package-path LifeOSKit` and read the names of the suites as a table of contents of what the app promises. Then open Appendix B and, for any row you cannot explain from memory, go back to the chapter it names.

Almanac Engineering Handbook, Revision E, 15 September 2026. Written by Shiv Vyas from the code on `main`. Source of record: the code; `docs/superpowers/specs/` for work not yet built. Built by `docs/handbook/build.py`; page numbers by `render-pdf.sh`; the Kindle edition by `render-epub.sh`.